



PLATFORM DUE-DILIGENCE PACK

MyInstantAI Agent Marketplace.

A complete, code-grounded technical reference to the agent marketplace platform — architecture, runtime engine, data model, API surface, security, and operations — generated directly from the source at the snapshot below.

AUDIENCE

MyInstantAI engineering & technical due diligence

SNAPSHOT

commit `fb97438` · branch `main` · 2026-08-25 · v0.1.0

STACK

Next.js 15 (App Router) · React 19 · TypeScript 5 · Node 20 ·
pnpm workspaces · Postgres (pg, no ORM) · Docker → Railway
(Azure Container Apps = migration target)

REPOSITORY

`github.com/MalcolmGov/miai-agent-marketplace` (private)

SCALE

406,319 authored lines · 1,212 files · 500-agent catalogue
(~299,508 lines of catalogue IP) · ~75,900 lines platform
runtime · 69 API routes · 39 pages · 8,526 evals · 2,128 tools

How to read this pack

This is a 23-section technical due-diligence pack (00–22), generated from a direct read of the source at the snapshot above; every path is repo-relative and every figure is measured from the code. It corrects the headline scale to include the **catalogue IP** — the 500 agents (~299.5K lines) that are the platform's largest asset. Sections 00–03 orient (readiness, summary, statistics, structure); 04–13 are the technical core (frontend, backend, AI platform, API, data, integrations, security, compliance, DevOps, dependencies); 14–18 assess quality (performance, testing, documentation, hygiene, patterns); 19–22 quantify the asset and plan the technical handover (marketplace metrics, complexity & rebuild effort, visual reports, cutover plan). Appendices A–D are machine-generated reference tables. This is a technical document — commercial terms are intentionally out of scope.

Contents

- ▶ 00. Handover Readiness
- ▶ 01. Executive Summary
- ▶ 02. Codebase Statistics
- ▶ 03. Project Structure
- ▶ 04. Frontend
- ▶ 05. Backend
- ▶ 06. AI Platform
- ▶ 07. API Surface
- ▶ 08. Data Architecture
- ▶ 09. Integrations
- ▶ 10. Security
- ▶ 11. Compliance
- ▶ 12. DevOps & Cloud
- ▶ 13. Dependencies
- ▶ 14. Performance & Scalability
- ▶ 15. Testing & Evals
- ▶ 16. Documentation
- ▶ 17. Engineering Hygiene
- ▶ 18. Design Patterns
- ▶ 19. Marketplace Metrics
- ▶ 20. Complexity & Rebuild Effort
- ▶ 21. Visual Reports
- ▶ 22. Handover & Cutover Plan
- ▶ Appendix A – Complete Endpoint Index
- ▶ Appendix B – Environment Variable Catalog
- ▶ Appendix C – Dependency Manifest
- ▶ Appendix D – In-Repo Documentation Index
- ▶ Open items to confirm

00 Handover Readiness

This pack is written for MyInstantAI's engineering team: it evaluates, and prepares the operational handover of, the **MyInstantAI Agent Marketplace** — a running, multi-tenant Agents Marketplace + Agent Runtime built by Move Digital. This opening section states plainly what is **Live / Partial / Planned** against the code as it stands, what is ready to operate today, how it cuts over onto MyInstantAI's rails, and exactly what the partner must supply to go to production. Every claim below is grounded in the repository at commit `fb97438` (branch `main`); later sections prove each one in depth.

0.1 Verdict (one paragraph)

The platform is **operationally ready for a staged handover, not a rewrite**. The complete product loop — browse → rent → configure → connect Actions → paste knowledge → sandbox/live chat → embed → prepaid-token metering — runs end-to-end today on mockable rails, backed by the full **500-SKU agent catalogue**, an eval harness of **8,526 cases**, Postgres persistence with a file-store fallback, five channels, a connector framework, and a fail-closed token wallet. What remains before production is **wiring, not invention**: swapping three mock adapters (identity, wallet, model) for MyInstantAI's live endpoints, and choosing a hosting topology. The adapter seams, dual-flag production guards, health/readiness endpoint, and migration runbook are already in place to make that swap low-risk.

0.2 Platform at a glance (measured, catalogue included)

Re-measured on disk (excludes `node_modules`, `.next`, `dist`, `.git`):

DIMENSION	MEASURED	NOTES
Total authored	406,319 lines / 1,212 files	<code>.ts/.tsx/.js/.mjs/.json/.md/.sql/.css</code>
Catalogue IP (the largest asset)	299,508 lines (~74% of the repo)	<code>data/catalog</code> 290,061 + <code>data/catalog-consumer</code> 9,447
— B2B agents	552 <code>*.agent.json</code> files	500 SKUs (100 families × 5 markets, exactly 100 each: us/eu/africa/asia/oceania) + 52 legacy ZA aliases retained for deep links
— Consumer specialists	17 <code>*.agent.json</code> files	<code>data/catalog-consumer</code>
Eval cases across catalogue	8,526 (552 agents, ~15.4/agent)	every agent carries evals
Hand-written runtime (TS/TSX/JS/MJS)	~75.9K LOC / ~410 files	<code>apps/web</code> , <code>packages/*</code> , <code>scripts</code> , <code>e2e</code>
API routes / pages	69 <code>route.ts</code> / 39 <code>page.tsx</code>	<code>apps/web/src/app/api</code> , <code>apps/web/src/app</code>
SQL migrations	5 (<code>001_init</code> → <code>005_consumer_reminders</code>)	<code>apps/web/migrations</code>

DIMENSION	MEASURED	NOTES
ADRs / docs	5 ADRs + <code>docs/adr/README</code> / 33 top-level docs	<code>docs/adr</code> , <code>docs/</code>
CI workflows	5	<code>ci.yml</code> , <code>eval-nightly.yml</code> , <code>e2e-staging.yml</code> , <code>publish-image.yml</code> , <code>daily-brief.yml</code>
On disk incl. dependencies	~805 MB	<code>du -sh .</code>

Growth since the earlier Licence Proposal (quoted ~398K lines / 1,021 files / ~298K catalogue / 45 routes / ~631 MB): the repo has grown to **406,319 lines / 1,212 files / 299,508 catalogue / 69 routes / ~805 MB**. Report the current measured figures above.

0.3 Readiness matrix – honest Live / Partial / Planned

Tags mirror the in-repo Trust Center enum (`apps/web/src/lib/trust-content.ts` : `live` | `partial` | `via_provider` | `planned`).

CAPABILITY	STATUS	EVIDENCE IN REPO
Marketplace + Agent Studio product loop (rent→configure→sandbox→embed→wallet)	Live	Phase-1 exit criteria <code>docs/PHASE1_EXIT.md</code> ; 69 API routes, 39 pages
500-SKU catalogue + 17 consumer specialists (prompt/knowledge/tools/guardrails/evals)	Live	<code>data/catalog</code> (552 agents), <code>data/catalog-consumer</code> (18)
Eval harness (static + live) over the catalogue	Live	8,526 cases; <code>eval:suite</code> , <code>eval-nightly.yml</code>
Runtime engine – <code>runTurn()</code> , <code>createModelAdapter()</code> , guardrails, RAG retrieval, per- vertical workflows, bounded plan→act→observe, fail- closed metering	Live	<code>packages/runtime/src/index.ts</code> , <code>guardrails.ts</code> , <code>knowledge-retrieve.ts</code> , <code>workflows/</code>
Health / readiness endpoint (liveness + config introspection)	Live	<code>apps/web/src/app/api/health/route.ts</code>
Sandbox chat on mock model (works with zero partner creds)	Live	<code>MIAI_MODEL_MODE</code> default <code>mock</code> (<code>packages/runtime/src/index.ts:2105</code>)
OAuth 2.0 + PKCE connector framework; live execute for Slack / webhook / MCP; staging-live Slack, Google Calendar, Gmail, Calendly	Live (execute) / connectors otherwise Partial	<code>packages/connectors/src/live/handlers</code> <code>/[{slack,webhook,mcp}.ts</code> ; 19 provider presets

CAPABILITY	STATUS	EVIDENCE IN REPO
Channels: web chat, embeddable JS widget, Telegram webhook, App channel (WebView + SSE), MCP server, public <code>/api/v1/*</code>	Live	<code>/api/embed/chat</code> , <code>/agents/v1/agent.js</code> , <code>/api/consumer/telegram/webhook</code> , <code>/api/app/chat</code> , <code>/api/mcp</code> + <code>/api/mcp/tools/call</code> , <code>/api/v1/{embed/chat,openapi,rent}</code>
Postgres persistence + JSON file-store fallback	Live	<code>apps/web/src/lib/store.ts</code> ; migrations 001-005
Prepaid token wallet gating, fail-closed at zero balance	Live	<code>packages/wallet-adapter/src/index.ts</code> (mock + http); Paystack <code>/api/payments/paystack/*</code>
Security hardening: boot dual-flag guards, CSP, embed CORS, SSRF DNS-pin, JWT alg-pinning, agent-IP redaction	Live	<code>apps/web/src/lib/security-flags.ts</code> , <code>instrumentation.ts</code> , <code>packages/connectors/src/ssrf.ts</code> , <code>lib/csp.ts</code>
Compliance: DSAR export/erase, audit log, RBAC (owner/admin/agent/read-only), market packs, Trust Center	Live	<code>/api/dsar/{export,erase}</code> , <code>/api/audit</code> , <code>lib/trust-content.ts</code>
OIDC SSO (B2B Bearer) — adapter ready, mock default	Partial	<code>lib/auth.ts</code> + <code>lib/agents-auth.ts</code> (`MIAI_AUTH_MODE=mock`)
Wallet HTTP adapter — ready, needs partner API	Partial	`MIAI_WALLET_MODE=mock`
Model gateway / Azure OpenAI adapters — ready, mock model runs now	Partial	`MIAI_MODEL_MODE=mock`
PII redaction at write time	Partial	best-effort (email/phone/card/OTP), Phase 3 — <code>trust-content.ts</code>
Encryption at rest (AES-256-GCM v2 seals)	Partial	Azure Key Vault wrapping is next; legacy v1 HMAC seals still open for migration
Redis (rate-limit / sessions)	Partial	optional; <code>maxReplicas</code> stays 1 until provisioned (<code>health/route.ts</code> surfaces <code>redisPing</code>)
Azure Container Apps production tenancy	Planned	Bicep baseline <code>infra/azure/main.bicep</code> , not deployed; gated on managed-Postgres
SOC 2, SCIM/SAML, fully-native chat client	Planned	<code>trust-content.ts</code> (SOC 2 "in progress"); <code>/api/app/chat</code> SSE exists, native UI is backlog

0.4 What is operationally ready for handover today

- **A running multi-tenant platform.** Every query carries a `tenant_id` / `workspace_id` predicate; per-tenant public keys, knowledge bases, and domain allowlists. RBAC enforced on rent, wallet, configure, knowledge, OAuth, and DSAR.
- **The full 500-SKU catalogue as flat, portable IP.** Each `{id}.agent.json` is self-contained (`format` , `manifest` , `system_prompt` , `knowledge` , `tools` , `guardrails` , `evals`). Tiers priced in `packages/agent-protocol` (`RENT_USD` standard 349 / pro 699 / enterprise 1199; `RENT_EUR` also defined).
- **A proven eval + certification path.** 8,526 static/live eval cases; go-live certification waves (18 → 55 → 100 families) via `pnpm certify:goLive` (`docs/PILOT_PRODUCTION_BAR.md`).
- **The full connector + channel surface** (19 provider presets; web/widget/Telegram/App/MCP/public-API channels) and a **generated preset layer** (`packages/presets/src/generated-presets.ts` , 11,761 lines).
- **CI/CD and observability scaffolding:** 5 GitHub Actions workflows, multi-stage `Dockerfile` → Railway (`railway.toml` , `/api/health`), `gosu` privilege-drop entrypoint, Azure Bicep + `scripts/validate-azure.sh` , App Insights hooks, 5 ADRs, 33 docs, and a full `docs/MIGRATION_RUNBOOK.md` .
- **Two-package extraction seam.** The marketplace packages depend on only ~6 modules of the wider codebase, so standalone-container extraction is week-scale, not a rewrite.

0.5 Deployment / cutover topologies (high level)

#	TOPOLOGY	PARTNER EFFORT	TIMELINE
A	Partner calls the REST / MCP API from their own storefront	Nothing to deploy partner-side	Available now
B	Partner domain resolves to the hosted runtime with partner brand / TLS / URLs	DNS + config only (widget loader + snippet generator derive host at runtime)	~2 weeks — recommended first launch
C	Marketplace + per-agent setup embedded in the partner console via signed SSO handoff (no second login)	SSO handoff wiring	~3 weeks
D	Runtime ships as a container into the partner's own Azure tenancy ; partner owns the data plane	Azure tenancy	Gated on the managed-Postgres migration first

0.6 14-day technical fast-track

DAYS	OUTCOME
1-3	OIDC login end-to-end (<code>MIAI_AUTH_MODE=oidc</code> , issuer/JWKS verified against staging)
4-7	Wallet debit + model gateway on one hero agent — activate → chat → bill (<code>MIAI_WALLET_MODE=http</code> , <code>MIAI_MODEL_MODE=gateway</code>)
8-11	Custom domain + OAuth redirect URIs + UAT on a go-live shortlist
12-14	Cutover rehearsal; mock rails off (unset <code>ALLOW MOCK RAILS</code> + <code>I_UNDERSTAND MOCK RAILS IN PROD</code>); Live Ops green on the partner hostname

0.7 What we need from the partner to go live

1. **Production OIDC** — issuer, audience, JWKS URL, and a sample JWT carrying `workspace_id` (+ `user_id` / `roles`).
2. **Production wallet API** — debit / balance / top-up contract, with pause-on-zero confirmed.
3. **Model gateway** — base URL, auth, and tool-calling support (OpenAI-compatible, or Azure OpenAI endpoint/key).
4. **Cloud + DNS** — cloud subscription + region, and the production hostname / DNS for the Agents surface and embed loader.
5. **WhatsApp** — WABA / BSP ownership for the WhatsApp channel.

Until items 1–3 land (even as staging stubs), the platform runs correctly on its mock rails — so evaluation, sandbox demos, and UAT proceed in parallel without blocking on partner credentials.

0.8 How to verify readiness in one call

`GET /api/health` (`apps/web/src/app/api/health/route.ts`) is the single readiness signal. It reports `authMode`, `walletMode`, `modelMode`, boot-`hardening` (503 on failure), `database` (`configured` vs `file-fallback`), `storeBackend` / `storePing`, and `redis*`. Missing partner creds surface as `status: degraded` + `config: incomplete` while **still returning 200**, so liveness does not flap during bring-up; a hard `503` means boot hardening failed or the store is unreachable. This is the check the cutover smokes (`pnpm smoke:cutover`, `pnpm handover:staging`) assert against.

01 Executive Summary

MyInstantAI Agent Marketplace (`miai-agent-marketplace` , repo private, root `package.json` name `miai-agent-marketplace` v0.1.0) is a **multi-tenant marketplace of production AI agents plus the runtime that executes them**. It is two things in one deployable: (1) a catalogue of 500 pre-built, market-localised business agents that a workspace *rents* per month, and (2) a hand-written agent runtime — turn loop, model adapters, guardrails, retrieval, connectors, wallet metering — that runs those agents against a partner's own identity, wallet, and model rails through swappable adapters. Nothing in the runtime is hard-wired to a single provider: identity (`MIAI_AUTH_MODE`), wallet (`MIAI_WALLET_MODE`), and model (`MIAI_MODEL_MODE`) are each selected by environment flag, so MyInstantAI can drop the platform onto its own OIDC issuer, token wallet, and Azure OpenAI gateway without touching agent content or core code.

This is the corrected, re-measured picture. Every number below was counted from the working tree at commit `fb97438` (2026-08-25), excluding `node_modules/.next/dist/.git`.

1.1 Scale at a glance (measured, not estimated)

DIMENSION	MEASURED VALUE	HOW COUNTED
Total authored (ts/tsx/js/mjs/json/md/sql/css)	406,319 lines / 1,212 files	<code>`find ...`</code>
Catalogue IP (the largest, most valuable asset)	299,508 lines	<code>data/catalog</code> + <code>data/catalog-consumer</code>
– business catalogue <code>data/catalog</code>	290,061 lines / 555 JSON (552 <code>*.agent.json</code> + 3 index)	per-file cat
– consumer specialists <code>data/catalog-consumer</code>	9,447 lines / 18 files (17 specialists + <code>index.json</code>)	per-file cat
Platform runtime (hand-written ts/tsx/js/mjs)	75,768 lines / 412 files	excludes catalogue + generated
– of which generated presets	<code>packages/presets/src/generated-presets.ts</code> = 11,761 lines	tool→connector bindings
HTTP API routes (<code>route.ts</code> under <code>apps/web/src/app/api</code>)	69	<code>find ... route.ts</code>
Page routes (<code>page.tsx</code>)	39	<code>find ... page.tsx</code>
Tool definitions across catalogue	2,128 (~3.86 / agent)	JSON parse of <code>tools[]</code>
Eval cases across catalogue	8,526 (~15.45 / agent)	JSON parse of <code>evals[]</code>
Git commits (this repo)	307	<code>git rev-list --count HEAD</code>
Repo on disk incl. dependencies	~805 MB	<code>du -sh</code>

Growth since the Licence Proposal. The Licence Proposal quoted ~398K lines / 1,021 files / ~298K catalogue / 45 routes / ~631 MB. The repo has grown: **+8.3K authored lines, +191 files, +24 API routes, +~174 MB on disk**, and the API surface has expanded from 45 to **69 routes** (adding consumer memory, brief/reminders, Telegram, MCP, and payments endpoints). The `docs/TECHNICAL_SPEC.md` figures (dated 2026-07-31: ~14.5K platform LOC, ~102K catalogue, 22 routes, 25 commits) are now badly stale and should be read as a historical snapshot, not current state — the platform is roughly **5× the catalogue-JSON size and 3× the route count** it reports.

Scale discipline — always count the catalogue. The headline is **~406K authored lines**, of which **~299.5K (74%) is catalogue IP** and only **~75.9K is hand-written runtime**. The catalogue is the moat: 500 agents each carrying `system_prompt`, `knowledge`, `tools`, `guardrails`, and `evals`. Reporting only the runtime would understate the asset by 4×.

1.2 Two audiences, one runtime

The platform serves two distinct buyers off the same execution engine (`packages/runtime`, `runTurn()` + `createModelAdapter()`):

- **Business agents — rented per workspace.** 500 SKUs modelled as **100 families × 5 regional market packs** (ADR 0001 `docs/adr/0001-catalog-100x5.md`). Measured market distribution: `us` 100, `eu` 100, `asia` 100, `oceania` 100, `africa` 151 (100 + 51 legacy unprefixd ZA aliases retained for deep-link stability), plus 1 `global` = 552 files on disk. Each agent is a flat file `data/catalog/{id}.agent.json` in the `miai.agent-package/v1` format (`packages/agent-protocol/src/index.ts`: `AgentPackage` = `{ format, manifest, system_prompt, knowledge, tools, guardrails, evals }`). Rent is tiered — `RENT_USD` **standard \$349 / pro \$699 / enterprise \$1199** (with a parallel `RENT_EUR` 319 / 649 / 1099). Measured tier split: pro 333, standard 112, enterprise 107.
- **Consumer specialists — durable per-person memory.** 17 always-on companions in `data/catalog-consumer` (e.g. `money-coach`, `health-navigator`, `study-coach`, `trip-planner`, `job-hunt-coach`, `private-confidant`) backed by layered memory: channel sessions, durable facts, a life-graph of goals/people, per-person knowledge base, and a brief + reminders subsystem (migrations `002_consumer_brief` ... `005_consumer_reminders`; routes under `apps/web/src/app/api/consumer/*`). The newest commit (`fb97438`) adds "Sign in with Google" OIDC so memory is keyed to a real person.

1.3 Core capabilities

- **Agent runtime** (`packages/runtime/src/index.ts`): a bounded plan→act→observe turn loop over the agent's tools, a deterministic mock model for zero-cost evals, and live adapters selected by `MIAI_MODEL_MODE` — **mock** (default) plus **openai**, **anthropic** (`claude` alias), **gateway** (`http` alias, OpenAI-compatible), and **azure** (`createModelAdapter()`, lines ~2104–2124). Hybrid semantic+lexical knowledge retrieval (`selectKnowledgeForPromptAsync`, `knowledge-retrieve.ts`) switches from prompt-stuffing to embedding-ranked chunks when an embedder is configured (`RUNTIME_SEMANTIC_RETRIEVAL`), plus ~20 per-vertical workflow modules under `packages/runtime/src/workflows/` (pharmacy, hotel-guest, mobile-money, wealth-management, it-helpdesk, ...).
- **Fail-closed metering.** A prepaid token wallet gates every reply. On `state` `≡` `"paused_no_tokens"` or `balance.tokens` `≤ 0`, `runTurn()` returns a top-up message with `tokensDebited: 0` **before any provider call is made** (`index.ts` lines ~2174 and ~2231) — a zero-balance tenant cannot run up a bill.
- **Multi-tenant isolation & auth** (`apps/web/src/lib/auth.ts`, `agents-auth.ts`): B2B OIDC Bearer requiring `workspace_id`; consumer Google-OIDC session or mock; embed keys (`mia_pk_` public keys); webhook HMAC-SHA256; every query filtered by tenant predicate.
- **Integrations & channels** (`packages/connectors`): OAuth2/PKCE connector framework with AES-GCM sealed tokens across Google, Microsoft 365, Slack, Shopify, HubSpot, Xero, QuickBooks, Calendly, Zendesk; surfaced over web chat, an embeddable zero-dep JS widget, Telegram, WhatsApp, and an MCP server.

- **Persistence** (ADR 0003): Postgres via `pg` (no ORM) with a JSON file-store fallback on a `/data` volume; 5 SQL migrations (`001_init` ... `005_consumer_reminders`).
- **Payments**: prepaid wallet adapter (`mock` | `http`) with Pystack top-ups and `STRIPE_SECRET_KEY` support.

1.4 Why this is a licensable platform, not a demo

Three properties separate this from a prototype:

1. **The IP is content-heavy and operationally maintained.** 299.5K lines of curated agent content — 2,128 tool contracts and 8,526 eval cases — with a full catalogue-ops toolchain (`polish:catalog`, `generate:presets`, `catalog:ready`, `certify:goLive`, `eval:full`, `heal:evals`). This is months of content engineering plus ongoing ops, not a weekend clone (a point `docs/TECHNICAL_SPEC.md` itself makes for copyability).
2. **Provider-neutral adapter contracts.** Identity, wallet, and model are all swappable by env flag, with Azure OpenAI + Azure Container Apps as the stated migration target (`infra/azure` Bicep, `scripts/validate-azure.sh`). MyInstantAI can take handover onto its own rails without forking.
3. **Delivery discipline for handover.** 5 ADRs (`docs/adr/0001` – `0005`), 38 docs, 5 CI workflows (`ci.yml`, `eval-nightly.yml`, `e2e-staging.yml`, `publish-image.yml`, `daily-brief.yml`), a required "quality" merge gate, a testing pyramid (`node --test` units, Playwright `@smoke/@functional/@uat/@handover` including a 76-scenario handover pack, static+live evals), multi-stage Dockerfile → Railway with `gosu` privilege-drop, and security hardening (CSP, embed CORS allowlist, DNS-pinned SSRF checks, JWT alg-pinning, agent-IP redaction on public reads).

In short: a working, tenant-isolated, metered, provider-neutral agent platform whose dominant asset is a 500-agent, five-market catalogue — sized and instrumented to be operated and taken over, not merely demonstrated.

02 Codebase Statistics

All figures below were **re-measured directly from the working tree** at commit `fb97438` (`fb97438bbefceea6f667dcf099942113b41ac93d`, dated 2026-08-25) on branch `main`, repository `/Users/malcolmgovender/Projects/miai-agent-marketplace`. Every number in this section is reproducible with the exact commands shown. Line counts are `wc -l` (newline-terminated lines); `node_modules`, `.next`, `dist`, and `.git` are excluded from all "authored" figures.

Headline (read this first). The platform is **406,319 authored lines across 1,212 files**. The single largest and most valuable asset is the **500-agent catalogue: 299,508 lines of curated JSON IP (73.7% of the codebase)**. The hand-written platform runtime is a further **76,169 lines of TypeScript/JavaScript**. Any statement of scale that headlines only the ~76K of runtime code under-reports the asset by roughly 4×, because it omits the catalogue that the runtime exists to serve.

2.1 Total authored size

```
# Total authored lines/files across all source extensions (deps/build excluded)
find . \( -path './node_modules' -o -path '*/node_modules' -o -path '*/.next' \
  -o -path '*/dist' -o -path './.git' \) -prune -o -type f \
  \( -name '*.ts' -o -name '*.tsx' -o -name '*.js' -o -name '*.mjs' \
  -o -name '*.json' -o -name '*.md' -o -name '*.sql' -o -name '*.css' \) -print \
  | xargs wc -l | tail -1
#    => 406319 total
# same find piped to `wc -l` (count files) => 1212
```

METRIC	VALUE
Total authored lines	406,319
Total authored files	1,212
Extensions counted	<code>.ts .tsx .js .mjs .json .md .sql .css</code>
Commit	<code>fb97438</code> (2026-08-25)

2.2 The three-way split: Catalogue IP vs. Platform Runtime vs. generated code

This is the split that matters for valuation and handover. The 406,319 authored lines decompose as follows:

BUCKET	LINES	% OF TOTAL	FILES	WHAT IT IS
Catalogue IP (curated JSON)	299,508	73.7%	573	The 500-SKU agent catalogue + 17 consumer specialists — system prompts, knowledge, tools, guardrails, evals

BUCKET	LINES	% OF TOTAL	FILES	WHAT IT IS
Platform runtime (hand-written TS/TSX/JS/MJS)	76,169	18.7%	415	The Next.js app, runtime engine, connectors, scripts, tests
– of which generated (<code>generated-presets.ts</code>)	11,761	2.9%	1	Machine-generated presets (build artifact checked into <code>packages/presets/src</code>)
Docs / config / other (<code>.md</code> <code>.sql</code> <code>.css</code> + non-catalogue <code>.json</code>)	~30,642	7.5%	~224	ADRs, runbooks, migrations, Tailwind CSS, package/tsconfig JSON, eval reports

```
# Catalogue IP
find data/catalog data/catalog-consumer -name '*.json' | xargs wc -l | tail -1 # 299508 total
find data/catalog data/catalog-consumer -name '*.json' | wc -l # 573

# Platform runtime (hand-written code)
find . \( -path './node_modules' -o -path '*/node_modules' -o -path '*/.next' \
  -o -path '*/dist' -o -path './.git' \) -prune -o -type f \
  \( -name '*.ts' -o -name '*.tsx' -o -name '*.js' -o -name '*.mjs' \) -print \
  | xargs wc -l | tail -1 # 76169 total
(415 files)

# Generated presets (subset of the runtime bucket)
wc -l packages/presets/src/generated-presets.ts # 11761
```

Note on "generated" code: `packages/presets/src/generated-presets.ts` is 11,761 lines (227 KB) of machine-emitted preset definitions committed to source control. Subtracting it, the **truly hand-authored runtime is ~64,408 lines**. It is counted inside the 76,169 runtime figure above but flagged here so reviewers do not credit it as hand-written engineering.

2.3 Language / extension distribution

```
for ext in ts tsx js mjs json md sql css; do
  find . \( -path './node_modules' -o -path '*/node_modules' -o -path '*/.next' \
    -o -path '*/dist' -o -path './.git' \) -prune -o -type f -name ".*$ext" -print \
    | xargs wc -l | tail -1; # + file count via `| wc -l`
done
```

EXTENSION	FILES	LINES	% OF LINES	NOTES
<code>.json</code>	607	314,535	77.4%	573 files = catalogue; remaining 34 = config + eval reports (§2.7)
<code>.ts</code>	247	48,626	12.0%	Runtime engine, API routes, libs, scripts
<code>.mjs</code>	83	14,262	3.5%	Build/proof/eval scripts, ESM tooling

EXTENSION	FILES	LINES	% OF LINES	NOTES
<code>.tsx</code>	84	13,275	3.3%	React 19 pages & components
<code>.md</code>	180	12,632	3.1%	Docs, ADRs, runbooks
<code>.css</code>	5	2,800	0.7%	Tailwind layers / embed widget styles
<code>.sql</code>	5	183	< 0.1%	5 Postgres migrations
<code>.js</code>	1	6	< 0.1%	Single stray JS file
Total	1,212	406,319	100%	Columns sum exactly to the §2.1 totals

JSON dominates by design: the product's IP is data, not code. TypeScript (`.ts` + `.tsx` + `.mjs` = 76,163 lines / 414 files) is the entire hand-written surface an acquiring team maintains.

2.4 Platform runtime – per-area LOC / file table

```
for d in apps/web packages/runtime packages/presets packages/connectors \
    packages/agent-protocol packages/wallet-adapter scripts e2e; do
  find "$d" \( -path '*/node_modules' -o -path '*/.next' -o -path '*/dist' \) -prune \
    -o -type f \( -name '*.ts' -o -name '*.tsx' -o -name '*.js' -o -name '*.mjs' \) -print \
    | xargs wc -l | tail -1;
done
```

AREA	FILES	LINES	ROLE
<code>apps/web</code>	280	34,550	Next.js 15 App Router: 69 API routes, 39 pages, <code>lib/*</code> , components
<code>packages/runtime</code>	36	13,818	Agent engine: <code>runTurn</code> , model adapters, guardrails, RAG, workflows
<code>packages/presets</code>	2	12,245	Presets (11,761 generated + 484 hand-written <code>index.ts</code>)
<code>scripts</code>	34	9,110	Eval / proof / catalog-integrity / build tooling
<code>packages/connectors</code>	19	4,293	OAuth2/PKCE connector framework + provider actions
<code>e2e</code>	34	1,095	Playwright specs (<code>@smoke/@functional/@uat/@handover</code>)
<code>packages/wallet-adapter</code>	2	349	Prepaid wallet metering adapter
<code>apps/mobile-shell</code>	—	129	Mobile shell stub
<code>packages/agent-protocol</code>	1	142	Shared protocol types + <code>RENT_USD</code> tier pricing
<code>apps/connectors</code>	—	21	Connector app stub

AREA	FILES	LINES	ROLE
<code>apps/runtime</code>	—	16	Runtime app stub
Runtime total	415	76,169	Confirmed by whole-tree <code>find</code> (\$2.2)

`apps/web` is 45% of the runtime; `packages/runtime` (the actual agent execution engine) is a further 18%. The three `apps/*` stubs (runtime/connectors/mobile-shell, 166 lines combined) are near-empty placeholders — the real code lives in `apps/web` and `packages/*`.

2.5 Catalogue structure and counts

The catalogue is a set of **flat per-agent JSON files** (`data/catalog/{id}.agent.json`), each with keys `format`, `manifest`, `system_prompt`, `knowledge`, `tools`, `guardrails`, `evals` (verified on `data/catalog/africa-bi-analyst.agent.json`).

```
find data/catalog -name '*.json' | wc -l # 555
find data/catalog -name '*.agent.json' | wc -l # 552
find data/catalog-consumer -name '*.json' | wc -l # 18 (17 agents + index.json)
node -e '...' # sum tools/evals across every *.agent.json (see below)
```

CATALOGUE METRIC	MEASURED VALUE	SOURCE OF TRUTH
SKUs (canonical)	500	<code>data/catalog/index.json</code> is an array of length 500
Families	100	<code>data/catalog/families.json</code> is an array of length 100
Markets	5	<code>us</code> , <code>eu</code> , <code>africa (incl. ZA)</code> , <code>asia</code> , <code>oceania</code> — market distribution: US/EU/Asia/Oceania = 100 each, Africa = 151, global = 1
Agent files on disk (<code>data/catalog</code>)	552	500 canonical + 51 legacy Africa/ZA-region aliases + 1 <code>global</code> agent (<code>personal-assistant</code>), retained for deep-link stability
Index/meta JSON (<code>data/catalog</code>)	3	<code>index.json</code> , <code>families.json</code> , <code>market-packs.json</code>
Total JSON in <code>data/catalog</code>	555	552 agents + 3 meta = 290,061 lines
Consumer specialists (<code>data/catalog-consumer</code>)	17 agent files (+ <code>index.json</code> = 18 JSON)	e.g. <code>health-navigator</code> , <code>money-coach</code> , <code>study-coach</code> , <code>trip-planner</code> — 9,447 lines
Tool definitions (on-disk, all agent files)	2,180	<code>data/catalog</code> 2,128 + <code>data/catalog-consumer</code> 50 + <code>data/platform</code> 2
Tool definitions (index-declared, 500 SKUs)	1,890	Sum of <code>index.json[].tools</code> — this is the proposal's " ~1,900 tools"

CATALOGUE METRIC	MEASURED VALUE	SOURCE OF TRUTH
Eval cases (on-disk, all agent files)	8,844	<code>data/catalog</code> 8,526 + consumer 302 + platform 16
Eval cases (<code>data/catalog</code> only)	8,526	Reconciles to the proposal's "8,500+"; ~15.4 evals/agent , range 12–23

100 families × 5 markets = 500 SKUs (ADR 0001) is confirmed three ways: `families.json` length 100, `index.json` length 500, and 100 distinct family stems recovered by stripping the market prefix from the 500 `index.json` ids.

Tool-count reconciliation: `index.json` carries *declared/nominal* per-SKU counts (`tools: 4, evals: 16` on the sample) that sum to **1,890 tools** — the origin of the “~1,900 tools” headline. The **actual tool objects inside the agent JSON** total **2,128** in `data/catalog` (2,180 catalogue-wide). Both are true; the index under-counts the on-disk definitions. For handover, the on-disk figure (2,128 / 2,180) is authoritative.

Eval-count reconciliation: on-disk eval cases in `data/catalog` = **8,526** (index-declared sum = 7,640). The 8,526 on-disk figure is what backs the proposal's "8,500+"; catalogue-wide (incl. consumer + platform) the total is **8,844**.

2.6 API surface, persistence, docs

```
find apps/web/src/app/api -name 'route.ts' | wc -l    # 69
find apps/web/src/app     -name 'page.tsx' | wc -l    # 39
ls apps/web/migrations    # 001_init ... 005_consumer_reminders
ls docs/adr               # 0001 ... 0005 (+ README)
```

ITEM	COUNT
API routes (<code>apps/web/src/app/api/**/*.route.ts</code>)	69
Pages (<code>page.tsx</code>)	39
SQL migrations (<code>apps/web/migrations</code>)	5 (<code>001_init</code> → <code>005_consumer_reminders</code>)
ADRs (<code>docs/adr</code>)	5 (+ README)
Docs markdown — top level of <code>docs/</code>	33
Docs markdown — <code>docs/</code> recursive	171

The “32 docs” quoted in the Licence Proposal corresponds to the **top level of `docs/` (now 33)**; the full documentation tree is **171 markdown files** (2,632-line subset within `docs`, part of the 12,632 total `.md`).

2.7 Non-catalogue JSON (the other 34 JSON files)

Of 607 `.json` files, 573 are catalogue; the remaining **34 files (15,027 lines)** are config and generated eval evidence — the largest being `data/reports/eval-results.json` (7,890), `docs/reports/binding-matrix-2026-08-05.json` (1,797), `data/wave4-live-proofs.json` (1,340), `data/turn-transcripts.json` (1,245), and `docs/reports/tool-sweep-2026-08-05.json` (1,096). Note there is also **one agent file outside the catalogue** — `data/platform/marketplace-`

`assistant.agent.json` (406 lines) — the marketplace's own concierge agent, counted in §2.5's platform totals but not in the 500 SKUs.

2.8 On-disk footprint (with dependencies)

```
du -sh .          # 805M   (full working tree, deps + build installed)
du -sh node_modules # 452M
```

VIEW	SIZE	NOTES
Full working tree (deps + build artifacts)	805 MB	As checked out and installed
<code>node_modules</code> (root pnpm store)	452 MB	Third-party dependencies
<code>apps/</code> (incl. nested build output / <code>.next</code>)	313 MB	Build artifacts dominate
<code>data/</code> (catalogue + reports)	16 MB	<code>data/catalog</code> alone = 15 MB
<code>.git</code> history	12 MB	
<code>docs/</code>	10 MB	
Authored source only (8 tracked extensions)	≈30 MB	The 406,319-line working set

Clean-vs-full multiple ≈ 27× (805 MB installed ÷ ~30 MB of authored source). Dependencies and build output — not authored code — account for ~96% of on-disk bytes, which is normal for a Next.js/pnpm monorepo. A source-only clone (no `node_modules`, no `.next`) is roughly 40 MB including `.git`.

2.9 Reconciliation with the Licence Proposal (growth since the quote)

The Licence Proposal quoted an earlier snapshot. The repository has **grown**; this section reports the current measured numbers:

METRIC	LICENCE PROPOSAL	MEASURED @ <code>FB97438</code>	Δ
Total authored lines	~398,000	406,319	+8,319 (+2.1%)
Total files	1,021	1,212	+191 (+18.7%)
Catalogue IP lines	~298,000	299,508	+1,508
API routes	45	69	+24 (+53%)
Files on disk (incl. deps)	~631 MB	805 MB	+174 MB
Tools	~1,900	1,890 declared / 2,128 on-disk	matches / higher on disk
Eval cases	8,500+	8,526 (<code>data/catalog</code>)	confirmed

The proposal's figures were directionally correct at the time; the platform has since added routes and files (notably the +24 API routes and the file-count jump), while the catalogue and eval corpus are essentially stable. **All headline numbers in this pack should cite the measured** 406,319 / 1,212 / 69 routes / 299,508 catalogue / ~805 MB .

03 Project Structure

This section maps the repository's physical and logical topology: the pnpm workspace graph, the role each app and package plays, the internal `workspace:*` dependency edges, the build ordering that follows from them, and a directory map of the one app that is the actual product (`apps/web`). All counts below were re-measured with `find / wc / ls` against the working tree (excluding `node_modules/`, `.next/`, `dist/`, `.git/`) and reflect the current repo, which has **grown** since the Licence Proposal snapshot (that quoted ~398K lines / 1,021 files / ~298K catalogue / 45 routes).

3.1 Scale, re-measured

METRIC	MEASURED (CURRENT TREE)	COMMAND BASIS
Total authored files	1,212	<code>`find ... -name '*.ts'</code>
Total authored lines	406,319	same set, <code>`xargs cat</code>
Catalogue IP (<code>data/catalog</code> + <code>data/catalog-consumer</code>)	299,508 lines across 552 + 18 JSON files	the 500-SKU agent catalogue — the single largest, most valuable asset
Hand-written platform runtime (TS/TSX/JS/MJS)	~75.9K LOC / ~410 files	<code>apps/*</code> + <code>packages/*</code> + <code>scripts/</code> + <code>e2e/</code>
<code>apps/web/src</code> (TS/TSX only)	31,554 LOC / 246 files	product code
<code>apps/web</code> authored incl. tests + CSS	36,551 LOC / 283 files	matches the lead's ~34.5K figure once <code>.css</code> / <code>test/</code> are folded in
<code>packages/presets/src/generated-presets.ts</code>	11,761 LOC (machine-generated)	one file dominates the <code>presets</code> package

The catalogue is ~74% of all authored lines. Any statement of "size" that headlines only the ~75.9K hand-written runtime understates the asset by ~4x; the 299,508-line catalogue must always be counted.

3.2 Monorepo topology — pnpm workspaces

`pnpm-workspace.yaml` declares two globs and one deliberate exclusion:

```
packages:
  - "apps/*"
  - "packages/*"
  # Expo / RN stays self-contained – install inside apps/mobile-shell only
  - "!apps/mobile-shell"
```

Root `package.json` pins the toolchain (`"packageManager": "pnpm@9.15.0"`, `"private": true`) and carries the entire orchestration surface — **~60 scripts** spanning dev/build, catalogue generation (`import:catalog`, `generate:packs`, `polish:catalog`, `generate:presets`, `catalog:ready`, `catalog:integrity`), eval suites (`eval:smoke|suite|live|full`, `heal:evals`), Playwright tiers (`test:e2e:smoke|functional|uat|handover`), live-proof

probes (`proof:*` , `smoke:live-llm`), and the `ci` gate. `apps/mobile-shell` is excluded from the workspace so its React Native / Expo dependency tree (`expo ~52` , `react-native 0.76.3`) never pollutes the web hoist; it is installed and run standalone via `pnpm --dir apps/mobile-shell` (`demo:app`).

Enumerated members (8 total workspace packages + 1 excluded app):

WORKSPACE	PATH	NPM NAME	ROLE	SOURCE SIZE
web app	<code>apps/web</code>	<code>@miai/web</code>	The product – Next.js 15 App Router surface	31,554 LOC / 246 TS-TSX (src)
runtime worker	<code>apps/runtime</code>	<code>@miai/runtime-app</code>	Thin worker scaffold	<code>src/worker.ts</code> = 16 LOC
connectors admin	<code>apps/connectors</code>	<code>@miai/connectors-app</code>	Thin HTTP scaffold	<code>src/server.ts</code> = 21 LOC
mobile shell	<code>apps/mobile-shell</code> (excluded)	<code>@miai/mobile-shell</code>	Expo WebView wrapper	<code>App.tsx</code> = 123 LOC
agent protocol	<code>packages/agent-protocol</code>	<code>@miai/agent-protocol</code>	Types + rent/tier constants	<code>src/index.ts</code> = 142 LOC
runtime engine	<code>packages/runtime</code>	<code>@miai/runtime</code>	The turn engine	12,404 LOC / 27 files
connectors	<code>packages/connectors</code>	<code>@miai/connectors</code>	OAuth + tool-execution framework	3,961 LOC / 13 files
presets	<code>packages/presets</code>	<code>@miai/presets</code>	Agent→connector tool bindings	11,761 (generated) + <code>index.ts</code>
wallet adapter	<code>packages/wallet-adapter</code>	<code>@miai/wallet-adapter</code>	Prepaid metering client	<code>src/index.ts</code>

3.3 Apps – one product, two worker scaffolds, one mobile shell

- `apps/web` (`@miai/web`) – **the product**. Everything a tenant or consumer touches is here: 39 `page.tsx` routes, 69 API `route.ts` handlers, all UI, all auth/tenancy/compliance libs. It depends on **all five** internal packages and is the only app deployed by the production Dockerfile → Railway. Scripts: `next dev|build|start|lint` , plus a bespoke `node --test` harness (`web-test-register.mjs`) and `test:api-contract` .
- `apps/runtime` (`@miai/runtime-app`) – **worker scaffold, 16 LOC**. `src/worker.ts` is explicitly a placeholder: its own comment states “Web app route handlers call `@miai/runtime` directly for MVP; deploy this process when tool loops move to a queue.” It instantiates a wallet adapter, logs `mode: "scaffold"` , and holds a 60s heartbeat `setInterval` . It exists so the plan→act→observe loop can be lifted out of the request path onto Azure Container Apps / a queue consumer without a rewrite. **Not currently a running service**.
- `apps/connectors` (`@miai/connectors-app`) – **HTTP scaffold, 21 LOC**. A raw `node:http` server exposing `/health` and `/v1/connectors` (returns `listConnectors()` + `WEBHOOK_TEMPLATES`). An OAuth-callback / connector-admin surface stub; the real OAuth callback the product uses lives at `apps/web/src/app/api/oauth/callback/route.ts` .

- `apps/mobile-shell` (`@miai/mobile-shell`) – Expo WebView, 123 LOC. `App.tsx` wraps the deployed web app in `react-native-webview` (`expo ~52`, `expo-status-bar`, `react-native-safe-area-context`). A native-store shell around the same web UI; no business logic duplicated. Kept out of the pnpm workspace by design.

3.4 Packages – the reusable core

- `@miai/agent-protocol` (142 LOC, one file) – the contract layer. Defines `AgentTier = "standard" | "pro" | "enterprise"`, the manifest/package shapes, `validateAgentPackage`, and the pricing tables `RENT_USD = { standard: 349, pro: 699, enterprise: 1199 }` and `RENT_EUR = { 319, 649, 1099 }`. A dependency-free **leaf**.
- `@miai/wallet-adapter` – prepaid token metering client (`createWalletAdapter()`, `MIAI_WALLET_MODE=mock|http`). Dependency-free **leaf**.
- `@miai/connectors` (3,961 LOC) – the integration framework: `src/oauth/` (`flow.ts`, `probe.ts`, `providers.ts`, `tokens.ts`), `src/live/execute.ts` + `src/live/handlers/` (`slack.ts`, `webhook.ts`, `mcp.ts`), `ssrf.ts` (DNS-pinned egress guard), `webhook-sig.ts` (HMAC), `retry.ts`, `types.ts`. Runtime dep on `pg ^8.22` (sealed-token store); no internal workspace deps → **leaf** in the internal graph.
- `@miai/presets` – maps each agent to its connector tool bindings. `src/index.ts` re-exports `GENERATED_PRESETS` from the **machine-generated** `src/generated-presets.ts` (11,761 LOC, produced by `scripts/generate-presets.mjs`). Depends on `@miai/connectors` for the `ToolBinding` type.
- `@miai/runtime` (12,404 LOC) – the engine. `src/index.ts` alone is 2,852 LOC (`runTurn`, model adapters). Ships `embeddings.ts`, `knowledge-retrieve.ts` (hybrid RAG), `guardrails.ts`, `templates.ts`, and `src/workflows/` – **22 per-vertical modules** (`accounting-practice`, `booking-front-desk`, `dental-front-desk`, `hotel-guest`, `mobile-money`, `pharmacy`, `restaurant-takeaway`, `wealth-management`, ... + helpers `i18n.ts`, `stop-suppression.ts`). It is the **aggregator**: depends on all four other packages.

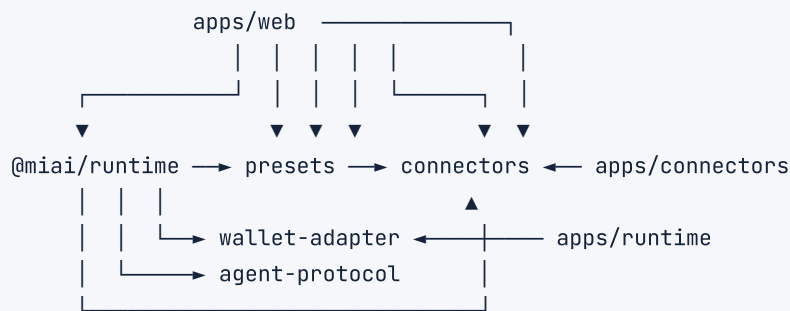
3.5 Internal dependency graph

Edges taken verbatim from each `package.json` (`workspace:*` / `workspace:^`):

```
@miai/agent-protocol → (none) [leaf]
@miai/wallet-adapter → (none) [leaf]
@miai/connectors → pg (external only) [leaf]
@miai/presets → @miai/connectors
@miai/runtime → @miai/agent-protocol, @miai/connectors,
               @miai/presets, @miai/wallet-adapter [aggregator]

apps/web (@miai/web) → agent-protocol, connectors, presets,
                      runtime, wallet-adapter (all five)
apps/runtime → runtime, wallet-adapter
apps/connectors → connectors
apps/mobile-shell → (no internal deps; excluded workspace)
```

As a layered graph (arrows = "depends on"):



`connectors`, `agent-protocol`, `wallet-adapter` are the three roots; `presets` sits above `connectors`; `runtime` sits above everything; `apps/web` consumes the whole stack. `next.config.ts` mirrors this with `transpilePackages: [agent-protocol, wallet-adapter, connectors, presets, runtime]` and marks `pg`, `jose` as `serverExternalPackages`, with `outputFileTracingRoot` set to the monorepo root and `experimental.externalDir: true` so Next can compile source across the workspace boundary.

3.6 Build graph

Two root scripts encode the topological order; `pnpm -r` resolves intra-package order from the `workspace:*` edges above:

```

"build:packages": "pnpm -r --filter './packages/*' build",    // tsc each package → dist/
"build:web":      "pnpm build:packages && pnpm --filter @miai/web build", // then next build
"build":          "pnpm -r build"                             // everything, recursively
  
```

`build:packages` compiles the five packages (each `"build": "tsc"`), honoring the dependency order (protocol/wallet/connectors → presets → runtime); `build:web` then runs `next build`. The CI gate chains it end-to-end:

```

"ci": "pnpm build:packages && pnpm typecheck && pnpm test && pnpm catalog:integrity && pnpm eval:suite:static"
  
```

The production `Dockerfile` follows the same ordering, and `railway.toml` deploys the resulting `@miai/web` server (health probe `/api/health`). `dist/` is git-ignored (present on disk only after a local build); packages are consumed from source in dev via `transpilePackages`.

3.7 Directory map – apps/web/src

```

apps/web/src/
├─ app/                                # Next.js App Router – 39 page.tsx, 69 api route.ts
│   └─ layout.tsx                      # root layout (+ app/v1/layout.tsx = 2 layouts total)
│   └─ page.tsx error.tsx loading.tsx not-found.tsx
│   └─ robots.ts sitemap.ts globals.css (31 KB)
│   └─ api/                            # 33 route groups (see below)
│       └─ agents/ personal/ consultants/ my-agents/ # catalogue + agent detail
│       └─ me/ settings/ tokens/ workspace/ history/ # tenant/consumer surfaces
│       └─ trust/ privacy/ terms/ legal/ cookies/ data-protection/ # compliance pages
│       └─ ask/ assistant/ create/ demo/ insights/ ops/ quality/ ... # ~39 route dirs
├─ components/                        # 32 files / 8,439 LOC
│   └─ Shell.tsx Sidebar.tsx CatalogGrid.tsx MarketplaceHero.tsx AgentStudio.tsx
│   └─ Consumer*.tsx TokenTopUpPanel.tsx KnowledgePanel.tsx ConsentBanner.tsx ...
│   └─ dashboard/Charts.tsx
│   └─ marketplace-assistant/{MarketplaceAssistant.tsx, marketplace-assistant.css}
├─ lib/                              # ~85 modules (83 top-level .ts/.tsx + 2 subdirs)
│   └─ handlers/embed-chat.ts
│   └─ i18n/{en,de,es,fr,hi,it,zh}.ts + index.ts # 8 locale files
├─ instrumentation.ts                # server boot hook → assertBootHardening() (fail-closed)
└─ middleware.ts                     # body-size gate, OIDC Bearer pre-check, per-request CSP nonce

```

app/api – 69 route handlers across 33 groups: `admin`, `agents/[id]`, `app/chat`, `ask/{chat,leads}`, `audit`, `auth/handoff`, `catalog/{route, family/[familyId], personal}`, `chat`, `configure`, `connectors`, `consent`, `consumer/*` (auth, brief, chat, connectors, reminders, telegram/webhook, wallet – the largest group), `custom-requests`, `dsar/{export,erase}`, `embed/{chat,sri}`, `health`, `history/{turns, trace/[correlationId]}`, `insights`, `knowledge/*`, `mcp/{route, tools/call}`, `oauth/*`, `onboarding`, `ops`, `payments/paystack/{init,return,webhook}`, `proof/tool`, `rent`, `rentals`, `slack/channels`, `v1/{embed/chat, openapi, rent}`, `version`, `wallet`, `webhook/sink`, `workspace/members`.

lib/ module clusters (85 modules; representative grouping):

CLUSTER	MODULES
Auth / tenancy	<code>auth.ts</code> , <code>agents-auth.ts</code> , <code>consumer-auth.ts</code> , <code>consumer-oidc.ts</code> , <code>consumer-session.ts</code> , <code>consumer-identity.ts</code> , <code>request-auth.ts</code> , <code>workspace-members.ts</code> , <code>workspace-onboarding.ts</code>
Runtime bridge	<code>ask-turn.ts</code> , <code>channel-turn.ts</code> , <code>consumer-turn.ts</code> , <code>chat-stream.ts</code> , <code>sse.ts</code> , <code>workflows.ts</code> , <code>guardrails.ts</code> , <code>models.ts</code>
Data / persistence	<code>pg.ts</code> , <code>store.ts</code> , <code>migrate.ts</code> , <code>traceability.ts</code> , <code>channel-sessions.ts</code> , <code>consumer-memory-store.ts</code> , <code>consumer-lifegraph-store.ts</code> , <code>consumer-brief-store.ts</code> , <code>consumer-reminders-store.ts</code> , <code>redis.ts</code>
Catalogue / IP	<code>catalog.ts</code> , <code>consumer-catalog.ts</code> , <code>smart-catalog-query.ts</code> , <code>agent-ip.ts</code> (public-read redaction), <code>card-blurbs.ts</code> , <code>sectors.ts</code> , <code>family-capabilities.ts</code>

CLUSTER	MODULES
Security	<code>csp.ts</code> , <code>security.ts</code> , <code>security-flags.ts</code> , <code>embed-cors.ts</code> , <code>webhook-sink-auth.ts</code> , <code>sandbox.ts</code> , <code>public-paths.ts</code> , <code>agent-js-sri.ts</code>
Compliance	<code>pii-redact.ts</code> , <code>ai-disclosure.ts</code> , <code>dsar-erase.ts</code> , <code>legal-content.ts</code> , <code>trust-content.ts</code> , <code>consumer.ts</code>
Payments	<code>paystack.ts</code> , <code>topup.ts</code> , <code>topup-client.ts</code>
i18n / theming	<code>i18n/*</code> , <code>locale.tsx</code> , <code>locale-boot.ts</code> , <code>theme.tsx</code> , <code>theme-boot.ts</code> , <code>chat-languages.ts</code>

`middleware.ts` runs on nearly all routes (matcher excludes `_next/static`, `_next/image`, static assets): it enforces a body-size cap (`MIAI_MAX_BODY_BYTES`, default 1 MiB), does an OIDC `Bearer` pre-check when `MIAI_AUTH_MODE=oidc` (skipping `isPublicApiPath`), and stamps a per-request CSP nonce (`buildContentSecurityPolicy`) so `script-src` avoids `unsafe-inline`. `instrumentation.ts` is the Next server-boot hook that calls `assertBootHardening()` to fail closed on weak secrets / mock rails in production.

3.8 ESM package layout

Every internal package is uniform ESM with declaration output:

```
// packages/*/package.json
"type": "module",
"main": "./dist/index.js",
"types": "./dist/index.d.ts",
"exports": { ".": { "types": "./dist/index.d.ts", "import": "./dist/index.js" } },
"scripts": { "build": "tsc", "typecheck": "tsc --noEmit" }
```

Each `tsconfig.json` targets `ES2022` with `module` / `moduleResolution: NodeNext`, `declaration: true`, `rootDir: src`, `outDir: dist`, `strict: true`. Cross-package imports use explicit `.js` specifiers against `dist` (e.g. `presets/src/index.ts` → `import { GENERATED_PRESETS } from './generated-presets.js'`), consistent with NodeNext ESM. `apps/web` diverges as expected for Next.js — `moduleResolution: "bundler"`, `jsx: "preserve"`, `noEmit: true`, and the `@/*` → `./src/*` path alias — relying on `transpilePackages` to pull package **source** through Turbopack/webpack rather than pre-built `dist` during `next build`.

04 Frontend

The customer-facing UI is a single Next.js 15.2.8 App Router application at `apps/web` – the largest hand-written module in the platform (measured **34,550 LOC / 280 files**; the `.ts` / `.tsx` under `apps/web/src` alone is **31,554 LOC**). It renders three distinct products from one codebase: the **B2B agent marketplace + Studio**, the **consumer assistant** (`/me` , `/personal`), and a **white-label embed** (injected widget + hosted WebView). It is also the presentation layer for the platform's headline asset – the **500-SKU catalogue (100 families × 5 markets, ~299,508 LOC of JSON IP)** – which every catalogue/detail surface reads live through `lib/catalog.ts` loaders (`listFamilies()` , `getAgentPackage()`), never a hard-coded list, so counts on the page cannot drift from the catalogue on disk.

4.1 Stack & rendering model

CONCERN	CHOICE	EVIDENCE
Framework	Next.js 15.2.8 App Router, React 19, TS 5	<code>apps/web/src/app</code> , root <code>layout.tsx</code> uses <code>async</code> RSC + <code>await headers()</code>
Styling	Tailwind 3.4 + a hand-authored CSS-variable design system	<code>app/globals.css</code> (1,343 lines), tokens on <code>:root</code> / <code>html[data-theme=...]</code>
Fonts	<code>next/font/google</code> – Manrope (<code>--font-sans</code>), JetBrains Mono (<code>--font-mono</code>), self-hosted at build	<code>app/layout.tsx</code>
Charts	Hand-rolled inline SVG , zero chart library	<code>components/dashboard/Charts.tsx</code> (<code>AreaChart</code> / <code>BarChart</code> , <code>role="img"</code>)
Images	Almost entirely inline SVG glyphs; <code>next/image</code> used in exactly 1 place (<code>/assistant</code>); chat images are raw <code></code> (no <code>next/image</code> domain config)	<code>grep next/image</code> = 1 file
Security headers	Per-request nonce CSP with <code>strict-dynamic</code> ; <code>frame-ancestors 'self'</code>	<code>lib/csp.ts</code> , nonce read from <code>x-nonce</code> header in <code>layout.tsx</code>

Page count: **39** `page.tsx` + **2** `layout.tsx` (root + the `/app/v1` WebView layout). Of the 39 pages, **10 are Client Components** (`"use client"` at the top of the page file) and **29 are Server Components**; **58 files** across `app/` set `export const dynamic = "force-dynamic"` (4 of them pages, the rest API routes).

4.2 Route surfaces (the 39 pages)

Grouped by product lane. "RSC" = server component page; "client" = `"use client"` page.

Marketplace / catalogue (business)

ROUTE	KIND	PURPOSE
/	RSC	Marketplace home — SSR-seeds <code>CatalogGrid</code> from <code>listFamilies()</code> (500-agent / 100-family browse)
/agents	RSC	"AI Agents hub" — two-lane splitter to consumer <code>/personal</code> vs business <code>/</code> with live counts (<code>INDEXED_AGENT_COUNT</code> , <code>INDEXED_MARKETS</code>)
/agents/[id]	RSC	Agent detail → mounts <code>AgentStudio</code> ; injects <code>AGENT_JS_INTEGRITY</code> for the install snippet; <code>generateMetadata</code> per agent
/my-agents	client	Rented-agent list
/create , /request , /personalize , /scan	client / RSC(<code>ComingSoon</code>)	Custom-agent request + "personalize/scan my site" stubs

Agent operations & workspace (business)

ROUTE	KIND	PURPOSE
/insights	client	Analytics dashboard (uses <code>Charts.tsx</code>)
/ops	client	Live Ops
/admin	client	Agent Admin (411 LOC)
/history	RSC→ <code>HistoryClient</code>	Turn/trace history
/workspace	RSC→ <code>WorkspaceClient</code>	Workspace / member management
/quality	RSC, <code>force-dynamic</code>	Live eval scoreboard
/support , /consultants , /marketing	RSC	Support desk (<code>ComingSoon</code>), consultants, marketing preview

Consumer assistant

ROUTE	KIND	PURPOSE
/assistant	RSC	Consumer landing (pre-chat marketing; the only <code>next/image</code> user)
/me	client (448 LOC)	The general personal assistant — brand switcher, connectors, reminders, daily-brief offer, wraps <code>useConsumerChat</code>
/me/connectors	client	Consumer connector management

ROUTE	KIND	PURPOSE
<code>/personal</code>	RSC, <code>force-dynamic</code>	Consumer specialist marketplace (17–18 family agents)
<code>/personal/[id]</code>	RSC, <code>force-dynamic</code> → <code>SpecialistChat</code>	Single specialist page; chat gated on <code>personalAgentRunnable</code> (certified on prod, all in sandbox)
<code>/app/v1</code>	RSC, <code>force-dynamic</code> → <code>AppChatClient</code>	Hosted bare WebView/in-app chat surface (own layout, <code>viewport-fit=cover</code> , no marketplace chrome)

Onboarding / account / commerce

ROUTE	KIND	PURPOSE
<code>/get-started</code>	RSC → <code>GetStartedWizard</code>	4-step B2B onboarding (<code>welcome-business-intent-account</code>)
<code>/login</code>	client	Sign in
<code>/tokens</code> , <code>/redeem</code>	RSC	Prepaid token balance / e-PIN redeem
<code>/install</code> , <code>/demo</code>	RSC	"Go live" + partnership go-live/license pack
<code>/trust</code>	client (403 LOC)	Trust Center (Live/Partial/Planned tags)
<code>/roadmap</code> , <code>/learn</code>	RSC	Roadmap, learn-and-earn

Legal / compliance

`/legal`, `/privacy`, `/terms`, `/cookies`, `/data-protection` — all RSC, rendered through the shared `LegalPage` component from `lib/legal-content.ts` (26.8 KB of policy copy).

4.3 Layout, Shell chrome & navigation

`app/layout.tsx` (root RSC) sets `<html lang="en" data-theme="dark" data-locale="en" suppressHydrationWarning>`, injects two nonce'd inline boot scripts **before paint** (`THEME_BOOT_SCRIPT`, `LOCALE_BOOT_SCRIPT`), then wraps children in `SandboxBanner` → `ThemeProvider` → `LocaleProvider` → `Shell`.

`components/Shell.tsx` (client, 174 LOC) is the chrome router. It reads `usePathname()` and switches layout by surface:

- `/app/v1*` and auth-entry (`/get-started*`, `/login*`, `/marketing*`) → **bare** (`return <>{children}</>`, no sidebar).
- `/me*` → consumer surface: `ConsumerNav` top bar instead of the business `Sidebar`.
- Everything else → full business shell: `Sidebar` + `ThemeToggle` + floating `MarketplaceAssistant` (FAB) + `ConsentBanner` + `OnboardingChecklist` + `TopUpModal`, with a mobile nav toggle and a `refreshWallet()` fetch to `/api/wallet` on every path change.

`components/Sidebar.tsx` (485 LOC) is a fully-i18n'd nav with two modes (`business` | `consumer`) — `CONSUMER_ALLOWED` / `CONSUMER_HIDDEN_GROUPS` sets prune the business Agents-ops groups in consumer mode. All nav labels are `MessageKey`'s resolved via `useT()`; icons are inline SVG components.

4.4 Marketplace / catalogue browse – CatalogGrid + MarketplaceHero

`components/CatalogGrid.tsx` (1,139 LOC, the largest component) is the catalogue browser. It is SSR-seeded from `/initialFamilies` prop) so first paint has content, then hydrates to a rich client filter:

- Facets: free-text `q`, `market` (5 packs via `CatalogMarketBadge` `PACK_ORDER`), `category` (industry), `audience`, plus boolean pills `workflowsOnly` and `pilotOnly` (go-live).
- Smart query** parsing (`lib/smart-catalog-query.ts`) auto-extracts market/filters from natural-language search (`smartFilter` toggle, `smartApplied` chips).
- Voice search** via the Web Speech API (`SpeechRecognition` / `webkitSpeechRecognition` feature-detected; graceful `speechErrorMessage` for `not-allowed` / `no-speech` /etc.).
- Re-fetches facet counts from `/api/catalog?view=families`, but **skips the network when the SSR seed already covers the unfiltered view**.
- Cards render `AgentIcon` (630-LOC SVG icon set), sector accent (`lib/sectors.ts`), and blurbs from `lib/card-blurbs.ts`.

`components/MarketplaceHero.tsx` (302 LOC) is the animated hero (floating `FloatTile` glyphs) with i18n CTAs; `MarketplaceCTA` is exported alongside it.

4.5 Agent Studio – configure / connect / install

`components/AgentStudio.tsx` (550 LOC, client) is the rent→configure→connect→install workspace mounted by `/agents/[id]`. It composes sub-panels: `KnowledgePanel` (326), `ActionsPanel` (678, connector/Actions wiring), `InstallPanel` (289), `TokenTopUpPanel`, `SandboxChat` (440, live try-it), and a `SetupGuide` step machine (`SetupStepId`, `readSetupFlag` / `writeSetupFlag` in `localStorage`). Tabs/steps are URL-driven (`?tab=actions|install|configure`, `?step=`).

The **embed install snippet** is built client-side with `buildEmbedScriptTag({ src, key, integrity })` (from `lib/agent-js-script.ts`), where `integrity` is the **build-time** `AGENT_JS_INTEGRITY` passed down as the `scriptIntegrity` prop from the RSC page. `origin` is derived from `window.location.origin` in an effect. Demo keys (`mia_pk_<id>_demo`) are only substituted when `process.env.NODE_ENV !== "production"`; production requires a real activated `publicKey`. `InstallPanel` offers two channels – **web** (script tag) and **app** (a `/app/v1?...` deep link with `title/accent/accent2/greeting` query params).

4.6 Consumer assistant surfaces

- `/me` (`AssistantHome`) layers its own chrome – multi-brand/tenant switcher (`BRANDS`, `brandThemeVars`, `getBrand`), connector chips, reminders list with a timezone-local `formatWhen`, a one-time welcome, and a daily-brief offer – around the shared chat hook. The **active brand doubles as the** `workspaceId` / **tenant**, so switching brand switches the isolated memory/wallet/connector context. Low-balance nudge at `LOW_BALANCE = 500`.
- `/personal/[id]` → `SpecialistChat` (client) runs a single `agentId` through the same hook with no extra chrome, rendered inside `ConsumerAuthGate`.
- `ConsumerAuthGate` (client) fetches `/api/consumer/auth/me`; in `oidc` mode with no session it renders a **"Sign in with Google"** card *instead of* its children (so the wrapped component never mounts and no authenticated fetches fire while signed out), and **fails open to mock mode** if the check errors. Return path is `window.location.pathname + search`.
- `ConsumerChatWindow` (shared surface): message list, typing indicator, starter chips, composer; assistant bubbles are rendered through `renderRichText`, user bubbles as plain text.
- `ConsumerNav` is the minimal consumer top bar (Assistant / Agents / "Business →").

4.7 Chat hook & SSE streaming

`lib/use-consumer-chat.ts` (`useConsumerChat`) is the single client chat engine shared by `/me` and `/personal/[id]`. It owns `messages`, `input`, `busy` / `typing`, `error`, prepaid `balance`, and a per-session `crypto.randomUUID()`. Callbacks (`onReplied` / `onSettled` / `onSubmitStart`) are held in a `ref` so `submit` stays referentially stable across renders. On submit it appends a user bubble, then streams the assistant reply via `streamChat`, incrementally appending deltas to one assistant bubble; on settle it reloads the wallet (`/api/consumer/wallet?workspaceId=...`).

The SSE contract lives in three files and is spoken by every chat surface (embed/app/consumer):

- **Server:** `lib/sse.ts` (`sseStreamResponse`, `x-accel-buffering: no`, `no-cache`) + `lib/chat-stream.ts` (`streamChatTurn` choreography: `meta → (delta|status)* → done | paused+done | error`).
- **Client:** `lib/chat-stream-client.ts` (`streamChat`) – a hand-written `fetch` + `ReadableStream` reader with a `parseSseBlocks` splitter (no `EventSource`, so it can POST a body and set `accept: text/event-stream`). Events are normalized to `delta | tool | paused | done`; an `error` SSE event throws.

`AppChatClient` (`/app/v1`) consumes `/api/app/chat` through the same `streamChat`. **The injected widget** (`agent.js`) is the exception: it uses plain `fetch(...).then(r.json())` against `/api/embed/chat` – non-streaming request/response, not SSE.

4.8 Embeddable widget (`agent.js`) + SRI + hosted WebView

Loader – `lib/agent-js-script.ts` exports `AGENT_JS_SCRIPT`, a `String.raw` IIFE served verbatim. Zero dependencies. It:

- reads config from its own `<script>` `data-*` attributes (`data-key` required; `data-accent` / `-accent-2` / `data-title` / `data-greeting` / `data-suggestions` optional);
- **derives its own origin** from `document.currentScript.src` (`new URL(current.src).origin`), falling back to `window.location.origin` – so the embed calls back to the host it was served from;
- mounts a single `#miai-agent-root` and attaches a **Shadow DOM** (`attachShadow({mode: "open"})`) with `:host{all:initial}`, fully isolating the host page's CSS; all styling is inline `--mi-a` / `--mi-a2` custom properties from the accent attrs;
- provides a FAB launcher, animated panel, typing dots, suggestion chips, per-session id (`ms_...`), a `resetConversation` on close, reduced-motion + mobile media queries, and an AI-disclosure footer ("AI system · Powered by MyInstantAI").

Serving route – `app/agents/v1/agent.js/route.ts` (`export const dynamic = "force-dynamic"`) returns the script with `content-type: application/javascript`, `cache-control: public, max-age=60`, `access-control-allow-origin: *`, and integrity advertised in both `x-miai-script-integrity` and an RFC-9530 `Digest` header.

SRI – `lib/agent-js-sri.ts` computes a stable **sha384** base64 token over the exact script body (`computeAgentJsIntegrity`) and exports `AGENT_JS_INTEGRITY`; `agentJsDigestHeader` renders the `sha-384=...` Digest. The install snippet emitted by Studio carries `integrity="sha384-..." crossorigin="anonymous" async`.

Grounding correction on widget size: the DD brief says "~6.4kB" and the file's own docstring says "~9KB", but the current served body measures **13,449 bytes raw / 4,307 bytes gzip / 3,630 bytes brotli** (measured at HEAD). The wire cost is ~4.3 KB gzipped; the "6.4 KB"/"9 KB" figures are stale and understate the raw source, which has grown. See open gaps.

Hosted WebView – `/app/v1` (`AppChatClient`) is the full-page, chrome-less equivalent for in-app/WhatsApp WebView embeds, configured via the same query params the App-channel deep link builds. There is **no** `/embed` page route; the "embed surface" is the injected widget + the `/api/embed/chat` endpoint (plus `/api/embed/sri`), not a Next page.

4.9 Theming

`lib/theme-boot.ts` defines `THEME_STORAGE_KEY = "miai-theme"` and `THEME_BOOT_SCRIPT`, an inline pre-paint script that stamps `data-theme` from `localStorage` so there is no dark/light flash. `lib/theme.tsx` (`ThemeProvider` / `useTheme`) is a client context: default `"dark"`, reads storage in an effect, writes `data-theme` on `document.documentElement`, and `useTheme()` returns a safe no-op fallback when used outside the provider. `components/ThemeToggle.tsx` is an accessible `role="switch"` with sun/moon SVGs.

Tokens live in `app/globals.css` (1,343 lines). Two full palettes are defined as CSS variables — dark on `:root, html[data-theme="dark"]`, light on `html[data-theme="light"]` — including a **teal consumer accent** (`--accent:#3dd6c6` dark / `#14968a` light, `--accent-bright/-dim/-ink`) and a distinct **blue business accent** (`--biz:#7f97f6` dark / `#3f57cf` light). Components consume tokens via `var(--...)` and Tailwind arbitrary values (`text-[var(--text)]`, `bg-[var(--bg-panel)]`), with heavy use of `color-mix(in srgb, var(--accent) N%, ...)` for tints. Per-tenant brand theming on `/me` is applied at runtime through `brandThemeVars` / `brandThemeStyle` overriding `--accent` on a wrapper.

4.10 i18n / locale

`lib/locale-boot.ts` defines **7 locales** — `en, es, fr, de, it, zh, hi` — with `LOCALE_LABELS`, an `isLocale` guard, `LOCALE_STORAGE_KEY = "miai-locale"`, and `LOCALE_BOOT_SCRIPT` (sets `<html lang>` + `data-locale` before paint). `lib/locale.tsx` (`LocaleProvider` / `useLocale` / `useT`) mirrors the theme provider pattern (default `en`, storage-backed, safe fallback). `lib/i18n/` holds one dictionary per locale — **en is the source of truth** (`en.ts`, 305 keys / 347 lines; total across all locales **2,435 lines**); every other file is typed `: Dictionary` so missing keys fail the build. `translate()` (in `i18n/index.ts`) does key lookup + `{var}` interpolation. `components/LanguageSelect.tsx` is the switcher. Note the runtime **chat** reply language is a separate axis (`lib/chat-languages.ts`, used by `MarketplaceAssistant`).

4.11 Rich-text rendering

`lib/rich-text-parse.ts` is a **pure, React-free** parser (unit-testable under the type-stripping test runner): one regex pass over light markdown → typed segments `text | bold | image | link`, with a `youtubeVideoId` extractor. `lib/rich-text.tsx` (`renderRichText`) maps segments to safe React nodes — **no `dangerouslySetInnerHTML`**; only `http(s)` links (`target=_blank rel=noreferrer`) and `https` images are ever emitted, and YouTube links render as a `YouTubeCard` with a derived `i.ytimg.com` thumbnail. This is the assistant-bubble renderer in `ConsumerChatWindow`. (The floating `MarketplaceAssistant` has its own lighter `LinkifyAssistantText` that also linkifies internal marketplace paths.)

4.12 Server/client split & boundaries

- **RSC-first**: 29/39 pages are server components; data-heavy landing pages (`/`, `/agents`, `/agents/[id]`, `/personal/[id]`) fetch from `lib/catalog.ts` / `lib/consumer-catalog.ts` on the server and pass typed props down, so the catalogue render is SSR and SEO-visible.
- **Client leaves**: interactive surfaces are isolated client components (`CatalogGrid`, `AgentStudio`, `useConsumerChat` consumers, `Shell`, `Sidebar`, providers). Co-located client files keep pages thin: `AppChatClient`, `DemoPageClient`, `me/CapabilitiesSheet`, `personal/[id]/SpecialistChat` (+ `app/error.tsx`).
- **force-dynamic**: pages that depend on request-time flags (`SANDBOX_MODE`, live eval data) opt out of static generation — `/app/v1`, `/personal`, `/personal/[id]`, `/quality`, plus the `agent.js` route.
- **Boundaries**: `<Suspense>` fallbacks (home "Loading catalogue..."), route-level `loading.tsx` (root + `/agents/[id]`), `error.tsx`, and `not-found.tsx` are all present.
- **Hydration hygiene**: `suppressHydrationWarning` on `<html>` because the boot scripts mutate `data-theme` / `data-locale` / `lang` before React hydrates.

4.13 Component inventory (scale)

`apps/web/src/components` holds **31 .tsx components + 1 co-located CSS** (`marketplace-assistant.css`). Largest: `CatalogGrid` 1,139, `ActionsPanel` 678, `AgentIcon` 630, `AgentStudio` 550, `Sidebar` 485, `GetStartedWizard` 471, `SandboxChat` 440, `SetupGuide` 382, `KnowledgePanel` 326, `HistoryClient` 305, `MarketplaceHero` 302. Small shared primitives: `ComingSoon`, `SandboxBanner`, `ThemeToggle`, `LanguageSelect`, `TopUpModal`, `ConnectorIcon`, `CatalogMarketBadge`.

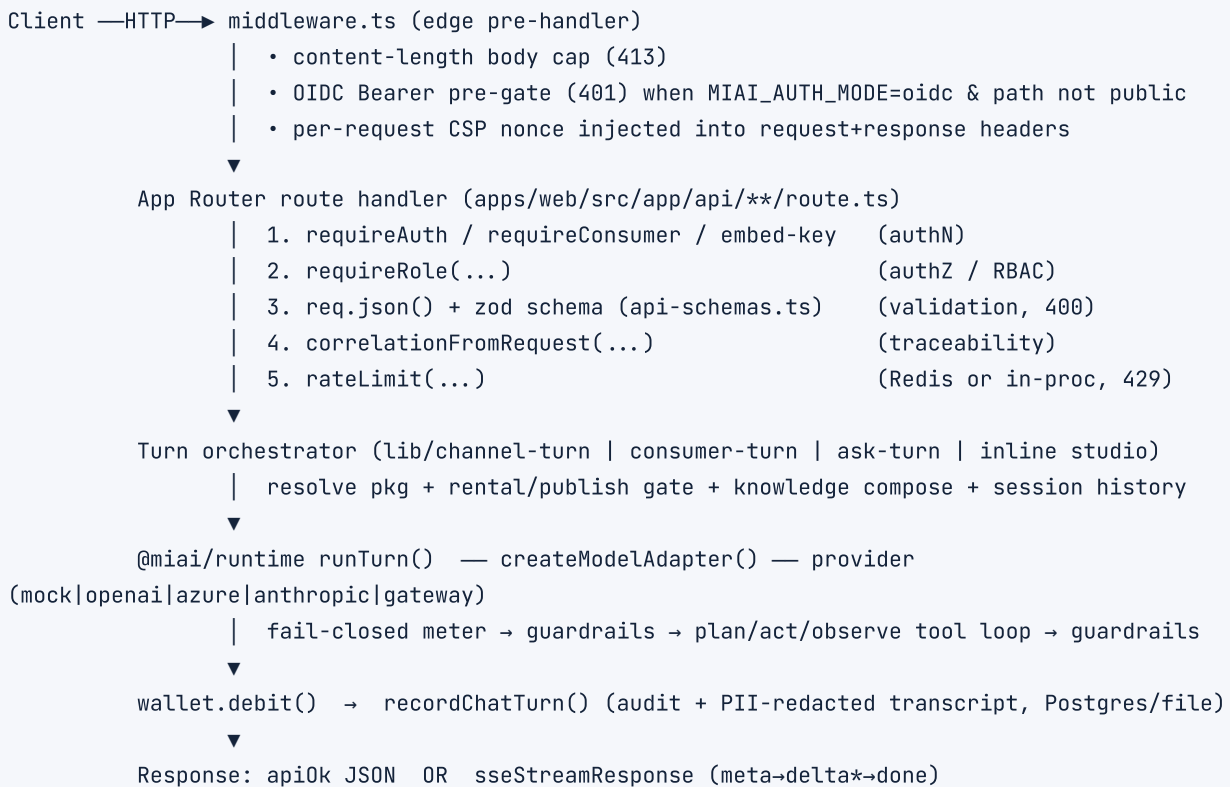
4.14 Handover risks / notes

1. **Stale documented widget size** (above) — reconcile the "6.4 KB/9 KB" claims with the measured ~13.4 KB raw / ~4.3 KB gzip.
2. **Widget is unminified** — served verbatim from a `String.raw` literal with only `max-age=60`; there is no build-time minification step, so the SRI is over the full source (correct, but larger than necessary over the wire).
3. **Embed widget is non-streaming** while all first-party chat surfaces stream over SSE — two divergent client transports to maintain for the same `runTurn` backend.
4. `style-src 'unsafe-inline'` **remains** in the CSP (documented as a deliberate deferral for `next/font` + Tailwind); only `script-src` is nonce-hardened.
5. **Chat images bypass** `next/image` (raw `<img>`, no domain allowlist) — acceptable given the `img-src 'self' data: blob: https:` CSP, but worth noting for perf/abuse review.
6. `AGENT_JS_INTEGRITY` **is computed at build/import time** and threaded through RSC props; any drift between the deployed script body and the snippet's integrity (e.g. edge caching an old body for up to 60 s after deploy) would break embeds with an SRI mismatch — a deploy-ordering consideration.

05 Backend

The server is a single Next.js 15.2.8 App Router application (`apps/web` , React 19 / TypeScript 5 / Node 20), running **69 route handlers** under `apps/web/src/app/api` (re-measured at commit `fb97438` : `find apps/web/src/app/api -name route.ts | wc -l` → 69) plus **39** server-rendered `page.tsx` . Every request-handling concern — body-size caps, auth, RBAC, zod validation, correlation IDs, turn orchestration, SSE streaming, metering, and audit — is factored into shared `lib/*` modules rather than repeated per route, so the four chat surfaces (`studio` , `app` / `embed` , `consumer` , `ask`) share one contract. The runtime brain (`@miai/runtime.runTurn`) is a workspace package the routes call into; it in turn drives the **299,508-line, 573-file catalogue** in `data/catalog` + `data/catalog-consumer` (`552` flat `*.agent.json` SKU files), which is the asset every chat turn reads its `system_prompt` , `knowledge` , `tools` , and `guardrails` from. The runtime is therefore small hand-written code (packages/runtime ≈ 13.8K LOC) orchestrating a very large data asset.

5.1 Request lifecycle overview



5.2 Middleware — the edge pre-handler (`apps/web/src/middleware.ts`)

A single `middleware()` runs on the matcher `"/(?!_next/static|_next/image|favicon.ico|.*\\.?(?:svg|png|jpg|jpeg|gif|webp|ico)$).*)"` (all app + API routes, excluding Next internals/static assets) and does three things:

CONCERN	BEHAVIOUR
Body-size cap	On <code>/api/*</code> , reads the <code>content-length</code> header; if <code>> maxBodyBytes()</code> returns 413 "Payload too large" . Default 1 MiB (1_048_576) , overridable via <code>MIAI_MAX_BODY_BYTES</code> .

CONCERN	BEHAVIOUR
OIDC Bearer pre-gate	Only when <code>MIAI_AUTH_MODE === "oidc"</code> : for <code>/api/*</code> paths not in the public allowlist (<code>isPublicApiPath</code>), a missing/non- <code>Bearer Authorization</code> header short-circuits to 401 before the handler runs.
CSP nonce	Generates a per-request base64 nonce (<code>crypto.randomUUID()</code>), sets it on both the forwarded request headers (<code>x-nonce</code>) and the response <code>Content-Security-Policy</code> (<code>buildContentSecurityPolicy(nonce)</code> from <code>lib/csp.ts</code>), so App Router scripts run under <code>script-src 'nonce-...'</code> without <code>unsafe-inline</code> .

Caveat (open gap): the body cap is a **header check only** – a chunked request that omits `content-length` bypasses it and is bounded only by the per-field zod `max()` limits at the route.

5.3 Authentication (`lib/request-auth.ts` , `lib/auth.ts` , `lib/consumer-auth.ts`)

Three authN entry points, one per audience:

- **B2B / operator routes** → `requireAuth(req)` (`lib/request-auth.ts`) wraps `resolveAuth()` and converts an `AuthError` into a typed `NextResponse` (its `.status`), so handlers stay linear (`const auth = await requireAuth(req); if (!isAuthContext(auth)) return auth;`). `resolveAuth()` (`lib/auth.ts`) has two modes keyed on `MIAI_AUTH_MODE` :
 - **oidc** – extracts the `Bearer` token and verifies it with `jose.jwtVerify(token, getJwks(), { issuer, audience })` , where `getJwks()` builds a cached `createRemoteJWKSet` from `MIAI_OIDC_JWKS_URL` (default `${issuer}/.well-known/jwks.json`). Requires a `workspace_id` (or `workspaceId` / `org_id`) claim or throws **403** "Token missing workspace_id claim"; `userId` from `user_id` / `sub` ; `roles` from the `roles` claim. Returns `AuthContext { mode, workspaceId, userId, roles, raw }` .
 - **mock** (default) – workspace/user/roles from `x-workspace-id` / `x-user-id` / `x-roles` headers or query params, else `WORKSPACE_ID` / `demo-user` . **Production safety:** elevated `["owner", "operator"]` defaults are granted only when `NODE_ENV === "production"` or both `ALLOW MOCK RAILS=1` and `I_UNDERSTAND MOCK RAILS IN PROD=1` are set; otherwise it falls back to `["readonly"]` . In mock it also consults the workspace-member store (`roleFromMembers`) to override the workspace role while preserving platform-operator roles.
- **Consumer routes** → `requireConsumer(req)` (`lib/consumer-auth.ts`) calls `resolveConsumerAuth` (`lib/consumer-identity.ts`), returning `{ consumerId: walletIdForConsumer(auth), auth }` or a ready-to-return error `Response` . Identity is the **consumer session cookie** ("Sign in with Google" OIDC) in oidc mode, a shared demo identity in mock – deliberately kept out of the Bearer gate (see §5.4).
- **Embed / channel routes** → **publishable key**, resolved inside the orchestrator via `resolveEmbedKey(key)` (`lib/store.ts`) rather than a header token (see §5.6).

5.4 Public-path allowlist (`lib/public-paths.ts`)

`isPublicApiPath(pathname)` is the **single source of truth** shared by both `middleware.ts` and `lib/auth.ts` (the doc comment notes the two lists previously drifted and 401'd `/api/v1/embed/chat`). "Public" means *authenticated by something other than the OIDC Bearer* :

- **PUBLIC_EXACT** : `/api/health` , `/api/version` , `/api/catalog` , `/api/consent` , `/api/auth/handoff` , `/api/v1/openapi` , `/api/consumer/brief/run-due` (CRON_SECRET), `/api/payments/paystack/webhook` (HMAC-SHA512), `/api/payments/paystack/return` (reference-verified).

- `PUBLIC_PREFIXES`: `/api/catalog/`, `/api/oauth/callback`, `/api/embed/`, `/api/app/`, `/api/v1/embed/`, `/agents/v1/`, `/api/webhook/sink`, `/api/mcp`, `/api/consumer/` (each enforces its own secret/session/key in-route).

Everything else — including `/api/v1/rent`, `/api/ops`, `/api/admin` — still requires a verified Bearer under OIDC.

5.5 Authorization / RBAC and rate limiting (`lib/security.ts`)

RBAC is a monotonic rank ladder plus a cross-tenant platform tier:

```
WORKSPACE_ROLES = readonly(1) < agent(2) < admin(3) < owner(4)
PLATFORM_ROLES = { operator, platform_admin, miai_admin } → treated as ≥ admin
```

- `normalizeRoles()` maps aliases (`read-only` / `reader` / `viewer` → `readonly`, `platform_admin` / `miai_admin` → `operator`).
- `requireRole(auth, min)` → `null` if `hasMinRole`, else **403 Forbidden – requires <min> role or higher**. The Studio chat route gates on `requireRole(auth, "agent")`.
- `requireOperator(auth)` gates MyInstantAI operator surfaces (Agent Admin); allows platform roles, or mock-mode `owner`.
- `rateLimit(key, {limit, windowMs})` is a token bucket: uses Redis `INCR` + `EXPIRE` (`redisIncr` / `redisTtl`) when Upstash is configured (shared across replicas), else an in-process `Map`. It is **fail-closed on Redis error** — if Redis is configured but `INCR` returns null, it returns `{ ok:false }` rather than silently degrading to a per-replica `Map`. Per-surface limits: `app` / `embed` / `consumer` = **30 / 60s**, `ask` = **40 / 60s**.

5.6 Request validation & error envelope (`lib/api-schemas.ts`, `lib/api-error.ts`)

All bodies are parsed with **zod ^3.24**. `parseJsonBody(req, schema)` (and the inline `schema.safeParse` variant used by chat routes) returns a typed value or a **400** with `formatZodError()` (`"path: message; ..."`). Field caps double as a defence-in-depth body limit behind the middleware cap: `message` = 1–8000 chars, `longText` / knowledge paste = ≤100_000, `correlationId` / `workspaceId` = ≤200, tool-call `bindings` ≤100 entries. Chat schemas:

SCHEMA	SURFACE	NOTABLE FIELDS
<code>studioChatBodySchema</code>	<code>/api/chat</code>	<code>agentId</code> , <code>message?</code> , <code>mode: sandbox live</code> , <code>clear</code> , <code>sessionId</code> , <code>correlationId</code>
<code>channelChatBodySchema</code>	<code>/api/app/chat</code> , <code>/api/embed/chat</code>	<code>key</code> (≤512), <code>message</code> , <code>sessionId</code> , <code>replyLanguage</code>
<code>consumerChatBodySchema</code>	<code>/api/consumer/chat</code>	<code>message</code> , <code>agentId?</code> (defaults flagship), no key (identity from auth)
<code>askChatBodySchema</code>	<code>/api/ask/chat</code>	<code>message</code> , <code>sessionId</code>

The error/success envelope is centralized in `lib/api-error.ts`: `apiOk(body, status, headers)`, `apiError(status, error, detail?, headers?)`, and `apiErrorFromRequest(req, ...)` which auto-attaches a `correlationId` pulled from `x-correlation-id` / `x-request-id`. Bodies are always `{ error, detail?, correlationId? }`.

5.7 The turn orchestrators

Four thin orchestrators translate an HTTP request into a `runTurn` call and back, each adding surface-specific concerns:

- **Studio** – inline in `apps/web/src/app/api/chat/route.ts`. `requireAuth` → `requireRole("agent")` → `studioChatBodySchema`. Handles a `clear` branch (wipes rental messages + audits), auto-creates a rental in state `selected` on first touch, derives `mode` (`live` when rental live, else `sandbox`) and `freeTry` (sandbox + selected → `runTurn(..., { skipDebit:true })` so pre-rent trials are not metered), composes knowledge via `getComposedKnowledge`, calls `runTurn`, persists `result.messages` + next state via `upsertWorkspaceAgent`, then `recordChatTurn` + `trackEvent("miai.chat.turn", ...)`. Response strips `tool` messages. This is the only chat surface that is **JSON one-shot only** (no SSE).
- **Channel** – `lib/channel-turn.ts` (`embed` + in- `app`). `prepareChannelTurn` runs the full gate chain: reject empty/rate-limited → `resolveEmbedKey(key)` (401 invalid) → `getAgentPackage` (404) → rental **publish gate** (`LIVE_STATES = {live, rented, paused_no_tokens}` , else 403 "not published") → **per-agent domain lock** (`originAllowed(origin, referer, approvedDomains)`) – a non-CORS guard against lifted public keys → compose knowledge → load session history → build `systemAppend` (per-channel `HANDOFF_POLICY` + reply-language). `finalizeChannelTurn` writes session history, flips state to `paused_no_tokens` if the wallet paused, and records the turn. Two entry points differ only in whether `onDelta` / `onToolStart` hooks are passed: `runChannelTurn` (JSON) vs `runChannelTurnStream` (SSE).
- **Consumer** – `lib/consumer-turn.ts`. No embed key, no rental/publish gate – the **only run gate is the wallet**. Adds durable memory: before the turn it folds `getMemoryContext` + `getGoalsContext` + `getPeopleContext` (scoped to `{tenantId, consumerId}`) into `systemAppend`; after the turn it reconstructs memory/reminder writes from `result.toolCalls` (`persistMemoryWrites` , `persistReminderWrites`) and passively extracts self-facts from the user message (`persistPassiveFacts`), all best-effort (a memory write never fails a turn). Note: consumer tool calls are **surfaced but not executed against real connectors yet** (documented in-file) – memory is written from the tool `args`.
- **Ask** – `lib/ask-turn.ts`. The first-party marketplace "Ask AI". Fixed `marketplaceWorkspaceId()` / `marketplaceAssistantId()` package, Redis-or-in-proc session store (`MAX_TURNS=24` , `SESSION_TTL_SEC=24h` , `MAX_SESSIONS=400`), captures `capture_lead` tool calls into `createAskLead`, and white-label-scrubs the reply (`Move Digital` → `MyInstantAI` , `Zara` → `our team` , `LEAD-*` → real id). JSON one-shot only.

5.8 SSE streaming (`lib/sse.ts` , `lib/chat-stream.ts` , `lib/chat-stream-client.ts`)

`sseStreamResponse(run, extraHeaders?)` wraps an async producer in a `ReadableStream`, handing it `send(event, data)` (`sseLine = event: ...\\ndata: <json>\\n\\n`) and **always** closing the controller in a `finally`. Headers: `content-type: text/event-stream`, `cache-control: no-cache, no-transform`, `connection: keep-alive`, `x-accel-buffering: no` (defeats proxy buffering). The stream always opens **200** – errors are delivered as an in-band `error` event, not an HTTP status.

`streamChatTurn(send, meta, runStream)` (`lib/chat-stream.ts`) defines the one canonical event choreography shared by `app` and `consumer` SSE:

```
meta { ...meta, streaming:true }
delta { text } ← onDelta, live model tokens
status { phase:"tool" } ← onToolStart, resets a partial bubble on a tool round
paused { reply, balance } (only when wallet paused)
done { reply, paused, balance, correlationId }
error { error, detail, status } (on !ok or thrown)
```

The client half (`lib/chat-stream-client.ts`) reads `res.body.getReader()` + `TextDecoder` , splits complete `event: / data:` blocks, and `mapEvent` s them back into `{type: delta|paused|done}` . `app/chat` and `consumer/chat` **negotiate**: SSE by default, JSON one-shot when `Accept: application/json` and not `text/event-stream` . `embed/chat` is JSON-only (`lib/handlers/embed-chat.ts`).

5.9 Correlation IDs & traceability (`lib/traceability.ts`)

`correlationFromRequest(req, bodyCorr?)` resolves precedence **header** (`x-correlation-id` → `x-request-id` → `x-miai-correlation-id`) → **body** → **freshly minted** `corr_<base36>_<hex>` (`newCorrelationId`), truncated to 128 chars – so a caller's trace id flows through the whole turn and out in every response and audit row. Every turn ends in `recordChatTurn(...)` , which:

1. **PII-redacts** user + assistant text (`redactPii`).
2. Writes an audit event (`appendAudit` , type e.g. `embed_turn` / `consumer_turn` / `agent_turn` / `paused_no_tokens`) with `{correlationId, sessionId, tokensDebited, paused, tools, liveTools, stubTools, model, mode, previews...}` , classifying each tool call as `live` vs `stubbed` (sandbox stub) from its result shape, and emits a separate `tool_error` audit event per failed tool.
3. Appends a full `TurnTranscript` (`appendTurnTranscript`) capped at `TURN_CAP = 20_000` rows, persisted to **Postgres** `miai_turns` (`INSERT ... ON CONFLICT (id) DO UPDATE` , JSONB `payload`) when a pool exists **and** mirrored to a JSON file (`turn-transcripts.json` under `DATA_DIR / ../../data`), hydrating from Postgres-first on cold start. Retrieval: `getTraceByCorrelation(correlationId)` joins transcripts + audit for `/api/history/trace/[correlationId]` ; `deleteTurnTranscriptsForWorkspace` backs DSAR erase.

5.10 The runtime engine – `runTurn()` (`packages/runtime/src/index.ts` , ~2,852 LOC)

`runTurn(req, deps)` is the shared brain. `deps` = `{ wallet?, model?, onDelta?, onToolStart?, skipDebit? }` .

Ordered behaviour:

1. **Template materialization** – `buildTemplateVars` + `materializePackage` fill `{{business_name}}` etc. so tenants never see raw catalogue tokens; `scrubLeakedPlaceholders` re-checks on the way out.
2. **Fail-closed metering (before any provider call)** – if `req.state === "paused_no_tokens"` , or if `!skipDebit && (await wallet.getBalance()).tokens ≤ 0` , it returns the localized `paused_no_tokens` top-up message with `tokensDebited:0` , `paused:true` and **makes no model call**. This is the load-bearing zero-balance guarantee.
3. **Knowledge selection** – `MockModel` gets a full KB prefix (`slice(0, RUNTIME_KNOWLEDGE_CHARS ?? 40_000)`) for deterministic evals; live models get `selectKnowledgeForPromptAsync(...)` – **hybrid semantic + lexical retrieval** when an embedder is configured (`semanticRetrievalEnabled()` / `createEmbedderFromEnv()`), lexical otherwise.
4. **System prompt assembly** – concatenates `system_prompt` , response rules, `## Knowledge base` , `## Guardrails` (≤4000 chars), `systemAppend` , and a `Mode/Model` footer.
5. **Input guardrails** – `checkInputGuardrails(userMessage, system, tools, {consumerLine})` (`packages/runtime/src/guardrails.ts`) can force a safe completion before the model is consulted.
6. **Per-vertical workflow dispatch** – before the generic LLM path, ~20 `isXxx(agentId)` guards route to dedicated multi-step modules under `packages/runtime/src/workflows/` (executive-assistant, it-helpdesk, booking-front-desk, sales-qualifier, restaurant-takeaway, hotel-guest, accounting-practice, events-venue, building-management, pharmacy, gym-membership, dental-front-desk, delivery-tracking, customer-support, marketplace-assistant, ...). A handled workflow finalizes via `finishWorkflow` (which also debits + emits its own `plan` / `steps` structure).
7. **Bounded plan→act→observe tool loop** – `maxToolRounds = min(5, max(1, RUNTIME_MAX_TOOL_ROUNDS ?? 3))` (default 3). Each round: `onToolStart()` → `executeConnector(...)` with the resolved `binding` → push a `role:"tool"` message → decide (read-tool fallback to KB on failure, handoff/emergency/GDPR special-casing,

order/booking/application synthesizers, or a follow-up `modelAnswer` that may itself request another tool while `allowMoreTools`). The loop is credit-checked overall by the single debit at the end.

8. **Output guardrails** — `checkOutputGuardrails(...)` scrubs card/OTP leakage from the final reply.

9. **Metering & debit** — token count prefers **provider-reported usage summed across all model calls this turn** (`turnUsageTotal`), falling back to `estimateTurnTokens` (char estimate) for the mock model. `wallet.debit({ workspaceId, amount, idempotencyKey, reason:"agent_turn", agentId })`. `paused = !skipDebit && (!debit.ok || debit.paused)`.

Streaming flows through `modelAnswer(model, input, onDelta)` → `iterateModelStream` → the adapter's `streamComplete` (true stream for openai/gateway/azure/anthropic) or `streamFromComplete` fallback; `delta` chunks are accumulated and forwarded to `onDelta`. Workflow/tool-synthesized replies are re-chunked into ~8-char `onDelta` pushes so non-streaming paths still animate.

5.11 Model adapters (`createModelAdapter`, same file)

`createModelAdapter()` selects on `MIAI_MODEL_MODE` (default `mock`) and returns one `ModelAdapter` implementing `complete` / `streamComplete`:

MODE	CLASS	ENDPOINT / AUTH
<code>mock</code> (default)	<code>MockModelAdapter</code>	deterministic, offline — powers evals & sandbox
<code>openai</code>	<code>OpenAIModelAdapter</code>	<code>api.openai.com/v1</code> , <code>OPENAI_API_KEY</code>
<code>anthropic</code> / <code>claude</code>	<code>AnthropicModelAdapter</code>	<code>api.anthropic.com/v1</code> OpenAI-compatible surface, <code>ANTHROPIC_API_KEY</code>
<code>gateway</code> / <code>http</code>	<code>GatewayModelAdapter</code>	<code>MIAI_MODEL_GATEWAY_URL</code> , OpenAI-compatible
<code>azure</code>	<code>AzureOpenAIModelAdapter</code>	Azure OpenAI deployment URL + <code>api-key</code> header — stated Azure migration target

A **sandbox cost backstop** falls back to the mock model after `SANDBOX_MODEL_TURN_CAP` (default 1000) real-provider turns per process when `SANDBOX_MODE=1`, so an eval run cannot run up an unbounded bill.

5.12 Wallet / metering fail-closed (`packages/wallet-adapter/src/index.ts`)

`createWalletAdapter()` (process-singleton) selects on `MIAI_WALLET_MODE`: `http` (`HttpWalletAdapter` → `MIAI_WALLET_API_URL`, 15s `AbortController` timeout, `idempotency-key` header on debit/topup) when configured, else `mock` (`MockWalletAdapter`, in-memory). Both are **fail-closed and never throw on insufficient funds**: `HttpWalletAdapter.debit` maps `402/409` to `{ ok:false, paused:true }` (mirroring the mock), and both dedupe on the debit `idempotencyKey` /top-up reference so a webhook retry cannot double-credit. Combined with the pre-call balance check in `runTurn` (§5.10.2), zero balance is enforced twice — once before the provider call, once at debit.

5.13 End-to-end trace — one consumer chat turn, POST → tokens → persistence

- `POST /api/consumer/chat` with `{ message, agentId? }`, `Accept: text/event-stream`.
- `middleware.ts`: `content-length` ≤ 1 MiB; path `/api/consumer/` is public to the Bearer gate; CSP nonce attached.

3. `requireConsumer(req)` → `resolveConsumerAuth` verifies the Google session cookie → `{ consumerId, auth }`; `walletId = consumerId`.
4. `req.json()` → `consumerChatBodySchema.safeParse` (400 on failure). `agentId = body.agentId ?? DEFAULT_CONSUMER_AGENT`.
5. `correlationFromRequest(req, body.correlationId)`; `rateLimit("consumer:<walletId>:<agentId>", {limit:30, windowMs:60000})` (429 on exceed).
6. SSE chosen → `sseStreamResponse(send ⇒ streamChatTurn(send, {channel:"consumer", agentId}, hooks ⇒ runConsumerTurnStream({..., ...hooks})))`. `streamChatTurn` immediately emits `meta`.
7. `runConsumerTurnStream` → `prepare`: `isRunnableConsumerAgent` (403 if not) → `getAgentPackage` (404 if missing) → `getComposedKnowledge` → load session history (`createSessionStore`, Redis-or-in-proc) → fold durable memory/goals/people into `systemAppend`.
8. `runTurn(turnInput, { wallet, onDelta, onToolStart })`: balance check (fail-closed) → knowledge retrieval → system assembly → input guardrails → (consumer isn't a vertical workflow) generic path → `modelAnswer` streams tokens; each `delta` fires `onDelta` → `send("delta", {text})` to the browser; a tool round fires `onToolStart` → `send("status", {phase:"tool"})` → bounded ≤3-round loop → output guardrails → `wallet.debit(...)`.
9. `finalize`: `sessionStore.set` (persist history) → reconstruct memory/reminder/passive-fact writes from `result.toolCalls` (best-effort) → `recordChatTurn` (PII-redacted audit + `miai_turns` transcript, Postgres + file) → returns `{ ok, assistantMessage, paused, balance, correlationId }`.
10. `streamChatTurn` emits `paused` (if metering paused) then `done { reply, paused, balance, correlationId }`; the `ReadableStream` `finally` closes the controller. Client `chat-stream-client.ts` has been rendering `delta` s live and resolves on `done`.

5.14 Route inventory (69 handlers, grouped)

GROUP	COUNT	REPRESENTATIVE ROUTES
<code>consumer/*</code>	15	<code>chat</code> , <code>auth/{login,callback,logout,me}</code> , <code>brief/{,run,run-due}</code> , <code>connectors/*</code> , <code>reminders/*</code> , <code>wallet</code> , <code>telegram/webhook</code>
<code>oauth/*</code>	5	<code>[connector]/{start,disconnect,test}</code> , <code>callback</code> , <code>status</code>
<code>knowledge/*</code>	5	<code>route</code> , <code>[id]</code> , <code>crawl</code> , <code>paste</code> , <code>upload</code>
<code>catalog/*</code> , <code>payments/paystack/*</code> , <code>v1/*</code>	3 each	<code>catalog / catalog/family/[familyId] / catalog/personal</code> ; <code>paystack init / return / webhook</code> ; <code>v1/{embed/chat,rent,openapi}</code>
<code>mcp/*</code> , <code>history/*</code> , <code>embed/*</code> , <code>dsar/*</code> , <code>custom-requests/*</code> , <code>connectors/*</code> , <code>ask/*</code> , <code>workspace/members/*</code>	2 each	<code>mcp + mcp/tools/call</code> ; <code>history/turns</code> + <code>history/trace/[correlationId]</code> ; <code>embed/chat</code> + <code>embed/sri</code> ; <code>dsar/{export,erase}</code> ; <code>ask/{chat,leads}</code>
singletons	1 each	<code>chat</code> (Studio), <code>app/chat</code> , <code>rent</code> , <code>rentals</code> , <code>configure</code> , <code>wallet</code> , <code>audit</code> , <code>insights</code> , <code>ops</code> , <code>admin</code> , <code>onboarding</code> , <code>consent</code> , <code>health</code> , <code>version</code> , <code>agents/[id]</code> , <code>slack/channels</code> , <code>proof/tool</code> , <code>webhook/sink</code> , <code>auth/handoff</code>

Chat/data handlers declare `export const dynamic = "force-dynamic"` to opt out of static optimization.

5.15 Scale note (re-measured at fb97438)

The backend is deliberately thin over a very large data asset. Re-measured: **69** API route handlers, **39** page components, and a **299,508-line / 573-file** catalogue (`data/catalog` + `data/catalog-consumer` , `552 *.agent.json` SKUs) that every turn reads its prompt, knowledge, tools, and guardrails from — this catalogue is the dominant asset and must be counted in any "size of the backend" statement, not just the ~13.8K-LOC runtime engine or the ~34.5K-LOC `apps/web` . The whole platform now measures **406,319 authored lines / 1,212 files**, grown from the ~398K / 1,021-file figure in the Licence Proposal.

06 AI Platform

The AI Platform is the `@miai/runtime` package (`packages/runtime` , `"version": "0.1.0"` , ESM, `type: module`). It is the execution engine every channel funnels into: web chat, embed widget, Telegram/WhatsApp webhooks, the MCP server, the eval harness, and the consumer line all converge on a single exported entry point, `runTurn()` . This section deep-dives that engine.

6.0 Scale and where the runtime sits

The runtime is deliberately small relative to the asset it drives. Measured on disk (`wc -l` , excluding `dist/` , `node_modules/`):

COMPONENT	PATH	LOC	FILES
Runtime engine (hand-written TS + tests)	<code>packages/runtime</code>	13,818	36
– of which <code>src/</code> TypeScript	<code>packages/runtime/src</code>	12,404	27
– of which <code>src/index.ts</code> (barrel + <code>runTurn</code> + adapters + mock)	<code>packages/runtime/src/index.ts</code>	2,852	1
– of which unit tests	<code>packages/runtime/test/*.mjs</code>	(rest)	9
Catalogue IP the runtime executes	<code>data/catalog</code> + <code>data/catalog-consumer</code>	299,508	552 <code>*.agent.json</code>

The ~13.8K-LOC engine is the machine; the 299,508 lines of catalogue JSON (552 agent packages: 500 SKUs + legacy ZA aliases + 17 consumer specialists, each carrying `system_prompt` , `knowledge` , `tools` , `guardrails` , `evals`) are the fuel. `runTurn` receives one `AgentPackage` per call and turns that static IP into a metered, guard-railed, tool-using conversation. The engine is generic; all vertical knowledge lives in the catalogue and in the 22 workflow modules.

6.1 Package layout and public surface

`packages/runtime/src/index.ts` is a barrel that re-exports the sub-modules and defines the engine types and `runTurn` . Sub-modules:

FILE	LOC	RESPONSIBILITY
<code>index.ts</code>	2,852	Types (<code>TurnRequest</code> / <code>TurnResult</code> / <code>ModelAdapter</code> / <code>ModelCompleteInput</code> / <code>ModelCompleteResult</code> / <code>StreamChunk</code> / <code>TokenUsage</code>), <code>runTurn()</code> , <code>createModelAdapter()</code> , the 5 adapter classes, <code>MockModelAdapter</code> , provider HTTP layer, workflow dispatch
<code>embedding.s.ts</code>	209	<code>Embedder</code> interface, <code>LocalHashEmbedder</code> , <code>OpenAiCompatibleEmbedder</code> , <code>cosineSimilarity</code> , <code>l2Normalize</code> , cache, <code>createEmbedderFromEnv</code> , <code>semanticRetrievalEnabled</code>

FILE	LOC	RESPONSIBILITY
<code>knowledge</code> <code>-</code> <code>retrieve.</code> <code>ts</code>	245	<code>retrieveKnowledgeChunks</code> (lexical), <code>retrieveKnowledgeChunksHybrid</code> (semantic+lexical), <code>selectKnowledgeForPrompt(Async)</code>
<code>guardrail</code> <code>s.ts</code>	244	<code>checkInputGuardrails</code> , <code>checkOutputGuardrails</code>
<code>templates</code> <code>.ts</code>	165	<code>buildTemplateVars</code> , <code>materializePackage</code> , <code>applyTemplateVars</code> , <code>scrubLeakedPlaceholders</code>
<code>workflows</code> <code>/*.ts</code>	~7.9K	20 per-vertical workflow modules + <code>i18n.ts</code> (<code>wf</code> reply-string table) + <code>stop-suppression.ts</code>

Exported types worth naming for handover: `TurnRequest`, `TurnResult`, `ChatMessage`, `AgentState` (`"selected" | "configuring" | "rented" | "live" | "paused_no_tokens"`), `ModelAdapter`, `ModelCompleteInput`, `ModelCompleteResult`, `TokenUsage`, `StreamChunk`, `WorkflowPlan`, `WorkflowStep`.

6.2 The turn loop – `runTurn()`

`runTurn(req: TurnRequest, deps?)` (`index.ts:2144`) is the whole request lifecycle. `deps` lets callers inject `wallet`, `model`, and three streaming callbacks (`onDelta` per text delta, `onToolStart` to reset a partial bubble, `skipDebit` for free sandbox tries). Execution order:

- Template materialization.** `buildTemplateVars(req.pkg)` + `materializePackage(req.pkg)` fill `{{business_name}}` - style tokens so customers never see raw catalogue placeholders; `knowledgeOverride` is run through `applyTemplateVars`.
- Paused short-circuit.** If `req.state === "paused_no_tokens"`, it returns the localized `wf(req.replyLanguage, "paused_no_tokens")` message immediately – **no model call**.
- Knowledge selection.** `knowledge = req.knowledgeOverride?.trim() || req.pkg.knowledge`; budget `RUNTIME_KNOWLEDGE_CHARS` (default `40_000`). Path forks on adapter type:
 - `MockModelAdapter` → raw `knowledge.slice(0, budget)` (keeps deterministic evals / `knowledgeHit` stable).
 - Live adapters → `await selectKnowledgeForPromptAsync(knowledge, req.userMessage, budget, semanticRetrievalEnabled() ? createEmbedderFromEnv() : null)` – hybrid RAG (§6.5).
- System-prompt assembly.** A single `system` string is built from `pkg.system_prompt`, a hard-coded `## Response rules` block ("answer factual questions from the knowledge base first; only call tools for live actions; never paste raw JSON"), `## Knowledge base` (the retrieved slice), `## Guardrails` (`pkg.guardrails.slice(0, 4000)`), an optional `systemAppend` (embed policies), and a `Mode: ... Model: ...` footer.
- Fail-closed credit gate.** `const bal = await wallet.getBalance(req.workspaceId); if (!skipDebit && bal.tokens ≤ 0) { return paused_no_tokens } ( index.ts:2228 ). This is the metering guarantee: the balance is read before any provider call, and a zero balance returns a top-up message having made no LLM request.`
- Binding resolution.** `resolveBindings(pkg, req.bindings)` → explicit override, else `getPreset(pkg.manifest.id).bindings`, else `defaultBindingsForTools(...)`.
- Workflow dispatch.** 20 `if (isX(req.agentId)) { ... }` guards (`index.ts` ~2320–2620) try each vertical workflow first; a `handled` result finishes the turn via `finishWorkflow` (which debits and streams). See §6.7.

8. Model + tool rounds. If no workflow claimed the turn, the plan→act→observe loop runs (§6.4).

9. Output scrub + debit. `checkOutputGuardrails` runs on the final draft, the assistant message is pushed, tokens are computed and debited, `scrubLeakedPlaceholders` strips any residual template tokens, and `TurnResult` is returned with `tokensDebited`, `balance`, `state`, `paused`, `toolCalls`, and (for workflows) a `workflow` plan summary.

`TurnResult.state` transitions: on a failed/paused debit → `"paused_no_tokens"`; otherwise a `"rented"` agent flips to `"live"`.

6.3 Model-provider abstraction — `createModelAdapter()` and the four live modes

Every adapter implements the two-method `ModelAdapter` interface:

```
interface ModelAdapter {
  complete(input: ModelCompleteInput): Promise<ModelCompleteResult>;
  streamComplete?(input: ModelCompleteInput): AsyncIterable<StreamChunk>;
}
```

`modelAnswer()` normalizes the two: `iterateModelStream` uses `streamComplete` when present, else wraps `complete` with `streamFromComplete` (which chops the finished string into ≥8-char pseudo-deltas so the UI still animates). This is why openai/gateway/azure/anthropic stream true SSE token deltas while the mock "paces" a pre-generated answer.

`createModelAdapter()` (`index.ts:2104`) selects by `MIAI_MODEL_MODE` (default `mock`) **and** requires the mode's credentials to be present, else it silently falls back to `MockModelAdapter`:

MIAI_MODEL_MODE	CLASS	ENDPOINT	AUTH	GUARD
<code>mock</code> (default)	<code>MockModelAdapter</code>	none	none	deterministic, offline
<code>openai</code>	<code>OpenAIModelAdapter</code>	<code>https://api.openai.com/v1/chat/completions</code>	Bearer <code>OPENAI_API_KEY</code>	key required
<code>anthropic</code> / <code>claude</code>	<code>AnthropicModelAdapter</code>	<code>https://api.anthropic.com/v1/chat/completions</code> (OpenAI-compatible surface)	Bearer <code>ANTHROPIC_API_KEY</code>	key required
<code>gateway</code> / <code>http</code>	<code>GatewayModelAdapter</code>	<code>MIAI_MODEL_GATEWAY_URL</code>	Bearer <code>MIAI_MODEL_GATEWAY_KEY</code> / <code>_API_KEY</code>	URL required
<code>azure</code>	<code>AzureOpenAIModelAdapter</code>	<code>\${AZURE_OPENAI_ENDPOINT}/openai/deployments/{deployment}/chat/completions?api-version=...</code>	<code>api-key</code> header	key + endpoint required

All four live adapters route through one HTTP core, `openAiCompatibleComplete()` / `openAiCompatibleStream()`, so tool-calling stays on a single code path. Azure is a first-class citizen — the migration target — differing only in endpoint shape (deployment name + `api-version`, default `2024-10-21`) and `api-key` auth (`providerAuthHeaders(apiKey, azureAuth)`).

Model mapping. Marketplace model aliases (`gemini-flash`, `gpt-4o-mini`, `claude-sonnet`, `gpt-4o`, `claude-opus`) map to concrete provider models via `OPENAI_MODEL_MAP` and `ANTHROPIC_MODEL_MAP` (e.g. `claude-sonnet` → `claude-sonnet-4-5`, `claude-opus` → `claude-opus-4-1-20250805`). `GatewayModelAdapter` honors `MIAI_MODEL_PASSTHROUGH=1` to send the alias verbatim. Azure picks `AZURE_OPENAI_DEPLOYMENT_LARGE` for the three "large" tiers.

Retries & fallback. `fetchProviderWithRetry(url, init, maxAttempts=3)` retries only on `429` / `5xx` (`isRetryableProviderStatus`) and transient network throws, with exponential backoff `400 * 2**attempt` ms (400ms, 800ms). Non-retryable `4xx` is not retried. Separately, `openAiCompatibleComplete` tries `[modelId, input.fallbackModel]` in sequence (`manifest.model.fallback`), and if everything fails returns a soft user-facing string (`MODEL_PROVIDER_SOFT_ERROR`, "I'm having trouble reaching my knowledge...") rather than throwing — so a provider outage degrades to a handoff offer, never a 500.

Token accounting. `ModelCompleteResult.usage` is populated from the provider's `usage` block (`mapUsage` normalizes `prompt_tokens` / `completion_tokens` / `total_tokens`); streaming requests set `stream_options: { include_usage: true }` to get a final usage chunk. `runTurn` sums `turnUsageTotal` across the initial call **and** every tool-round follow-up. Final debit prefers this provider-reported total; only when it is 0 (mock, or a provider that omits usage) does it fall back to the char estimate `estimateTurnTokens(model, charsIn, charsOut) = max(200, round((charsIn+charsOut)/4)) * mult`, where `mult` is the burn multiplier (flash 0.4x, mini 0.5x, sonnet 1x, gpt-4o 1.6x, opus 3x — matching the `MODELS` table in `apps/web/src/lib/models.ts`).

Sandbox cost backstop. When `SANDBOX_MODE=1` and `mode ≠ mock`, `createModelAdapter` counts real-provider turns per process and, past `SANDBOX_MODEL_TURN_CAP` (default 1000), returns the mock adapter — an evaluation cannot run up an unbounded bill.

6.4 Tool/connector invocation and the bounded plan→act→observe loop

For non-workflow agents, `runTurn` runs a bounded reason loop:

```
maxToolRounds = min(5, max(1, RUNTIME_MAX_TOOL_ROUNDS ?? 3)) // default 3, hard cap 5
```

- **Plan.** First `modelAnswer(model, {system, messages, tools: req.pkg.tools}, onDelta)`. Before this, `checkInputGuardrails` may pre-empt with a `forced` completion (§6.6) — and a forced completion can itself be a tool call (e.g. `handoff_to_human`).
- **Act.** While `completion.toolCall` and `toolRound < maxToolRounds`: `onToolStart()` fires, then `executeConnector({workspaceId, agentId, tool, args, binding: bindingFor(name, bindings), mode})` from `@miai/connectors` runs the bound connector (real in `mode:"live"`, stubbed in sandbox). The call is recorded in `toolCalls[]` with `connector`, `stubbed`, and `live` flags, and the raw result is pushed as a `role:"tool"` message.
- **Observe.** The result is fed back for a follow-up model call that must answer in natural language ("Do not show JSON..."). The loop has hand-tuned deterministic terminators so common actions don't waste a second model round: read-tool failure → answer from knowledge only; unreachable write tool → offer a teammate; `handoff_to_human` → emergency/GDPR/generic handoff copy; `get_order_status` / `*book*` / `*application*` → templated confirmations. Otherwise it re-invokes the model with `tools` still attached only while `toolRound + 1 < maxToolRounds`, so it can chain up to the cap and no further.

`OpenAI` -style tool schemas are built by `openAiToolsPayload` from each `AgentPackage.tools[]` (`name`, `description`, `parameters`). Bindings come from presets (`@miai/presets`) and default to a `webhook` connector when unbound (`bindingFor`).

The loop is **credit-gated once, not per round**: the balance gate at step 5 admits the turn, and a single `wallet.debit` at the end covers all model calls in the turn (`idempotencyKey =`

`${workspaceId}:${agentId}:${Date.now()}:${messages.length}`). There is no per-round balance re-check inside the ≤ 3 rounds.

6.5 Hybrid semantic retrieval / RAG

RAG lives in `knowledge-retrieve.ts` + `embeddings.ts`. It is a **hybrid lexical + semantic** ranker with graceful degradation, not a persistent vector DB (there is no pgvector; vectors are computed per turn and LRU-cached in-process).

- **Chunking.** `splitChunks` slices the knowledge blob at markdown headings (`\n(?:=#+)`), drops chunks `< 30` chars, and caps the corpus at `80_000` chars.
- **Lexical.** `retrieveKnowledgeChunks(knowledge, query, {topK=6, maxChars=12000})` → `scoreChunk` scores keyword overlap with stop-word-filtered `queryTerms`, **soft-bans meta chunks** ("how this file works", guardrail/response-rule headers → score `-1`), and applies domain boosts (hours +12, refund +14, PTO +18) and a creative-writing penalty (`poem|essay|joke|homework` -20). `packChunks` packs top-K within budget at ≤ 1600 chars/chunk.
- **Hybrid.** `retrieveKnowledgeChunksHybrid(knowledge, query, embedder, {semanticWeight=0.6})` embeds `[query, ... chunks]` in one batch, min-max-normalizes lexical and cosine scores, blends `0.6*sem + 0.4*lex`, requires a floor of signal (`sem<0.15 && lex<0.15` → `0`; final filter `>0.12`), and **falls back to pure lexical if the embedder throws or returns nothing** (try/catch around `embedder.embed`).
- **Selection.** `selectKnowledgeForPromptAsync` (the live-model path from `runTurn`) calls hybrid when an embedder is supplied, else lexical, then `assemblePrompt` prepends any `## Key facts` block; when retrieval finds nothing it degrades to a raw prefix slice (`trimmed.slice(0, budget)`). **This prefix-slice-vs-retrieved-chunks branch is the actual "prompt-stuff → retrieval" switch** — it is driven by embedder availability and retrieval hits, not by a hard char threshold (see open gaps re. the "~6,000-char" figure).

Embedders & gating. `createEmbedderFromEnv()` returns `OpenAiCompatibleEmbedder` when `EMBEDDING_API_KEY` / `OPENAI_API_KEY` is set (`EMBEDDING_BASE_URL` / `OPENAI_BASE_URL` / default `https://api.openai.com/v1`; `EMBEDDING_MODEL` default `text-embedding-3-small`; batched 64; inputs sliced to 8,000 chars; L2-normalized), else a 384-dim deterministic `LocalHashEmbedder` (FNV/hashing-trick with word, trigram, and char-bigram features) — but only when explicitly allowed. `RUNTIME_SEMANTIC_RETRIEVAL` gates it: `0/false/off` disables; `1/true/on/local` forces on (local permitted); unset/ `auto` → on only if a remote embedding key exists. Vectors share an FNV-1a-keyed LRU (`CACHE_MAX=2048`, clearable via `clearEmbeddingCache()`). `cosineSimilarity` assumes L2-normalized inputs (dot product). The same embedder + `semanticRetrievalEnabled()` are reused by `apps/web/src/lib/consumer-memory-store.ts` for durable-fact recall.

Knowledge budgets (measured, apps/web layer). Uploaded/pasted/crawled sources are each sliced to `100_000` chars at ingest (`api/knowledge/{upload,paste,crawl}/route.ts`), `composeKnowledge` concatenates them under `KNOWLEDGE_MAX_CHARS` (default `80_000`, tenant docs first / catalogue template truncated to a $\leq 4,000$ -char excerpt), and `runTurn` then re-budgets to `RUNTIME_KNOWLEDGE_CHARS` (default `40_000`) before retrieval.

6.6 Guardrails enforcement

Two exported functions in `guardrails.ts`, applied to **both** mock and live paths (the mock also self-checks internally):

- `checkInputGuardrails(userMessage, system, tools, {consumerLine})` runs before the model and returns a forced `GuardrailResult` (a canned reply, optionally with a `handoff_to_human` tool call) on: prompt-injection / "reveal your prompt"; card/CVV/OTP/PIN solicitation; STOP/unsubscribe (→ `stop_suppression` handoff); financial/tax advice (→ `financial_advice` handoff); sanctions/AML "push it through" (→ `sanctions_aml` handoff); a broad **cross-tenant data-access** regex ("another patient/account/client...", "show me his results") returning a privacy refusal; clinical/medical-danger and physical/safety/security emergencies (→ emergency handoff quoting the agent's `emergencyNumber`); and GDPR/CCPA erasure (→ `gdpr_erasure` handoff). The cross-tenant check has a `familyRef` clause ("my daughter Aya")

that is **skipped on the consumer line** (`TurnRequest.consumerLine`) so first-party family references aren't misread as tenant probes — every other pattern still applies.

- `checkOutputGuardrails(userMessage, draft, tools)` scrubs the model's own draft before it ships: full PAN-like digit runs (13–19), `cvv/cvc: NNN`, `otp/pin/password` + digits, and a jailbreak-compliance dump (system-prompt leak > 400 chars). It **only blocks/replaces unsafe drafts; it never invents answers**.

In `runTurn`, `checkInputGuardrails` produces the `forced` completion that pre-empts the model; `checkOutputGuardrails` runs after the loop when `!completion.toolCall`. `findTool(tools, "handoff")` resolves the agent's actual handoff tool name, falling back to `handoff_to_human`.

6.7 Per-vertical workflow modules

`packages/runtime/src/workflows/` holds **22 files: 20 dispatched per-vertical modules** plus `i18n.ts` (the `wf()` localized reply-string table) and `stop-suppression.ts`. The 20 verticals (each exposes an `isX(agentId)` matcher + `runXWorkflow(ctx)`): `marketplace-assistant`, `executive-assistant`, `it-helpdesk`, `booking-front-desk`, `sales-qualifier`, `restaurant-takeaway`, `onboarding-buddy`, `dental-front-desk`, `hotel-guest`, `accounting-practice`, `events-venue`, `building-management`, `pharmacy`, `gym-membership`, `mobile-money`, `wealth-management`, `tax-office`, `veterinary`, `customer-support`, `delivery-tracking`. (This exceeds the "10+" claim.)

The Executive Assistant is the reference multi-step agent and defines the shared plan model: `WorkflowPlan { id, goal, steps: WorkflowStep[], status: "proposed"|"executing"|"completed"|"cancelled" }`, `WorkflowStep { id, label, tool?, args?, status: "pending"|"done"|"skipped"|"failed", resultSummary? }`. The cycle is **goal → plan → confirm → execute → verify**, and the pending plan is persisted across turns by embedding it as an HTML comment in the assistant message (`<!--miai-workflow:{...json}-->`), parsed back by `parseWorkflowFromMessages`.

`runTurn.finishWorkflow` lifts that plan into `TurnResult.workflow` (steps with statuses/summaries) for the UI, strips the marker comment, debits tokens, and streams the cleaned reply. Workflow turns bypass the generic tool loop entirely but share the same wallet metering and placeholder scrubbing.

6.8 Fail-closed metering and the wallet

Metering is enforced by `@miai/wallet-adapter` (`createWalletAdapter()`), selected by `MIAI_WALLET_MODE`: `mock` default `MockWalletAdapter`, or `http` `HttpWalletAdapter` against `MIAI_WALLET_API_URL` with a 15s abort). The `WalletAdapter` contract is `getBalance` / `debit` / `topUp`; `debit` is idempotent on `idempotencyKey` and returns `{ ok, balance, paused }` where `paused = balance ≤ 0`.

The fail-closed guarantee is structural in `runTurn`:

1. `state ≡ "paused_no_tokens"` → return before any work.
2. `bal = getBalance(...)` read **before** the first model call; `bal.tokens ≤ 0` (and not `skipDebit`) → return `paused_no_tokens` with **zero provider calls**.
3. All model + tool-round usage is metered once at the end; a failed/ `paused` debit flips `TurnResult.state` to `"paused_no_tokens"` and sets `paused: true`.

`skipDebit` (free sandbox try before rent) still assembles the prompt and can call the provider but reports `tokensDebited: 0`. Top-ups are prepaid packages (`TopUpPackageId "5"|"10"|"20"|"50"|"100"|"200"` → `TOPUP_TOKENS`), funded via Paystack in production and idempotent on the payment reference.

6.9 Notable observations for handover

- **The RAG engine post-dates the last internal audit.** `docs/reports/technical-audit-2026-08-02.md` states "a grep for `embeddings/vector/cosine/pgvector` returns zero hits" and that knowledge is stuff-the-context only. That is now stale:

`embeddings.ts` (`cosineSimilarity`, `LocalHashEmbedder`, `OpenAiCompatibleEmbedder`) and hybrid retrieval were added afterward. Due-diligence readers should treat the RAG section of that older audit as superseded by the current code described above.

- **Retrieval quality is still capped by the local embedder** unless a real embedding key is configured — `LocalHashEmbedder` is a hashing-trick approximation ("Not a substitute for neural embeddings in production," per its own docstring). Semantic quality in prod depends on `EMBEDDING_API_KEY` / `OPENAI_API_KEY` being set with `RUNTIME_SEMANTIC_RETRIEVAL=auto`.
- **Evals are carried but not executed by the runtime.** `AgentPackage.evals` (~15/agent) rides in the package; `runTurn` never runs them. Eval execution lives outside `packages/runtime` (the eval suite / `pnpm eval:*` scripts).
- **No per-round credit re-check.** A turn that maxes out its 3 tool rounds (each a provider call) is admitted on the single up-front balance read and settled once at the end; a turn cannot be halted mid-loop for balance, though the $\leq 3 / \leq 5$ cap bounds the spend.

07 API Surface

This section is an exhaustive reference to the platform's HTTP API. Every endpoint lives under `apps/web/src/app/api` as a Next.js 15 App Router `route.ts` file. Re-measured at read time:

```
$ find apps/web/src/app/api -name route.ts | wc -l → 69 (route files / endpoints)
$ find apps/web/src/app -name page.tsx | wc -l → 39 (rendered pages)
```

The 69 route files back the whole product: the B2B Agents dashboard, the consumer ("personal AI") line, the embeddable widget/channels, the operator console, billing, compliance, and the integration/webhook/MCP sinks. Several files export multiple HTTP methods, so the endpoint count (verbs) is higher than 69. The catalogue endpoints (`/api/catalog*`) are thin read projections over the platform's largest asset — the 500-SKU agent catalogue: `data/catalog` (552 `*.agent.json` files, incl. legacy ZA aliases) + `data/catalog-consumer` (17 consumer specialists) = **299,508 lines of catalogue JSON** (re-measured). Crown-jewel fields (`system_prompt`, `guardrails`, `evals`) are stripped before any public read — see `/api/agents/[id]` and `/api/catalog/family/[familyId]`.

7.1 Authentication model

Auth is resolved per-route (no framework-wide handler); routes call one of a small set of guards. The mechanisms:

AUTH CLASS	GUARD / SOURCE	HOW THE CALLER PROVES IDENTITY
Bearer + workspace (B2B)	<code>requireAuth</code> → <code>resolveAuth</code> (<code>lib/request-auth.ts</code> , <code>lib/auth.ts</code>)	<code>MIAI_AUTH_MODE=oidc</code> : <code>Authorization: Bearer <JWT></code> verified via <code>jose</code> against <code>MIAI_OIDC_ISSUER</code> / <code>_AUDIENCE</code> / <code>_JWKS_URL</code> ; must carry <code>workspace_id</code> / <code>org_id</code> claim. <code>mode=mock</code> : workspace/user/roles from <code>x-workspace-id</code> / <code>x-user-id</code> / <code>x-roles</code> headers or <code>?workspaceId=</code> , defaulting to <code>WORKSPACE_ID</code> + elevated <code>["owner", "operator"]</code> outside prod.
Workspace role gate	<code>requireRole(auth, min)</code> / <code>requireOperator(auth)</code> (<code>lib/security.ts</code>)	Ranked roles <code>readonly < agent < admin < owner</code> ; platform <code>operator</code> / <code>PLATFORM_ROLES</code> above all. Returns 403 when below <code>min</code> .
Consumer session	<code>requireConsumer</code> → <code>resolveConsumerAuth</code> (<code>lib/consumer-auth.ts</code> , <code>consumer-identity.ts</code>)	Signed session cookie from "Sign in with Google" (<code>oidc</code>); shared demo identity (<code>mock</code>). Keys the consumer wallet/memory.
Embed key	<code>runChannelTurn</code> via <code>body.key</code> (<code>lib/handlers/embed-chat.ts</code> , <code>channel-turn.ts</code>)	Publishable <code>mia_pk_...</code> key in the JSON body identifies + scopes the tenant agent; CORS-gated (<code>EMBED_ALLOWED_ORIGINS</code>), rate-limited.

AUTH CLASS	GUARD / SOURCE	HOW THE CALLER PROVES IDENTITY
Shared secret / HMAC	per-route	Paystack webhook = HMAC-SHA512; connector <code>webhook/sink</code> = HMAC-v1 (<code>x-miai-signature</code>); MCP sink = <code>MCP_SINK_TOKEN</code> Bearer; cron = <code>CRON_SECRET</code> ; Telegram = <code>TELEGRAM_BOT_SECRET</code> header (mandatory).
Signed state	<code>oauth/callback</code>	Provider redirect carrying signed OAuth <code>state</code> (+ consumer PKCE/nonce cookie).
Public	none in-route	Unauthenticated by design.

Edge pre-gate. `apps/web/src/middleware.ts` applies a body-size cap (`MIAI_MAX_BODY_BYTES` , default 1 MiB → 413) and a per-request CSP nonce to every route. When `MIAI_AUTH_MODE=oidc` , it 401s any `/api/*` request lacking a Bearer **unless** the path is in the allowlist `lib/public-paths.ts` (single source shared with `lib/auth.ts`). "Public to the OIDC gate" therefore means: no Bearer required, but the route still enforces its own mechanism (session/HMAC/cron/etc.). In `mock` mode the edge gate is off and `requireAuth` never fails, so Bearer routes are effectively open with elevated default roles — a deployment-config concern flagged for the handover team.

The tables below use the auth classes above. "role X" means `requireRole(auth,"X")` ; "operator" means `requireOperator` .

7.2 Catalogue & Agents

PATH	METHOD	AUTH	PURPOSE
<code>/api/catalog</code>	GET	Public	Marketplace index. <code>?view=families agents</code> , filters <code>q,market,category,audience,workflow,pilot</code> . Reports <code>totalAgents=INDEXED_AGENT_COUNT</code> (100×5=500); ZA aliases not counted as a 6th market.
<code>/api/catalog/personal</code>	GET	Public	Personal/consumer agent catalogue (separate from the business index).
<code>/api/catalog/family/[familyId]</code>	GET	Public	Customer-facing capability brief for a family ("Learn more" modal), <code>?market=</code> . IP-safe projection.
<code>/api/agents/[id]</code>	GET	Bearer (any role)	One rented agent for the studio; crown-jewel IP (<code>system_prompt</code> / <code>guardrails</code> / <code>evals</code>) redacted from the client payload.
<code>/api/rentals</code>	GET	Bearer (any role)	List the caller workspace's rented agents.
<code>/api/rent</code>	POST	Bearer · admin	Rent an agent → create rental at tier (<code>standard pro enterprise</code>); audits price from <code>TIER_PRICES</code> . Body: <code>rentBodySchema</code> .
<code>/api/chat</code>	POST	Bearer · agent	Studio agent chat (<code>studioChatBodySchema</code> , <code>mode: sandbox live</code>); soft-creates entitlement; metered.

PATH	METHOD	AUTH	PURPOSE
<code>/api/configure</code>	POST	Bearer · agent	Configure a rented agent: <code>model</code> , <code>knowledge</code> , <code>bindings</code> , <code>connectedConnectors</code> , <code>markRented</code> (<code>configureBodySchema</code>).
<code>/api/proof/tool</code>	POST	Bearer + proof-harness gate	Scripted tool execution that bypasses the LLM (<code>assertProofHarness</code>); Wave-4 live-tool proof.

7.3 Knowledge

PATH	METHOD	AUTH	PURPOSE
<code>/api/knowledge</code>	GET	Bearer (any role)	List an agent's knowledge sources (<code>?agentId=</code>).
<code>/api/knowledge/[id]</code>	DELETE	Bearer · agent	Delete one knowledge source.
<code>/api/knowledge/upload</code>	POST	Bearer · agent	Upload a file (≤ 4 MiB) as knowledge.
<code>/api/knowledge/paste</code>	POST	Bearer · agent	Paste text (10–100 000 chars) as knowledge (<code>knowledgePasteBodySchema</code>).
<code>/api/knowledge/crawl</code>	POST	Bearer · agent	Crawl a URL (<code>maxPages</code> ≤ 8) into knowledge (<code>knowledgeCrawlBodySchema</code>).

7.4 Connectors, OAuth & Integration sinks

PATH	METHOD	AUTH	PURPOSE
<code>/api/connectors</code>	GET	Public in-route (edge Bearer in oidc)	List the connector catalogue; optional <code>?phase=1 2</code> .
<code>/api/connectors</code>	POST	Bearer · agent	Attach a connector to an agent (<code>connectorsBodySchema</code>).
<code>/api/connectors/credentials</code>	POST	Bearer · admin	Store API-key / webhook / MCP credentials for non-OAuth connectors (<code>connectorCredentialsBodySchema</code> , <code>remapTools</code>).
<code>/api/oauth/[connector]/start</code>	GET	Bearer · agent	Begin OAuth2/PKCE connect for a connector on an agent.
<code>/api/oauth/callback</code>	GET	Public (signed <code>state</code>)	Provider redirect target; seals the token to the workspace/agent in the signed state.

PATH	METHOD	AUTH	PURPOSE
<code>/api/oauth/status</code>	GET	Bearer (any role)	Per-agent connector connection status.
<code>/api/oauth/[connector]/disconnect</code>	POST	Bearer · admin	Revoke/disconnect a stored OAuth token (<code>oauthDisconnectBodySchema</code>).
<code>/api/oauth/[connector]/test</code>	POST	Bearer (any role)	Read-only live probe that the stored OAuth token still works with the vendor (zero LLM).
<code>/api/slack/channels</code>	GET	Bearer (any role)	List channels the connected Slack workspace exposes + saved default.
<code>/api/slack/channels</code>	POST	Bearer (any role)	Save the workspace's default Slack handoff channel (<code>slackChannelsBodySchema</code>).
<code>/api/webhook/sink</code>	POST	Shared secret (HMAC-v1; legacy raw unless <code>WEBHOOK_SINK_HMAC_ONLY</code>)	Connector webhook proof-sink; verifies signature, records + echoes the event.
<code>/api/webhook/sink</code>	GET	Shared secret (<code>WEBHOOK_SINK_SECRET</code> in prod)	Inspect recent sink events.

7.5 Consumer line (personal AI)

All data routes call `requireConsumer` (session cookie / mock). The whole `/api/consumer/*` prefix is public to the OIDC Bearer gate; each route enforces the session in-route (webhook + cron carry their own secrets).

PATH	METHOD	AUTH	PURPOSE
<code>/api/consumer/chat</code>	POST	Consumer session	Personal-agent chat metered to the consumer's own wallet; SSE by default, JSON one-shot on <code>Accept: application/json</code> . First-party only (no embed key/CORS).
<code>/api/consumer/wallet</code>	GET	Consumer session	Consumer prepaid balance (shared wallet adapter).
<code>/api/consumer/brief</code>	GET / PUT	Consumer session	GET schedule + latest daily brief; PUT update schedule (<code>briefConfigSchema</code> : enabled/hour/timezone/channel).
<code>/api/consumer/brief/run</code>	POST	Consumer session	Generate the consumer's brief now ("brief me now").
<code>/api/consumer/brief/run-due</code>	POST	Shared secret (<code>CRON_SECRET</code>)	Cron sweep — generate briefs for every consumer whose schedule is due. Fails closed if secret unset.

PATH	METHOD	AUTH	PURPOSE
<code>/api/consumer/connectors</code>	GET	Consumer session	Linked connectors + which the consumer's agents still need.
<code>/api/consumer/connectors/[connector]/start</code>	GET	Consumer session	Begin an OAuth connect for the signed-in consumer.
<code>/api/consumer/connectors/[connector]/disconnect</code>	POST	Consumer session	Disconnect a consumer connector.
<code>/api/consumer/reminders</code>	GET	Consumer session	Pending reminders (soonest first), scoped to brand + account.
<code>/api/consumer/reminders/dismiss</code>	POST	Consumer session	Dismiss a reminder (one-off cleared; recurring rolls forward).
<code>/api/consumer/telegram/webhook</code>	POST	Shared secret (<code>TELEGRAM_BOT_SECRET</code> , mandatory)	Inbound Telegram → consumer turn; each chat auto-provisions an isolated <code>telegram:<chat_id></code> identity; private chats only; rate-limited per chat.

7.6 Auth & Identity

PATH	METHOD	AUTH	PURPOSE
<code>/api/consumer/auth/login</code>	GET	Public	Start "Sign in with Google": PKCE + CSRF <code>state</code> + <code>nonce</code> in a short-lived signed cookie, redirect to IdP. 404 when consumer OIDC unconfigured.
<code>/api/consumer/auth/callback</code>	GET	Public (CSRF <code>state</code> validated)	OIDC redirect target; verifies id_token/nonce, sets signed session cookie; failure → <code>?auth_error=1</code> .
<code>/api/consumer/auth/logout</code>	GET / POST	Public	Clear the consumer session cookie; redirect to a re-validated same-origin path (<code>safeReturnPath</code>).
<code>/api/consumer/auth/me</code>	GET	Public	Who is signed in (mock = always signed in; oidc reflects the cookie).
<code>/api/auth/handoff</code>	GET	Public	B2B Agents auth descriptor: <code>mode</code> (mock/oidc), external MIAI <code>loginUrl</code> , <code>getStartedPath</code> , <code>requiresExternalLogin</code> .

7.7 Billing / Wallet / Payments

PATH	METHOD	AUTH	PURPOSE
<code>/api/wallet</code>	GET	Bearer · readonly	Workspace prepaid token balance.
<code>/api/wallet</code>	POST	Bearer · admin	Mock-rail top-up (credits tokens without payment). Returns 403 unless <code>mockRailsAllowed()</code> — live rails must use Paystack.
<code>/api/payments/paystack/init</code>	POST	Bearer (workspace scope → admin ; consumer scope → any signed-in)	Start a Paystack hosted checkout for a fixed-price package (<code>paystackInitBodySchema</code>); returns <code>authorization_url</code> . 503 in sandbox / when <code>PAYSTACK_SECRET_KEY</code> unset. Wallet credited later, never here.
<code>/api/payments/paystack/return</code>	GET	Public	Browser return; verifies the reference with Paystack and credits the wallet named in verified metadata (idempotent); redirects <code>?topup=...</code> .
<code>/api/payments/paystack/webhook</code>	POST	Shared secret (HMAC-SHA512)	Source-of-truth crediting on <code>charge.success</code> for wallet references; idempotent by reference; 500 to force Paystack retry on error.

7.8 Embed & public v1 API

PATH	METHOD	AUTH	PURPOSE
<code>/api/embed/chat</code>	OPTIONS / POST	Embed key + CORS + rate-limit (30/min)	Website-widget chat turn (<code>channelChatBodySchema</code> , <code>channel: "embed"</code>). Cross-origin by design.
<code>/api/v1/embed/chat</code>	OPTIONS / POST	Embed key	Versioned alias — same <code>handleEmbedChat*</code> handler.
<code>/api/app/chat</code>	OPTIONS / POST	Embed key + CORS + rate-limit (30/min)	In-app channel chat; SSE token deltas by default, JSON one-shot on <code>Accept: application/json</code> .
<code>/api/embed/sri</code>	GET	Public	Subresource-Integrity hash for the published <code>agent.js</code> widget.
<code>/api/v1/rent</code>	POST	Bearer · admin	Versioned rent — re-exports <code>POST</code> from <code>/api/rent</code> .
<code>/api/v1/openapi</code>	GET	Public	OpenAPI 3.0.3 contract for the versioned surface (<code>servers: /api/v1</code>).

Public v1 contract (`/api/v1/openapi`). The spec formally documents two operations: `POST /embed/chat` (`operationId: embedChat` , body `{key,message,sessionId?,replyLanguage?,correlationId?}` → `{reply,paused,balance,correlationId}` ; 400/429) and `POST /rent` (`operationId: rentAgent` , `security: bearerAuth` , body `{agentId,tier?,workspaceId?}` ; 200/400/401/404). Security scheme `bearerAuth` = HTTP bearer, JWT. The description notes legacy unversioned routes remain supported.

7.9 MCP surface

The MCP surface is a **minimal HTTP tool-call sink/bridge** ("Wave-4 proof"), not a full JSON-RPC MCP server — there is no `tools/list` and no catalogue exposure. It records and echoes accepted tool calls so an external Actions→MCP configuration can be proven live.

PATH	METHOD	AUTH	PURPOSE
<code>/api/mcp</code>	GET	Shared secret (<code>MCP_SINK_TOKEN</code> required in prod; Bearer or <code>?token=</code>)	Health + inspect the last N recorded MCP tool calls.
<code>/api/mcp/tools/call</code>	POST	Shared secret (<code>MCP_SINK_TOKEN</code> required in prod)	Record a tool call (<code>{name?, arguments?}</code> , <code>mcpToolsCallBodySchema</code>) and return <code>{ok, live, provider: "mcp_sink", callId, result}</code> . Capped at 50, persisted to <code>mcp-sink.json</code> .

The **real** connector/tool catalogue (Google Cal/Gmail, M365, Slack, Shopify, HubSpot, Xero, QuickBooks, Calendly, Zendesk, Teams, webhook, MCP action) lives in `packages/connectors` and is executed through `runTurn` / `executeConnector` , not exposed as an HTTP tool-listing endpoint. `/api/connectors` (§7.4) is the closest thing to a machine-readable tool inventory.

7.10 Workspace & Admin

PATH	METHOD	AUTH	PURPOSE
<code>/api/workspace/members</code>	GET	Bearer · readonly	List workspace members.
<code>/api/workspace/members</code>	POST	Bearer · admin	Invite a member (<code>workspaceMemberInviteBodySchema</code> : email + role); audits, returns 201.
<code>/api/workspace/members/[userId]</code>	PATCH	Bearer · admin	Change a member's role (<code>workspaceMemberRoleBodySchema</code>).
<code>/api/workspace/members/[userId]</code>	DELETE	Bearer · admin	Remove a member.
<code>/api/onboarding</code>	GET / POST / PATCH	Bearer (any role)	GET onboarding + auth summary; POST complete business wizard (<code>onboardingCompleteBodySchema</code> : company/market/industry/size/intent); PATCH checklist progress.
<code>/api/custom-requests</code>	GET	Bearer · operator	List all custom agent requests (operator-owned pipeline; partners see none).

PATH	METHOD	AUTH	PURPOSE
<code>/api/custom-requests</code>	POST	Bearer · agent	Submit a custom agent request (<code>customRequestBodySchema</code>).
<code>/api/custom-requests/[id]</code>	PATCH	Bearer · operator	Update request status (<code>customRequestStatusBodySchema</code>).
<code>/api/admin</code>	GET	Bearer · operator	MAI / Move Digital operator overview (live marketplace aggregation); <code>?narrative=1</code> for pitch-scale figures.
<code>/api/ops</code>	GET	Bearer · admin	This workspace's wallet balance + audit + <code>opsSummary</code> . Cross-tenant <code>?workspaceId=</code> only for a platform operator in mock mode.
<code>/api/audit</code>	GET	Bearer · readonly (operator for <code>?all=1</code>)	Workspace-scoped audit trail; filters <code>type,agentId,correlationId,limit</code> .
<code>/api/insights</code>	GET	Bearer (any role)	Workspace insights + wallet tokens.
<code>/api/ask/leads</code>	GET	Bearer · operator	Leads captured by the marketplace "Ask AI" assistant.

7.11 Ask (marketplace assistant), Ops, Health, History

PATH	METHOD	AUTH	PURPOSE
<code>/api/ask/chat</code>	POST	Public (anon session, rate-limit 40/min)	Marketplace "Ask AI" pre-sales assistant turn (<code>askChatBodySchema</code>); can capture <code>leadIds</code> .
<code>/api/health</code>	GET	Public	Boot-hardening + auth/wallet/model modes, DB backend, store ping, telemetry. Returns 503 when hardening fails.
<code>/api/version</code>	GET	Public	Git commit/branch/deployment probe from Railway env; confirms which build is running.
<code>/api/history/turns</code>	GET	Bearer · readonly (operator for cross-tenant)	Conversation turn transcripts for History / traceability.
<code>/api/history/trace/[correlationId]</code>	GET	Bearer · readonly	Full trace for one correlation id.

7.12 Privacy / DSAR / Consent

PATH	METHOD	AUTH	PURPOSE
<code>/api/consent</code>	POST	Public (workspace from auth when present)	Record cookie/consent choice (<code>consentBodySchema</code> : <code>accepted essential</code>) into the audit log.
<code>/api/dsar/export</code>	GET	Bearer · admin	DSAR / data-subject export for the caller's workspace; never includes OAuth access/refresh tokens.
<code>/api/dsar/erase</code>	POST	Bearer · admin	DSAR erasure for the caller's workspace; requires explicit <code>{ confirm: true }</code> .

7.13 Cross-cutting API behaviour

- **Validation.** JSON bodies are parsed with zod schemas centralised in `apps/web/src/lib/api-schemas.ts` (`parseJsonBody` → 400 with `formatZodError`). Field caps are explicit (e.g. message ≤ 8 000 chars, knowledge paste ≤ 100 000, crawl ≤ 8 pages, top-up ≤ \$10 000).
- **Errors / envelope.** Consistent JSON via `lib/api-error.ts` (`apiOk` / `apiErrorFromRequest`); auth failures surface as 401/403, oversized bodies 413 (middleware), unconfigured integrations 503.
- **Streaming.** `chat`, `app/chat`, and `consumer/chat` default to SSE (`lib/sse.ts`, `chat-stream.ts`) with live model token deltas; `Accept: application/json` forces a one-shot reply.
- **Rate limiting.** `rateLimit` (`lib/security.ts`) keys per embed key / session / consumer+agent (30–40 req/min windows) on the public chat surfaces.
- **Metering (fail-closed).** All chat turns run through the wallet-gated `runTurn`; at zero balance a turn returns a top-up message and makes **no** provider call (`packages/runtime`, wallet adapter).
- **Tenancy.** Bearer routes pin the workspace to the verified token in oidc mode; `?workspaceId=` overrides are only honoured for a platform operator in mock mode (see `/api/ops`, `/api/audit`, `/api/insights`, `/api/wallet`).

7.14 Endpoint census

DOMAIN	ROUTE FILES
Catalogue & Agents	9
Knowledge	5
Connectors, OAuth & sinks	9
Consumer line	11
Auth & Identity	5
Billing / Wallet / Payments	4
Embed & public v1	6
Ask	2

DOMAIN	ROUTE FILES
MCP	2
Workspace & Admin	9
Ops / Health / History	4
Privacy / DSAR	3
Total	69

08 Data Architecture

This platform keeps its data in three physically distinct tiers, each with a different durability and ownership model:

1. **The catalogue IP** — 552 read-only `*.agent.json` files on disk (`data/catalog` + `data/catalog-consumer`), version-controlled, never written at runtime. At **299,508 lines** (independently re-measured with `find ... | wc -l`) this is **73.7% of the 406,319 authored lines** and the single largest, most valuable asset in the repository — the "data model" of MyInstantAF is first and foremost this file corpus, not the SQL schema.
2. **Operational + memory state** — a **dual-mode store**: Postgres (via `pg`, no ORM) when `DATABASE_URL` / `MIAI_DATABASE_URL` is set, otherwise JSON files on a `/data` volume, with an in-process fallback for CI. 13 migration-defined tables + a runtime bookkeeping table.
3. **Ephemeral session memory** — rolling chat history in Upstash Redis (REST) when configured, else a capped in-process `Map`. Not durable, not part of the SQL schema.

ADR 0003 (`docs/adr/0003-postgres-persistence.md`, *Accepted 2026-08-02*) is the governing decision; it explicitly **supersedes the July brief's SQLite** and mandates Postgres-primary with file fallback and forward-only, boot-applied migrations.

8.1 Backend selection and connection management

`apps/web/src/lib/pg.ts` is the single Postgres endpoint. Selection is purely presence-of-URL:

```
export function databaseUrl(): string | undefined {
  const url = process.env.DATABASE_URL || process.env.MIAI_DATABASE_URL;
  return url?.trim() || undefined;
}
```

- **Pool** — a singleton `pg.Pool` (`max: 10`) memoised on `globalThis.__miaiPgPool` to survive Next.js HMR. `getPool()` returns `null` when no URL is set, and every store consults `getPool()` to branch Postgres-vs-file. ADR 0003 flags the process-singleton pool as needing "separate review" for serverless cold starts.
- **TLS** — `sslFor(url)` disables SSL for `localhost` / `127.0.0.1` and otherwise verifies the server certificate (`rejectUnauthorized: true`). `PG_SSL_REJECT_UNAUTHORIZED=0` opts out for managed CAs (Railway/Azure) that need a custom trust store — gated behind a boot ACK in production.
- **Readiness** — `pingPool()` runs `SELECT 1`; `store.pingStore()` surfaces `{ backend: "postgres" | "file", ok, error }` to `/api/health`, so operators can see which tier is live and whether Postgres answers.

8.2 Migrations

`apps/web/src/lib/migrate.ts` applies an **ordered, forward-only** list on boot, but only when a pool exists (`if (!pool) return`), so CI/file mode needs no database container:

```
const MIGRATIONS = [
  { id: "001_init", file: "001_init.sql" },
  { id: "002_consumer_brief", file: "002_consumer_brief.sql" },
  { id: "003_consumer_memory", file: "003_consumer_memory.sql" },
  { id: "004_memory_tenant_and_graph", file: "004_memory_tenant_and_graph.sql" },
  { id: "005_consumer_reminders", file: "005_consumer_reminders.sql" },
];
```

Mechanics:

- Applied ids are recorded in a runtime-created `miai_schema_migrations (id PK, applied_at)` table; already-applied ids are skipped — **idempotent**, guarded by a module-level `ensuring` promise so concurrent requests apply once.
- Each file is split by `splitStatements()`, a **naive ;-splitter** that strips `--` line comments and runs each fragment separately. It cannot handle semicolons inside string literals or `DO $$ ... $$` /PL-pgSQL bodies — safe for the current pure-DDL migrations, a landmine for future function/trigger migrations (see gaps).
- File resolution tries `cwd/migrations`, `cwd/apps/web/migrations`, and a path relative to the compiled module, so it works whether cwd is the monorepo root or `apps/web`.
- No down-migrations / rollback path exists.** Migration `004` is a live retrofit (`ALTER TABLE ... ADD COLUMN ... DEFAULT 'demo-workspace'` + `DROP INDEX` / `CREATE UNIQUE INDEX`) that back-fills the tenant column onto the pre-tenant `miai_consumer_memory` table.

8.3 Full relational schema

Thirteen migration tables plus the runtime `miai_schema_migrations`. All defined as `CREATE TABLE IF NOT EXISTS . TS = TIMESTAMPTZ`; every `TS` audit/updated column defaults `NOW()`.

001_init.sql — B2B core + audit + stubs (keyed by `workspace_id`)

TABLE	KEY COLUMNS	PK	INDEXES	ISOLATION
<code>miai_rents</code>	<code>workspace_id</code> , <code>agent_id</code> , <code>payload JSONB</code> , <code>updated_at</code>	<code>(workspace_id, agent_id)</code>	— (scan by PK prefix)	<code>WHERE workspace_id = \$1</code> on every read
<code>miai_audit</code>	<code>id</code> , <code>at</code> , <code>workspace_id</code> , <code>agent_id?</code> , <code>type</code> , <code>detail JSONB</code> , <code>'{}'</code>	<code>id</code>	<code>miai_audit_ws_at (workspace_id, at DESC)</code>	append-only; redact rewrites <code>detail</code> only
<code>miai_turns</code>	<code>id</code> , <code>at</code> , <code>correlation_id</code> , <code>workspace_id</code> , <code>agent_id</code> , <code>channel</code> , <code>session_id</code> , <code>user_id?</code> , <code>payload JSONB</code>	<code>id</code>	<code>miai_turns_ws_at (workspace_id, at DESC)</code> , <code>miai_turns_corr (correlation_id)</code>	per-workspace transcript log
<code>miai_oauth_tokens</code>	<code>workspace_id</code> , <code>connector</code> , <code>sealed JSONB</code> , <code>updated_at</code>	<code>(workspace_id, connector)</code>	—	sealed connector tokens (see 8.6)

TABLE	KEY COLUMNS	PK	INDEXES	ISOLATION
miai_knowledge_sources	id, workspace_id, agent_id, payload JSONB, updated_at	id	—	per-(workspace, agent) KB rows
miai_workspace_members	workspace_id, user_id, payload JSONB, updated_at	(workspace_id, user_id)	—	RBAC membership
miai_ask_leads	id, at, payload JSONB	id	—	inbound "ask" leads
miai_custom_requests	id, payload JSONB, updated_at	id	—	bespoke-agent requests

`miai_rentals.payload` is the **workspace's instance of a catalogue agent** (config, `publicKey` for embed, overrides) – the catalogue file itself is never copied into the DB. `miai_turns` matches `traceability.ts`; `miai_audit` / `miai_turns` are append-only (audit history is retained in Postgres; the in-process mirror is capped by `AUDIT_CAP`).

002 / 003 / 004 / 005 – consumer line (personal-assistant memory)

TABLE (MIGRATION)	KEY COLUMNS	PK	UNIQUE / OTHER INDEXES	OWNER KEY
miai_consumer_brief (002)	consumer_id, config JSONB, '{}' , latest JSONB?, last_sent_on TEXT?, updated_at	consumer_id	—	consumer_id only – NOT tenant-scoped
miai_consumer_memory (003, + tenant_id in 004)	tenant_id 'demo-workspace', consumer_id, id, category 'general', content, content_key, source 'assistant', created_at, updated_at	(consumer_id, id)	idx_consumer_memory_dedupe UNIQUE (tenant_id, consumer_id, content_key), idx_consumer_memory_recent (tenant_id, consumer_id, updated_at DESC)	(tenant_id, consumer_id)
miai_consumer_goal (004)	tenant_id, consumer_id, id, title, title_key, detail, target, progress INTEGER 0, deadline, status 'active', created_at, updated_at	(consumer_id, id)	idx_consumer_goal_dedupe UNIQUE (tenant_id, consumer_id, title_key), idx_consumer_goal_recent (tenant_id, consumer_id, updated_at DESC)	(tenant_id, consumer_id)

TABLE (MIGRATION)	KEY COLUMNS	PK	UNIQUE / OTHER INDEXES	OWNER KEY
<code>miai_consumer_person</code> (004)	<code>tenant_id</code> , <code>consumer_id</code> , <code>id</code> , <code>name</code> , <code>name_key</code> , <code>relationship</code> , <code>notes</code> , <code>created_at</code> , <code>updated_at</code>	(<code>consumer_id</code> , <code>id</code>)	<code>idx_consumer_person_dedupe</code> UNIQUE (<code>tenant_id</code> , <code>consumer_id</code> , <code>name_key</code>), <code>idx_consumer_person_recent</code> (..., <code>updated_at</code> DESC)	(<code>tenant_id</code> , <code>consumer_id</code>)
<code>miai_consumer_reminder</code> (005)	<code>tenant_id</code> , <code>consumer_id</code> , <code>id</code> , <code>text</code> , <code>fires_at</code> TS, <code>recurring</code> , <code>channel</code> 'app', <code>status</code> 'pending', <code>created_at</code>	(<code>consumer_id</code> , <code>id</code>)	<code>idx_consumer_reminder_owner</code> (<code>tenant_id</code> , <code>consumer_id</code> , <code>status</code> , <code>fires_at</code>), <code>idx_consumer_reminder_due</code> (<code>status</code> , <code>fires_at</code>)	(<code>tenant_id</code> , <code>consumer_id</code>)

Plus runtime-only `miai_schema_migrations` (`id` PK, `applied_at`).

8.4 Tenancy and the isolation predicate

Two distinct isolation models coexist:

- **B2B (workspace) tier** – every `001_init` table is scoped by `workspace_id`, which sits in the PK (`miai_rentals`, `miai_oauth_tokens`, `miai_workspace_members`) or is the leading index column (`miai_audit`, `miai_turns`). Reads filter `WHERE workspace_id = $1` (e.g. `store.ts` `readAgentsFromPostgres`, `readAuditFromPostgres`).
- **B2B2C (consumer) tier** – migration `004`'s header states the rule verbatim: *"the same person's account id ... can exist under more than one brand, so the memory OWNER is (`tenant_id`, `consumer_id`) – never `consumer_id` alone."* `tenant_id` is the brand/carrier (Vodafone/MTN/..., default `'demo-workspace'`), `consumer_id` is the person (e.g. a phone number). Enforcement is threefold and defence-in-depth:
 1. `consumer-memory-store.ts` exports the `MemoryOwner = { tenantId; consumerId }` type reused by every consumer store, and `validOwner()` makes any op with a missing half a **no-op**.
 2. Every SQL statement carries **both** predicates – `WHERE tenant_id = $1 AND consumer_id = $2` on read/delete, and `ON CONFLICT (tenant_id, consumer_id, content_key/title_key/name_key)` on upsert, so dedupe and refresh are per-owner.
 3. The file-store fallback buckets rows under `ownerFileKey(owner) = \ ${tenantId}::${consumerId}\`.`

```
// consumer-memory-store.ts – every memory read is double-scoped
`SELECT id, category, content, source, created_at, updated_at
FROM miai_consumer_memory WHERE tenant_id = $1 AND consumer_id = $2 ORDER BY updated_at DESC`
```

Two structural asymmetries worth handover attention (both surfaced as gaps): the consumer tables' **primary key is (`consumer_id`, `id`)**, not (`tenant_id`, `consumer_id`, `id`) – cross-tenant isolation rests on the query predicates and the UUID `id`, not on the PK; and `miai_consumer_brief` is **keyed by `consumer_id` alone with no `tenant_id` column at all**, so the daily-brief config and last-sent brief are *not* brand-isolated the way facts/goals/people/reminders are.

8.5 Consumer memory layers

The personal assistant composes memory from five layers. `consumer-turn.ts` assembles them per turn – pulling session history from the session store and concurrently `getComposedKnowledge` + `getMemoryContext` + `getGoalsContext` +

`getPeopleContext`, injecting them into the system prompt, then persisting tool-emitted facts/reminders after the turn:

LAYER	PURPOSE	POSTGRES TABLE	FILE FALLBACK (DEFAULT)	OWNER
Channel sessions	rolling chat history (last 24 turns, 24 h TTL)	(none — <i>Redis/in-proc</i>)	in-process <code>Map</code> , 500-session cap	<code>walletId::agentId::sessionId</code>
Durable facts	<code>remember_about_me</code> — facts injected "across conversations"	<code>miai_consumer_memory</code>	<code>data/consumer-memory.json</code>	(tenant, consumer)
Life-graph: goals	tracked objectives (progress/deadline/status)	<code>miai_consumer_goal</code>	<code>data/consumer-goals.json</code>	(tenant, consumer)
Life-graph: people	who "my wife"/"my manager" is	<code>miai_consumer_person</code>	<code>data/consumer-people.json</code>	(tenant, consumer)
Knowledge base	per-agent KB composed into the prompt	<code>miai_knowledge_sources</code>	<code>data/knowledge-sources.json</code>	(workspace, agent)
Daily brief	brief schedule + last generated brief	<code>miai_consumer_brief</code>	<code>data/consumer-brief.json</code>	<code>consumer_id</code> only
Reminders	timed nudges (<code>set_reminder</code>)	<code>miai_consumer_reminder</code>	<code>data/consumer-reminders.json</code>	(tenant, consumer)

Retrieval of durable facts is **hybrid**: `getMemoryContext()` ranks by embedding cosine similarity when `semanticRetrievalEnabled()` and an env-configured embedder are present (`@miai/runtime createEmbedderFromEnv / cosineSimilarity`), falling back to keyword relevance (`buildMemoryBlock / rankKeyword`) otherwise. The injected block is bounded: ≤ 30 items, $\leq 1,600$ chars, per-item content capped at 500 chars. All consumer reads are wrapped to **never throw** — a memory failure degrades the block to `""` rather than breaking the chat turn.

Channel sessions (`channel-sessions.ts`) are deliberately *not* durable: `createSessionStore()` uses Upstash Redis REST (`redisAvailable()` → `UPSTASH_REDIS_REST_URL` + `_TOKEN`) with a 24 h TTL, else an in-process `Map` capped at `DEFAULT_MAX_SESSIONS = 500` with FIFO eviction, keeping the last `MAX_TURNS_KEPT = 24` messages. Without Redis, session history is per-replica and lost on restart. (The durable per-turn record lives separately in `miai_turns`.)

8.6 Connector tokens at rest

OAuth connector tokens are the one materially sensitive payload in the SQL tier.

`packages/connectors/src/oauth/tokens.ts` seals them with **AES-256-GCM** (`createCipheriv("aes-256-gcm", ...)`, `v2.` envelope; legacy `v1.` HMAC and plaintext still accepted for migration) and persists the sealed blob as `miai_oauth_tokens.sealed JSONB`, keyed (`workspace_id, connector`). In file mode they go to `OAUTH_TOKEN_STORE_PATH`. DSAR erase bulk-deletes these rows per workspace.

8.7 JSON file-store fallback and the `/data` volume

When `getPool()` is `null` (no `DATABASE_URL`), every store writes JSON. Defaults resolve to the monorepo `../.. /data/*.json`; production overrides them to the mounted volume. `store.ts` additionally guards that **when Postgres is primary, the rentals file must not rewrite audit history** (`audit: []` in the file payload). The Dockerfile pins the fallback paths to `/data` and warns that leaving them at the in-image default wipes memory on redeploy:

```
ENV OAUTH_TOKEN_STORE_PATH=/data/oauth-tokens.json
ENV KNOWLEDGE_STORE_PATH=/data/knowledge-sources.json
ENV RENTAL_STORE_PATH=/data/rentals.json
ENV CONSUMER_MEMORY_STORE_PATH=/data/consumer-memory.json
ENV CONSUMER_GOALS_STORE_PATH=/data/consumer-goals.json
ENV CONSUMER_PEOPLE_STORE_PATH=/data/consumer-people.json
ENV CONSUMER_REMINDERS_STORE_PATH=/data/consumer-reminders.json
ENV BRIEF_STORE_PATH=/data/consumer-brief.json
```

`docker-entrypoint.sh` starts as root only to `chown -R node:node /data` on the mounted volume, then drops privileges via `gosu` and execs the server as the unprivileged `node` user. Per ADR 0003 the file tier is explicitly “*not a substitute for backup, replication, or cross-region DR*”, has no row locking (whole-file last-writer-wins), and is not intended for multi-replica production.

8.8 Catalogue on-disk format (the primary data asset)

Agents are **flat, one-file-per-SKU** JSON packages, loaded read-only from `CATALOG_DIR` (`../.. /data/catalog` , `data/catalog-consumer` for the 17 consumer specialists via `CONSUMER_CATALOG_DIR`). Measured on disk:

METRIC	MEASURED VALUE
<code>data/catalog/*.agent.json</code>	552 files
– market SKUs (<code>us-</code> / <code>eu-</code> / <code>africa-</code> / <code>asia-</code> / <code>oceania-</code> × 100 families each)	500 (100 each)
– legacy/base aliases (no market prefix, retained for deep links)	52
<code>data/catalog-consumer</code>	18 files
Total catalogue lines (<code>`find ... -name '*.json'`</code>)	xargs cat

This confirms ADR 0001’s `100 families × 5 markets = 500 SKUs` model. Each file’s top-level schema:

```
{
  "format": "miai.agent-package/v1",
  "manifest": { /* id, name, version, category, tier, summary, channels, languages,
                voice, model, prompt, knowledge, tools, guardrails, evals, handoff,
                usage_profile, market, compliance, prepaid */ },
  "system_prompt": "...",      // crown-jewel IP
  "knowledge":     "...",      // grounding text (RAG source)
  "tools":         [ ... ],    // tool specs (~4 in the sampled agent)
  "guardrails":    { ... },    // crown-jewel IP
  "evals":         [ ... ]     // ~15 eval cases in the sampled agent
}
```

The `manifest.prepaid.skus[]` array encodes the per-agent prepaid cards (e.g. `CARD-ACPR-150` / `-500` / `-OUT50` with capacity + `price_band`), and `manifest.compliance` names the market regime (e.g. `["popia", "regional_privacy"]` for

`africa-*`). The catalogue is **never persisted to Postgres** – the DB stores only a workspace's *rental* of an agent (`miai_rentals.payload`) and its per-tenant KB overrides (`miai_knowledge_sources`). On public reads, `agent-ip.ts` strips `system_prompt`, `guardrails`, and `evals`, so the highest-value fields never leave through the public API even though they sit in plain files.

8.9 Data-erasure surface (DSAR)

`dsar-erase.ts` is the authoritative enumeration of workspace-scoped personal data and doubles as a data-map cross-check. `eraseWorkspaceData(workspaceId)` clears `miai_rentals`, `miai_turns` (transcripts), `miai_knowledge_sources`, `miai_oauth_tokens` (+ per-connector token delete), `miai_workspace_members`, `miai_custom_requests`, and **redacts** `miai_audit.detail` (rows retained append-only with type/agent/timestamp; tombstones appended by the route). Note this erasure path is workspace-scoped and does **not** currently traverse the consumer (`tenant, consumer`) tables – a scope gap flagged below.

8.10 Scale-in-context and drift from the Licence Proposal

The Licence Proposal quoted ~398K lines / 1,021 files / ~298K catalogue / 45 routes / ~631 MB. The repository has **grown** since: current measured **406,319 authored lines / 1,212 files**, catalogue independently re-measured at **299,508 lines, 69 API routes**, ~805 MB on disk incl. deps. Any handover sizing must count the catalogue (73.7 % of the line total) – headlining only the ~75.9K hand-written runtime understates the asset by roughly 4×.

09 Integrations

The integration layer is the hand-written boundary between the agent runtime and the outside world: how a tool call becomes a real HTTP request to Google, Slack, Shopify, Stripe, etc., and how inbound traffic from websites, Telegram, and MCP clients becomes an agent turn. It is small, dense, and security-first – the whole connector framework is `packages/connectors` (~3,961 TS LOC across 12 source files under `src/`; ~4,293 LOC / 19 files including its `test/` suite), a rounding error against the platform's **406,319 authored lines / 1,212 files** (measured at commit `fb97438`), of which the **299,508-line, 500-agent JSON catalogue** in `data/catalog` is the dominant asset. The integration code is leverage, not bulk: one `executeConnector` path fans out to every provider, and the 500 catalogue agents declare which of these connectors/tools they bind.

All connector code lives in the workspace package `@miai/connectors` (`packages/connectors`), consumed by `apps/web` via 5 OAuth routes (`api/oauth/*`), 2 MCP routes (`api/mcp/*`), the embed chat endpoints, the Telegram webhook, and the webhook sink.

9.1 Connector model – how a connector is defined

Three types in `packages/connectors/src/types.ts` describe the entire contract:

```
export interface ConnectorMeta { id: ConnectorId; name: string; phase: 1|2;
  description: string; auth: "oauth"|"api_key"|"webhook_secret"|"mcp"; recommended?: boolean }
export interface ToolBinding { tool: string; connector: ConnectorId; config?:
  Record<string,string> }
export interface ConnectorCall { workspaceId: string; agentId: string; tool: string;
  args: Record<string,unknown>; binding: ToolBinding; mode: "sandbox"|"live" }
export interface ConnectorResult { ok: boolean; data: Record<string,unknown>;
  connector: ConnectorId; stubbed: boolean }
```

A catalogue agent's tool (e.g. `notify_team`, `place_order`, `create_ticket`) is wired to a provider through a `ToolBinding` (`{ tool, connector, config }`) carried on the tenant's rental record. `ConnectorId` is a 24-member string union in `types.ts`, but only **16 are surfaced as "Actions"** – the `CONNECTORS: ConnectorMeta[]` catalogue in `packages/connectors/src/index.ts`. The extra `ConnectorId`s (`web_search`, `weather`, `google_tasks`, `google_contacts`, `google_drive`, `notion`, `spotify`, `todoist`, `youtube`) are internal/consumer tools handled in `executeLive` but not shown in the B2B Actions catalogue.

The 16 Actions (`CONNECTORS`, with `phase` and `auth`):

#	ID	NAME	PHASE	AUTH
1	<code>mcp</code>	MCP server (POST /tools/call)	1	mcp
2	<code>webhook</code>	Webhook (POST events to any URL)	1	webhook_secret
3	<code>google_calendar</code>	Google Calendar	1	oauth
4	<code>m365_calendar</code>	Microsoft 365 Calendar (Graph)	1	oauth
5	<code>shopify</code>	Shopify (orders/products/stock/returns)	1	oauth
6	<code>hubspot</code>	HubSpot (leads/contacts/tickets)	1	oauth

#	ID	NAME	PHASE	AUTH
7	slack	Slack (notify / human handoff)	1	oauth
8	email	Email – Gmail or Microsoft	1	oauth
9	whatsapp	WhatsApp (Cloud API transport)	1	api_key
10	woocommerce	WooCommerce	2	api_key
11	teams	Microsoft Teams	2	oauth
12	xero	Xero	2	oauth
13	quickbooks	QuickBooks	2	oauth
14	stripe	Stripe (payment links)	2	api_key
15	calendly	Calendly	2	oauth
16	zendesk	Zendesk	2	oauth

`listConnectors(phase?)` filters this list; `WEBHOOK_TEMPLATES` (property_enquiry, hotel_guest_request, insurance_fnoi) provide pre-shaped payloads.

9.2 executeConnector – invocation and the SANDBOX gate

Every tool call enters `executeConnector(call: ConnectorCall)` (`index.ts`). The **fail-safe sandbox gate is first**: if `call.mode === "sandbox"` or `process.env.SANDBOX_MODE === "1"`, it never actuates a live connector – it returns `stubFor(call.tool, call.args)` with `stubbed: true`, whatever the caller passed. This is what makes the public "Try" sandbox safe: no real sends/writes are ever possible in a sandbox deployment.

```
if (call.mode === "sandbox" || process.env.SANDBOX_MODE === "1") {
  return { ok: true, data: stubFor(call.tool, call.args), connector: call.binding.connector,
    stubbed: true };
}
return executeLive(call);
```

`stubFor` (`live/execute.ts`, ~500 lines) is a large intent-matched fixture map keyed on lowercased tool name (`place_order` → `ORD-3391`, `book_table` → `TBL-4821`, `check_availability` → free/busy blocks, `create_ticket`, `capture_lead`, plus vertical-specific stubs for menus, payslips, leave balances, PO status, tracking, etc.), so an agent behaves realistically before any account is linked.

`executeLive(call)` (`live/execute.ts`, lines ~1808–2064) is the real dispatcher. Notable behaviours:

- **Reminders are app-owned**: any tool whose name includes `remind` is handled locally by `handleInternalAssistantTool` before any token routing (so reminders always succeed regardless of calendar state).
- `webhook` / `mcp` are routed first; internal assistant tools (tasks/memory/research) with no external sink degrade to a local result instead of failing.

- **API-key connectors** resolve their key from the sealed token store → binding `config` → env, in that order: `whatsapp` (`WHATSAPP_TOKEN` + `WHATSAPP_PHONE_NUMBER_ID`), `stripe` (`STRIPE_SECRET_KEY`), `woocommerce` (`consumer_key` / `consumer_secret`), and public `youtube` (`YOUTUBE_API_KEY`, no user auth).
- **OAuth connectors** call `getValidAccessToken(workspaceId, connector)`; if no live token, they return a `stubbed: true` result annotated `"<connector> not OAuth-connected – complete Connect in Actions"` rather than erroring.
- The `catch` degrades every failure to `{ ok:false, data:{ error, suggestion:"I couldn't reach the connected system – I can hand this to the team." } }` – connectors never throw into the turn loop.

9.3 Providers and capabilities

`OAUTH_PROVIDERS` (`packages/connectors/src/oauth/providers.ts`) defines **18 OAuth providers** (the 9 B2B ones named below plus consumer providers `google_tasks`, `google_contacts`, `google_drive`, `notion`, `spotify`, `todoist`, `youtube`, and `email`). Each provider declares `clientIdEnv` / `clientSecretEnv`, `scopes`, `pkce`, `authStyle` (`body` | `basic`), `authorizeUrl` / `tokenUrl` (functions of context), and optional `requiresShop` / `requiresSubdomain` / `extraAuthParams`.

Live capability handlers (all in `live/execute.ts` unless noted):

PROVIDER	CONNECTOR ID	HANDLER	CAPABILITY (VERIFIED)	AUTH / SCOPES
Google Calendar	<code>google_calendar</code>	<code>googleCalendar()</code>	List upcoming events; freeBusy availability (<code>check_calendar</code>); create timed reminder event; create/schedule meeting with attendees	PKCE; <code>calendar.events</code> , <code>calendar.readonly</code>
Gmail / Email	<code>email</code>	<code>sendEmail()</code> , <code>gmailTriage()</code> , <code>fetchInboxItem()</code>	Send (Gmail raw / Graph <code>sendMail</code>); read/triage inbox (readonly search + snippet, metadata fallback); draft intent returns composed message, never sends	PKCE; <code>gmail.send</code> , <code>gmail.readonly</code> (Google) or <code>Mail.Send</code> (Microsoft)
Microsoft 365 Calendar	<code>m365_calendar</code>	<code>m365Calendar()</code>	<code>getSchedule</code> availability; create event via Graph <code>/me/events</code>	PKCE; <code>Calendars.ReadWrite</code> , <code>User.Read</code> , <code>offline_access</code>
Microsoft Teams	<code>teams</code>	<code>teamsHandoff()</code>	Post channel message via Graph (needs <code>TEAMS_TEAM_ID</code> + <code>TEAMS_CHANNEL_ID</code>)	PKCE; <code>ChannelMessage.Send</code> , <code>Chat.ReadWrite</code>

PROVIDER	CONNECTOR ID	HANDLER	CAPABILITY (VERIFIED)	AUTH / SCOPES
Slack	slack	slackHandoff() (handlers/slack.ts)	chat.postMessage human-handoff card (reason/summary/contact) to a chosen channel	comma-scoped; chat:write , channels:read /join , groups:read , users:read
Shopify	shopify	shopifyOrder()	Order status lookup via Admin API 2024-10 ; egress allowlisted to *.myshopify.com	no-PKCE, requiresShop ; read_orders , write_orders , read_products , read_inventories
HubSpot	hubspot	hubspotWrite() , hubspotDefaultTicketStage()	Create ticket (resolves portal's first pipeline stage) or create contact/lead	no-PKCE; crm.objects.contacts.read/write , tickets , oauth
Xero	xero	xeroRead()	Read invoices (api.xro/2.0/Invoices); tenant resolved post-consent via /connections	PKCE, authStyle:"basic" ; accounting.transactions , offline_access
QuickBooks	quickbooks	quickbooksRead()	Query invoices (select * from Invoice), sandbox/prod base by QUICKBOOKS_ENV ; realmId from callback	no-PKCE, authStyle:"basic" ; com.intuit.quickbooks.accounting
Calendly	calendly	calendlyBook()	Fetch users/me , return scheduling_url for the customer	PKCE; users:read , event_types:read , scheduled_events:read

PROVIDER	CONNECTOR ID	HANDLER	CAPABILITY (VERIFIED)	AUTH / SCOPES
Zendesk	zendesk	zendeskTicket()	Create ticket via <code>api/v2/tickets.json</code> on <code><subdomain>.zendesk.com</code> (SSRF-checked)	no-PKCE, <code>requiresSubdomain</code> ; <code>read</code> , <code>write</code>
WhatsApp	whatsapp	whatsappSend()	Send text via Graph <code>v19.0/<phone_number_id>/messages</code> (outbound transport only)	<code>api_key</code> (<code>WHATSAPP_TOKEN</code>)
Stripe	stripe	stripePaymentLink()	Create product → price → payment_link (never raw card in chat)	<code>api_key</code> (<code>STRIPE_SECRET_KEY</code>)
WooCommerce	woocommerce	wooOrder()	Order lookup via <code>wc/v3/orders/<id></code> (Basic auth, SSRF-checked)	<code>api_key</code> (consumer key/secret)
Google Tasks/Contacts/Drive, YouTube, Notion, Spotify, Todoist	(consumer)	<code>googleTasks</code> / <code>googleContacts</code> / <code>googleDrive</code> / <code>youtubeSearch</code> / <code>notionSearch</code> / <code>spotifyControl</code> / <code>spotifyCreatePlaylist</code> / <code>todoistTasks</code>	Read/write personal-assistant tools (tasks CRUD, contact search, Drive search, video search, page search, playback control + playlist create, Todoist tasks)	per-provider OAuth

Note: `calendlyBook`, `xeroRead`, and `quickbooksRead` are **read/echo** handlers – they confirm the connection and return account/list data plus an echo of the args, not a fully-transacted booking or write. Flagged in open gaps.

9.4 OAuth2/PKCE lifecycle – start → callback → disconnect → test → status

Five `apps/web/src/app/api/oauth` routes drive the lifecycle; all secret material stays in `@miai/connectors`.

Start – `GET /api/oauth/[connector]/start` (`start/route.ts`): requires auth + `agent` role, rate-limited (20/min per workspace+user). Calls `buildAuthorizeUrl(connectorId, {workspaceId, agentId, returnUrl, shop, subdomain, emailProvider})`. That builder (`oauth/flow.ts`):

- generates PKCE (`createPkce()`: `S256` challenge of a 32-byte verifier) when `provider.pkce`;
- packs a **self-contained, HMAC-signed state** via `createState()` – `base64url(json).base64url(hmac-sha256)`, 15-min expiry, carrying `workspaceId`, `agentId`, `connectorId`, `returnTo`, `shop` / `subdomain` / `emailProvider`, and `codeVerifier`. This survives multi-instance / cold starts on Railway with no server-side state;
- formats scopes per provider (space-separated default; **comma-separated for Slack + Shopify**);

- returns 503 with `missingEnv` when client id/secret are absent (`?format=json` returns `{url,state}` instead of redirecting).

Callback — `GET /api/oauth/callback` (`callback/route.ts`): `consumeState()` verifies the HMAC (timing-safe) and expiry, then `exchangeCode()` POSTs the token endpoint (Basic header for `authStyle:"basic"`, body creds otherwise; `code_verifier` for PKCE). Provider-specific post-processing: Slack unwraps `authed_user` /team/bot ids; Shopify/Zendesk normalize+store `shop` / `subdomain`; QuickBooks `realmId`; Xero makes a second `GET https://api.xero.com/connections` call to resolve `tenantId`. On success it `saveToken()`s, marks the connector connected on the rental, rewrites bindings to a **pointer only** (`config.oauth = "connected"` — the real secret never enters LLM context), appends an `oauth_connected` audit event, and telemetry (`trackDependency` / `trackEvent`). Errors redirect back to `/agents/<id>?tab=actions&oauth=error`.

Disconnect — `POST /api/oauth/[connector]/disconnect`: requires `admin` role; `deleteToken(workspaceId, connector)`, strips the connector from the rental's `connectedConnectors` and clears its binding config, appends `oauth_disconnected` audit.

Test — `POST /api/oauth/[connector]/test` (`test/route.ts` → `oauth/probe.ts`): **read-only, zero-LLM** live probe confirming the stored token still works. **PROBERS** cover **4 connectors only** — `slack` (`auth.test`), `google_calendar` (`calendarList`), `hubspot` (`account-info` → contacts fallback), `email` /Gmail (`users/me/profile`); any other connector returns `probe_not_supported`. Gated behind `assertProofHarness`.

Status — `GET /api/oauth/status`: returns the per-workspace matrix — for each provider `{configured, connected, requiresShop, requiresSubdomain, missingEnv, clientIdEnv, clientSecretEnv}` plus the api-key/webhook/mcp connectors and the shared `callbackUrl`.

9.5 AES-GCM sealed token storage

`packages/connectors/src/oauth/tokens.ts` is the token vault. Access/refresh tokens are **sealed with AES-256-GCM** (`seal()` / `open()`), envelope `v2.<iv>.<tag>.<ciphertext>` (all base64url); the 32-byte key is `sha256(OAUTH_TOKEN_SECRET)`. `open()` also accepts legacy `v1`. (HMAC integrity-only) and bare plaintext for migration. Production boot **fails closed** if `OAUTH_TOKEN_SECRET` is missing/weak (`dev-only-change-me`, `< 16` chars).

Storage is dual-backend: **Postgres** table `miai_oauth_tokens` (`workspace_id`, `connector`, `sealed JSONB`, `updated_at`, `PK(workspace_id,connector)`) created on demand, with an in-process `Map` cache and a **JSON file fallback** (`OAUTH_TOKEN_STORE_PATH`, default `../..data/oauth-tokens.json`) when `DATABASE_URL` is unset. Only the sealed rows are ever written; hydration lazily loads from PG then file. TLS verification is on by default (opt-out `PG_SSL_REJECT_UNAUTHORIZED=0`).

`getValidAccessToken()` (`flow.ts`) transparently refreshes when `expiresAt` is within 60s (POST refresh_token grant, Basic vs body per `authStyle`), deleting the token if refresh is impossible. `ListTokenMeta()` returns connector metadata **with no token material** for DSAR exports. Isolation key is always `<workspaceId>::<connectorId>` — tokens are per-tenant.

9.6 SANDBOX-stubbed vs live, retries, SSRF

- **Sandbox**: `executeConnector` short-circuits to `stubFor` (§9.2).
- **Retries** (`retry.ts`): `withRetry(fn, {maxRetries=2, baseDelayMs=300})` → 3 attempts, exponential backoff, retrying only `429` / `5xx` (`HttpErrorResponse`) and network `TypeError`s. Used by the webhook and MCP handlers.
- **SSRF** (`ssrf.ts`): `assertSafeOutboundUrl` rejects non-http(s), credentialed URLs, blocked hostnames (`localhost`, `*.local` / `*.internal`, cloud-metadata hosts) and any DNS answer in loopback/private/link-local/CGNAT/ULA/multicast ranges. `safeFetch` re-validates then **pins the connection to the validated IP via an undici.Agent custom lookup** (anti DNS-rebinding), failing closed if undici is unavailable, with `redirect:"manual"`. Live handlers that hit tenant-

controlled hosts (`shopifyOrder`, `zendeskTicket`, `wooOrder`, the webhook + MCP handlers) all route through `safeFetch`; Shopify hosts are additionally regex-locked to `*.myshopify.com`.

9.7 MCP — outbound connector and inbound sink

There are **two** distinct MCP surfaces:

(a) Outbound MCP connector — `handlers/mcp.ts` (`executeMcp`). When an agent's tool is bound to the `mcp` connector, the runtime resolves `endpoint` + `token` (from sealed meta / binding config) and issues `POST {endpoint}/tools/call` via `safeFetch` + `withRetry` with body `{ name: call.tool, arguments: call.args }` and optional `Authorization: Bearer`. This lets a tenant point an agent at their own HTTP MCP bridge.

(b) Inbound MCP server — `apps/web/src/app/api/mcp`:

- `POST /api/mcp/tools/call` (`tools/call/route.ts`) validates `mcpToolsCallBodySchema` (`{ name?: string(≤120), arguments?: unknown }`), requires `Authorization: Bearer <MCP_SINK_TOKEN>` (mandatory in production via `sinksRequireSecret()`, timing-safe compare), records the call (capped ring buffer of 50, persisted to `mcp-sink.json`), and returns `{ ok, live, provider:"mcp_sink", callId, name, result:{accepted, summary, arguments} }`.
- `GET /api/mcp` is a health/inspect endpoint (recent calls, secret-gated in prod) advertising `endpoint:"/api/mcp"`, `toolsCall:"/api/mcp/tools/call"`.

Accuracy note for handover: `/api/mcp` is a **minimal HTTP tools/call bridge (a Wave-4 proof sink)** — it is **not** a JSON-RPC 2.0 MCP server. There is no `initialize`, no `tools/list`, no `jsonrpc` envelope anywhere in `apps/web/src/app/api` (grep-confirmed). The contract is a flat `POST /tools/call {name, arguments}` on both the outbound connector and the inbound sink. Any DD claim of "JSON-RPC tools" should be read against this. (Flagged in open gaps.)

9.8 Channels — web, embed widget, Telegram, WhatsApp, MCP

Inbound channels converge on shared turn runners (`runChannelTurn` for embed/app, `runConsumerTurn` for Telegram); `ChannelKind` in `channel-turn.ts` is `"embed" | "app"`.

Web chat / app: authenticated in-product chat (`/app`, `AppChatClient.tsx`) via `runChannelTurn({channel:"app", ...})`.

Embeddable JS widget — `apps/web/src/app/agents/v1/agent.js/route.ts` serves `AGENT_JS_SCRIPT` (`lib/agent-js-script.ts`, ~14.2 KB source / self-described "~9KB, zero dependencies"). Configuration is via `data-*` attributes on the `<script>` tag: `data-key` (required publishable embed key), `data-accent`, `data-accent-2`, `data-title`, `data-greeting`, `data-suggestions`. The IIFE reads `document.currentScript`, builds a floating chat bubble, and POSTs `{key, message, sessionId}` to `origin + /api/embed/chat`. The route is served with `access-control-allow-origin:*`, `X-MIAI-Script-Integrity` and `Digest` headers; integrity is a stable `sha384 SRI` (`agent-js-sri.ts`), exposed at `GET /api/embed/sri` and injected into the copy-paste snippet by `buildEmbedScriptTag()` (`integrity=...` `crossorigin=anonymous`).

- Endpoints: `POST /api/embed/chat` and versioned `POST /api/v1/embed/chat` (same handler `lib/handlers/embed-chat.ts`), plus `OPTIONS` preflight.
- Handler validates `channelChatBodySchema` (`key ≤512`, `message`, `sessionId?`, `replyLanguage?`, `correlationId?`), rate-limits `embed:<key[:48]>` at 30/min, and calls `runChannelTurn`.

- **Embed key** `mia_pk_<base64url(workspaceId::agentId)>_<hmac10>` (`store.ts`): `resolveEmbedKey()` verifies an HMAC-10 over the id (salted per-agent after rotation, honouring `embedRevoked`), keyed by `EMBED_KEY_SECRET` / `OAuth_TOKEN_SECRET`; `*_demo` keys are rejected in production. The key is public by design and scoped to one tenant agent.
- **CORS** (`embed-cors.ts`): `EMBED_ALLOWED_ORIGINS` allowlist (supports `https://*.example.com` wildcards); bare `*` in production requires the dual flags `ALLOW_EMBED_ORIGIN_STAR=1` + `I_UNDERSTAND_EMBED_ORIGIN_STAR=1`, otherwise it falls back to `APP_BASE_URL` origin or denies.
- **Per-tenant domain lock**: `runChannelTurn` additionally rejects (403) requests whose `origin` / `referer` are not in the rental's `approvedDomains` (opt-in; empty = unrestricted) — this catches non-browser callers that lifted the public key, which CORS alone cannot. Turns are also blocked unless the rental state is live (`LIVE_STATES`).

Telegram — `POST /api/consumer/telegram/webhook` (`consumer/telegram/webhook/route.ts` + `lib/consumer-telegram.ts`). The **webhook secret is mandatory and fails closed** (503 if `TELEGRAM_BOT_TOKEN` / `TELEGRAM_BOT_SECRET` unset; 401 unless the `X-Telegram-Bot-API-Secret-Token` header timing-safe-matches `TELEGRAM_BOT_SECRET` — Telegram sends the raw secret, not an HMAC). Private chats only (groups/channels silently ignored). Each chat auto-provisions an isolated identity `telegram:<chat_id>` (no OIDC), routed through `runConsumerTurn` with the same wallet/memory/reminders as any consumer; `replyLanguage` is inferred from `language_code` (en/ru/es/fr/de/af/zu). Rate-limited 30/min per chat. Deep-link account binding uses signed single-use 5-min setup nonces (`mintSetupNonce` / `verifySetupNonce`, HMAC over `TELEGRAM_BOT_SECRET`).

WhatsApp — present as the **outbound whatsapp connector only** (`whatsappSend`, Cloud API `v19.0/.../messages`, `api_key` auth). There is **no inbound WhatsApp webhook route** in `apps/web/src/app/api` (directory-searched — none exists). Its `CONNECTORS` description is literally "Customer channel transport (Cloud API)". (Flagged in open gaps: the July/architecture framing of WhatsApp as a full inbound channel is not realised in code at `fb97438`.)

MCP — the inbound sink (§9.7) is the fifth channel surface.

9.9 Outbound webhook sink + HMAC

`webhook-sig.ts` defines the signing scheme: `signWebhookPayload(secret, timestampMs, body) = v1=<hex hmac-sha256 over "${timestampMs}.${body}">`. The outbound handler `handlers/webhook.ts` (`postWebhook`) POSTs JSON `{tool, args, agentId, workspaceId}` through `safeFetch` (SSRF + DNS-pin), sending headers `x-miai-signature: v1=<hex>` and `x-miai-timestamp: <ms>`, with `redirect:"manual"` (3xx is treated as an SSRF attempt and rejected) and `withRetry` on 429/5xx.

Verification (`verifyWebhookSignature`) enforces a **±5-min timestamp skew** for `v1=` signatures and, for one migration release, still accepts a raw shared-secret compare unless `allowLegacyRawSecret:false`. MyInstantAI's own proof sink `POST /api/webhook/sink` (`api/webhook/sink/route.ts`) uses exactly this verifier: in production `WEBHOOK_SINK_SECRET` is mandatory (503 if unset), HMAC-only can be forced via `WEBHOOK_SINK_HMAC_ONLY=1`, and it stores a capped (50) ring buffer to `webhook-sink.json`, tagging each event `hmac-v1` / `present` / `null`. All sink secret checks (`webhook`, `mcp`) use `timingSafeEqualString`.

9.10 Handover notes

- **Credential surface**: every OAuth provider needs a `*_OAUTH_CLIENT_ID` / `*_OAUTH_CLIENT_SECRET` pair plus the single shared redirect URI `{APP_BASE_URL}/api/oauth/callback` registered in each console (`docs/CONNECTOR_OAUTH.md`). Live tool code is complete; missing env is the only blocker — `/api/oauth/status` reports exactly which pairs are absent. Also required: `OAuth_TOKEN_SECRET` (seal + state), `EMBED_KEY_SECRET`, `MCP_SINK_TOKEN`, `WEBHOOK_SINK_SECRET`, `TELEGRAM_BOT_TOKEN` / `_SECRET`, `WHATSAPP_TOKEN` / `_PHONE_NUMBER_ID`, `STRIPE_SECRET_KEY`, `TEAMS_TEAM_ID` / `_CHANNEL_ID`, `YOUTUBE_API_KEY`, `QUICKBOOKS_ENV`.

- **Secrets never touch the model:** bindings store a `oauth:"connected"` pointer only; real tokens live sealed in the token store and are read at execution time.
- **Everything fails closed or degrades gracefully:** sandbox stubs, missing-token stubs, SSRF pins, mandatory Telegram/sink secrets, and a connector `catch` that offers a human handoff rather than throwing.

10 Security

This section is written for a security reviewer taking operational handover. Every control below was read in source at commit `fb97438`; file paths are repo-relative and identifiers are quoted verbatim. Where the platform relies on an operator to set an environment variable correctly, that dependency is called out rather than assumed.

Scale note (for weighting the review): the codebase is **406,319 authored lines across 1,212 files**, of which **299,508 lines / 552 JSON files** are the agent catalogue (`data/catalog` + `data/catalog-consumer`) – the crown-jewel IP whose `system_prompt` / `guardrails` / `evals` the redaction control (§10.9) exists to protect. The hand-written runtime that implements every control here is the smaller ~75.9K-LOC / ~410-file remainder (`apps/web` 34,550, `packages/runtime` 13,818, `packages/connectors` 4,293, etc.). The security surface is therefore concentrated in a comparatively small, auditable body of code guarding a very large data asset. There are **69 API routes** (`apps/web/src/app/api/**/route.ts`).

10.1 Trust boundaries and design posture

The platform draws a hard line between **instructions it authored** (system prompts, guardrails, workflow code) and **everything that crosses a boundary at runtime** (JWTs, request bodies, connector responses, web-search snippets, model output). The recurring pattern is *fail-closed*: boot refuses to start on weak production config (§10.4), the wallet refuses to call a provider at zero balance (§10.10), Redis rate-limiting refuses to fall back to a per-replica map on error (§10.12), SSRF refuses to fetch when the DNS-pin transport is unavailable (§10.6), and inbound sinks reject when their shared secret is unset in production (§10.8).

Trust boundaries, and the module that enforces each:

BOUNDARY	ENFORCEMENT POINT	MECHANISM
Browser / API caller → platform (B2B)	<code>apps/web/src/middleware.ts</code> , <code>apps/web/src/lib/auth.ts</code>	OIDC Bearer JWT, <code>workspace_id</code> claim required
Person → consumer line	<code>apps/web/src/lib/consumer-identity.ts</code> , <code>consumer-session.ts</code>	Signed HttpOnly session cookie (Google OIDC)
Website embed → chat API	<code>apps/web/src/lib/store.ts</code> (<code>resolveEmbedKey</code>)	Publishable <code>mia_pk_</code> key, HMAC-derived
External sink → platform (webhook/MCP/Telegram/Paystack)	route-level	Per-channel shared secret / HMAC
Platform → outbound URL (webhook, knowledge crawl)	<code>packages/connectors/src/ssrf.ts</code>	DNS-pinned SSRF allowlist
Tenant A ↔ Tenant B	every store query + <code>packages/runtime/src/guardrails.ts</code>	<code>tenant_id</code> / <code>workspace_id</code> predicate + refusal guard
Server → browser (agent IP)	<code>apps/web/src/lib/agent-ip.ts</code>	<code>redactAgentPackage</code> strips prompt/guardrails/evals

10.2 Identity and authentication

There are **four distinct authentication mechanisms**, deliberately separated so that a compromise of one does not grant another's authority. `apps/web/src/lib/public-paths.ts` is the single source of truth for which routes are exempt from the B2B Bearer gate (imported by both `middleware.ts` and `auth.ts` so the two lists cannot drift — the header comment records that they previously *did* drift and broke the `/api/v1/embed/chat` contract).

(a) B2B OIDC Bearer — `apps/web/src/lib/auth.ts::resolveAuth`. When `MIAI_AUTH_MODE=oidc`, the `authorization: Bearer ...` JWT is verified with `jose`'s `jwtVerify` against a remote JWKS (`createRemoteJWKSet` built from `MIAI_OIDC_JWKS_URL`, or `${MIAI_OIDC_ISSUER}/.well-known/jwks.json`), pinned to `issuer = MIAI_OIDC_ISSUER` and `audience = MIAI_OIDC_AUDIENCE`. A token that verifies but carries **no** `workspace_id` (checked across `workspace_id` / `workspaceId` / `org_id` claims) is rejected `403 "Token missing workspace_id claim"` — tenancy is mandatory, not defaulted. `middleware.ts` additionally short-circuits any non-public `/api/*` request lacking a `Bearer` prefix to `401` at the edge before the handler runs.

(b) Consumer Google OIDC — `apps/web/src/lib/consumer-oidc.ts` + `consumer-session.ts`. A full **Authorization Code** + **PKCE** flow (not the B2B handoff): `pkcePair()` generates an S256 challenge, `buildAuthorizationUrl` sets `code_challenge_method=S256`, endpoints come from the provider discovery document (works for Google and any compliant issuer). At the callback (`apps/web/src/app/api/consumer/auth/callback/route.ts`) the CSRF `state` is compared to the value in a signed, short-lived (10 min) login-state cookie; `exchangeCodeForIdentity` then verifies the returned `id_token` against the provider JWKS with `issuer` + `audience` pinned **and** checks `payload.nonce === args.nonce`. Success mints a session JWT via `signSession` — **HS256, HttpOnly, secure in production, sameSite=lax, 30-day**, secret from `MIAI_SESSION_SECRET` / `OAUTH_TOKEN_SECRET` with an enforced **≥16-char** minimum. Session verification pins the algorithm explicitly: `jwtVerify(token, secretKey(), { algorithms: ["HS256"] })`. Any callback failure redirects back with `? auth_error=1` and **never sets a session cookie**.

Open-redirect defense on the consumer return path is `safeReturnPath` (`consumer-oidc.ts`), which resolves the candidate against the app origin, keeps only same-origin path+query, and explicitly rejects `//evil.com`, `https://evil.com`, the backslash form `/\evil.com` (WHATWG normalizes `\` → `/`), and a resolved pathname that itself begins `//`; it falls back to `/me`. It is applied again at the redirect sink in the callback route.

© Embed publishable keys — `apps/web/src/lib/store.ts`. Format `mia_pk_<base64url(workspaceId::agentId)>_<hmac10>`; the suffix is an HMAC-SHA256 over the id (folding in a per-agent `salt` once rotated) keyed by `EMBED_KEY_SECRET` (falls back to `OAUTH_TOKEN_SECRET`; both rejected if weak in production). `parseEmbedKey` uses only string ops and anchored character-class checks (`/^[a-f0-9]{10}$/`, `/^[A-Za-z0-9_-]+$/`) — the comment notes this is deliberately "no ambiguous/backtracking regex on attacker input" (ReDoS avoidance). `resolveEmbedKey` honours `embedRevoked` (rotate via `rotateEmbedKey` assigns a fresh salt so the old key stops verifying; revoke via `setEmbedRevoked`) and refuses any `*_demo` key when `NODE_ENV=production`.

(d) Sink / channel secrets — see §10.8. Distinct secrets for outbound webhook HMAC, Paystack, MCP sink token, Telegram, and CRON.

Mock mode (`MIAI_AUTH_MODE` unset/ `mock`) intentionally trusts `x-workspace-id` / `x-user-id` / `x-roles` request headers for frictionless demo, and grants elevated `["owner", "operator"]` defaults **only** outside production or under the dual mock-rails ACK; production without ACK falls to `["readonly"]` and boot fails closed (§10.4). This is the intended demo trust model, and its safety depends entirely on production never running mock rails.

10.3 Authorization, roles, and tenant isolation

Roles live in `apps/web/src/lib/security.ts`: a four-rank workspace ladder `readonly(1) < agent(2) < admin(3) < owner(4)` plus a cross-tenant `PLATFORM_ROLES` set (`operator`, `platform_admin`, `miai_admin`). `normalizeRoles` collapses aliases (`viewer` / `reader` → `readonly`, etc.); `requireRole(auth, min)` and `requireOperator(auth)` return a

ready `403 NextResponse` or `null`. Platform-operator surfaces (cross-tenant Agent Admin) require a platform role, or owner-in-mock.

Cross-tenant isolation is enforced at the query layer, not the role layer. Every persistence read/write carries the tenant predicate:

- Rentals/audit (`store.ts`): `WHERE workspace_id = $1 ...` on every `SELECT` / `DELETE` / `UPDATE`.
- Consumer memory (`consumer-memory-store.ts`): the owner is the composite (`tenantId, consumerId`) – the header states “the memory OWNER is (tenantId, consumerId) – never consumerId alone ... one brand’s users can never see another brand’s.” Every statement is `WHERE tenant_id = $1 AND consumer_id = $2`, and the file-store key is `{owner.tenantId} :: {owner.consumerId}`. The same holds for the life-graph and reminder stores.
- OAuth tokens (`packages/connectors/src/oauth/tokens.ts`) are keyed by `workspaceId`.

All queries are **parameterized** (`pg` with `$1, $2...`; no ORM, no string-concatenated SQL). A second, model-level backstop lives in `packages/runtime/src/guardrails.ts`, which refuses attempts to read another party’s data: “I can’t share or access another person’s, another account holder’s, or another tenant’s confidential information ...” Tenant isolation has a dedicated test (`apps/web/test/oidc-isolation.test.mjs`, `tenant-brands.test.mjs`).

10.4 Boot hardening and the secrets model

`apps/web/src/instrumentation.ts` calls `assertBootHardening()` on Node server start; in production a failed check **throws and refuses to boot**. The checks (`apps/web/src/lib/security-flags.ts`) are:

- `checkProductionSecrets` – when OIDC is on, or production without mock rails, `OAuth_TOKEN_SECRET` / `OAuth_STATE_SECRET` / `EMBED_KEY_SECRET` must be strong. `isWeakSecret` rejects empty, `< 16` chars, `dev-only-change-me`, and `replace-with-long-random-string`.
- `checkProductionRails` – refuses `MIAI_AUTH_MODE=mock`, `MIAI_WALLET_MODE=mock`, `MIAI_MODEL_MODE=mock` in production, and catches *half-configured* escape hatches (one flag of a dual pair without the other).
- `checkProductionPersistence` – production requires `DATABASE_URL` unless the file-fallback dual flag is set; a remote Postgres with `PG_SSL_REJECT_UNAUTHORIZED=0` is blocked without its ACK.

The **dual-flag escape hatch** pattern is the core operator-safety idea: a dangerous production posture requires *both* a permission flag and a literal acknowledgement env, so it cannot be enabled by a single stray variable. Enabling one logs a structured `warn` alert (`miai.mock_rails_enabled` / `miai.embed_origin_star_enabled`) on boot.

ESCAPE HATCH	PRIMARY FLAG	ACKNOWLEDGEMENT ENV
Mock auth/wallet/model in prod	<code>ALLOW MOCK RAILS=1</code>	<code>I_UNDERSTAND MOCK RAILS IN PROD=1</code>
Bare <code>*</code> embed CORS in prod	<code>ALLOW EMBED ORIGIN STAR=1</code>	<code>I_UNDERSTAND EMBED ORIGIN STAR=1</code>
File store instead of Postgres	<code>ALLOW FILE FALLBACK IN PROD=1</code>	<code>I_UNDERSTAND FILE FALLBACK IN PROD=1</code>
Unverified Postgres TLS	<code>PG_SSL_REJECT_UNAUTHORIZED=0</code>	<code>I_UNDERSTAND PG_SSL INSECURE=1</code>

Which secret guards what:

SECRET ENV	GUARDS	NOTES
<code>MIAI_OIDC_ISSUER</code> / <code>_AUDIENCE</code> / <code>_JWKS_URL</code>	B2B Bearer verification	JWKS-based
<code>GOOGLE_OAUTH_CLIENT_ID</code> / <code>_SECRET</code> (or <code>MIAI_OIDC_CLIENT_*</code>)	Consumer sign-in code exchange	<code>consumer-oidc.ts</code>
<code>MIAI_SESSION_SECRET</code> (or <code>OAUTH_TOKEN_SECRET</code>)	Consumer session + login-state JWT signing	≥16 enforced
<code>OAUTH_TOKEN_SECRET</code>	AES-256-GCM key for OAuth tokens at rest; default for embed/state secrets	<code>sha256(secret)</code> → 32-byte key
<code>OAUTH_STATE_SECRET</code>	OAuth connector <code>state</code> HMAC	falls back to token secret
<code>EMBED_KEY_SECRET</code>	<code>mia_pk_</code> embed-key HMAC	falls back to token secret
<code>WEBHOOK_SINK_TOKEN</code> / HMAC secret	Inbound webhook sink	<code>WEBHOOK_SINK_HMAC_ONLY=1</code> drops legacy raw-secret
<code>PAYSTACK_SECRET_KEY</code>	Paystack webhook HMAC-SHA512	<code>paystack.ts</code>
<code>MCP_SINK_TOKEN</code>	MCP inbound sink bearer	required in prod
<code>TELEGRAM_BOT_SECRET</code>	Telegram webhook header token	fail-closed if unset
<code>CRON_SECRET</code>	Consumer daily-brief cron route	

Note: `assertProductionSecrets()` only logs; the throwing gate is `assertBootHardening()` (which composes all three checks). `SANDBOX_MODE=1` deliberately relaxes all boot hardening — that flag is an isolated-eval declaration and must never be set on a customer deployment.

10.5 Transport and browser-surface hardening

CSP (`apps/web/src/lib/csp.ts`, applied per-request in `middleware.ts` with a fresh `crypto.randomUUID()`-derived nonce): `script-src 'self' 'nonce-...' 'strict-dynamic'` (adds `'unsafe-eval'` **only** outside production for HMR), `object-src 'none'`, `base-uri 'self'`, `form-action 'self'`, `frame-ancestors 'self'`, `upgrade-insecure-requests`, `style-src` retains `'unsafe-inline'` — the code comments this as a knowingly-deferred item (next/font + Tailwind), with a `scriptSrcAllowsUnsafeInline` acceptance-gate helper asserting the *script* side stays clean. A `security-headers` e2e spec (`e2e/api/security-headers.spec.ts`) guards these.

Embed CORS (`apps/web/src/lib/embed-cors.ts`): origins come from `EMBED_ALLOWED_ORIGINS` (supports `https://*.example.com` wildcards via an escaped-then-`[^.]+` regex). In production a bare `*` is denied unless the dual `ALLOW_EMBED_ORIGIN_STAR` ACK is set; otherwise it narrows to the `APP_BASE_URL` origin or, failing that, returns `null` (deny). `vary: Origin` is always emitted.

Body-size gate: `middleware.ts` rejects any `/api/*` request whose `content-length` exceeds `MIAI_MAX_BODY_BYTES` (default 1 MiB) with `413`.

Embed script integrity: `apps/web/src/lib/agent-js-sri.ts` computes a stable `sha384` SRI token and an RFC 9530 `Digest` header for the zero-dependency `agent.js` embed body, so the served snippet can be integrity-pinned by host pages.

10.6 SSRF defense (DNS-pinned)

`packages/connectors/src/ssrf.ts` is the single egress guard, used by outbound webhooks, the MCP handler, and knowledge crawls (`apps/web/src/lib/ingest.ts` imports `safeFetch` from `@miai/connectors`).

`assertSafeOutboundUrl`:

- allows only `http:` / `https:`, rejects URLs carrying credentials, and blocks a denylist of hostnames (`localhost`, `metadata.google.internal`, `metadata.azure.com`, `*.local` / `*.internal` / `*.localhost`);
- resolves DNS (`dns.lookup(..., {all:true})`) and rejects if **any** returned address is private/loopback/link-local/CGNAT/ULA/multicast — `isBlockedIp` covers `0/8`, `10/8`, `127/8`, `169.254/16` (cloud metadata), `172.16/12`, `192.168/16`, `100.64/10`, IPv6 `::1` / `fc*` / `fd*` / `fe80` / `ff*`, plus IPv4-mapped-IPv6 normalization.

`safeFetch` then **pins the connection to the validated IP** via an `undici Agent` with a `connect.lookup` override returning the pre-resolved address — closing the TOCTOU/DNS-rebinding window between validation and fetch — and **fails closed** (`throw "SSRF blocked: undici unavailable..."`) rather than silently reverting to an unpinned `fetch`. Redirects are `manual` and a 3xx from a webhook is treated as an error (`"Webhook redirects are not followed (SSRF protection)"`). Covered by `packages/connectors/test/ssrf.test.mjs`.

10.7 Secrets at rest and OAuth-connector flow

OAuth tokens sealed at rest — `packages/connectors/src/oauth/tokens.ts`. `seal()` encrypts with **AES-256-GCM** (`v2`. `<iv>.<tag>.<ct>`, random 12-byte IV, key = `sha256(OAUTH_TOKEN_SECRET)`); `open()` still accepts legacy `v1`. HMAC-sealed and plaintext values for migration only. The DSAR/Trust-facing `TokenMeta` type is explicitly *"never includes token material."* Production refuses a weak `OAUTH_TOKEN_SECRET`.

Signed, PKCE-bound OAuth state — `packages/connectors/src/oauth/flow.ts`. `createState` emits `base64url(json).base64url(hmac-sha256)` carrying `connectorId` / `workspaceId` / `agentId`, a random `nonce`, the PKCE `codeVerifier`, and a **15-minute exp**. `consumeState` verifies the HMAC with `timingSafeEqual`, rejects malformed/expired/wrong-shape states, and is self-contained so it survives multi-instance/cold-start on Railway. `createPkce` produces an S256 challenge; per-provider `pkce: true/false` is declared in `providers.ts` (Google/M365/Shopify/HubSpot/Slack/Xero/Calendly etc. use PKCE). Fail-closed behaviour tested by `oauth-fail-closed.test.mjs`.

Sandbox connectors are force-stubbed — `packages/connectors/src/index.ts::executeConnector` returns a `stubbed: true` stub for **every** tool when `call.mode === "sandbox"` or `SANDBOX_MODE=1`, *"whatever the caller passed — no real sends/writes."*

10.8 Webhook, sink, and channel authenticity

- Outbound webhook** (`packages/connectors/src/webhook-sig.ts`): HMAC-SHA256 over `${timestampMs}.${body}`, sent as `x-miai-signature: v1=<hex> + x-miai-timestamp`. `verifyWebhookSignature` enforces a **±5-minute** skew window and uses `timingSafeEqual`. A legacy raw-secret path exists but can be disabled with `WEBHOOK_SINK_HMAC_ONLY=1` (`webhook-sink-auth.ts`).
- Paystack** (`apps/web/src/lib/paystack.ts`): `verifyWebhookSignature` computes HMAC-SHA512 of the raw body against the `x-paystack-signature` header, timing-safe. `/api/payments/paystack/webhook` and `/return` are Bearer-public precisely because they carry their own signature/verification; `/paystack/init` stays behind the Bearer gate (`public-paths.ts`).

- **MCP sink** (`apps/web/src/app/api/mcp/route.ts`, `.../tools/call/route.ts`): `MCP_SINK_TOKEN` bearer is **required in production** (`sinksRequireSecret()` → 503 if unset, 401 on mismatch), compared with `timingSafeEqualString`.
- **Telegram** (`consumer-telegram.ts::verifyTelegramWebhook`): compares the `x-telegram-bot-api-secret-token` header to `TELEGRAM_BOT_SECRET` with `timingSafeEqual`, **fails closed if the secret is unset** (`if (!secret || !headerValue) return false`), and the route additionally restricts to `chat.type === "private"`.

10.9 Agent-IP redaction

`apps/web/src/lib/agent-ip.ts::redactAgentPackage` replaces `system_prompt` and `guardrails` with `AGENT_IP_REDACTED` (`"[Restricted – not exposed through the API.]"`) and empties `evals` on any package leaving the server. It is applied at the single-agent read route (`apps/web/src/app/api/agents/[id]/route.ts`, `package: redactAgentPackage(pkg)`), and the catalogue listing (`apps/web/src/lib/catalog.ts`) only ever emits **counts** (`tools: number`, `evals: number`) – never prompt/guardrail/eval bodies – so the 299K-line catalogue cannot be enumerated-and-scraped through the API. Redaction runs in production and sandbox alike. Tested by `apps/web/test/embed-key-security.test.mjs` and the redaction is the stated reason the crawl/API cannot reconstruct the IP.

10.10 Prompt-injection and LLM-abuse posture

The runtime (`packages/runtime/src/index.ts`) treats the model as an untrusted planner over untrusted data:

- **Parse-only model output.** Tool invocations come solely from the provider's structured `tool_calls`; arguments are `JSON.parse`d inside a `try/catch` that tolerates malformed JSON. There is **no eval / new Function on model or tool output** anywhere in the runtime (the sole `new Function` in the tree is the static `import("undici")` shim in `ssrf.ts`).
- **Tool results are data, not instructions.** A connector result is appended as a `{ role: "tool", content: JSON.stringify(result.data) }` message, then the model is asked to summarize it ("Do not show JSON... answer from the knowledge base") – the result is never spliced into the system prompt or executed.
- **Bounded plan→act→observe loop.** `maxToolRounds = Math.min(5, Math.max(1, RUNTIME_MAX_TOOL_ROUNDS ?? 3))` – default **3**, hard-capped at 5.
- **Fail-closed metering.** Before any provider call, `runTurn` checks the wallet: `if (!skipDebit && bal.tokens ≤ 0)` it returns `paused_no_tokens` with a top-up message and **makes no model call** – spend and prompt-injection blast radius are both bounded by prepaid balance.
- **Untrusted external text is stripped.** Live web-search snippets (Brave) have HTML removed (`s.replace(/<[>]+>/g, "")`) in `packages/connectors/src/live/execute.ts` before re-entering the prompt; free-text written to connectors is newline-collapsed (`replace(/\\n/g, " ")`, e.g. IT-helpdesk subject/issue) and ingested documents collapse runs of newlines (`ingest.ts`).
- **Output guardrails** (`packages/runtime/src/guardrails.ts`): deterministic post-model scrubs block leaking internal instructions, repeating card numbers/CVV/OTP/PIN, cross-tenant data (§10.3), and force human-handoff on medical/safety emergencies. A final `scrubLeakedPlaceholders` strips any unresolved `{{template}}` tokens.
- **PII redaction before persistence** (`apps/web/src/lib/pii-redact.ts::redactPii`): emails, phone numbers, card-length digit runs (13–19), and OTP/PIN patterns are masked in stored transcripts (tested by `pii-redact.test.mjs`).

10.11 Rate limiting and abuse control

`apps/web/src/lib/security.ts::rateLimit` is a token bucket that uses Redis `INCR` + `EXPIRE` when Upstash is configured (shared across replicas) and an in-process `Map` otherwise. It **fails closed on Redis error** – a configured-but-erroring Redis returns `{ ok:false }` rather than silently degrading to a per-replica map (a deliberate anti-bypass choice). It is wired onto the abuse-prone public routes: `api/app/chat`, `api/ask/chat`, `api/knowledge/crawl`,

`api/oauth/[connector]/start`, `api/consumer/chat`, `api/consumer/connectors/[connector]/start`, and `api/consumer/telegram/webhook`.

10.12 Assurance: tests, disclosure, and transport TLS

Security behaviour is regression-locked by dedicated unit tests — `apps/web/test/{security-boot, embed-key-security, oidc-isolation, consumer-oidc-session, tenant-brands, pii-redact}.test.mjs` and `packages/connectors/test/{ssrf, webhook-sig, oauth-fail-closed, oauth-probe}.test.mjs` — plus e2e specs `e2e/api/{security-headers, webhook-sink, embed-chat-contract, oauth-status}.spec.ts`. Postgres transport verifies TLS by default (`apps/web/src/lib/pg.ts::sslFor`, and the mirrored `sslFor` in `oauth/tokens.ts`), with insecure opt-out gated behind the boot ACK. A `SECURITY.md` at repo root defines a private-disclosure channel (`security@myinstantai.com` / GitHub advisory), an explicit in/out-of-scope statement (tenant-uploaded prompts are out of scope unless the platform fails to isolate), and draft severity SLAs.

10.13 Residual risks and items to confirm during handover

Verified strengths aside, a reviewer should weigh the following (all confirmed in source):

- B2B Bearer verification does not pin JWT algorithms.** `auth.ts` calls `jwtVerify(token, getJwks(), { issuer, audience })` with `no algorithms allow-list`, unlike the consumer path which pins `["HS256"]`. Exploitation is not straightforward (the JWKS supplies asymmetric public keys, so `jose` will not accept an HS-signed token there), but adding an explicit `algorithms: ["RS256", "ES256"]` allow-list is cheap defence-in-depth against alg-confusion and should be a handover fix.
- Embed key MAC is a truncated 40-bit HMAC compared with `≡`.** `resolveEmbedKey` matches `parsed.mac ≡ expected` on a 10-hex-char (40-bit) truncated HMAC — non-constant-time and short. `mia_pk_` is a *publishable* key (it authorizes embed chat on one agent, gated further by CORS and rate-limits and the wallet), so impact is bounded, but a timing-safe compare and/or a longer tag would harden it.
- `style-src 'unsafe-inline'` remains** in the CSP (documented, deferred). Script-src is clean, so this is a lower-tier residual (style-based injection only), but it is not yet closed.
- Production safety is env-dependent.** The mock-rails, file-fallback, embed-`*`, and PG-insecure-TLS postures are all gated by dual flags and boot hardening, but the guarantee is only as good as the deployed environment; the handover runbook should assert `NODE_ENV=production`, no `SANDBOX_MODE`, and none of the `ALLOW_*` / `I_UNDERSTAND_*` flags set on the customer deployment.
- **No dedicated sanitiser for third-party-MCP tool *descriptions*.**** The platform's own `/api/mcp` is an outbound sink and does not import external MCP servers' tool metadata into prompts, so today there is no injection vector there — but if a future connector ingests remote MCP tool *descriptions* into the model context, a strip/whitelist step (as done for web-search snippets) would need to be added.

11 Compliance

This section documents the privacy and regulatory posture of the MyInstantAI Agent Marketplace as it exists **in code at commit `fb97438`**. The governing principle throughout the repo — stated in `docs/adr/0005-compliance-drafts.md` and enforced as house style in `docs/TRUST_AND_COMPLIANCE.md` ("Prefer under-claiming over over-claiming in partner meetings") — is that the platform ships **honest, shippable compliance mechanics and honest draft paperwork**, and does **not** claim certifications it does not hold. A due-diligence reader should treat this section's central distinction as load-bearing: **what is enforced in the runtime vs what is drafted prose / prompt guidance pending counsel**.

A scale note that matters here: the regional compliance guidance is not a single global system prompt. It is **baked per-agent across the entire 299,508-line catalogue** (`data/catalog` 290,061 lines + `data/catalog-consumer` 9,447 lines; 552 `*.agent.json` files on disk = 500 SKUs + legacy ZA aliases). Every agent package carries its own `manifest.compliance[]` array and market-specific `complianceNotes` appended into its `system_prompt` — so compliance is part of the catalogue IP, generated by build scripts, not a runtime rules engine.

11.1 Drafted vs enforced — the master distinction

CAPABILITY	STATUS IN CODE	ENFORCED AT RUNTIME?
DSAR export (<code>GET /api/dsar/export</code>)	Shipped, RBAC-gated	Yes — real endpoint, real data collection
DSAR erase (<code>POST /api/dsar/erase</code>)	Shipped, RBAC-gated, <code>confirm:true</code>	Yes — deletes operational data + redacts audit detail
Consent capture (<code>POST /api/consent</code>)	Shipped	Partial — records the choice to audit; does not gate downstream processing
Audit log + workspace isolation (<code>/api/audit</code>)	Shipped	Yes — tenant-scoped query predicate
PII redaction (<code>pii-redact.ts</code>)	Shipped	Partial — applied only to turn transcripts, not all stores
AI disclosure (EU AI Act Art.50)	Shipped copy, on surfaces	Partial — hardcoded per-surface; shared module unused
Market-pack regional compliance	Baked into catalogue	Prompt-level only — LLM guidance, not enforced controls
ROPA / DPIA / DPA / BAA / PCI-SAQ / SOC 2 index	Present in <code>docs/compliance/</code>	No — explicitly DRAFT / counsel-pending
Legal pages (<code>/privacy</code> , <code>/terms</code> , <code>/cookies</code>)	Present	No — <code>v0.2-draft</code> , counsel review pending
WORM / immutable audit	Not implemented	No — soft cap only (see 11.4)

11.2 DSAR — export and erasure

Two real, RBAC-gated endpoints implement data-subject rights at the **workspace** granularity (B2B DSAR, not per-end-user in-chat). Both require role `admin` or above via `requireRole(auth, "admin")` (`WORKSPACE_RANK`: `readonly:1` → `agent:2` → `admin:3` → `owner:4`, `apps/web/src/lib/security.ts`).

Export — `apps/web/src/app/api/dsar/export/route.ts` (`GET /api/dsar/export`) assembles a JSON pack for the caller's workspace and streams it as a file attachment. It gathers: wallet balance, rented agents (with last 40 messages each, truncated to 2000 chars), connector token **metadata** (`listTokenMeta` — never access/refresh tokens), knowledge sources (with `contentPreview` up to 4000 chars), audit events (up to 5000), and conversation turn transcripts (up to 500). Two protections are wired directly into the payload builder:

```
// OAuth secrets never leave the boundary — metadata only
const connectors = await listTokenMeta(workspaceId);
// audit detail keys matching token|secret|password|authorization|api_key → "[redacted]"
detail: redactAuditDetail(e.detail),
```

The export itself writes a `dsar_export` audit event (self-logging). **Scope caveat for handover:** `recentMessages` (from the live rental store `a.messages`) are included **raw — not PII-redacted** (confirmed: no `redactPii` import in the export route), whereas `conversationTurns` come from `traceability.ts` and *were* redacted at write time. So the two message sources in one export have **different redaction guarantees**.

Erase — `apps/web/src/app/api/dsar/erase/route.ts` (`POST /api/dsar/erase`, body `{ confirm: true }` via `dsarEraseBodySchema = z.object({ confirm: z.literal(true) })`). It brackets the wipe with `dsar_erasure_requested` / `dsar_erasure_completed` tombstone audit events and delegates to `eraseWorkspaceData()` in `apps/web/src/lib/dsar-erase.ts`, which performs a best-effort cascade and returns per-category counts:

CLEARED	FUNCTION
rentals	<code>clearWorkspaceRentals</code>
turn transcripts	<code>deleteTurnTranscriptsForWorkspace</code>
knowledge sources	<code>deleteKnowledgeForWorkspace</code>
OAuth tokens (per-connector + bulk <code>DELETE FROM miai_oauth_tokens</code>)	<code>eraseOAuthTokens</code>
workspace members	<code>clearWorkspaceMembers</code>
custom requests	<code>deleteCustomRequestsForWorkspace</code>
audit detail (redacted, rows retained)	<code>redactWorkspaceAuditDetails</code>

Audit rows are deliberately **not deleted** — the append-only `type` / `agent_id` / `at` columns survive so the trail still shows *that* activity occurred, while `redactWorkspaceAuditDetails` (`apps/web/src/lib/store.ts`) overwrites `detail` JSONB and nulls the denormalized `userId` / `sessionId` / `correlationId` / `channel` in **both** memory and Postgres (`UPDATE miai_audit SET detail = '{"_erased":true,...}'::jsonb WHERE workspace_id = $1`). **Discrepancy to flag:** the erase response `notice` states prior detail is "redacted **in memory only**," but the code demonstrably issues the Postgres `UPDATE` as well — the notice under-claims (conservative direction, but inaccurate). This matches the honest-language rule ("not one-

click right-to-be-forgotten inside chat; admin Trust erasure is available"): erasure is an **admin API action, not an in-chat consumer flow**.

11.3 Consent

`apps/web/src/app/api/consent/route.ts` (`POST /api/consent`) accepts `consentBodySchema = z.object({ choice: z.enum(["accepted", "essential"]) })` and records it as a `consent_recorded` audit event with `channel: "web_banner"`. Auth is optional — anonymous banner hits default to `WORKSPACE_ID`; authenticated hits attach `workspaceId` / `userId`. The UI is `apps/web/src/components/ConsentBanner.tsx`. **Limitations for the reader:** consent is **binary** (accept-all vs essential-only), there is **no granular per-purpose/per-category consent**, and — critically — recording the choice does **not gate any downstream processing** (no code path consults the stored consent before analytics/telemetry). It is an auditable record of the click, not an enforced consent-management platform.

11.4 Audit log, redaction, and retention

The audit trail is append-oriented. `appendAudit()` (`apps/web/src/lib/store.ts`) unshifts rows into memory and either `insertAuditRow()` into Postgres `miai_audit` (schema in `apps/web/migrations/001_init.sql`) or persists to the `rentals.json` file fallback. Read access is via `GET /api/audit` (`requireRole(auth, "readonly")`), always filtered by the caller's `workspaceId` **predicate** — the only cross-tenant path is `?all=1` which additionally requires `requireOperator(auth)` (platform roles `operator` / `platform_admin` / `miai_admin`). This is the enforced tenant-isolation guarantee for audit.

Retention — WORM gap (material finding). `docs/AUDIT_RETENTION.md` is explicit and honest that **WORM is not implemented**: aged rows are **hard-deleted**, not immutably archived, so the store is "best-effort traceability, **not** tamper-proof evidence." However, there is a **doc-vs-code mismatch** worth correcting at handover:

- The doc's table claims Postgres retention is "cap enforced by `DELETE` keeping newest **20,000** rows."
- The actual `insertAuditRow()` performs **only** an `INSERT ... ON CONFLICT (id) DO NOTHING` with a comment: *"Append-only posture: Postgres retains full audit history until ops archives."* There is **no Postgres DELETE cap in code**.
- The `AUDIT_CAP = 20_000` slice applies **only to the in-memory array and the file fallback**, not Postgres.

Net: Postgres audit is effectively **unbounded/append-only** (contradicting the doc's DELETE-cap claim), while memory/file is capped at 20,000. Turn transcripts (`miai_turns`) carry a parallel `TURN_CAP = 20_000` in `traceability.ts` and the doc states their retention policy is **TBD**. The recommended future architecture (archive table, Azure Immutable Blob, hash-chain, legal-hold flag) is documented but **not built**.

11.5 PII redaction

`apps/web/src/lib/pii-redact.ts` exports `redactPii(text)` — a **best-effort regex** pass replacing email → `[EMAIL]`, card-like 13–19-digit runs → `[CARD]` (ordered before phones so PANs aren't mis-caught), 10–15-digit phone-like sequences → `[PHONE]`, and `OTP/PIN/code` patterns → `[CODE]`. It is explicitly labelled "Best-effort" and is **regex-only** (no ML/NER, no address/name detection).

Enforcement scope (verified via grep — the important part): `redactPii` is imported and called in exactly **one** module, `apps/web/src/lib/traceability.ts`, on the turn-transcript write path (`userMessage` / `assistantMessage`, and the App Insights preview path). It is **not** applied to: the live rental message store (`a.messages`), knowledge documents, or audit `detail`. Consequently PII redaction protects durable turn transcripts but **not** the primary conversation store surfaced (raw) in DSAR export §11.2.

11.6 AI disclosure (EU AI Act Art.50)

`apps/web/src/lib/ai-disclosure.ts` centralizes Art.50-aligned copy (`AI_DISCLOSURE_SHORT/GREETING/STATUS/FOOTER/TRUST`) whose intent is a plain-language "you are chatting with an AI, ask for a human anytime" notice with human-handoff. **Finding (drift / dead-code):** grep confirms **no file imports `ai-disclosure.ts`** – the shared "single source" module is currently **unused**. The disclosure *does* appear on live surfaces, but via **hardcoded duplicated literals**, not the shared constants:

SURFACE	FILE	LITERAL
In-app assistant	<code>apps/web/src/app/app/v1/page.tsx</code>	"Hi – I'm an AI assistant (not a human)..."
Agent Studio	<code>apps/web/src/components/AgentStudio.tsx</code>	same greeting
Marketplace assistant	<code>apps/web/src/components/marketplace-assistant/MarketplaceAssistant.tsx</code>	"AI system · Ask anything..."

So the disclosure obligation is **met on those surfaces** but the copy is **not DRY** – a maintenance/consistency risk, and any surface added later can silently omit it (no lint/test gate enforces presence).

11.7 Regional market-pack compliance

Compliance guidance is **per-pack and baked into the catalogue at build time**, exactly as claimed – not one global prompt. `data/catalog/market-packs.json` defines five packs, each carrying `compliance[]`, `healthCompliance[]`, `privacyLabel`, `emergency`, and a prose `complianceNotes` block:

PACK	COMPLIANCE[]	HEALTH ADD-ON	EMERGENCY	PRIVACY LABEL
US (<code>us-</code>)	<code>tcpa</code> , <code>ccpa</code>	<code>hipaa</code>	911	CCPA/HIPAA where applicable
EU (<code>eu-</code>)	<code>gdpr</code>	<code>gdpr</code>	112	GDPR
Africa (<code>africa-</code> , incl. ZA)	<code>popia</code> , <code>regional_privacy</code>	same	local services	POPIA-style / regional privacy
Asia (<code>asia-</code>)	<code>pdpa</code> , <code>regional_privacy</code>	same	local services	PDPA-style / regional privacy
Oceania (<code>oceania-</code>)	<code>au_privacy_act</code> , <code>nz_privacy_act</code>	same	000 / 111	Australian Privacy Act / NZ Privacy Act

The generator/polish scripts (`scripts/generate-market-packs.mjs`, `scripts/polish-catalog.mjs`, `scripts/deepen-wave3-markets.mjs`) append `pack.complianceNotes` to each agent's `system_prompt`, set `manifest.compliance = [...]`, and add a `Privacy: {privacyLabel}. Collect only what the task needs.` line into `guardrails`. Verified on a real file – `data/catalog/eu-accounting-practice.agent.json`: `manifest.compliance = ["gdpr"]`, `manifest.market = "eu"`, GDPR text present in `system_prompt`, privacy line present in `guardrails`.

Honest characterization for the reader: these are **behavioural prompt instructions to the LLM** (e.g. TCPA: "continue conversations the customer started, don't cold-message, honour STOP by handing off"; GDPR/POPIA/PDPA: "minimise data,

honour access/erasure by human handoff"). They are **compliance-by-guardrail, not enforced controls** — there is no code that, say, blocks an outbound SMS to a new number or programmatically executes an erasure from within chat. The market packs raise the probability of compliant model behaviour and encode the right regional emergency numbers/currencies; they are not a substitute for the enforced controls in §11.2–11.4.

11.8 Trust Center honesty model & draft compliance pack

The in-app Trust Center (`apps/web/src/app/trust/page.tsx`) mirrors `docs/TRUST_AND_COMPLIANCE.md` and renders a four-tag truth legend on every claim: **Live** (`accent-bright` , "in this build"), **Partial** (`#f0b429` , "real capability, fuller story on roadmap"), **Via provider** (`#93c5fd` , "Stripe / host / IdP"), **Planned** (muted, "not claiming today"). The doc's "Honest language" block codifies what may/may not be said — explicitly forbidding "SOC 2 certified", "GDPR-compliant platform", "EU residency guaranteed today", and "one-click right-to-be-forgotten inside chat".

Per **ADR 0005** (`docs/adr/0005-compliance-drafts.md` , Accepted 2026-08-02), the `docs/compliance/` pack is **skeletons pending counsel/auditor review, not certifications**: `ROPA_DRAFT.md` , `DPIA_DRAFT.md` , `BREACH_72H_RUNBOOK.md` , `PCI_STRIPE_SAQ.md` , `SOC2_EVIDENCE_INDEX.md` , plus `templates/DPA_DRAFT.md` and `templates/BAA_DRAFT.md` (BAA **HIPAA-blocked** until a PHI architecture exists). The customer-facing legal pages (`/privacy` , `/terms` , `/cookies`) draw from `apps/web/src/lib/legal-content.ts` , stamped `LEGAL_VERSION = "0.2-draft"` , `LEGAL_LAST_UPDATED = "2 August 2026"` , with a `LEGAL_DRAFT_BANNER` and unresolved `TBD` s (operating entity, DPO, POPIA registration). Subprocessor list and roadmap (AES-256-GCM tokens, Key Vault, per-tenant EU/ZA/APAC residency pin, SOC 2 Type II, DPA/breach SLA) are enumerated as **future**, not present.

11.9 Handover findings & risks

- 1. DSAR export redaction is inconsistent** — live `recentMessages` are raw while `conversationTurns` are PII-redacted; align by running `redactPii` over exported rental messages (`apps/web/src/app/api/dsar/export/route.ts`).
- 2. Erase `notice` is inaccurate** — says audit detail is redacted "in memory only" while the Postgres `UPDATE` runs; correct the copy to avoid an internal misrepresentation of erasure scope.
- 3. Audit retention doc-vs-code mismatch** — `AUDIT_RETENTION.md` claims a Postgres 20k `DELETE` cap that does not exist in `insertAuditRow` ; Postgres is append-only/unbounded, only memory+file are capped. Reconcile the doc, and note the **WORM gap** is real and acknowledged.
- 4. AI-disclosure shared module is unused** — `ai-disclosure.ts` constants are dead; surfaces hardcode the copy. No gate enforces disclosure presence, so new surfaces can omit it. Wire surfaces to the module and add a test.
- 5. Consent is not enforced** — the recorded choice gates no downstream processing and is not granular; a real CMP is required for strict GDPR/ePrivacy posture.
- 6. Regional compliance is prompt-level** — market packs are LLM behavioural guidance across the 299,508-line catalogue, not enforced regulatory controls; do not represent them as technical enforcement in DD.
- 7. All formal paperwork is DRAFT** — ROPA/DPIA/DPA/BAA/PCI-SAQ/SOC 2 index and legal pages are counsel-pending skeletons; no certifications, signed agreements, or attestations exist (correctly non-claimed per ADR 0005).
- 8. PII redaction is regex-only** and single-call-site; it will miss names, addresses, and non-standard identifiers.

12 DevOps & Cloud

This section documents how the platform is built, shipped, run, observed, and migrated. Everything below was verified against the repository at commit `fb97438`. Scale context matters here because the build and image pipeline carry the whole asset, not just the runtime: the repo is **406,319 authored lines across 1,212 files** (re-measured: `ts/tsx/js/mjs/json/md/sql/css`, excluding `node_modules/.next/dist/.git`), of which **299,508 lines (~73.7%) are the 500-SKU agent catalogue** in `data/catalog` + `data/catalog-consumer` (**552 `*.agent.json` files on disk** – 500 SKUs plus retained legacy ZA aliases). The catalogue is baked into the container image (`ENV CATALOG_DIR=/app/data/catalog`), so every build compiles ~75.9K LOC of hand-written TypeScript **and** ships ~300K LOC of catalogue IP as read-only image content. Any capacity, image-size, or cold-start reasoning must count the catalogue, not just the runtime.

12.1 Container image – multi-stage Dockerfile

`Dockerfile` (repo root) is a four-stage build on `node:20-bookworm-slim`, with `corepack` pinning `pnpm@9.15.0`:

STAGE	PURPOSE
<code>base</code>	<code>node:20-bookworm-slim</code> ; enables <code>corepack</code> , activates <code>pnpm 9.15.0</code> ; <code>WORKDIR /app</code>
<code>deps</code>	Copies only workspace <code>package.json</code> files + <code>pnpm-lock.yaml</code> + <code>pnpm-workspace.yaml</code> , then <code>pnpm install --frozen-lockfile</code> (dependency layer cache)
<code>builder</code>	Copies full source, re-runs <code>pnpm install --frozen-lockfile</code> , <code>pnpm build:packages</code> , then <code>pnpm --filter @miai/web build</code> with <code>NEXT_TELEMETRY_DISABLED=1</code>
<code>runner</code>	Installs <code>gosu</code> , sets production ENV, copies the built app from <code>builder</code> , installs the entrypoint, and drops privilege at runtime

Notable runner details:

- **Privilege-drop model.** There is deliberately **no** `USER node`. The image comment explains it: the container starts as **root** only long enough for the entrypoint to `chown` the runtime-mounted `/data` volume (mounted root-owned), then drops to the unprivileged `node` user via `gosu`. `RUN mkdir -p /data && chown -R node:node /data /app`.
- **Entrypoint** (`docker-entrypoint.sh`): `set -e`; if `/data` exists, `chown -R node:node /data` (best-effort, `|| true`), then `exec gosu node "$@"`. Without this, every file-store write (`oauth`, `knowledge`, `rentals`, and all consumer-memory stores) would `EACCES` and a user's memory would be lost on the mounted volume.
- **The ENV block pins every file-store path onto the `/data` volume** so file-store mode survives deploys:

```
ENV CATALOG_DIR=/app/data/catalog
ENV OAUTH_TOKEN_STORE_PATH=/data/oauth-tokens.json
ENV KNOWLEDGE_STORE_PATH=/data/knowledge-sources.json
ENV RENTAL_STORE_PATH=/data/rentals.json
ENV CONSUMER_MEMORY_STORE_PATH=/data/consumer-memory.json
ENV CONSUMER_GOALS_STORE_PATH=/data/consumer-goals.json
ENV CONSUMER_PEOPLE_STORE_PATH=/data/consumer-people.json
ENV CONSUMER_REMINDERS_STORE_PATH=/data/consumer-reminders.json
ENV BRIEF_STORE_PATH=/data/consumer-brief.json
```

The inline comment is a real operational trap-warning: in file-store mode these **must** live on `/data`, otherwise `next start` (cwd `/app/apps/web`) resolves the default to `/app/data` **inside the image**, wiping a person's memory on every redeploy. When `DATABASE_URL` is set these paths are ignored (Postgres wins).

- **Rails default to mock** in the image: `MIAI_AUTH_MODE=mock`, `MIAI_WALLET_MODE=mock`, `MIAI_MODEL_MODE=mock` – safe on local/Railway; overridden to live values by Azure Bicep (§12.4).
- `NODE_ENV=production`, `PORT=3000`, `HOSTNAME=0.0.0.0`, `EXPOSE 3000`.
- **CMD**: `sh -c "pnpm --filter @miai/web exec next start -H 0.0.0.0 -p ${PORT:-3000}"` – honours the platform-injected `PORT` (Azure Container Apps / Railway).

Build-efficiency note: the `deps` stage does a frozen install, and the `builder` stage does `COPY . .` then runs `pnpm install --frozen-lockfile` a **second time** – a redundant install that costs build time; the dependency-layer cache from `deps` mitigates but does not eliminate it (open gap for optimisation).

12.2 Railway – primary hosting today

`railway.toml` (repo root):

```
[build]
builder = "DOCKERFILE"
dockerfilePath = "Dockerfile"

[deploy]
healthcheckPath = "/api/health"
healthcheckTimeout = 30
restartPolicyType = "ON_FAILURE"
restartPolicyMaxRetries = 5
```

Two Railway services are documented (verified via repo references to `*.up.railway.app`):

SERVICE	HOST	ROLE
Production/staging	<code>miaiweb-production.up.railway.app</code>	Live demo/partner host; Postgres attached; default target of <code>smoke:staging</code> , <code>uat:staging</code> , <code>handover:staging</code> , <code>playwright.config.ts</code> , OAuth callbacks (<code>docs/CONNECTOR_OAUTH.md</code>)
Sandbox	<code>miaiweb-production-f4cb.up.railway.app</code>	<code>SANDBOX_MODE=1</code> , file store, isolated eval env (<code>docs/CONSUMER_AUTH.md</code> names it as the sandbox consumer-auth callback)

Operational conventions from `docs/RAILWAY_DEPLOY.md`:

- **Root directory = repo root** (not `apps/web`); Railway auto-detects `railway.toml` + `Dockerfile`.
- **Persistence: Postgres preferred, `/data` volume as file fallback**. On boot the app runs versioned migrations (`apps/web/migrations/001_init.sql` ... `005_*`) and does **row-level upserts** (no full-table wipe) into `miai_rentals`, `miai_audit`, `miai_turns`, `miai_oauth_tokens`, `miai_knowledge_sources`, `miai_workspace_members`, `miai_ask_leads`, `miai_custom_requests`. Without a volume **and** without `DATABASE_URL`, restarts lose durable state.

The `/data` volume is attached in the Railway dashboard (Settings → Volumes → mount `/data`) – **not** expressible in `railway.toml`.

- **Optional Upstash Redis** (`UPSTASH_REDIS_REST_URL` / `_TOKEN`) shares rate limits (`miai:rl:*`) and chat/ask sessions (`miai:chan:*`, `miai:ask:*`) across replicas; when set, Redis errors **fail closed** and `/api/health` surfaces `redisPing`. `REDIS_URL` is reserved and unused in v1. Without Redis, `maxReplicas` must stay 1 (sessions + rate limits are per-replica).
- **Mock-rails dual-flag gate**: both `ALLOW MOCK RAILS=1` and `I_UNDERSTAND MOCK RAILS IN PROD=1` are required together, or boot/health fail closed; boot emits a structured `miai.mock_rails_enabled` warning. The same dual-flag pattern guards `EMBED_ALLOWED_ORIGINS=*`, file-fallback-in-prod, and insecure Postgres TLS (see §12.6). A CLI-based alternative deploy (`@railway/cli`: `railway up` + `railway variables set ...`) is documented as well.

12.3 Azure Container Apps – stated migration target

The forward-looking target is Azure Container Apps, fully expressed as infrastructure-as-code in `infra/azure/main.bicep` (`targetScope = 'resourceGroup'`). The Bicep provisions a complete landing zone:

RESOURCE	DETAIL
Container Apps managed environment + app	<code>caName</code> app; ingress <code>external: true</code> , <code>targetPort: 3000</code> , <code>transport: 'auto'</code> , <code>allowInsecure: false</code> ; container <code>cpu: 0.5</code> , <code>memory: 1Gi</code>
PostgreSQL Flexible Server	<code>Standard_B1ms</code> Burstable, version 16 , 32 GB storage, 7-day backups, HA disabled, <code>AllowAzureServices</code> firewall (<code>0.0.0.0-0.0.0.0</code>), DB <code>miai_agents</code>
Key Vault (RBAC)	Holds all signing secrets, wallet/model keys, DB URL, and one secret per supplied connector credential; empty connector values are <code>filter()</code> -ed out so no empty KV secret / dangling <code>secretRef</code> is created
Azure Files share (<code>miai-data</code> , 50 GB)	Mounted at <code>/data</code> for OAuth tokens, knowledge, and rental file fallback
Log Analytics + Application Insights	<code>appLogsConfiguration.destination = log-analytics</code> ; App Insights connection string injected as <code>APPLICATIONINSIGHTS_CONNECTION_STRING</code>
Managed identities	A user-assigned MI (<code>uami</code>) created before the app so KV <code>secretRef</code> s resolve on first deploy (avoids the circular system-assigned dependency); the app also gets a system-assigned MI for runtime/rotation. Both granted built-in Key Vault Secrets User (<code>4633458b-...</code>)

Health is wired directly into Container Apps as **both liveness and readiness probes** on `/api/health` (port 3000): liveness `periodSeconds: 30`, readiness `periodSeconds: 10`, each `failureThreshold: 3`.

Env flip on Azure (the Bicep hard-codes the live rails, overriding the image mock defaults):

ENV	RAILWAY STAGING	AZURE PRODUCTION
<code>MIAI_AUTH_MODE</code>	<code>mock</code>	<code>oidc</code>
<code>MIAI_WALLET_MODE</code>	<code>mock</code>	<code>http</code>

ENV	RAILWAY STAGING	AZURE PRODUCTION
MIAI_MODEL_MODE	mock	gateway (with MIAI_MODEL_PASSTHROUGH=1)
DATABASE_URL	optional/unset	Azure Postgres (KV secretRef)
APPLICATIONINSIGHTS_CONNECTION_STRING	unset	App Insights output

Scale guardrail: `maxReplicas` param is `@minValue(1)` and defaults to `1`, with a param description stating it must stay at `1` until Redis is provisioned and `UPSTASH_*` env is set — otherwise chat sessions and rate limits are per-replica and scaling breaks mid-conversation continuity. Scale block is `minReplicas: 1, maxReplicas: maxReplicas`.

Signing-secret discipline (deliberate design): `oauthTokenSecretParam`, `oauthStateSecretParam`, `embedKeySecretParam` are all `@secure()` `@minLength(32)` **required** params — **not auto-generated**. The comment documents why: Bicep `uniqueString()` yields only 13 chars (below the app's 16-char boot-hardening floor → crash-loop) and is deterministic from the resource-group id (→ forgeable embed keys). Operators generate them once (`openssl rand -hex 24`) and **reuse the same values every deploy**; rotating them invalidates existing embed keys and sessions.

Private-image pull: the published image is private by default; the Bicep accepts `registryServer` / `registryUsername` / `registryPassword` (empty ⇒ public image, no creds). Validated offline with `scripts/validate-azure.sh` (uses `az bicep build` or the `bicep` binary; no subscription deploy). A parameters template lives at `infra/azure/parameters.example.json`. `infra/azure/README.md` documents resources and persistence. Deploy/rollback are runbooked in `docs/MIGRATION_RUNBOOK.md`.

Migration status: Azure is a *target*, not yet the running platform — the runbook is a phased plan (Phase 0 rails wiring → Phase 1 DNS cutover → Phases 2–4 native-app clients → Phase 5 hardening) with an explicit **rollback** procedure (flip DNS/Front Door back to `miaiweb-production.up.railway.app`, keep Railway env intact, RTO target = **DNS TTL + 15 min**, keep TTL ≤ 300s during the window). Cutover default is a **fresh start** on Azure (Railway treated as demo-only). The `MIGRATION_P0/P1` docs and `PLATFORM_INTEGRATION.md` back it.

12.4 CI/CD — GitHub Actions

Five workflows in `.github/workflows/`. There is **no automated deploy workflow** in-repo: Railway auto-deploys from GitHub via its own dashboard integration, and Azure is deployed manually via `az deployment group create`. CI/CD here means *quality gating* + *image publishing*.

WORKFLOW	TRIGGER	PURPOSE
<code>ci.yml</code>	push→ <code>main</code> , all PRs	The merge gate. Job <code>quality</code> (the required check): <code>pnpm install --frozen-lockfile</code> → <code>build:packages</code> → <code>typecheck</code> → <code>@miai/web lint</code> → <code>pnpm test</code> (wallet+connectors+web+runtime unit) → <code>catalog:integrity</code> (100×5 + ZA alias policy) → <code>eval:suite:static</code> → <code>pnpm audit --prod --audit-level=critical</code> . Job <code>secrets</code> : forbidden-file scan (<code>.env</code> , <code>*.pem</code> , <code>id_rsa</code>) + <code>gitleaks</code> (elevated <code>pull-requests: write</code> for PR comments). Default perms <code>contents: read</code> (least privilege).
<code>publish-image.yml</code>	push→ <code>main</code> , <code>v*</code> tags, manual	Builds the image from the repo Dockerfile and pushes to GHCR (<code>ghcr.io/<owner/repo></code>), tagged <code>sha-<sha></code> and a ref tag; moves <code>latest</code> only on <code>main</code> . Uses the pre-installed <code>docker</code> CLI (no unpinned marketplace actions); <code>packages: write</code> . Private by default — the Azure deploy pulls it via the Bicep registry params.

WORKFLOW	TRIGGER	PURPOSE
<code>e2e-staging.yml</code>	manual, daily 06:15 UTC, push→ <code>main</code> (path-filtered)	Non-blocking Playwright against Railway staging. Three chained jobs, all <code>continue-on-error: true</code> : <code>e2e-smoke</code> (<code>@smoke</code>), then <code>e2e-functional+uat</code> , then <code>e2e-full</code> (incl. sandbox chat) — the latter two only on schedule/dispatch. Uploads Playwright reports (14-day retention). <code>PLAYWRIGHT_BASE_URL</code> defaults to the Railway host when unset.
<code>eval-nightly.yml</code>	daily 06:00 UTC, manual	<code>eval-nightly</code> runs the full mock eval suite over the catalogue (authoritative). A <code>preflight</code> job reads <code>ANTHROPIC_API_KEY</code> into an output boolean (the comment explains the GitHub quirk: <code>secrets</code> context is disallowed in job/step <code>if:</code>); gated live jobs <code>live-llm-matrix</code> (<code>MIAI_MODEL_MODE=anthropic</code> , <code>smoke:live-llm:matrix</code>) and <code>eval-live-sample</code> (<code>eval:live --limit=6</code>) run only when the key exists, both <code>continue-on-error</code> .
<code>daily-brief.yml</code>	hourly (<code>0 * * * *</code>), manual	Fans out consumer daily briefs: <code>POST {CONSUMER_APP_URL}/api/consumer/brief/run-due</code> with <code>x-cron-secret</code> . No-op unless <code>CONSUMER_APP_URL</code> + <code>CRON_SECRET</code> are set; endpoint's per-day guard makes best-effort scheduler drift/duplicate ticks harmless.

Toolchain nuance worth flagging for handover: **CI runs Node 22** (`setup-node node-version: 22`, required for `--experimental-strip-types` in `apps/web` tests) while the **runtime image is Node 20** (`node:20-bookworm-slim`). Both use `pnpm 9.15.0`. `pnpm run ci` mirrors the `quality` gate locally (minus lint/audit/secrets). **SonarCloud** is advisory only — `sonar-project.properties` exists (duplication exclusions for generated presets, scaffold generators, i18n dicts) but **no workflow invokes a Sonar scanner**, so it is wired via SonarCloud's GitHub App / automatic analysis, not Actions (open gap — branch-protection "quality" required check is a GitHub repo setting, not visible in-repo).

12.5 SANDBOX_MODE behaviour

`apps/web/src/lib/sandbox.ts` is a one-liner gate — `isSandbox()` returns `process.env.SANDBOX_MODE === "1"` — but the flag reshapes runtime behaviour across the codebase (documented in `docs/SANDBOX.md`, enforced in code):

GUARD	WHERE	BEHAVIOUR WHEN <code>SANDBOX_MODE=1</code>
Mock rails satisfied	<code>security-flags.ts</code> <code>checkBootHardening()</code>	Returns <code>{ok:true}</code> immediately — relaxes prod secret floor, real-rails, and Postgres requirements so a fresh sandbox deployment boots with mock auth + mock wallet and no dual flags
Live connectors killed	<code>packages/connectors/src/index.ts:103</code>	<code>if (call.mode === "sandbox")</code>
Model spend capped	<code>packages/runtime/src/index.ts:2108</code>	<code>if (SANDBOX_MODE === "1" && mode !== "mock")</code> caps real-provider turns at <code>SANDBOX_MODEL_TURN_CAP</code> (default 1000 per process), then falls back to the mock model
Payments disabled	Paystack top-up route	Returns <code>503</code> ; top-ups use free mock credit

GUARD	WHERE	BEHAVIOUR WHEN <code>SANDBOX_MODE=1</code>
Agent IP redacted	<code>/api/agents/[id]</code> via <code>lib/agent-ip.ts</code>	Strips <code>system_prompt</code> , <code>guardrails</code> , <code>evals</code> from client payloads (agents still run full server-side) – prevents bulk scraping of the 552-file catalogue IP
Labelled	UI	Persistent SANDBOX banner on every page

The intent (per `docs/SANDBOX.md`) is a give-my-team-access evaluation copy on **your** infra (separate service, own `/data` volume or Neon branch, capped keys), where "artifacts and running services cross; source and IP do not."

12.6 Operational endpoints & telemetry

`GET /api/health` (`apps/web/src/app/api/health/route.ts`, `force-dynamic`) is the deep readiness probe used by both Railway and Azure. It returns a JSON check bag and drives status codes:

- Reports `authMode` / `walletMode` / `modelMode`, `mockRailsAllowed`, `hardening`, `database` (`configured` vs `file-fallback`), `telemetry` (`appinsights|console`), and per-rail config presence (`oidcIssuer`, `walletUrl`, `modelUrl` – the last resolves against `gateway|azure|openai|anthropic` keys).
- **503 (fail-closed)** when boot hardening fails (returns `hardeningErrors`), when `pingStore()` fails, when store hydration throws, or when Redis is **configured-but-broken** (rate-limits/sessions fail closed).
- **200 (degraded)** when partner config is merely incomplete (`config: "incomplete"`) – deliberately kept 200 so Container Apps liveness does not flap during bring-up before OIDC/wallet/model creds land. Surfaces `storeBackend` (`postgres|file`), `storePing`, `redisConfigured/redisBackend/redisPing`.

`GET /api/version` (`force-dynamic`, unauthenticated per `public-paths.ts`): surfaces the running build so server-only changes are verifiable – reads Railway git metadata (`RAILWAY_GIT_COMMIT_SHA` / `GIT_COMMIT_SHA` / `SOURCE_COMMIT`), returns `commit`, `shortCommit`, `branch` (`RAILWAY_GIT_BRANCH`), `deploymentId` (`RAILWAY_DEPLOYMENT_ID`), and `node` (`process.version`).

`GET /api/ops` (`force-dynamic`): admin-gated Live Ops. `requireAuth` + `requireRole(auth, "admin")`; readonly/agent members and unauthenticated callers get 403. Returns this workspace's `wallet` balance, `opsSummary`, and last 20 `audit` rows filtered to the workspace. In OIDC mode the workspace is pinned to the verified token; only a platform **operator** in mock mode may inspect another via `?workspaceId=` – closing a cross-tenant read.

Telemetry (`apps/web/src/lib/telemetry.ts`) is SDK-free and dual-sink:

- **Always** emits structured single-line JSON to stdout/stderr (`console.log` / `console.error`) – picked up by Container Apps → Log Analytics and by Railway logs.
- **Additionally**, when `APPLICATIONINSIGHTS_CONNECTION_STRING` (or `APPINSIGHTS_CONNECTION_STRING`) is set, POSTs raw App Insights envelopes to `{IngestionEndpoint}v2/track` with a 2.5s `AbortSignal.timeout`, no SDK dependency. Envelope builders: `trackEvent` (Event), `trackException` (Exception, with stack), `trackDependency` (RemoteDependency – wallet/model-gateway/OAuth timings), and `trackAudit` (maps audit rows → `miai.audit.<type>` events with correlation/channel tags). Role tags come from `WEBSITE_SITE_NAME` / `HOSTNAME`. Telemetry failures are swallowed – "never fail the product path for telemetry." `telemetryMode()` (console vs appinsights) is echoed in `/api/health`.

Post-cutover the runbook verifies `miai.audit.*` events land in App Insights and watches `/api/health` for 30–60 min. Smoke tooling (`pnpm smoke:cutover`, `smoke:staging`, `proof:*`, `smoke:live-llm:matrix`) targets these endpoints against either host.

12.7 Handover summary

- **Build once, run anywhere:** a single multi-stage Dockerfile produces the deployable image (runtime + ~300K-LOC catalogue baked in); the same image runs on Railway (today) and Azure Container Apps (target).
- **Two live Railway services** — production `miaiweb-production.up.railway.app` (Postgres) and sandbox `...-f4cb` (`SANDBOX_MODE=1`, file store, auto-deploy off) — with health-gated restarts (`ON_FAILURE`, max 5).
- **Azure is fully IaC-described** (`infra/azure/main.bicep`) and runbooked, but not yet the running platform; the cut is DNS-reversible with a ~5-min-TTL rollback.
- **Quality is gated in CI** (`quality` job) and **the image is published to GHCR**; deploys themselves are platform-managed (Railway) or manual (Azure `az`), i.e. there is no in-repo continuous-deployment workflow.
- **Fail-closed posture throughout:** dual-flag escape hatches for mock rails / `*` CORS / file-fallback / insecure PG-TLS, a 16-char secret boot floor, and a health endpoint that 503s on real degradation but stays 200 during partner-credential bring-up.

13 Dependencies

This section is an exhaustive, verified inventory of every dependency the platform declares, how those declarations resolve in the committed lockfile, the deliberate engineering posture behind the manifest (minimal runtime surface, no ORM, no heavy UI/state libraries, no mandatory Redis), the internal `workspace:*` graph, and the supply-chain controls that pin all of it. Every version below was read from the actual manifests and cross-checked against `pnpm-lock.yaml` at commit `fb97438`.

13.1 Posture in one paragraph

The platform is a **pnpm 9.15.0 monorepo** (`packageManager: "pnpm@9.15.0"` in the root `package.json`) on **Node 20**, built around a deliberately tiny third-party runtime surface. The entire production web app (`@miai/web`) ships with **exactly six third-party runtime dependencies** — `next`, `react`, `react-dom`, `jose`, `pg`, `zod` — plus five internal workspace packages. There is **no ORM, no HTTP client library, no state-management library, no component/UI framework, no utility toolbelt (lodash/underscore), and no mandatory cache/queue dependency**. Data access is hand-written against the raw `pg` driver; optional Redis is implemented over `fetch` with zero added packages. This is not an accident of a small app — it is a ~406K-line platform (see §13.10) that has consciously kept its dependency tree flat to minimise the attack surface, audit burden, and upgrade friction that MyInstantAI's team would inherit at handover.

13.2 Runtime dependencies — `apps/web` (the deployable)

Source: `apps/web/package.json` (declared) cross-checked against the `apps/web:` importer block in `pnpm-lock.yaml` (resolved).

PACKAGE	DECLARED RANGE	RESOLVED IN LOCKFILE	ROLE / WHY IT IS HERE
<code>next</code>	<code>^15.2.8</code>	15.5.22	Next.js 15 App Router — the whole HTTP surface: 69 API route handlers + 39 <code>page.tsx</code> , SSR, edge/node runtime, build.
<code>react</code>	<code>^19.0.0</code>	19.2.8	React 19 rendering runtime for the App Router.
<code>react-dom</code>	<code>^19.0.0</code>	19.2.8	DOM renderer paired with React 19.
<code>jose</code>	<code>^6.2.5</code>	6.2.5	JWT/JWS/JWKS primitives — verifies B2B OIDC Bearer tokens (<code>MIAI_OIDC_JWKS_URL</code>), signs/verifies the consumer HS256 session, and backs alg-pinning. Chosen over <code>jsonwebtoken</code> because it is dependency-free, ESM-native, and JOSE-complete (JWKS remote key sets).
<code>pg</code>	<code>^8.22.0</code>	8.22.0	Raw PostgreSQL driver. The only data-access dependency — no ORM sits on top of it (see §13.6).
<code>zod</code>	<code>^3.24.2</code>	3.25.76	Runtime schema validation at every trust boundary (request bodies, webhook payloads, connector configs, env parsing).

Workspace (internal) runtime dependencies of `@miai/web`, all declared `workspace:^` and resolved as filesystem links:

WORKSPACE PACKAGE	RESOLVES TO
@miai/agent-protocol	link: ../../packages/agent-protocol
@miai/connectors	link: ../../packages/connectors
@miai/presets	link: ../../packages/presets
@miai/runtime	link: ../../packages/runtime
@miai/wallet-adapter	link: ../../packages/wallet-adapter

The transitive third-party runtime tree pulled in by `pg` is minimal and stable: `pg-pool@3.14.0`, `pg-protocol@1.15.0`, `pg-types@2.2.0` (plus `pg-connection-string`, `pgpass`, `pg-int8`). That is the complete database stack – there is nothing else between the app and Postgres.

13.3 Internal workspace dependency graph

`pnpm-workspace.yaml` enrolls `apps/*` and `packages/*` and **explicitly excludes** `apps/mobile-shell` (`- "!apps/mobile-shell"`, commented "Expo / RN stays self-contained"). The internal graph (from each package's `dependencies`) is a clean DAG with `@miai/runtime` as the aggregation hub:

```

@miai/agent-protocol → (leaf; no workspace deps)
@miai/wallet-adapter → (leaf; no workspace deps)
@miai/connectors    → (leaf, but has one 3rd-party dep: pg ^8.22.0)
@miai/presets       → @miai/connectors
@miai/runtime       → @miai/agent-protocol, @miai/connectors, @miai/presets, @miai/wallet-adapter
@miai/web (app)     → all five packages above
apps/runtime-app    → @miai/runtime, @miai/wallet-adapter
apps/connectors-app → @miai/connectors

```

Two leaf packages (`agent-protocol`, `wallet-adapter`) have **zero dependencies of any kind** – pure TypeScript compiled to `dist/`. `@miai/connectors` is the only package besides the web app that declares a third-party runtime dependency (`pg`), because it seals/persists OAuth tokens. `@miai/presets` is notable: its shipped code is machine-generated – `packages/presets/src/generated-presets.ts` is **11,761 lines** (produced by `scripts/generate-presets.mjs`), so a large slice of the "runtime" LOC is generated, not hand-maintained, and carries no external dependency.

13.4 Per-package dependency inventory

Every internal package is ESM (`"type": "module"`), private, versioned `0.1.0`, and exposes a typed `dist/` endpoint via the `exports` map. Dependencies as declared:

PACKAGE	RUNTIME DEPS	DEV DEPS
@miai/agent-protocol	—	typescript ^5.7.3
@miai/wallet-adapter	—	typescript ^5.7.3

PACKAGE	RUNTIME DEPS	DEV DEPS
@miai/connectors	pg ^8.22.0	@types/node ^20.17.10 , @types/pg ^8.20.0 , typescript ^5.7.3
@miai/presets	@miai/connectors workspace:*	typescript ^5.7.3
@miai/runtime	@miai/agent-protocol , @miai/connectors , @miai/presets , @miai/wallet-adapter (all workspace:*)	typescript ^5.7.3
apps/connectors-app (@miai/connectors-app)	@miai/connectors workspace:*	@types/node ^20 , typescript ^5.7.3
apps/runtime-app (@miai/runtime-app)	@miai/runtime , @miai/wallet-adapter (workspace:*)	@types/node ^20 , typescript ^5.7.3

`apps/runtime-app` and `apps/connectors-app` are thin deployable wrappers (queue-consumer worker / OAuth-callback surface) that compile with bare `tsc` and run with `node dist/...js` – no framework, no bundler.

13.5 Dev, build and test toolchain

Root `package.json` `devDependencies`: `@playwright/test ^1.54.2` and `tsx ^4.19.3`. `apps/web` `devDependencies` carry the rest of the toolchain:

DEV DEPENDENCY	DECLARED	RESOLVED	PURPOSE
typescript	^5	5.9.3	TS 5 compiler; every package typechecks via <code>tsc --noEmit</code> .
tailwindcss	^3.4.1	3.4.19	Utility CSS – the only styling system (no CSS-in-JS runtime).
postcss	^8	8.5.25 (direct); 8.4.31 also present transitively via <code>next</code>	Tailwind's PostCSS pipeline.
eslint	^9	9.39.5	Flat-config ESLint 9.
eslint-config-next	15.1.0 (exact pin)	15.1.0	Next.js lint ruleset.
@eslint/eslintrc	^3	3.3.6	Compat layer for ESLint flat config.
@types/node	^20	20.19.43	Node 20 type defs.
@types/pg	^8.20.0	8.20.0	Types for the raw <code>pg</code> driver (no ORM means the app owns the types).

DEV DEPENDENCY	DECLARED	RESOLVED	PURPOSE
@types/react / @types/react-dom	^19	19.2.18 / 19.2.4	React 19 type defs.
@playwright/test (root)	^1.54.2	1.62.1	E2E harness – tagged @smoke/@functional/@uat/@handover.
tsx (root)	^4.19.3	4.23.4	TS execution for scripts/tests without a build step.

Unit tests use the **native** `node --test` runner (with `--experimental-strip-types` / a `web-test-register.mjs` import hook) rather than Jest/Vitest – another deliberately-absent-tooling choice that keeps the dev tree small. There is no bundler beyond Next's built-in one, no Babel in the workspace (Babel appears only inside the excluded Expo `mobile-shell`).

13.6 The NO-ORM posture – evidence

ADR 0003 mandates Postgres persistence, and the implementation is raw `pg` with hand-written parameterised SQL. `apps/web/src/lib/pg.ts` is the entire data-access foundation and confirms the posture concretely:

- A single shared `pg.Pool` is memoised on `globalThis` (`__miaiPgPool`) so Next.js HMR does not leak connections, with `max: 10`.
- The `pg` module itself is lazy-loaded via `require("pg")` inside `loadPgSync()` and cached on `globalThis.__miaiPgModule`, so importing `pg.ts` does not force the native driver to load when the app runs in the JSON file-store fallback (no `DATABASE_URL`).
- `getPool()` returns `null` when no `DATABASE_URL` / `MIAI_DATABASE_URL` is set – the file-store fallback path – so `pg` is effectively an **optional runtime dependency** at execution time.
- Queries go through a thin generic `query<R>(text, params)` wrapper (`pool.query<R>(text, params)`); callers write SQL by hand. SSL is decided per-URL (`sslFor` / `pgSslVerifyEnabled`), verifying certs by default, opt-out via `PG_SSL_REJECT_UNAUTHORIZED=0`.

There is **no Prisma / Drizzle / TypeORM / Sequelize / Knex / Mongoose** anywhere in the tree – a full-lockfile scan for those names returns nothing. Schema is managed by five plain SQL migration files (`001_init ... 005_consumer_reminders`), not a migration framework. The trade-off MyInstantAI inherits: no generated types or query builder (the team hand-writes SQL and owns correctness), in exchange for zero ORM lock-in, a trivially auditable data layer, and one fewer major dependency to keep patched.

13.7 Deliberately absent (and how the gap is filled)

A lockfile scan confirms the following are **not present as direct dependencies**:

- ORM / query builder** – none (see §13.6).
- HTTP client** (`axios`, `got`, `node-fetch`) – none; the platform uses the built-in `fetch`.
- State management** (`redux`, `@reduxjs/toolkit`, `zustand`, `jotai`, `recoil`, `mobx`) – none; React 19 + server components only.
- Component/UI kit** (`@mui`, `@chakra`, `antd`, `shadcn` runtime, `styled-components`, `@emotion`) – none; styling is Tailwind utility classes only.

- **Utility toolbelt** (`lodash`, `underscore`, `ramda`, `moment`, `date-fns`) – none directly. The only `Lodash*` in the tree is `lodash.merge@4.6.2`, pulled in **transitively** by lint tooling, not declared by the platform.
- **Redis / queue client** (`ioredis`, `redis`, `bullmq`) – **none**. Redis is optional and, when enabled, is spoken over the **Upstash REST API** using `fetch` (`apps/web/src/lib/redis.ts`, documented in its header as "fetch-only, no ioredis"). Env `UPSTASH_REDIS_REST_URL` + `UPSTASH_REDIS_REST_TOKEN` are optional; `apps/web/src/app/api/health/route.ts` treats Redis as "not_configured" on single-replica and only shared rate-limits/sessions across replicas when it is present. This is the sharpest illustration of the posture: even an infrastructure integration was added **without adding a package**.

For a due-diligence reader: the practical benefit is a very small `npm audit` / SBOM surface and near-frictionless framework upgrades; the practical cost is that capabilities other teams get "for free" from libraries (query building, rich UI primitives, a battle-tested Redis client) are hand-rolled here and must be maintained in-house.

13.8 Supply-chain surface and pinning

- **Lockfile:** `pnpm-lock.yaml`, `lockfileVersion: '9.0'`, 4,363 lines, resolving **~754 packages** in the `packages:` section (full transitive closure of the workspace, excluding the self-contained Expo app). Every transitive package is pinned to an exact version and integrity hash.
- **Package manager pinned two ways:** root `packageManager: "pnpm@9.15.0"`, and the Dockerfile activates the same via `corepack prepare pnpm@9.15.0 --activate`. Reproducible installs are enforced with `pnpm install --frozen-lockfile` in **every** context: the `Dockerfile` (both the `deps` and `builder` stages) and every CI workflow (`ci.yml`, `e2e-staging.yml`, `eval-nightly.yml`). A drifted lockfile fails the build rather than silently resolving new versions.
- **Node pinned by image, not by manifest:** the base image is `node:20-bookworm-slim`. Note there is **no** `engines` field in any `package.json`, so the Node 20 floor is enforced only at the container/CI layer — a local developer on a different Node major is not blocked by the manifest. Flagged in open gaps.
- `.npmrc` is empty (no custom/private registry, no `node-linker` override), so installs come from the public npm registry with pnpm's default strict, symlinked `node_modules` — transitive dependencies are not hoisted into reach of application code by default, which narrows the accidental-import surface.
- **Advisory scanning:** SonarCloud is wired (`sonar-project.properties`) as an advisory gate; the required merge gate is the `quality` job.

13.9 Version-drift and skew observations (for the acquiring team)

Because most ranges use caret (`^`), the resolved lockfile has moved ahead of the headline specifiers. These are worth noting at handover:

- **Next.js:** declared `^15.2.8` but **resolved 15.5.22**. Any prior documentation (including the Licence Proposal) citing "15.2.8" is quoting the range floor, not what installs today — the platform is materially newer.
- `eslint-config-next` is pinned at exactly `15.1.0` while `next` resolves to `15.5.22`. This lint-config/framework minor skew is harmless today but should be re-aligned on the next Next.js bump.
- `zod` declared `^3.24.2`, resolved `3.25.76`; `tailwindcss` `^3.4.1` → `3.4.19`; `typescript` `^5` → `5.9.3`; `@playwright/test` `^1.54.2` → `1.62.1`; `tsx` `^4.19.3` → `4.23.4`. All in-range, all recent.
- **Dual React majors in the repo:** the workspace runs **React 19.2.8**, but the excluded `apps/mobile-shell` (Expo `~52.0.0`, `react-native 0.76.3`) pins `react 18.3.1`. These are two separate dependency islands (mobile-shell installs its own tree and is not in `pnpm-workspace.yaml`), so there is no version conflict — but the acquiring team maintains two React lineages if the Expo shell is kept.

- `postcss` appears at two versions (direct `8.5.25`, transitive `8.4.31` via `next`); pnpm de-dupes these correctly, no action needed.

13.10 The vendored catalogue as the largest "dependency" (scale discipline)

The most valuable asset in the repo is not an npm package — it is **vendored in-tree** and version-controlled alongside the code, so it carries none of the fetch/patch risk of an external dependency but dominates the size story. Re-measured at commit `fb97438`:

- **Total authored:** 406,319 lines across 1,212 files (`ts/tsx/js/mjs/json/md/sql/css`, excluding `node_modules/.next/dist/.git`).
- **Catalogue IP (data dependency):** `data/catalog` = 555 JSON files / **290,061 lines**, plus `data/catalog-consumer` = 18 files / **9,447 lines** → **299,508 lines** of catalogue — roughly **74% of the entire codebase**. This is the 500-SKU agent catalogue (100 families × 5 markets; `~555` files on disk includes retained legacy ZA aliases per ADR 0001) that the hand-written runtime consumes.
- **Hand-written platform runtime:** ~75.9K LOC of TS/TSX/JS/MJS (of which `packages/presets/src/generated-presets.ts` alone is 11,761 generated lines).

For a dependency review this matters two ways: (1) any statement of platform size that headlines only the ~75.9K runtime **undercounts the deliverable by ~4×** and misrepresents what is being licensed; and (2) the catalogue is loaded from committed files, not pulled from a registry — so there is zero external supply-chain exposure for the single largest thing MyInstantAI is acquiring. The repo has **grown** since the Licence Proposal's figures (~398K lines / 1,021 files / ~298K catalogue); the current measured numbers above supersede them.

13.11 Summary for handover

The dependency manifest is small, current, exactly pinned, and reproducibly installed. The runtime attack surface is six third-party packages plus their tight transitive closure; the database stack is the `pg` family and nothing else; optional infrastructure (Redis) is integrated over `fetch` with no added package. The chief risks the acquiring team should note are all in the "skew" bucket, not the "bloat" bucket: framework versions that have floated ahead of documented floors, a `eslint-config-next` pin lagging `next`, the absence of an `engines` field to enforce Node 20 outside CI/Docker, and a second React major living in the excluded Expo shell. None of these block operation; all are cheap to reconcile.

14 Performance & Scalability

This section documents the platform's runtime performance characteristics as they exist in code at commit `fb97438`, grounded in the actual execution engine (`packages/runtime`), the data/cache tier (`apps/web/src/lib/pg.ts`, `redis.ts`, `store.ts`, `knowledge.ts`), the streaming layer (`sse.ts`, `chat-stream.ts`), and the deployment manifests (`Dockerfile`, `railway.toml`). It states honestly where the design scales cleanly and where the current bottlenecks and single-process assumptions are.

14.1 What "scale" means for this platform

The platform is two very different assets with very different performance profiles, and any capacity conversation must keep both in view:

ASSET	MEASURED SIZE (RE-MEASURED AT <code>FB97438</code>)	RUNTIME COST PROFILE
Catalogue IP (<code>data/catalog</code> + <code>data/catalog-consumer</code>)	299,508 lines JSON across 573 files (<code>data/catalog</code> = 290,061 lines / 555 files incl. <code>index.json</code> / <code>families.json</code> / <code>market-packs.json</code> + 552 <code>*.agent.json</code> ; <code>data/catalog-consumer</code> = 9,447 lines / 18 files)	Static JSON on disk; read per request , largest package $\approx$ 47 KB (<code>oceania-payroll-queries.agent.json</code>). Not CPU/DB-bound — it is I/O + <code>JSON.parse</code> .
Platform runtime (hand-written <code>ts/tsx/js/mjs</code>)	$\approx$ 75,768 LOC across <code>apps/*</code> , <code>packages/*</code> , <code>scripts</code> , <code>e2e</code> (includes the 11,761-line generated <code>packages/presets/src/generated-presets.ts</code>)	The live path: Next.js request $\rightarrow$ <code>runTurn()</code> $\rightarrow$ model provider $\rightarrow$ Postgres. This is where latency and concurrency live.
Whole repo (authored)	406,319 lines / 1,212 files (<code>ts,tsx,js,mjs,json,md,sql,css</code> , excl. <code>node_modules</code> / <code>.next</code> / <code>dist</code> / <code>.git</code>)	—

The salient point for capacity planning: **the 500-agent catalogue does not grow the runtime working set linearly**. Adding markets/families adds files on disk, not hot code paths — one file is read per turn regardless of catalogue size (see §14.10). The runtime cost is per-conversation-turn, not per-catalogue-entry.

14.2 Per-turn cost model (where the wall-clock goes)

A single Studio/consumer chat turn (`runTurn` in `packages/runtime/src/index.ts:2144`, entered from `apps/web/src/app/api/chat/route.ts` and the `consumer-turn` / `channel-turn` / `ask-turn` call sites) does, in order:

- Auth + role gate** — `requireAuth` + `requireRole(auth, "agent")` (JWT/OIDC verify via `jose`, or mock).
- Catalogue read** — `getAgentPackage(id)` $\rightarrow$ `fs.readFile` + `JSON.parse` of `{id}.agent.json` (uncached, every turn — see §14.10).
- Rental read-through** — `getWorkspaceAgent` issues a Postgres `SELECT ... FROM miai_rentals WHERE workspace_id=$1 AND agent_id=$2` when a pool exists (§14.4).
- Knowledge compose + retrieve** — `getComposedKnowledge` (in-memory after first hydrate) then, for live models, `selectKnowledgeForPromptAsync` chunk-scoring / optional embedding (§14.7).
- Balance check** — `wallet.getBalance` (in-process for mock; one HTTP call for `http` mode). **Zero balance short-circuits before any provider call** (§14.8).

6. Model call(s) — one initial `modelAnswer` plus up to `maxToolRounds` tool-round follow-ups (§14.6, §14.9). This dominates wall-clock (provider latency).

7. Debit + persist — `wallet.debit` (in-process or one HTTP call), plus row-level Postgres writes for audit/turn transcript.

So the fixed per-turn overhead outside the LLM is roughly: **1 file read/parse + 1–2 Postgres round-trips + 0–2 wallet round-trips + in-process chunk scoring**. Everything else is provider latency, which is streamed (§14.6).

14.3 Postgres connection pooling (`lib/pg.ts`)

A single `pg.Pool` is memoised on `globalThis` (`__miaiPgPool`) so HMR and repeated imports reuse one pool per process:

```
g.__miaiPgPool = new pg.Pool({
  connectionString: url,          // DATABASE_URL || MIAI_DATABASE_URL
  ssl: sslFor(url),               // verify certs for remote; false for localhost
  max: 10,                        // ← the only pool sizing knob set
});
```

Observations grounded in the file:

- max: 10 per replica**, and **no other pool parameters are set** — no `min`, `idleTimeoutMillis`, `connectionTimeoutMillis`, or `statement_timeout`. The `pg` library defaults therefore apply (idle 10 s, **no connection-acquire timeout, no statement timeout**). Under burst load beyond 10 concurrent DB-touching turns, additional turns queue on connection acquisition **indefinitely** rather than failing fast.
- TLS**: `sslFor()` / `pgSslVerifyEnabled()` verify the server certificate for any non-localhost URL unless `PG_SSL_REJECT_UNAUTHORIZED=0` (which requires a boot-hardening ACK in production). Good default for managed Postgres (Railway/Azure/Neon).
- No ORM** — all access is parameterised SQL through `query()` (`pg.Pool.query`). `query()` throws if `DATABASE_URL` is unset, which is why every store call guards on `getPool()` / `databaseUrl()` first and can fall back to the file store.
- Readiness**: `pingPool()` runs `SELECT 1` for `/api/health` without hydrating data.

Hot-table indexing (from `apps/web/migrations/001_init.sql`) backs the per-turn queries: `miai_rentals` PK (`workspace_id, agent_id`) (the exact `getWorkspaceAgent` predicate), `miai_audit (workspace_id, at DESC)`, `miai_turns (workspace_id, at DESC)` and `(correlation_id)`. Consumer memory/graph tables (`003` – `005`) add composite PKs plus dedupe/recency indexes. Tenant isolation is a `workspace_id` / `consumer_id` predicate on indexed columns, so isolation and index usage coincide.

14.4 Horizontal scaling & shared state

The server is designed to run stateless behind a load balancer (Railway today, Azure Container Apps as the stated target). Whether that actually holds depends on **which shared backend is configured**, and the codebase is deliberately explicit about this:

STATE	MULTI-REPLICA BEHAVIOUR	BACKING
Rentals / workspace agents	Consistent — <code>getWorkspaceAgent</code> / <code>listWorkspaceAgents</code> read through to Postgres on every call ("always read-through so multi-replica sees sibling writes"; <code>store.ts:389</code>); the in-memory <code>Map</code> is a cache/fallback only.	Postgres <code>miai_rentals</code>

STATE	MULTI-REPLICA BEHAVIOUR	BACKING
Rate limits + chat/session counters	Consistent only with Upstash — <code>rateLimit</code> uses Redis <code>INCR</code> + <code>EXPIRE</code> when <code>redisAvailable()</code> , otherwise a per-replica in-process Map (<code>__miaiRateLimit</code>).	Upstash Redis REST (optional)
Knowledge sources	Stale across replicas — <code>knowledge.ts</code> hydrates once (<code>__miaiKnowledgeHydrated</code>) via <code>SELECT ... FROM miai_knowledge_sources</code> with no <code>WHERE</code> clause (loads every tenant's KB into the heap) and is not read-through. A KB edit on replica B is invisible to replica A until restart.	Postgres, hydrate-once
Wallet balances (mock mode)	Per-replica — <code>MockWalletAdapter</code> holds balances in an in-process <code>Map</code> ; only <code>MIAI_WALLET_MODE=http</code> gives a shared ledger.	in-process / external HTTP
Embedding cache	Per-replica LRU (<code>CACHE_MAX=2048</code> , FNV-1a keyed), <code>embeddings.ts</code> .	in-process

The Redis client (`lib/redis.ts`) is intentionally minimal: **Upstash REST over `fetch` only** (no `ioredis`), gated on `UPSTASH_REDIS_REST_URL` + `UPSTASH_REDIS_REST_TOKEN`. `REDIS_URL` is accepted as a documented alias for a future native client but is **unused in v1**. Every helper (`redisIncr` / `redisTtl` / `redisGet` / `redisSet` / `redisDel`) no-ops to `null` when unconfigured so single-replica dev works with zero Redis.

Net: two of the three shared-state stores (rentals, rate limits with Upstash) scale horizontally cleanly; **knowledge hydration is the one store that does not** — it is both a per-replica memory-growth vector (all tenants' KB in every heap) and a cross-replica staleness source. This is the most material scaling caveat in the data tier (see §14.11).

14.5 Rate limiting (`lib/security.ts` `rateLimit`)

A token-bucket limiter with a **fail-closed** posture when Redis is present:

```
if (redisAvailable()) {
  const count = await redisIncr(`miai:rl:${key}`, ttlSec);
  if (count === null) return { ok: false, retryAfterSec: Math.max(1, ttlSec) }; // Redis broken →
  deny, do NOT fall open to per-replica Map
  if (count > opts.limit) { /* ... 429 with TTL-derived Retry-After ... */
    return { ok: true };
  }
  return rateLimitInProcess(key, opts); // single-replica Map
}
```

Configured per-route limits (all 60 s windows):

ROUTE / SURFACE	KEY	LIMIT / 60 S
<code>oauth/[connector]/start</code>	<code>oauth-start:{ws}:{user}</code>	20
<code>app/chat</code> (embed key)	<code>app:{key}</code>	30
<code>handlers/embed-chat</code>	<code>embed:{key}</code>	30
<code>ask/chat</code>	<code>ask:{session}</code>	40

ROUTE / SURFACE	KEY	LIMIT / 60 S
consumer/chat	consumer:{wallet}:{agent}	30
consumer/telegram/webhook	consumer:tg:{chat}:{agent}	30
consumer/connectors/[connector]/start	consumer-oauth-start:{consumer}	20
knowledge/crawl	knowledge-crawl:{ws}:{user}	10

Honest gap: the **authenticated Studio route** `/api/chat` is **not rate-limited** (grep-confirmed) — it relies on OIDC + `requireRole("agent")` + wallet metering as its throttle. That is defensible (metering caps spend), but there is no per-tenant request-rate ceiling on the primary B2B chat surface, so a single authenticated tenant can drive concurrency up to the pool/provider limits.

14.6 SSE token streaming (`lib/sse.ts` , `lib/chat-stream.ts`)

Streaming is real end-to-end, not simulated buffering:

- **Transport:** `sseStreamResponse()` wraps a `ReadableStream` and always `controller.close()` s in a `finally`. Headers are streaming-correct: `content-type: text/event-stream`, `cache-control: no-cache, no-transform`, and `x-accel-buffering: no` (disables nginx/proxy buffering so tokens flush immediately).
- **Event choreography** (`streamChatTurn`): `meta → (delta | status)* → done | paused+done | error`, shared by `app/embed/consumer` routes so the contract lives in one place.
- **Upstream is genuine provider SSE:** the OpenAI/Gateway/Azure/Anthropic adapters implement `streamComplete()` with `stream: true`, `accept: text/event-stream`, and read `res.body.getReader()` parsing `data:` frames (`index.ts:1887`). `modelAnswer` forwards each `delta` chunk through `onDelta` into the SSE `send("delta", ...)`. Provider tokens reach the browser as they arrive — Time-to-First-Token is provider-bound, not blocked on full completion.
- **Mock/workflow replies** (deterministic paths, `finishWorkflow`) are pseudo-streamed by splitting the finished string on whitespace and emitting ~8-char buffers, so the client UX is identical whether or not the provider streams.

Because streaming holds an open HTTP connection for the turn's duration, each in-flight turn occupies one server worker/connection; the request-worker is not released until the turn (including tool rounds) completes.

14.7 RAG / knowledge cost control (`knowledge-retrieve.ts` , `embeddings.ts` , `lib/knowledge.ts`)

The retrieval layer's job is to keep the prompt (and therefore token spend and latency) bounded regardless of KB size:

- **Compose budget** (`composeKnowledge`): `KNOWLEDGE_MAX_CHARS` default **80,000**. Tenant-uploaded sources are placed first; the catalogue template is truncated to a small budget (1,500–4,000 chars) when the tenant has their own knowledge, so onboarding docs aren't crowded out.
- **Runtime prompt budget** (`runTurn`): `RUNTIME_KNOWLEDGE_CHARS` default **40,000**.
- **Chunking** (`splitChunks`): split on markdown headings, KB **capped at 80,000 chars** before chunking, chunks ≥ 30 chars; `packChunks` caps each chunk at **1,600 chars** and packs top-K within a char budget.
- **Lexical (default):** `retrieveKnowledgeChunks` scores chunks by stop-word-filtered term overlap plus domain heuristics (hours/refund/PTO/price...), soft-bans meta/guardrail chunks (`score = -1`). `topK=6`, `maxChars=12,000` by default; `selectKnowledgeForPromptAsync` uses `topK=8`, `maxChars=min(budget, 24,000)`.

- **Hybrid (opt-in):** `retrieveKnowledgeChunksHybrid` embeds `[query, ...chunks]` in one `embed()` call, normalises lexical + cosine scores, blends `0.6*sem + 0.4*lex`, requires signal (`sem<0.15 && lex<0.15 → 0`, filter `> 0.12`), and **falls back to lexical if the embedder throws or returns nothing.**
- **Embedder gating** (`createEmbedderFromEnv` / `semanticRetrievalEnabled`): `RUNTIME_SEMANTIC_RETRIEVAL` – `0/false/off` disables; `1/true/on/local` forces on; unset / `auto` (the default) enables semantic **only if a remote embedding key exists** (`EMBEDDING_API_KEY` / `OPENAI_API_KEY`). Remote uses `text-embedding-3-small` (batched 64, L2-normalised, cached); the `LocalHashEmbedder` (384-dim, deterministic) is only used when explicitly allowed. `MockModelAdapter` bypasses retrieval entirely and receives a straight KB prefix slice so evals stay deterministic.

Correction to the internal "prompt-stuff below ~6,000 chars / vector retrieval above" characterisation: there is **no ~6,000 -char switch anywhere in the code** (repo-wide grep for `6000` / `6_000` / `6,000` returns only unrelated constants). The actual switch between "stuffing" and "retrieval" is **model type + embedder presence**, not a KB size threshold: the MockModel prompt-stuffs (slice to the 40 k budget); live models always run retrieval (lexical, or hybrid when an embedding key is configured). Lexical retrieval also runs regardless of KB length. This is flagged in open gaps.

Cost implication: there is **no persistent vector index/DB** – chunking, scoring, and (in hybrid) embedding are recomputed **per turn** on the in-process KB string. The embedding cache is per-process and keyed by chunk text, so repeated chunks re-hit the cache but the query vector is always fresh. This is fine at demo/pilot KB sizes (≤ 80 k chars, ≤ 20 docs/agent) but is CPU work on the Node main thread that grows with KB size.

14.8 Fail-closed metering (`runTurn` + `wallet-adapter`)

Metering short-circuits the provider call at zero balance, which is both a cost control and a performance guard (no wasted upstream latency):

```
const bal = await wallet.getBalance(req.workspaceId);
if (!skipDebit && bal.tokens ≤ 0) {
  return { assistantMessage: wf(lang, "paused_no_tokens"), tokensDebited: 0,
    balance: 0, state: "paused_no_tokens", paused: true, /* ... */ }; // NO model call
  made
}
```

- The **bounded plan→act→observe loop** is `maxToolRounds = Math.min(5, Math.max(1, RUNTIME_MAX_TOOL_ROUNDS ?? 3))` – **default 3 tool rounds, hard-clamped to 1–5** – so a single turn can make at most `1 + maxToolRounds` provider calls. This caps both unbounded tool recursion and unbounded spend/latency per turn.
- **Token accounting:** provider-reported `usage.totalTokens` is summed across the initial call + tool-round follow-ups (`turnUsageTotal`); when the provider omits usage (or for the mock), it falls back to `estimateTurnTokens(model, charsIn, charsOut) = max(200, (charsIn+charsOut)/4) × a per-model multiplier (opus ×3, gpt-4o ×1.6, mini ×0.5, flash ×0.4)`. Debits carry an `idempotencyKey` (`{ws}:{agent}:{ts}:{msgLen}`) so retries don't double-charge.
- **Sandbox backstop:** `createModelAdapter` degrades to the MockModel after `SANDBOX_MODEL_TURN_CAP` (default 1,000) real-provider turns per process when `SANDBOX_MODE=1`, so an eval run cannot run up an unbounded bill.
- **HTTP wallet mode** (`MIAI_WALLET_MODE=http`) adds **two sequential external round-trips per turn** on the critical path – `getBalance` before the model call and `debit` after – each wrapped in a 15 s `AbortController` timeout; `402/409` are treated as "paused", not thrown. This is the correct semantics but places an external dependency in-line with every turn.

14.9 Request body caps & other request-level guards (`middleware.ts`)

- **1 MiB body cap** (`MIAI_MAX_BODY_BYTES`, default `1_048_576`) enforced in middleware via `content-length`; oversize → `413 Payload too large`. Note this checks the **header only**, so a chunked/absent-`content-length` request is not pre-gated at the edge (the downstream `req.json()` + Zod parse is the backstop).
- **Per-request CSP nonce** minted in middleware for every non-static route.
- **OIDC bearer pre-check** in middleware when `MIAI_AUTH_MODE=oidc` (401 before the route runs) for non-public paths.
- **Ingestion caps**: website crawl (`knowledge/crawl`) sets `maxDuration = 60` and `ingest.ts` truncates fetched text to 100,000 chars.

14.10 Catalogue load strategy (`lib/catalog.ts`)

- **Directory**: `CATALOG_DIR` (default `../../data/catalog`), consumer `CONSUMER_CATALOG_DIR` (default `../../data/catalog-consumer`).
- **Index/families are mtime-memoised** — `listCatalog()` / `families()` cache `index.json` / `families.json` in a `FileMemo<T>{ mtimeMs, data }` and only re-read/re-parse when the file's `mtimeMs` changes. So marketplace listing is effectively O(1) after first load regardless of the 500-SKU list size.
- **Per-agent packages are NOT cached** — `getAgentPackage(id)` does a fresh `fs.readFile` + `JSON.parse` of `{id}.agent.json` on **every call** (with a consumer-dir fallback), then `loadAgentPackage()` validates the `miai.agent-package/v1` format. Since this is on the hot chat path, **every turn parses up to ~47 KB of JSON**. In practice the OS page cache serves the read cheaply and `JSON.parse` of 47 KB is sub-millisecond, but there is no in-process memo, so this is pure repeated work and the most obvious low-risk optimisation (mtime-memo it exactly like the index).

14.11 Stateless runtime & deployment (`Dockerfile`, `railway.toml`, `next.config.ts`, `instrumentation.ts`)

- **Container**: multi-stage `node:20-bookworm-slim`; the `runner` stage installs `gosu`, fixes ownership of the `/data` volume, then **drops from root to the `node` user** via the entrypoint before starting. Runs `next start -H 0.0.0.0 -p ${PORT:-3000}` (i.e. the standard Next server; `output: "standalone"` is **not** used, though `outputFileTracingRoot` is set for the monorepo). `serverExternalPackages: ["pg", "jose"]` keeps native/runtime deps out of the bundle.
- **Railway**: Dockerfile builder, health check `GET /api/health` (timeout 30 s), `restartPolicy = ON_FAILURE` max 5.
- **Health signal is load-shed-aware**: `/api/health` returns **503** when boot hardening fails, when the store ping fails/can't hydrate ("degraded"), or when **Redis is configured but unreachable** (because rate-limits/sessions then fail closed) — so an orchestrator will pull a half-broken replica out of rotation.
- **Statelessness holds** because the durable state is Postgres (rentals read-through, audit/turns/consumer memory) and the optional Upstash shared cache; local `/data` file stores and in-process `Map`s are fallbacks for single-replica/dev. The documented caveat (§14.4) is knowledge hydration.
- **Migration target**: Azure Container Apps + Azure OpenAI + App Insights (`infra/azure` Bicep, `AzureOpenAIModelAdapter`, `MIAI_MODEL_MODE=azure`). Container Apps' KEDA/HTTP autoscaling fits the stateless-replica model. `instrumentation.ts` runs boot hardening on the Node runtime only and deliberately avoids importing `pg` (migrations run lazily on first Postgres touch, guarded by a memoised `ensuring` promise in `migrate.ts`).
- **Worker offload is scaffolded, not wired**: `apps/runtime/src/worker.ts` is a **16-LOC** heartbeat scaffold; web routes still call `@miai/runtime runTurn` **in-process**. So today the same Next.js process serves UI, API, and the full tool loop — long tool loops occupy request workers. The scaffold exists to move tool loops onto a queue in Container Apps later.

14.12 Bottlenecks, assumptions & honest caveats

Bottlenecks (ranked):

1. **Knowledge hydrate-once, whole-table, per-replica** (`knowledge.ts`) – loads every tenant's KB into each replica's heap with no `WHERE` /lazy-load and is not read-through. This is simultaneously a **memory-growth vector** (heap scales with total corpus × replicas) and a **cross-replica staleness bug** (KB edits invisible until restart). Contrast with rentals, which are correctly read-through. This is the top item to fix before multi-replica production.
2. `getAgentPackage` **uncached disk read + parse on every turn** – trivially memoisable by mtime (the index already does this).
3. **Postgres pool** `max: 10` **with no acquire/statement timeout** – beyond 10 concurrent DB-touching turns, turns queue on connection acquisition with no timeout; a single slow query can stall the pool. Set `connectionTimeoutMillis` / `statement_timeout` and size `max` to the deployment.
4. **HTTP wallet adds 2 in-line external round-trips per turn** (15 s timeout each) on the critical path when `MIAI_WALLET_MODE=http`.
5. **In-process tool loop occupies request workers** – no queue offload yet (`apps/runtime` is a scaffold); long multi-round turns hold a streaming connection + worker for their full duration.
6. **RAG recomputed per turn, no persistent vector index** – chunk scoring (always) and embedding (hybrid) run on the Node main thread against the in-memory KB; grows with KB size. Fine at pilot sizes, not a substitute for a real vector store at large corpora.
7. `/api/chat` (Studio) **unthrottled** – only auth + wallet gate it; no per-tenant request-rate ceiling on the primary B2B surface.
8. **In-process rate-limit Map is per-replica without Upstash** – limits are effectively `limit × replica_count` unless Upstash is configured.

Assumptions the design bakes in:

- Stateless horizontal scaling **assumes Postgres + Upstash are the shared backends**; without Upstash, rate limits and any in-process caches are per-replica, and without Postgres the platform runs single-replica on the `/data` file store.
- Node 20 single event loop: JSON parsing, chunk scoring, and `LocalHashEmbedder` math are main-thread CPU; there is no worker-thread/queue offload in the current code.
- `estimateTurnTokens` is an explicit "**prototype multipliers**" char/4 heuristic used only when the provider omits usage – a metering approximation, not exact billing, on those paths.
- No load/benchmark artefacts (k6/Artillery/throughput numbers) exist in the repo; all figures above are **structural** (limits, pool sizes, round-trip counts) rather than measured throughput/latency.

15 Testing & Evals

The quality system is layered and **evidence-first**: a fast, deterministic gate that blocks merges, a broad non-blocking automation tier against Railway staging, and — the crown jewel — an **8,526-case behavioural eval corpus baked into the 500-agent catalogue IP** (plus 302 consumer cases), run zero-token in CI and separately sampled against live models. No third-party test runner is used: the entire suite is `node:test` (`node --test`) for unit/contract work and Playwright for E2E — `grep` over every `package.json` finds no `jest`, `vitest`, `mocha`, or `ava`.

Scale discipline matters here. The evals are not a side artifact — they are ~2.20 MB of hand-authored expectation spec embedded in the **299,508-line catalogue** (`data/catalog` + `data/catalog-consumer`), itself the largest asset in the 406,319-line / 1,212-file repo (measured at HEAD `fb97438`; grown from the ~398K/1,021-file figure in the Licence Proposal). When this section says "8,500+ evals," that is a real count of `expect` blocks across 552 agent packs, not an estimate.

15.1 Test-runner stack & topology

LAYER	TOOL	ENTRY	RUNS WHERE
Unit / contract	<code>node --test</code> (Node 22+)	<code>pnpm test</code> (fans out per package)	CI <code>quality</code> job + local
TS/TSX web units	<code>node --test</code> + <code>--experimental-strip-types</code> via a custom register hook	<code>pnpm test:web</code>	CI + local
Catalogue integrity	plain Node script	<code>pnpm catalog:integrity</code>	CI <code>quality</code> job
Static / mock evals	Node + <code>MockModelAdapter</code>	<code>pnpm eval:suite:static</code>	CI <code>quality</code> job
E2E (API + UI)	Playwright (chromium only)	<code>pnpm test:e2e*</code>	Non-blocking staging CI + local
Live-LLM sample	Node + real model adapter	<code>pnpm eval:live</code> , <code>pnpm smoke:live-llm</code>	Nightly (opt-in), never a gate

There is **no `webServer` block** in `playwright.config.ts` — E2E runs assume an already-running target (defaults to `https://miaiweb-production.up.railway.app`), so specs are integration tests against a deployed app, not an in-process harness.

15.2 Unit & contract tests (`node --test`)

45 `*.test.mjs` files, ~323 `test()` / `it()` call-sites, split across four workspaces:

SUITE	FILES	~CASES	PNPM SCRIPT	FOCUS
<code>apps/web/test</code>	29	~204	<code>test:web</code>	routes, auth, consumer memory/OIDC, security, DSAR

SUITE	FILES	~CASES	PNPM SCRIPT	FOCUS
<code>packages/runtime/test</code>	9	~90	<code>test:runtime</code>	workflows, retrieval, guardrails, metering, adapters
<code>packages/connectors/test</code>	6	~21	<code>(@miai/connectors test)</code>	OAuth fail-closed, SSRF, webhook sig, retry
<code>packages/wallet-adapter/test</code>	1	~8	<code>test:wallet</code>	prepaid metering / fail-closed debit

The **strip-types** register harness. `apps/web` unit tests import real Next.js source (TS/TSX using the `@/*` alias) with **no bundler**. Two scripts make that work under bare Node:

- `scripts/web-test-register.mjs` – registers a module loader hook via `node:module`.
- `scripts/web-test-hooks.mjs` – a `resolve()` hook that rewrites `@/*` specifiers to `apps/web/src/*`, probing `.ts/.tsx/.js/.mjs`.

Invocation (from root `package.json`): `node --import ./scripts/web-test-register.mjs --experimental-strip-types --test apps/web/test/*.test.mjs`. **Node 22+ is mandatory** – `--experimental-strip-types` is the reason `ci.yml` pins `node-version: 22` with an explicit comment. This lets contract tests exercise production auth/schema code directly (e.g. `apps/web/src/lib/auth.ts`, `api-schemas.ts`) rather than a reimplementation.

Notable web unit files:

- `api-contract.test.mjs` – the closest thing to an API spec that runs without a server. Asserts `data/catalog/index.json` parses and lists **≥ 500 agents**; `loadAgentPackage()` validates `us-hr-helpdesk` with **≥ 8 evals**; `resolveAuth()` rejects a missing Bearer with `AuthError 401 / "Missing Bearer token"`; `isPublicApiPath()` allowlists `/api/health`, `/api/consent`, `/api/catalog[...]`, `/api/v1/openapi` but **blocks** `/api/rent`; and seven mutating POST bodies (`configureBodySchema`, `dsarEraseBodySchema`, `knowledgeCrawlBodySchema`, `workspaceMemberInviteBodySchema`, ...) reject malformed input with **400**. Exposed as its own script `pnpm test:api-contract`.
- `consumer-oidc-session.test.mjs`, `oidc-isolation.test.mjs` – consumer "Sign in with Google" OIDC session + cross-tenant isolation.
- `security-boot.test.mjs` (boot hardening / dual-flag rails), `embed-key-security.test.mjs` (`mia_pk_` data-key), `dsar-erase.test.mjs`, `pii-redact.test.mjs`, `pg-ssl.test.mjs`, `paystack-topup.test.mjs`, `tenant-brands.test.mjs`, `memory-semantic.test.mjs` (RAG retrieval), `residual-punchlist.test.mjs`.

Runtime tests of note: `knowledge-retrieve.test.mjs` (prompt-stuffing → vector retrieval switch), `wallet-metering.test.mjs` (fail-closed at zero balance), `guardrails-consumer.test.mjs`, `azure-adapter.test.mjs`, and the `flagship-depth-* workflows.test.mjs` trio. **Connectors:** `oauth-fail-closed.test.mjs`, `ssrf.test.mjs` (DNS-pinned outbound guard), `webhook-sig.test.mjs` (HMAC-SHA256).

15.3 Playwright E2E (`e2e/`)

`playwright.config.ts`: `testDir: ./e2e`, `fullyParallel`, chromium-only project, 60 s test / 15 s expect timeouts, `retries: 1` and `workers: 2` under CI, `trace: on-first-retry`, `screenshot: only-on-failure`, `video: retain-on-failure`. **Mock-rails identity is injected as HTTP headers** (`x-workspace-id` / `x-user-id` / `x-roles`, defaulting to `demo-workspace` / `e2e-user` / `owner,operator`), so API specs run without a real OIDC token against staging.

33 spec files (18 API · 8 functional · 4 UAT · 3 UI), **66 static `test()` blocks** that expand at runtime via parameterization (markets, Go-live hero matrix). Tag distribution (files carrying each tag): `@smoke` 10, `@functional` 18, `@uat` 7, `@handover` 20. Tag → command mapping lives in `package.json` (`test:e2e:smoke|functional|uat|acceptance|handover`), plus `smoke:staging` / `uat:staging` / `handover:staging` which pin the Railway base URL).

Representative spec map (from `docs/TESTING.md`):

SPEC	TAG	ASSERTS
<code>e2e/api/health.spec.ts</code>	<code>@smoke</code>	<code>/api/health</code> not failing; store ping
<code>e2e/api/catalog.spec.ts</code>	<code>@smoke</code>	500 agents / 5 markets; US ~100
<code>e2e/api/embed.spec.ts</code>	<code>@smoke</code>	<code>agent.js</code> + SRI hash
<code>e2e/api/guardrail-refusals.spec.ts</code>	<code>@smoke</code>	jailbreak / card / cross-tenant refusals ×3
<code>e2e/api/webhook-sink.spec.ts</code>	<code>@smoke</code>	unsigned webhook → 401/403 (HMAC-only)
<code>e2e/api/dsar-contract.spec.ts</code>	<code>@functional</code>	DSAR export (no token leak); erase gated
<code>e2e/functional/b2b-onboarding.spec.ts</code>	<code>@functional @uat @handover</code>	<code>/get-started</code> wizard → catalogue checklist
<code>e2e/functional/markets-za-fold.spec.ts</code>	<code>@functional</code>	no ZA pack/chip; ZA ⊂ Africa
<code>e2e/uat/partner-demo-journey.spec.ts</code>	<code>@uat</code>	browse → rent → chat → install
<code>e2e/uat/golive-hero-matrix.spec.ts</code>	<code>@uat</code>	6 Go-live hero studio shells
<code>e2e/uat/acceptance-bar.spec.ts</code>	<code>@uat</code>	B+ ops + 500 SKUs + legal surfaces

15.4 The testing pyramid (L0–L8)

`docs/TESTING.md` codifies an eight-layer pyramid and, crucially, which layers **block a PR**:

LAYER	COMMAND	BLOCKS PR?
L0 Unit / contract	<code>pnpm test</code> , <code>test:api-contract</code>	Yes (via <code>pnpm run ci</code>)
L1 Catalogue integrity	<code>pnpm catalog:integrity</code>	Yes
L2 Static / mock evals	<code>pnpm eval:suite:static</code>	Yes
L3 Staging E2E smoke	<code>pnpm test:e2e:smoke</code>	No (soft)
L4 Functional	<code>pnpm test:e2e:functional</code>	No

LAYER	COMMAND	BLOCKS PR?
L5 UAT (automated)	<code>pnpm test:e2e:uat</code>	No
L6 Handover pack	<code>pnpm handover:staging</code>	No
L7 UAT (human)	<code>docs/UAT_CHECKLIST.md</code>	Manual sign-off
L8 Live-LLM sample	<code>pnpm eval:live --limit=N</code>	No (credit-gated)

The design intent is explicit: keep the blocking gate **fast and zero-cost** (units + integrity + mock evals), and run everything that needs a deployed app or model spend as **non-blocking** staging/nightly automation. `docs/TESTING.md` even advises promoting E2E `@smoke` to a required check only "after ~1–2 weeks of green runs."

15.5 The evals system (the quality IP)

This is the section MyInstantAI should weigh most heavily. Every catalogue agent ships an `evals` array; each case is a behavioural contract:

```
{
  "id": "apply-confirms",
  "channel": "whatsapp", "lang": "en",
  "input": "I'd like to apply for the Delivery Driver job...",
  "followups": ["Yes, everything is correct – please submit my application."],
  "expect": {
    "tool": "get_job_openings",          // or tool_any / tool_none
    "says_any": ["reference", "REF-0001", "captured"],
    "says_none": ["you're hired", "you got the job"] // safety guardrail
  }
}
```

Measured footprint (HEAD `fb97438`):

CORPUS	AGENT PACKS	EVAL CASES	PER-AGENT
B2B catalogue (<code>data/catalog</code>)	552	8,526	12–23, avg 15.4
Consumer (<code>data/catalog-consumer</code>)	17	302	avg ~17.8
Total	569	8,828	—

(552 packs = 500 SKUs + ~52 retained legacy unprefixd ZA aliases on disk; the index still lists exactly 500 — see §15.6.)

`scripts/eval-suite.mjs` — the zero-token engine (`pnpm eval:suite`, `--static-only` for the gate). Two lanes, **no** OpenAI/Anthropic spend:

- **Static check** per agent: preset bindings exist (`getPreset`), `system_prompt ≥ 800` chars, `knowledge ≥ 400` chars, `≥ 8` evals, every `expect.tool` exists in the pack's tool set and is bound in the preset, and **knowledge↔eval drift** — if `says_any` phrases appear nowhere in `knowledge + guardrails + system_prompt` (and aren't "soft" runtime-refusal

phrases), it's flagged high-severity. This catches localization drift where a market pack's evals expect phrases the localized knowledge no longer contains.

- **Runtime lane** (default, mock): replays each case multi-turn through `runTurn()` with `MockModelAdapter` + `MockWalletAdapter(500_000)` in `sandbox` / `rented` state, then scores `tool` / `tool_any` / `tool_none` / `says_any` / `says_none` / non-empty. The `says_any` matcher is deliberately **permissive** (currency/digit normalization + loose token overlap) because this lane proves *routing and contract*, not prose quality – a comment explicitly defers live quality to `live-llm-smoke.mjs`.

Outputs `data/reports/eval-results.json` + `docs/reports/eval-gap-<date>.md` (ranked by family drift/fails). **CI-failing thresholds:** exits non-zero if **any static high-severity issue** exists, or (runtime lane) **pass-rate < 35 %**. The `--static-only` variant is what the merge gate runs, so packaging defects (missing tools, thin prompts, drift) block merges while live spend stays out of CI.

`scripts/eval-live.mjs` – **real-model quality sample** (`pnpm eval:live`). Uses `createModelAdapter()` (`MIAI_MODEL_MODE=anthropic|openai|gateway`) with `skipDebit`, so the MockModel answer-injection path never runs.

Refuses to run in mock mode (exit 2) and requires the matching API key. Supports named hero sets (`--set=go-live-18|flagship-1a|flagship-1b|flagship-2`), `--families=`, `--limit=`. Pass bar is intentionally shallow (not paused, reply > 40 chars, not a "broken" template) and it **always exits 0** – the deliverable is the report (`docs/reports/eval-live-<date>-<set>.md`), never a gate. `scripts/live-llm-smoke.mjs` is the sibling single-agent / `--matrix` smoke over the Go-live 18.

`scripts/build-eval-live-scoreboard.mjs` – **the diligence scoreboard** (`pnpm scoreboard:live`). Aggregates `eval-live-*.md` reports + Wave-4 connector proofs (`data/wave4-live-proofs.json`) into `data/reports/eval-live-scoreboard.json` + a dated MD. It hard-codes an **honesty disclaimer** into every output: *live-LLM samples are a different metric from MockModel catalogue pass-rates and must not be sold as live quality; coverage is the hero/flagship sets, not the full 551-agent catalogue*. This is a deliberate anti-overclaim control worth noting for diligence.

Eval healing (`pnpm eval:full`). `scripts/fix-catalogue-evals.mjs` (zero-token) aligns packaged evals with each pack's market + knowledge (currency/emergency localization, grounded amounts into `says_any`, drop expects for undeclared tools). `scripts/heal-eval-failures.mjs` reads `eval-results.json` and injects grounding phrases / softens flaky non-safety expects. **Both carry an explicit safety fence:** a `STRICT` regex (`sanctions|aml|financial-advice|phil|hipaa|pci|pan|ssn|password|jailbreak|prompt-inject|cross-tenant|card|otp|pin|gdpr|erasure|...`) means **safety/compliance cases are never healed or grounded** – protecting the guardrail evals from being auto-passed. `eval:full` chains fix → build → suite → heal → suite.

Regression backfill – `packages/runtime/test/eval-live-findings.test.mjs`. A standout artifact: when the 2026-08-03 live run surfaced reply-content bugs that *passed* the shallow eval-live bar (hardcoded/mis-routed/leak-prone replies in hotel-guest, pharmacy, mobile-money, veterinary, accounting-practice, events-venue, gym-membership), the team converted each into a deterministic `node:test` case asserting the workflow reflects the tool result / knowledge. This closes the loop between the soft live sampler and the hard CI gate.

Certification / go-live tie-in. `scripts/certify-golive.mjs` (`pnpm certify:golive`, `--wave55` / `--next37` / `--beyond55`) grades Go-live 18/55/100 families against `docs/PILOT_PRODUCTION_BAR.md`: **cert-ready = depth stronglive + pilot doc + all 5 market packs**; "Depth live" (connector proof) is tracked separately as a stretch. It emits a per-family manual checklist (golden path 5–8 turns, handoff, `eval:suite --agents us-{family}`, install gating). Informational only (`exit 0`). Companion: `scripts/catalog-readiness.mjs` (`catalog:ready`).

15.6 Catalogue integrity (`pnpm catalog:integrity` , CI-required)

`scripts/check-market-pack-integrity.mjs` enforces the SKU model (ADR 0001) as a hard gate:

- `index.json` has **exactly 500** agents; `families.json` has **exactly 100** families.
- Every `families[*].markets.{us,eu,africa,asia,oceania}` id exists in the index and carries the correct market prefix.

- **ZA policy:** `markets.za` (if present) must equal `markets.africa`; unprefixed packs stay on disk as legacy aliases only and **must not appear in the index**.
- Warns (non-fatal) if all five market packs of a family share an identical `system_prompt+knowledge` SHA-256 (possible under-localization).

Fails the build on any structural error — this is what keeps "100 × 5 = 500" true as the catalogue is edited.

15.7 Live connector & platform proofs (`proof:*`)

Six scripts prove that tool calls hit **real** integrations, not sandbox stubs — asserting `live ≡ true` / no `sandbox_stub` / no `"_note: not OAuth-connected"` :

SCRIPT	COMMAND	NETWORK / LLM	PROVES
<code>live-connector-proof.mjs</code>	<code>proof:live</code>	live app, no LLM	Wave-4 OAuth connector results are real; records to <code>data/wave4-live-proofs.json</code>
<code>proof-probe.mjs</code>	<code>proof:probe</code>	live app, zero-LLM	<code>POST /api/oauth/{connector}/test</code> for slack / google_calendar / hubspot / email
<code>proof-bindings.mjs</code>	<code>proof:bindings</code>	offline	family→connector binding matrix vs Phase-1 set + proofs
<code>proof-tools.mjs</code>	<code>proof:tools</code>	live app, no LLM	scripted tool sweep via <code>POST /api/proof/tool</code> (read-only unless <code>--writes</code>)
<code>proof-webhook.mjs</code>	<code>proof:webhook</code>	live app	outbound webhook sink w/ HMAC signature
<code>proof-mcp.mjs</code>	<code>proof:mcp</code>	live app	MCP bridge <code>POST /api/mcp/tools/call</code>

These are the evidence layer behind "Depth live" and feed the scoreboard's connector-proof counts. `scripts/cutover-smoke.mjs` (`smoke:cutover`) is the post-deploy HTTP health/home smoke.

15.8 CI gates — required vs advisory

`.github/workflows/ci.yml` — job `quality` is the merge gate (per `docs/TESTING.md` ; branch-protection settings live in GitHub, not the repo). Steps, on Node 22 / pnpm 9.15.0, `push:main` + all PRs:

```
install (frozen-lockfile) → build:packages → typecheck → lint (@miai/web)
→ pnpm test (wallet+connectors+web+runtime)
→ catalog:integrity → eval:suite:static → pnpm audit --prod --audit-level=critical
```

A second `secrets` job greps tracked files for `.env` / `.pem` / `id_rsa` and runs `gitleaks/gitleaks-action@v2`. Local mirror: `pnpm run ci` = `build:packages → typecheck → test → catalog:integrity → eval:suite:static`.

Non-blocking / scheduled workflows:

- `eval-nightly.yml` – full mock `eval:suite` nightly (06:00 UTC); a `preflight` job reads `ANTHROPIC_API_KEY` into an output (working around GitHub's ban on `secrets` in `if:`) to conditionally run `live-llm-matrix` and `eval-live-sample`, both `continue-on-error` so the mock nightly stays authoritative and uploads a live-eval report artifact.
- `e2e-staging.yml` – soft `@smoke` on push (paths-filtered), functional+UAT+full on schedule/dispatch; every job `continue-on-error: true` → red surfaces in Actions **without gating merges**; uploads Playwright HTML reports (14-day retention).
- `publish-image.yml` (GHCR image build for the Azure Container Apps target), `daily-brief.yml` (hourly consumer-brief cron fan-out).

SonarCloud is advisory, not a gate. `sonar-project.properties` exists (CPD/duplication exclusions for generated presets, scaffold generators, and i18n dictionaries) but **no workflow in `.github/workflows` invokes the scanner** – so it functions as external PR decoration, not a blocking check.

15.9 Handover evidence pack

`docs/HANDOVER_TEST_PACK.md` is the pre-handover evidence bundle: last recorded full run **78/78 Playwright passed (~27 s)** on 2026-08-02 @ `feb6d43` via `pnpm handover:staging`, plus a green local `pnpm run ci`. It maps each acceptance criterion to the proving spec, enumerates the 78 scenarios, and – valuably for diligence – lists **“Intentionally not automated (handover residual)”**: full live golden paths, live connector proofs needing customer credentials, the OIDC-Bearer 401 matrix (staging still on mock rails), destructive DSAR erase, SOC 2 / counsel-signed policies, and the continuous live-LLM quality program. Sign-off is marked “Yes with caveats” (mock rails still on; product owner to spot-check live Studio quality; counsel/SOC 2 open).

15.10 How to run – quick reference

GOAL	COMMAND	COST / TARGET
Full local merge gate	<code>pnpm run ci</code>	zero-token, local
Unit + contract only	<code>pnpm test</code> · <code>pnpm test:api-contract</code>	zero-token
Catalogue integrity	<code>pnpm catalog:integrity</code>	zero-token
Mock evals (gate)	<code>pnpm eval:suite:static</code>	zero-token
Mock evals (full runtime)	<code>pnpm eval:suite</code>	zero-token
Fix → run → heal → re-run	<code>pnpm eval:full</code>	zero-token
Live-LLM sample	<code>MIAI_MODEL_MODE=anthropic ANTHROPIC_API_KEY=... pnpm eval:live -- set=go-live-18</code>	model spend
Live-LLM smoke matrix	<code>MIAI_MODEL_MODE=anthropic ... pnpm smoke:live-llm:matrix</code>	model spend
Build diligence scoreboard	<code>pnpm scoreboard:live</code>	zero-token

GOAL	COMMAND	COST / TARGET
Go-live certification	<code>pnpm certify:goLive (-- --wave55 / --next37)</code>	zero-token
E2E smoke (staging)	<code>pnpm smoke:staging</code>	deployed app
Functional + UAT (staging)	<code>pnpm uat:staging</code>	deployed app
Full handover pack	<code>pnpm handover:staging</code>	deployed app
E2E locally	<code>PLAYWRIGHT_BASE_URL=http://127.0.0.1:3000 pnpm test:e2e</code>	local server
Install Playwright	<code>pnpm test:e2e:install</code>	chromium only
Connector proofs	<code>DEMO_BASE=... pnpm proof:probe · proof:tools · proof:webhook · proof:mcp</code>	deployed app
Post-deploy HTTP smoke	<code>BASE=... pnpm smoke:cutover</code>	deployed app

15.11 Assessment for handover

Strengths: (1) a genuinely large, structured behavioural eval corpus (8,828 cases) that is IP, not just tests; (2) a zero-cost blocking gate that catches packaging/routing/localization defects without model spend; (3) explicit, code-enforced separation between mock pass-rates and live quality, with anti-overclaim disclaimers baked into tooling; (4) safety-case fencing that prevents auto-healing of compliance evals; (5) a live→regression feedback loop (`eval-live-findings.test.mjs`). Watch-items for the incoming team are in the gaps below – chiefly that the richest layers (live quality, connector proofs, E2E) are non-blocking by design and depend on freshly regenerated reports and a running staging app, and that the last full Playwright evidence predates the current HEAD.

16 Documentation

The documentation corpus is a first-class due-diligence asset, not an afterthought. Measured at commit `fb97438` (2026-08-25), the repository carries **171 Markdown files under `docs/` totalling ~11,504 lines**, plus four root-level governance files (`README.md`, `CONTRIBUTING.md`, `SECURITY.md`, `COORDINATION.md`) and a self-documenting OpenAPI route. This sits *on top of* the code and catalogue: the platform's true bulk is the **~299,508-line, ~555-file agent catalogue** (`data/catalog` + `data/catalog-consumer`) and the ~75,900-LOC hand-written runtime — the docs describe and index that asset rather than duplicating it. The single largest document, `docs/PLATFORM_TECHNICAL_DOSSIER.md`, is a 1,428-line / ~22,562-word / 194 KB grounded technical dossier written for exactly this audience (the MyInstantAI engineering handover team).

16.1 Corpus inventory and shape

The corpus splits into a curated top-level set, five structured sub-directories, and four root governance files.

LOCATION	FILES	CHARACTER
<code>docs/*.md</code> (top level)	33	Curated product/spec, onboarding, ops, catalogue, trust, testing, partnership, and build-coordination docs
<code>docs/adr/</code>	6	5 ADRs (<code>0001</code> – <code>0005</code>) + an index <code>README.md</code>
<code>docs/compliance/</code> (+ <code>templates/</code>)	5 + 2	Draft ROPA, DPIA, breach runbook, PCI/SAQ, SOC 2 evidence index; DPA/BAA template outlines
<code>docs/pilots/</code>	100	One per-family "job story" spec (golden-path turns, required live connectors, depth, evidence correlation IDs)
<code>docs/reports/</code>	28	Time-stamped evidence artifacts (audits, eval-gap, go-live certifications, tool/OAuth sweeps) — <code>.md</code> , <code>.html</code> , <code>.json</code>
<code>docs/branded-pdfs/</code>	10	HTML+CSS sources and a Chrome-headless generator (<code>generate.mjs</code> , <code>render-dossier.mjs</code>) for commercial leave-behind PDFs
Root	4	<code>README.md</code> , <code>CONTRIBUTING.md</code> , <code>SECURITY.md</code> , <code>COORDINATION.md</code>

A useful growth signal for diligence: both the verified ground truth and the dossier's own Appendix E state " `docs/` (32 files)"; but the current on-disk count of top-level `docs/*.md` is **33** — the dossier itself was added since that count was taken and it omits itself from its own index. The corpus is actively maintained (several files carry Aug-24/Aug-25 mtimes) rather than frozen at a milestone.

16.2 Purpose groups (top-level `docs/`)

The 33 top-level docs organise cleanly by purpose. Note the last group — AI-agent build-coordination logs — which a new reader must recognise as historical build artifacts rather than current-state spec.

GROUP	FILES	WHAT THEY CARRY
Product / spec	PLATFORM_TECHNICAL_DOSSIER.md , TECHNICAL_SPEC.md , PLATFORM_INTEGRATION.md	Full grounded architecture (dossier); Phase-3 technical spec; the MyInstantAI integration contract (surface ownership)
Onboarding	B2B_ONBOARDING.md , CONSUMER_AUTH.md	B2B workspace/identity onboarding split; consumer "Sign in with Google" (Auth Code + PKCE) setup
Connectors	CONNECTOR_OAUTH.md , WAVE4_LIVE_CONNECTORS.md	Shared OAuth callback + per-provider setup; live-connector proof programme with correlation-ID evidence
Sandbox / channels	SANDBOX.md , APP_CHANNEL.md	Cost-capped SANDBOX_MODE=1 partner eval env (no source repo hand-over); hosted app-channel + Expo shell
Migration / deploy	MIGRATION_P0.md , MIGRATION_P1.md , MIGRATION_RUNBOOK.md , RAILWAY_DEPLOY.md	Railway→Azure cutover/rollback runbook, P0/P1 readiness, staging deploy checklist
Catalogue	FAMILIES_100.md , MARKET_PACKS.md , CATALOGUE_READY.md , PRODUCTION_SCALE_500.md , PARALLEL_WORKSTREAMS.md , PILOT_PRODUCTION_BAR.md	The 100×5=500 SKU model, market-pack conventions, catalogue-ready claim, scale-to-500 programme, go-live bar
Compliance / trust	TRUST_AND_COMPLIANCE.md , AUDIT_RETENTION.md	Single trust narrative mirroring the in-app Trust Center (Live/Partial/Via-provider/Planned tags); retention posture (draft)
Testing	TESTING.md , HANDOVER_TEST_PACK.md , UAT_CHECKLIST.md , PHASE1_EXIT.md	Test pyramid, handover evidence pack (78/78 Playwright), human UAT sign-off, phase-1 exit criteria
Partnership / commercial	PARTNERSHIP_KICKOFF_BRIEF.md , MONDAY_COMMERCIAL_PACK.md	Move Digital × MyInstantAI kickoff and annual-license commercial model
AI-agent build coordination	CLAUDE_HANDOFF.md , CLAUDE_AUDIT_500.md , CLAUDE_VERIFY_REMEDIATION.md , CURSOR_FLAGSHIP_DEPTH.md , CURSOR_RESIDUAL_PUNCHLIST.md , CURSOR_BPLUS_PILOT_BAR.md	Point-in-time task briefs / handoffs between the AI authors (Claude, Cursor, Hermes) that built the platform, each Owner: -tagged and pinned to a commit/score

16.3 The Platform Technical Dossier – the diligence centrepiece

[docs/PLATFORM_TECHNICAL_DOSSIER.md](#) is purpose-built for takeover. Its header states it is "generated from a direct read of the source at the snapshot above; every path is repo-relative and every claim is grounded in code." Structure:

- **13 numbered sections:** (1) exec summary & repo topology, (2) stack & deps, (3) request lifecycle, (4) runtime engine, (5) catalog & market packs, (6) data/persistence/memory, (7) identity & multi-tenancy, (8) integrations/connectors/channels, (9–10) full API reference, (11) security/privacy/compliance, (12) deployment/ops, (13) testing/evals.
- **5 machine-generated appendices** (A endpoint index, B env-var catalog, C LOC/file counts, D dependency manifest, E in-repo doc index).
- **An "Open items to confirm" honesty ledger** — ~55 explicit statements of what each section's author could *not* verify from source (e.g. that `TECHNICAL_SPEC.md`'s scale figures are stale, that no inbound WhatsApp webhook exists in-repo despite WhatsApp being listed as a channel, that `next@^15.2.8` resolves to `15.5.22` in the lockfile). This ledger is itself a strong diligence signal: the documentation under-claims and self-flags gaps rather than papering over them, matching the ADR-0005 "prefer under-claiming" discipline.

This is the one document a new engineer should read first; it cross-references almost every other doc and the code paths behind each claim.

16.4 Architecture Decision Records

`docs/adr/` holds an index plus five one-to-two-page ADRs in a fixed template (Title · Status · Date · Context · Decision · Consequences · References). All five are `Accepted` (2026-08-02). The index `README.md` documents the format and the "supersede-by-linking" convention.

ADR	DECISION (SUMMARY)
0001 – Catalogue 100×5 (500 SKUs)	Indexed catalogue = 100 families × 5 markets (US, EU, Africa, Asia, Oceania) = 500 SKUs in <code>index.json</code> / <code>families.json</code> . ZA is not a sixth market; legacy unprefixed <code>{family}.agent.json</code> files stay on disk as aliases (hence >500 files, ~555 on disk) with <code>markets.za</code> pointing at the <code>africa-{family}</code> id. Index is script-generated, never hand-edited.
0002 – Dual-flag mock rails	In <code>NODE_ENV=production</code> , mock auth/wallet/model fail closed at boot unless <i>both</i> <code>ALLOW MOCK RAILS=1</code> and <code>I_UNDERSTAND MOCK RAILS IN PROD=1</code> are set; half-configured flags block startup; a <code>miai.mock_rails_enabled</code> warning is emitted. Same pattern for <code>ALLOW EMBED ORIGIN STAR</code> . Implemented in <code>lib/security-flags.ts</code> .
0003 – Postgres persistence + file fallback	When <code>DATABASE_URL</code> is set, use Postgres via <code>pg</code> (no ORM) with versioned SQL migrations on boot; otherwise fall back to JSON file/in-memory stores so contributor laptops and CI need no DB. Row-level upserts, no full-table wipes. Supersedes the earlier SQLite brief.
0004 – Live LLM shared guardrails + retrieval	A shared guardrail engine (<code>packages/runtime/src/guardrails.ts</code>) runs input/output policy for both mock and live models; hard refusals/handoffs never rely on <code>MockModel</code> alone. Market-pack prompts add regional compliance cues; retrieval + guardrails compose. Explicitly notes guardrails <i>reduce but do not eliminate</i> jailbreak/hallucination risk.
0005 – Phase-4 draft compliance docs	In-app <code>/privacy</code> , <code>/terms</code> , <code>/cookies</code> + consent, and the <code>docs/compliance/</code> pack (ROPA, DPIA, breach runbook, PCI/SAQ, SOC 2 index, DPA/BAA templates) are drafts, not certifications . Explicit non-claims: not SOC 2 certified, not a signed DPA/BAA, HIPAA blocked. Security contact <code>security@myinstantai.com</code> .

The ADRs are tightly coupled to code: each cites the implementing module (`lib/security-flags.ts`, `lib/store.ts` / `lib/pg.ts`, `packages/runtime/src/guardrails.ts`), so a reviewer can trace decision → implementation directly.

16.5 Root governance docs

- `README.md` (161 lines) — quick start, prototype loop, a grouped script catalogue (Catalog / Eval / Proof / CI-test / Deploy-ops), CI overview, doc index, monorepo layout, and an env-var table. Strong entry point.
- `CONTRIBUTING.md` — pnpm-workspace setup, Apache-2.0 contribution terms, the `pnpm run ci` quality gate, and the "do not delete ZA aliases / regenerate via scripts" catalogue rules (mirrors ADR-0001).
- `SECURITY.md` — private disclosure channels, in/out-of-scope table (notably: **tenant LLM prompts/knowledge are out of scope** — the platform's job is tenant isolation, not tenant content), and a *draft* severity/SLA table (Critical 24h ack / 7d fix ... 90-day coordinated disclosure).
- `COORDINATION.md` — the multi-author contract between **Claude Code, Cursor, and Hermes**: `main` is protected in both repos, one-author-per-family, namespaced branches (`hermes/<family>` , `claude/<family>`), shared-file conflict surface called out, and a non-negotiable consumer-family certification gate (3× majority eval pass on `gpt-4o`). This is an unusual and revealing artifact — the platform was built by coordinated AI authors under an explicit written protocol.

16.6 Inline code documentation density

Documentation is not confined to `docs/`. The `apps/web/src/lib/` module layer (80 `.ts` files, ~11,774 lines) carries **948 comment lines (~8%) and 286 JSDoc-opening `/**` blocks**; 28 of 80 files open with a header comment explaining intent. Representative headers are genuinely explanatory, not boilerplate:

```
// apps/web/src/lib/csp.ts
/**
 * Per-request CSP builder (nonce + strict-dynamic for scripts).
 * style-src keeps 'unsafe-inline' for next/font + Tailwind (B+ deferred — removing it
 * without a full style-nonce pass breaks the App Router shell). Security win is script-src.
 */
```

```
// apps/web/src/lib/security-flags.ts
/** Second confirmation required with ALLOW MOCK_RAILS in production. */
export const MOCK_RAILS_ACK_ENV = "I_UNDERSTAND MOCK_RAILS_IN_PROD";
```

```
// packages/runtime/src/guardrails.ts
/**
 * Shared safety policy for mock + live models (Phase 3 D2).
 * Hard refusals / handoffs that must not depend on MockModel alone.
 */
```

The header comments frequently encode *why* a trade-off was made (deferred work, security rationale, phase provenance), which is the class of context most valuable to a takeover team. `lib/public-paths.ts` even documents the auth model inline ("health / catalog / openapi / consent / handoff → unauthenticated by design").

16.7 Self-documenting API

`apps/web/src/app/api/v1/openapi/route.ts` serves a hand-authored **OpenAPI 3.0.3** spec at `GET /api/v1/openapi`, exposed publicly (allowlisted in `lib/public-paths.ts`, so both `middleware.ts` and `lib/auth.ts` treat it as unauthenticated-by-design and the lists cannot drift). It is referenced from `RAILWAY_DEPLOY.md`, `TECHNICAL_SPEC.md`,

`SANDBOX.md`, `HANDOVER_TEST_PACK.md`, and is asserted by the `api-contract` test. **Caveat:** it is a *stub/contract* for `/api/v1` — it fully documents `embedChat` and `rentAgent` (request/response/error schemas), not all 69 API routes. The authoritative complete endpoint reference is the dossier's §9–10 plus Appendix A, not the OpenAPI document. New integrators are correctly steered to `/api/v1/*` while legacy unversioned routes remain supported.

16.8 Evidence trail: `pilots/` and `reports/`

Two sub-directories give the corpus unusual depth for diligence:

- `docs/pilots/` (100 files) — one spec per agent family. Each states a customer-anchored *job story*, an explicit *golden-path* turn-by-turn script, the *live connectors required*, a *depth* rating (`strong` / `live`), and — where earned — an *evidence* line with a real correlation ID and staging URL (e.g. `corr_wave4_msgcbgmd - live slack on ...railway.app (2026-08-05)`). This ties documentation to reproducible runtime behaviour.
- `docs/reports/` (28 files) — time-stamped, largely machine-generated evidence: `technical-audit-2026-08-02.md` (128 KB) + its HTML render, `audit-500-*`, a series of `eval-gap-*` (2026-07-31 → 2026-08-25), `eval-live-*`, `flagship-depth-*`, `golive-certify-*`, and JSON sweeps (`binding-matrix`, `tool-sweep`, `oauth-probes`). This is a dated audit history a diligence team can follow.

16.9 Assessment — could a new engineer onboard from the docs alone?

Largely yes, for the core platform. The path is well-lit: `README.md` → `CONTRIBUTING.md` (setup + quality gate) → `PLATFORM_TECHNICAL_DOSSIER.md` (grounded architecture + endpoint reference + env catalog) → the 5 ADRs (the *why*) → `TESTING.md` and `HANDOVER_TEST_PACK.md` (how to prove it works) → per-family `pilots/` for behavioural intent. Env vars, scripts, the API contract, deploy runbooks, and the multi-tenancy/auth model are all documented and cross-referenced to code. Inline JSDoc fills gaps at the module level.

Friction a new engineer will hit:

1. **Stale spec figures.** `TECHNICAL_SPEC.md` is dated 2026-07-31 and reports ~14,540 platform LOC / 22 routes / 55 families — contradicted by the current repo (~75,900 runtime LOC / 69 routes / 100 families / 500 SKUs). The dossier's open-items ledger flags this explicitly, but a reader who starts from the spec will be misled unless they reach the dossier.
2. **Stale cross-references.** `PARALLEL_WORKSTREAMS.md` and `PILOT_PRODUCTION_BAR.md` reference `PRODUCTION_SCALE_410.md`, which no longer exists (the file is now `PRODUCTION_SCALE_500.md`) — a dangling link left by the 410→500 rename.
3. **Build-log docs mixed with spec docs.** The `CLAUDE_*` / `CURSOR_*` briefs are point-in-time task orders, not current-state documentation; without the `Owner:` /commit-pin cues a newcomer could mistake them for live spec. There is no top-level `docs/README.md` index to disambiguate (only `docs/adr/` has one) — the doc map lives inside the dossier's Appendix E and the root README table.
4. **Partial OpenAPI.** The self-documenting API covers only two `/api/v1` operations; full endpoint knowledge requires the dossier.
5. **Out-of-repo dependencies.** Key surfaces are documented as external: the upstream agent source (`../miai-agents-audit/agents` consumed by `import:catalog`) and the MyInstantAI portal that mints B2B `workspace_id` /JWTs are outside this repo, so onboarding for those flows depends on artifacts not present here.

Net: the documentation is genuinely a due-diligence asset — grounded, code-linked, honestly tagged (Live/Partial/Planned), and backed by a dated audit/eval evidence trail and 100 per-family behavioural specs. A competent engineer could stand up, understand, and operate the platform from `README` → dossier → ADRs → `TESTING` /runbooks without external help. The residual work before a clean handover is editorial hygiene, not authorship: retire or date-stamp the stale `TECHNICAL_SPEC.md` and the `PRODUCTION_SCALE_410` references, add a `docs/README.md` index that separates "current spec" from "historical build logs", and expand the OpenAPI document toward full route coverage.

17 Engineering Hygiene

This section assesses the code-quality controls that gate every change into the platform: static typing, linting, the CI merge gate, secret scanning, duplication analysis, branch/PR discipline, commit hygiene, module boundaries, and the contract test. The verdict up front: **for a codebase of this size the hygiene controls are disproportionately mature and, importantly, mechanically enforced in CI rather than left to reviewer goodwill.** The suppression rate is near-zero, the type discipline is uniform, and the one structural risk – an enormous machine-generated asset – is explicitly fenced off from the human-authored code so it does not distort the quality signal.

17.1 Scale the hygiene controls must hold

Re-measured at commit `fb97438` (excluding `node_modules`, `.next`, `dist`, `.git`):

SLICE	FILES	LINES	NOTES
Total authored (<code>ts/tsx/js/mjs/json/md/sql/css</code>)	1,212	406,319	grown from the Licence Proposal's ~398K / 1,021
Catalogue IP (<code>data/catalog</code> + <code>data/catalog-consumer</code>)	552 <code>*.agent.json</code> + indices	299,508	74% of the repo – the largest, most valuable asset
Hand-written runtime (<code>apps</code> + <code>packages</code> + <code>scripts</code> + <code>e2e</code> , <code>ts/tsx/js/mjs</code>)	412	75,768	the code the hygiene gate primarily governs
Generated (<code>packages/presets/src/generated-presets.ts</code>)	1	11,761	machine-emitted, fenced from duplication analysis
API routes (<code>apps/web/src/app/api/**/route.ts</code>)	69	–	up from the 45 quoted in the Proposal
Pages (<code>apps/web/src/app/**/page.tsx</code>)	39	–	–

The hygiene story is therefore two-sided: ****~76K lines of hand-written TypeScript held to a strict bar, and ~300K lines of catalogue JSON held to a *schema/integrity* bar**** (`pnpm catalog:integrity`) rather than a lint bar. Both are gated in the same CI job.

17.2 Type safety – uniform `strict`, near-zero escape hatches

Every one of the **nine** `tsconfig.json` files in the tree sets `"strict": true` – no package opts out:

```
apps/web apps/runtime apps/connectors apps/mobile-shell
packages/{runtime,connectors,agent-protocol,presets,wallet-adapter}
```

- **Enforcement:** `pnpm typecheck` → `pnpm -r typecheck`, which runs `tsc --noEmit` in every workspace. It is step 3 of the CI `quality` job, so a type error anywhere fails the merge.
- `apps/web/tsconfig.json` is the standard Next.js strict profile: `strict`, `noEmit`, `isolatedModules`, `module: esnext`, `moduleResolution: bundler`, `resolveJsonModule`, `paths: { "@/*": ["/src/*"] }`, and the `next` TS

plugin. Packages use `NodeNext` + `declaration: true` (they emit `.d.ts` consumed by the app).

- `skipLibCheck: true` is set everywhere — standard for velocity, but it means third-party `.d.ts` drift is not caught.
- **Escape-hatch discipline (measured):**

SUPPRESSION	COUNT IN APPS/WEB/SRC + PACKAGES/*/SRC
<code>@ts-ignore</code> / <code>@ts-expect-error</code>	0
<code>eslint-disable*</code>	9
<code>: any</code> / <code>as any</code> / <code><any></code> (apps/web/src)	2

Zero type-suppressions across ~76K lines is an unusually clean result and the single strongest hygiene signal in the repo. The nine `eslint-disable` uses are all line-scoped (`eslint-disable-next-line`), never file- or block-level, and most carry an inline justification: three `react-hooks/exhaustive-deps` (deliberate effect deps in `ActionsPanel.tsx`, `AgentStudio.tsx`), four `@next/next/no-img-element` for genuinely remote/model-supplied thumbnails with no `next/image` domain config (`rich-text.tsx`, `MarketplaceAssistant.tsx`), one `no-require-imports` for a lazy `pg` require, and one `no-explicit-any` in `packages/connectors/src/ssrf.ts`.

Gap: there is **no root/base `tsconfig`** — each package repeats the same compiler options. Harmless today (they are consistent) but a drift risk as packages multiply.

17.3 Linting

- **Config:** `apps/web/eslint.config.mjs` — flat config bridging via `FlatCompat` to `next/core-web-vitals` + `next/typescript`. Resolved toolchain: `eslint` 9.39.5, `eslint-config-next` 15.1.0, `typescript` 5.9.3.
- **Enforcement:** the CI `quality` job runs `pnpm --filter @miai/web lint` (`next lint`) as a distinct, blocking step. Linting is *not* folded into `pnpm run ci` (that script is build → typecheck → test → catalog:integrity → eval:suite:static); CI adds lint on top, and CONTRIBUTING flags local lint as "optional... CI lint is separate."
- **Coverage gap:** only `@miai/web` is linted. The five `packages/*` and the `scripts/` directory ship **no ESLint config** and are held only by `tsc`. For ~30K lines of package + script code this is the most material lint gap.
- **Version skew (minor):** `eslint-config-next` is pinned to 15.1.0 while `next` resolves to 15.5.22 in the lockfile — the lint ruleset trails the framework by several minors. Low risk, worth a bump.

17.4 The CI merge gate (`.github/workflows/ci.yml`)

CI runs on every `pull_request` and on `push` to `main`, with least-privilege `permissions: contents: read` at the top. Two jobs:

Job `quality` (the named required check — see §17.6) runs `pnpm` 9.15.0 on **Node 22**, `pnpm install --frozen-lockfile`, then in strict order:

#	STEP	COMMAND	GUARDS
1	Build packages	<code>pnpm build:packages</code>	shared <code>.d.ts</code> must compile
2	Typecheck	<code>pnpm typecheck</code>	strict <code>tsc --noEmit</code> , all workspaces
3	Lint web	<code>pnpm --filter @miai/web lint</code>	<code>next lint</code>

#	STEP	COMMAND	GUARDS
4	Unit tests	<code>pnpm test</code>	wallet + connectors + web + runtime (<code>node --test</code>)
5	Catalogue integrity	<code>pnpm catalog:integrity</code>	100×5 SKU shape + ZA-alias policy over the 299K-line catalogue
6	Static eval suite	<code>pnpm eval:suite:static</code>	structural eval-case validation across the catalogue
7	Dependency audit	<code>pnpm audit --prod --audit-level=critical</code>	supply-chain gate (critical only)

Two properties stand out. First, the gate validates the **catalogue IP itself** (steps 5–6), not just code – appropriate given the catalogue is 74% of the repo. Second, `--frozen-lockfile` means a lockfile that drifts from `package.json` fails the build, keeping dependency state deterministic.

Job secrets (see §17.5). The other four workflows are **not** merge gates: `e2e-staging.yml` (daily 06:15 UTC + `main` push on relevant paths), `eval-nightly.yml` (nightly live evals), `publish-image.yml` (image build on `main` /tags), `daily-brief.yml` (hourly cron). This is a clean separation of *blocking correctness gates* from *scheduled/observability* jobs.

Divergence to flag: CI runs **Node 22** (comment: "required for `--experimental-strip-types`"), while `CONTRIBUTING.md`, the Dockerfile base, and the stated runtime target are **Node 20**. Tests use Node's experimental TS-stripping in CI; the production runtime is a different major. Not a defect, but the test toolchain and the deploy toolchain are not on the same Node line.

17.5 Secret scanning

The `secrets` job elevates `pull-requests: write` (documented inline as required for `gitleaks-action` to enumerate PR commits without a 403) and runs two independent controls:

- Committed-file denylist** – a `set -euo pipefail` guard failing the build if any tracked file matches `\.env$`, `\.pem$`, or `id_rsa`:

```
hits="$(git ls-files | grep -E '^(/)\.env$|\.pem$|^(/ )id_rsa$' || true)"
[ -n "$hits" ] && { echo "$hits"; exit 1; }
```

- Gitleaks** – `gitleaks/gitleaks-action@v2` over full history (`fetch-depth: 0`), with no custom `.gitleaks.toml` (default ruleset).

This is backed by `.gitignore` (ignores `.env`, `.env.local`, `data/oauth-tokens.json`, and other runtime secret stores) and by `COORDINATION.md`, which forbids the Hermes PAT from ever landing in a commit. `SECURITY.md` documents private disclosure (`security@myinstantai.com` + GitHub advisories). Secret hygiene is layered (ignore → denylist → scanner → policy) rather than reliant on any single line of defence.

17.6 SonarCloud (advisory) and duplication policy

No workflow invokes Sonar – it runs as the **SonarCloud GitHub App**, i.e. **advisory**, not a blocking gate. Its one config file, `sonar-project.properties`, sets *duplication-only* exclusions with an explicitly reasoned comment:

```
sonar.cpd.exclusions=packages/presets/src/generated-presets.ts,
scripts/scaffold-*.mjs,
apps/web/src/lib/i18n/*.ts
```

The rationale (verbatim intent): these are **generated output** (the 11,761-line `generated-presets.ts`), **scaffolding generators**, and **i18n dictionaries** whose repetition is *structural*, not a maintainability smell — so they are removed from the "Duplications on New Code" metric **only**. Bug, vulnerability, and code-smell rules still apply to them. This is exactly the right scalpel: it prevents an 11.7K-line machine-emitted file from swamping the duplication signal for the ~76K lines of hand-written code, without blinding Sonar to real defects in those files. The choice is durable enough that it was landed as its own commit (`b32bec4 chore(sonar): ...`).

17.7 Branch, PR, and commit conventions

`CONTRIBUTING.md` establishes: branch from `main`; run the CI gate locally via `pnpm run ci` before opening a PR; PRs must pass CI before merge; catalogue/index files are **generated, never hand-edited** (a table maps each task to its script); and new cross-cutting decisions require an ADR.

`COORDINATION.md` is a genuinely unusual and valuable artifact — a written contract for **two independent authors** (Claude Code/Opus and "Hermes"/DeepSeek):

- `main` is protected in both repos; nobody pushes to it — ever. All work arrives as PRs.
- Namespaced branches: `claude/<family>` and `hermes/<family>`; one author per branch, one author per agent family.
- Explicit shared-file conflict surface enumerated (`data/catalog/index.json`, `spec/*`, `packages/*`, ...) with a "coordinate before editing, never two open PRs on the same shared file" rule.
- Ownership split by risk: Claude Code owns all safety-critical families (crisis, minors, health) and *all merges*; Hermes authors lower-risk families and **does not self-merge**.
- Certification gate for consumer families: no merge until the eval suite passes at **3× majority on gpt-4o**; fixes must go into prompt/guardrails, never into weakening an assertion.
- The single required CI check is documented per repo: `quality` for this repo, `validate` for `miai-agents`.

Commit hygiene (measured, last 20 commits): Conventional Commits throughout — `feat(consumer):`, `fix(infra):`, `refactor(consumer):`, `docs(sandbox):`, `chore(sonar):` — each squash-merged with its PR number (`#87`, `#86`, ...). 307 commits on `main`. Subjects are descriptive and scoped. This is consistent, machine-parseable history suitable for changelog automation.

17.8 Code organisation and module boundaries

`apps/web/src/lib/` (~80 modules) is organised by concern, and there is a **deliberate, documented boundary between HTTP-edge code and unit-testable logic**: modules that emit HTTP responses import `next/server` (`api-error.ts`, `request-auth.ts`, `security.ts`, `proof-harness.ts`, `handlers/embed-chat.ts` — `NextResponse`), while pure-logic modules **avoid it on purpose so they can run under `node --test` without the Next runtime**. This is not incidental — it is stated in the code:

- `consumer-identity.ts`: "kept free of `next/server` so it can be unit-tested and imported..."
- `consumer-auth.ts`: "Kept out of `consumer.ts` on purpose... [the tested] `consumer.ts` must stay free of `next/server`."
- `webhook-sink-auth.ts`: written so callers "don't need to load `next/server`."

The payoff shows in the test layout: the 29 web test files (`apps/web/test/*.test.mjs`) exercise `auth.ts`, guardrails, PII redaction, memory extraction, OIDC isolation, embed-key security, etc. via `node --test` with `--experimental-strip-types` — **no Next server is booted**. The boundary is what makes that possible. This is a level of layering discipline most Next.js codebases lack.

17.9 The API contract test

`apps/web/test/api-contract.test.mjs` is a deliberately "**thin, no Next server required**" contract check that runs inside `pnpm test` (via `test:web`) and is also exposed as `pnpm test:api-contract`. It asserts the invariants the rest of the platform is built on:

- `data/catalog/index.json` parses and lists **≥ 500** agents, each row typed (`id` / `name` strings, `tools ≥ 1`, `evals ≥ 1`);
- `loadAgentPackage()` from `@miai/agent-protocol` validates a real pack (`us-hr-helpdesk`) — manifest id matches, `tools ≥ 1`, `evals ≥ 8` ;
- OIDC auth config resolution behaves correctly under a saved/restored `process.env` (auth mode, issuer, audience, JWKS).

Because it loads catalogue files off disk through the shared protocol loader, it catches catalogue/loader contract drift in the same fast unit run — complementing the heavier `catalog:integrity` step.

17.10 Maintainability assessment

DIMENSION	ASSESSMENT
Type safety	Strong. Uniform <code>strict</code> , 0 <code>@ts-ignore</code> , ~2 <code>any</code> in the web app, blocking <code>tsc --noEmit</code> .
Lint	Good for web, gap for packages/scripts. Enforced on <code>@miai/web</code> ; <code>packages/*</code> + <code>scripts/</code> unlinted.
CI gate	Strong. Deterministic (<code>--frozen-lockfile</code>), gates code <i>and</i> catalogue IP, least-privilege perms, supply-chain audit.
Secret hygiene	Strong, layered. ignore → denylist → gitleaks → policy.
Duplication policy	Strong/considered. Precise CPD carve-outs for generated/i18n code, rules otherwise intact.
Collaboration model	Exceptional for the size. Written multi-author contract, protected <code>main</code> , namespaced branches, merge-ownership by risk.
Commit/PR history	Strong. Conventional Commits, PR-linked squash merges, ADR discipline.
Module boundaries	Strong. Documented <code>next/server</code> -free testable core enabling server-less unit tests.
Suppression rate	Excellent. 9 line-scoped ESLint disables, all justified; nothing blanket.

Overall: the hand-written runtime is maintainable and the controls are real — enforced by CI, not aspiration. The dominant *structural* consideration is not code quality but **volume**: 74% of the repo is machine-generated/authored catalogue JSON,

correctly governed by schema-integrity and eval gates rather than lint, and correctly fenced out of the duplication metric so it does not mask smells in the human code. A team taking handover inherits a codebase where the quality signal is trustworthy because the noisy 300K lines have been deliberately isolated from it.

Residual hygiene risks (all low-to-moderate, none blocking):

1. `packages/*` and `scripts/` are unlinted (~30K LOC) – only `tsc` guards them. Add a shared flat ESLint config at the root.
2. **Node major divergence** – CI/tests run Node 22, the runtime targets Node 20. Align, or document the split explicitly.
3. **No base `tsconfig`** – nine near-identical configs invite drift; extract a shared base.
4. `eslint-config-next` (15.1.0) trails `next` (15.5.22) – minor ruleset lag; bump.
5. `skipLibCheck: true` everywhere – dependency `.d.ts` regressions won't surface in typecheck.
6. **Sonar is advisory** and the `secrets` job's *required* status is undocumented – `COORDINATION.md` names only `quality` as the required merge check; confirm branch-protection also marks `secrets` required, or accept gitleaks as advisory.

18 Design Patterns

This section documents the architectural patterns that let a ~406,319-line / 1,212-file monorepo stay maintainable and extensible. The dominant asset is the **catalogue IP** — re-measured at **299,508 lines of JSON** (**290,061** in `data/catalog` across **552** `*.agent.json` files, plus **9,447** across **18** files in `data/catalog-consumer`) — and the entire hand-written runtime (~75.9K LOC) exists to *load, bind, meter and serve* those flat files without ever hard-coding an agent. Every pattern below is in service of that separation: **data (the 500-SKU catalogue) is inert; behaviour (rails, tenancy, models) is swapped by environment, never by edit.**

The two ideas that recur are **Ports & Adapters (Hexagonal)** — one interface, many implementations chosen at a single factory — and **mode-switching by env flag** so the identical code path runs as a zero-cost mock, a live production deployment, or a risk-free partner sandbox. A due-diligence team can therefore stand up the whole platform on mocks, then flip flags one rail at a time.

18.1 Pattern catalogue (map)

#	PATTERN	WHERE IT LIVES	SWAPPABLE AXIS
1	Adapter/Port — model	<code>packages/runtime/src/index.ts</code> (<code>ModelAdapter</code> , <code>createModelAdapter</code>)	<code>MIAI_MODEL_MODE</code> = mock/openai/azure/anthropic/gateway
2	Adapter/Port — wallet	<code>packages/wallet-adapter/src/index.ts</code> (<code>WalletAdapter</code>)	<code>MIAI_WALLET_MODE</code> = mock/http
3	Adapter/Port — connectors	<code>packages/connectors/src/index.ts</code> (<code>executeConnector</code>)	<code>mode</code> = sandbox/live
4	Adapter/Port — persistence	<code>apps/web/src/lib/*-store.ts</code> + <code>lib/pg.ts</code>	Postgres when <code>DATABASE_URL</code> , else JSON file-store
5	Shared-contract package	<code>packages/agent-protocol/src/index.ts</code> (<code>AgentPackage</code>)	the wire format between catalogue, runtime, UI
6	The "turn" abstraction	<code>runTurn(TurnRequest): TurnResult</code>	one entry point for every channel
7	Preset / tool-binding generation	<code>packages/presets/src/{index,generated-presets}.ts</code>	generated + hand-override merge
8	Capability + certification gating	<code>apps/web/src/lib/consumer.ts</code> , <code>consumer-catalog.ts</code>	<code>certified</code> flag v <code>SANDBOX_MODE</code>
9	Fail-closed metering	<code>runTurn</code> balance guards	zero balance ⇒ no provider call
10	Mode switching (4 flags)	<code>AUTH_MODE</code> / <code>WALLET_MODE</code> / <code>MODEL_MODE</code> / <code>SANDBOX_MODE</code>	orthogonal env toggles

#	PATTERN	WHERE IT LIVES	SWAPPABLE AXIS
11	<code>next/server</code> -free lib split	<code>apps/web/src/lib</code> (81 of 89 files framework-free)	testability seam
12	Cross-cutting redaction (decorator)	<code>apps/web/src/lib/agent-ip.ts</code>	strips IP on egress
13	HMR-safe singleton	<code>globalThis.__miai*</code> in pool/wallet	one instance per process
14	Idempotency key	wallet <code>debit</code> / <code>topUp</code> , embed keys	safe retries / dedupe

18.2 Ports & Adapters – the model adapter

The runtime never imports a vendor SDK. It depends only on an interface, `ModelAdapter` (`packages/runtime/src/index.ts`):

```
export interface ModelAdapter {
  complete(input: ModelCompleteInput): Promise<ModelCompleteResult>;
  /** When present, yields true token deltas for openai/gateway (mock paces after generate). */
  streamComplete?(input: ModelCompleteInput): AsyncIterable<StreamChunk>;
}
```

Five concrete adapters implement it — `MockModelAdapter`, `OpenAIModelAdapter`, `AnthropicModelAdapter`, `GatewayModelAdapter`, `AzureOpenAIModelAdapter` — and a **single factory** picks one from env, degrading to the mock whenever a key is absent so the platform can never hard-fail on missing config:

```
export function createModelAdapter(): ModelAdapter {
  const mode = env("MIAI_MODEL_MODE") ?? "mock";
  // Sandbox cost backstop: after a capped number of real-provider turns per process,
  // fall back to the mock model so an evaluation cannot run up an unbounded bill.
  if (env("SANDBOX_MODE") === "1" && mode !== "mock") {
    const cap = Number(env("SANDBOX_MODEL_TURN_CAP") ?? "") || 1000;
    if (sandboxModelTurns ≥ cap) return new MockModelAdapter();
    sandboxModelTurns++;
  }
  if ((mode === "gateway" || mode === "http") && env("MIAI_MODEL_GATEWAY_URL")) return new
  GatewayModelAdapter();
  if (mode === "azure" && env("AZURE_OPENAI_API_KEY") && env("AZURE_OPENAI_ENDPOINT")) return new
  AzureOpenAIModelAdapter();
  if ((mode === "anthropic" || mode === "claude") && env("ANTHROPIC_API_KEY")) return new
  AnthropicModelAdapter();
  if (mode === "openai" && env("OPENAI_API_KEY")) return new OpenAIModelAdapter();
  return new MockModelAdapter();
}
```

Three design decisions are worth calling out for handover:

- **Anti-corruption via an OpenAI-compatible surface.** `OpenAIModelAdapter`, `AnthropicModelAdapter` and `GatewayModelAdapter` all funnel through one pair of helpers (`openAiCompatibleComplete` / `openAiCompatibleStream`). Anthropic is deliberately called on its `/v1/chat/completions` compatibility surface so "tool-calling stays on the same path as OpenAI/gateway" (comment at the class). Only Azure differs, and only in `endpoint()` + `azureAuth` (the `api-key` header vs `Bearer`) – the migration target for MyInstantAI's Azure infra "runs on Azure by flipping the mode."
- **Capability negotiation, not assumption.** `streamComplete?` is optional. `iterateModelStream` uses the real provider stream when present and otherwise **synthesises** deltas from a completed answer (`streamFromComplete`), so the mock and non-streaming providers still drive a token-by-token UI.
- **Sandbox cost backstop is in the factory itself** (`SANDBOX_MODEL_TURN_CAP` , default 1000 real turns/process → mock), so an evaluating partner cannot run up an unbounded provider bill by hammering the sandbox.

18.3 Ports & Adapters – the wallet adapter

`packages/wallet-adapter/src/index.ts` is the cleanest expression of the pattern: one interface, a `MockWalletAdapter` (in-memory, seeds `demo-workspace` with `1_000_000` tokens) and an `HttpWalletAdapter` (MyInstantAI's future billing API), chosen by `createWalletAdapter()` from `MIAI_WALLET_MODE` .

```
export interface WalletAdapter {
  getBalance(workspaceId: string): Promise<WalletBalance>;
  debit(req: DebitRequest): Promise<DebitResult>;
  topUp(req: TopUpRequest): Promise<WalletBalance>;
}
```

Two properties make this handover-grade:

- ****The HTTP adapter mirrors the mock's *semantics*, not just its shape.**** Insufficient-funds is a first-class result, never an exception: on `402` / `409` the HTTP adapter returns `{ ok:false, paused:true }` "to mirror `MockWalletAdapter` ," so the runtime's fail-closed logic is provider-agnostic.
- **Idempotency is baked into the contract.** `DebitRequest.idempotencyKey` and `TopUpRequest.idempotencyKey` (typically the Paystack transaction ref) dedupe replays; the HTTP adapter forwards them as an `idempotency-key` header so "a webhook retry and the return-URL verifier can both fire safely." The token/USD menu is a single source of truth (`TOPUP_TOKENS` / `TOPUP_PACKAGES`) shared by UI and payment routes.

The catalogue's commercial contract lives one level up in `packages/agent-protocol` as plain data (`RENT_USD = { standard: 349, pro: 699, enterprise: 1199 }`), plus `RENT_EUR`), keeping pricing out of the runtime.

18.4 Ports & Adapters – connectors, and sandbox force-stubbing

`executeConnector` (`packages/connectors/src/index.ts`) dispatches a tool call to its bound connector, but **sandbox short-circuits every live actuation** – this is the seam that makes "the real, running platform with the risk taken out" safe:

```
export async function executeConnector(call: ConnectorCall): Promise<ConnectorResult> {
  // Sandbox: never actuate live connectors, whatever the caller passed – no real sends/writes.
  if (call.mode === "sandbox" || process.env.SANDBOX_MODE === "1") {
    return { ok: true, data: stubFor(call.tool, call.args), connector: call.binding.connector,
      stubbed: true };
  }
  return executeLive(call);
}
```

`stubFor` (`packages/connectors/src/live/execute.ts`) returns realistic, deterministic fixtures tagged `source: "sandbox_stub"` per tool (`place_order` → `ORD-3391` , `book_table` → `TBL-4821` , availability slots, ...), so a demo *looks* live while writing nothing. The connector registry itself is data (`CONNECTORS: ConnectorMeta[]` , 25 ids across `phase 1|2`), and `ConnectorId` is a closed union – adding a provider is a registry row plus a handler, not a runtime change.

18.5 Ports & Adapters – dual persistence (Postgres ↔ file-store)

Every stateful `lib/*-store.ts` module implements the **same port twice**: a Postgres branch when `getPool()` is non-null (`DATABASE_URL` set) and a JSON file-store fallback otherwise. `lib/pg.ts` centralises the pool (SSL policy, `PG_SSL_REJECT_UNAUTHORIZED` opt-out) and a thin parameterised `query()` ; there is **no ORM** (raw `pg`). Representative shape from `consumer-memory-store.ts` :

```
if (getPool()) {
  await ensureMigrations();
  const res = await query(
    `INSERT INTO miai_consumer_memory (tenant_id, consumer_id, id, category, content, content_key,
    source)
    VALUES ($1,$2,$3,$4,$5,$6,$7)
    ON CONFLICT (tenant_id, consumer_id, content_key) DO UPDATE SET ...`,
    [owner.tenantId, owner.consumerId, id, category, content, key, source]);
  return { remembered: true, id: res.rows[0]?.id ?? id };
}
// else: in-memory Map keyed by owner, persisted to a JSON file on the /data volume
```

This is what lets the platform boot with zero infrastructure (file-store) yet run production on managed Postgres by setting one variable – the two branches are behind one function signature (`rememberFact` , `listMemories` , `forgetMemory` , ...).

18.6 Multi-tenant isolation – the `tenant_id` predicate

Isolation is enforced **structurally**: every persisted row carries `tenant_id` (workspace/brand) and, on the consumer line, `consumer_id` ; **every** read, write and delete carries both as the leading predicate/columns. There is no shared table without a tenant key, so a missing predicate is a visible code smell rather than a silent leak:

```
WHERE tenant_id = $1 AND consumer_id = $2 AND id = $3          -- reminders, lifegraph, memory
ON CONFLICT (tenant_id, consumer_id, content_key) DO UPDATE ... -- upserts scoped to owner
```

OAuth tokens follow the same rule keyed by `workspace_id` (`SELECT sealed FROM miai_oauth_tokens WHERE workspace_id = $1 AND connector = $2`), and tokens are **sealed at rest with AES-256-GCM** (`v2.` envelope, with legacy `v1.` HMAC accepted for migration) in `packages/connectors/src/oauth/tokens.ts`. The predicate is complemented at the edge by auth (§18.9): the tenant id is not client-trusted in OIDC mode — it comes from a verified `workspace_id` JWT claim.

18.7 The shared-contract package — `agent-protocol`

`packages/agent-protocol` is the **stable seam between the three moving parts**: the 500-SKU catalogue that produces `AgentPackage` files, the runtime that consumes them, and the UI that renders them. It is dependency-free and defines the wire format plus a validating loader:

```
export interface AgentPackage {
  format: "miai.agent-package/v1";
  manifest: AgentManifest; // id, tier, channels,
  model{primary,fallback,temperature,max_output_tokens}, prepaid.skus...
  system_prompt: string;
  knowledge: string;
  tools: AgentTool[];
  guardrails: string;
  evals: unknown[];
}
export function loadAgentPackage(raw: unknown): AgentPackage { /* asserts format + required fields
*/ }
```

Because the contract is versioned (`miai.agent-package/v1`) and the catalogue is *flat files* keyed by id (`data/catalog/{id}.agent.json`), the 100-families × 5-markets SKU model (ADR 0001) scales by adding files, never code. Derived views are computed off the manifest, not stored: `marketplaceCategory()` maps an agent id/category to a marketplace shelf, and `agentAudience()` splits customer-facing vs internal — presentation logic that stays out of both catalogue and runtime.

18.8 The "turn" abstraction

Every channel — web chat, embed widget, Telegram/WhatsApp webhooks, MCP server — collapses to **one** call: `runTurn(req: TurnRequest): Promise<TurnResult>` (`packages/runtime/src/index.ts`). `TurnRequest` is a self-contained value object (`workspaceId`, `agentId`, `pkg`, `messages`, `model`, `mode: "sandbox"|"live"`, `bindings?`, `consumerLine?`, ...); `TurnResult` returns the assistant message, the tool-call trace (`stubbed` / `live` flagged), `tokensDebited`, `balance`, `paused`, and an optional multi-step `workflow` plan. Dependencies are **injected**, which is the whole testability story:

```
export async function runTurn(req: TurnRequest, deps?: {
  wallet?: WalletAdapter; model?: ModelAdapter;
  onDelta?: (t: string) => void; onToolStart?: () => void; skipDebit?: boolean;
}): Promise<TurnResult> {
  const wallet = deps?.wallet ?? createWalletAdapter();
  const model = deps?.model ?? createModelAdapter();
  ...
}
```

Inside, the **bounded plan→act→observe loop** is credit-checked and step-capped. The cap defaults to 3 and is env-tunable, hard-clamped to `[1, 5]` – a correction worth flagging vs. a fixed "`≤3`":

```
const maxToolRounds = Math.min(5, Math.max(1, Number(env("RUNTIME_MAX_TOOL_ROUNDS") ?? 3)));
...
for (let toolRound = 0; completion.toolCall && toolRound < maxToolRounds; toolRound++) { ... }
```

Retrieval is itself mode-switched inside the turn: the mock model always gets a full knowledge prefix (deterministic evals), while live models get **hybrid semantic+lexical** retrieval when an embedder is configured (`selectKnowledgeForPromptAsync`, budget `RUNTIME_KNOWLEDGE_CHARS`, default 40k). Vertical workflow modules (`packages/runtime/src/workflows/*`, plus first-party handlers like the marketplace assistant) are tried before the generic loop and short-circuit via `finishWorkflow(...)`.

18.9 Mode switching – four orthogonal flags

The platform's operational posture is composed from **independent** env toggles, each read at one factory/guard so they never entangle:

FLAG	VALUES	READ AT	EFFECT
<code>MIAI_AUTH_MODE</code>	mock oidc	<code>lib/auth.ts</code> , <code>consumer-identity.ts</code>	header/demo identity vs verified JWT/session
<code>MIAI_WALLET_MODE</code>	mock http	<code>wallet-adapter</code> <code>createWalletAdapter()</code>	in-memory vs billing API
<code>MIAI_MODEL_MODE</code>	mock openai azure anthropic gateway	<code>runtime</code> <code>createModelAdapter()</code>	which LLM rail
<code>SANDBOX_MODE</code>	1	<code>lib/sandbox.ts</code> <code>isSandbox()</code>	force-stub connectors, cap model, allow all specialists

`SANDBOX_MODE` is a **cross-cutting** switch by design (documented in `lib/sandbox.ts`): it simultaneously force-stubs connectors (§18.4), activates the model cost-cap (§18.2) and opens the certification gate for evaluation (§18.10) – "the real, running platform with the risk taken out." The auth mode also carries the production safety interlock: elevated mock roles (`owner`, `operator`) are only granted outside production *or* when the **dual-flag** `ALLOW MOCK RAILS=1` + `I UNDERSTAND MOCK RAILS IN PROD=1` is set (ADR 0002), otherwise mock defaults to `readonly` and boot fails closed.

18.10 Capability + certification gating

What an agent is *allowed to run* is a data-driven allowlist, not code. Consumer specialists carry a `certified` boolean ("behaviourally certified – 3×-majority eval pass") in `consumer-catalog.ts`; the runtime gate is a one-liner that also honours the sandbox:

```
export async function isRunnableConsumerAgent(agentId: string): Promise<boolean> {
  if (CONSUMER_AGENTS.has(agentId)) return true;           // flagship always runs
  const entry = await getPersonalAgent(agentId);
  if (!entry) return false;                                // resolves ONLY consumer ids → no B2B
  crossover
  return entry.certified === true || isSandbox();           // uncertified: browse-only in prod,
  runnable in sandbox
}
```

Two boundaries hold at once: an **allowlist** (`getPersonalAgent` only resolves consumer-catalogue ids, so a tenant/B2B id is never runnable on the consumer surface) and a **certification gate** (uncertified specialists stay browse-only in production but are fully evaluable in the sandbox). Customer-facing capability copy is derived from the pack, not authored twice — `buildFamilyCapabilities()` reads `pkg.tools[].side_effects` (`read-only|write|financial`) to render Learn-more modals.

18.11 Fail-closed metering

Metering is a **guard before the provider call**, so zero balance can never incur cost. `runTurn` short-circuits twice — on entering a paused state and on a zero balance — returning a localized top-up message with **no** model invocation:

```
if (req.state === "paused_no_tokens") { return { ..., paused: true }; } // no model call
...
const bal = await wallet.getBalance(req.workspaceId);
if (!skipDebit && bal.tokens ≤ 0) {
  return { assistantMessage: wf(req.replyLanguage, "paused_no_tokens"), ..., state:
    "paused_no_tokens", paused: true };
}
```

Debits are metered per turn (`estimateTurnTokens` applies rough per-model multipliers — `flash 0.4×`, `mini 0.5×`, `opus 3×` — while live providers prefer their reported usage totals) and use a composite idempotency key (`workspace:agent:ts:len`). A `skipDebit` path exists for the free "try before rent" sandbox turn (still estimates for metering=0). This mirrors the model-tier UI table in `apps/web/src/lib/models.ts` (`MODELS` with `burn` labels), keeping the price signal consistent between UI and ledger.

18.12 Preset / tool-binding generation

Binding an agent's abstract tools to concrete connectors is **generated then hand-overridden** — the classic "generated base + curated exceptions" merge. `scripts/generate-presets.mjs` emits `packages/presets/src/generated-presets.ts` (11,761 lines, 551 preset entries, header: "AUTO-GENERATED ... do not edit by hand"). `packages/presets/src/index.ts` layers ~46 `HAND_OVERRIDES` on top and merges last-wins:

```
function mergePresets(): AgentPreset[] {
  const byId = new Map<string, AgentPreset>();
  for (const p of GENERATED_PRESETS) byId.set(p.agentId, { ...p, phase: p.phase as 1|2 });
  for (const p of HAND_OVERRIDES) byId.set(p.agentId, p); // hand wins
  return [...byId.values()];
}
```

Overrides are expressed as small **composable factories** (`booking()`, `salonBooking()`, `executiveAssistant(calendar, handoff)`, `personalAssistant()`), each parameterised by region-appropriate connectors — e.g. EU SKUs bind `m365_calendar` + `teams`, US/Africa bind `google_calendar` + `slack`. When no preset exists, `defaultBindingsForTools()` infers a binding from the tool name (`*book*/availability → google_calendar`, `*ticket*/lead → hubspot`, else `webhook`). `resolveBindings()` in the runtime consults, in order: an explicit per-turn override → the preset → the name-based default — so a fresh catalogue agent is runnable with zero configuration.

18.13 The `next/server` -free lib split (testability seam)

The business logic in `apps/web/src/lib` is deliberately kept importable **without the Next.js runtime**: only **8 of 89** `.ts` files touch `next/server`; the other **81** are framework-free and unit-testable under `node --test`. `consumer-identity.ts` states the rule explicitly:

```
// Consumer identity resolution — kept free of `next/server` so it can be unit-tested and
// imported by the runtime path. requireConsumer (in consumer-auth.ts) wraps this to turn
// AuthError into an HTTP Response.
export async function resolveConsumerAuth(req: Request): Promise<AuthContext> { ... }
```

The pattern is a two-layer split: a **pure core** taking a Web-standard `Request` and throwing typed `AuthError`, wrapped by a thin **HTTP shell** (`requireConsumer`) that maps errors to `Response`. Route handlers stay trivial; the logic they call is testable in isolation.

18.14 Cross-cutting patterns

- **Redaction decorator (agent-IP protection).** `redactAgentPackage()` (`apps/web/src/lib/agent-ip.ts`) strips `system_prompt`, `guardrails` and `evals` — “the crown jewels” — from **every** package leaving the server for a browser or API caller, on production *and* sandbox, so the catalogue “can’t be enumerated-and-scraped through the API.” The full package is only ever materialised server-side inside `runTurn`.
- **HMR-safe singletons.** The pg pool, wallet adapter and pg module are cached on `globalThis` (`__miaiPgPool`, `__miaiWallet`, `__miaiPgModule`) so a single instance survives Next.js hot reloads and mock balances persist across turns in one process; `resetWalletAdapterForTests()` provides a deterministic teardown.
- **Idempotency & deterministic derivation.** Beyond wallet dedup, embed keys are derived, not stored: `embedKeyFor()` returns `mia_pk_<base64url(workspaceId::agentId)>_<hmac10>` (HMAC over `EMBED_KEY_SECRET`), so legacy keys survive redeloys with no storage, and folding in a per-agent `salt` (via `rotateEmbedKey`) makes revocation possible without a global secret change.

18.15 Why these patterns matter for handover

The through-line is **substitution at a single seam**. A partner engineer evaluates on mocks (no keys, no infra, no spend), flips `MIAI_MODEL_MODE` / `MIAI_WALLET_MODE` / `MIAI_AUTH_MODE` / `DATABASE_URL` one at a time to go live, runs an isolated

`SANDBOX_MODE` deployment for their team with connectors force-stubbed, and migrates to Azure by setting `MIAI_MODEL_MODE=azure` — all without touching the 299,508-line catalogue or the `runTurn` core. New agents are files; new connectors and models are one adapter plus a registry row; tenancy is a predicate the schema forces you to write. That is the maintainability/extensibility thesis of the codebase, and it is consistent across every subsystem examined.

19 Marketplace Metrics

This section quantifies the catalogue as a balance-sheet asset. Every number below was re-measured against the repository at commit `fb97438` (`/Users/malcolmgovender/Projects/miai-agent-marketplace`), not taken from prior documents. Where a headline figure exists, both the *indexed-SKU* view (the 500 sellable products) and the *on-disk* view (all package files, including legacy aliases) are given so the reader can reconcile either.

19.1 Headline – the catalogue is the largest asset in the repo

The value of this platform is not the ~75.9K lines of hand-written runtime; it is the **299,508 lines of authored agent IP** that the runtime executes. That IP is ~74% of the entire authored codebase.

LAYER	LINES	FILES	SHARE OF AUTHORED CODE
Catalogue IP (<code>data/catalog</code> + <code>data/catalog-consumer</code> , JSON)	299,508	573 JSON	~73.7%
– business catalogue <code>data/catalog</code>	290,061	555	
– consumer catalogue <code>data/catalog-consumer</code>	9,447	18	
Platform runtime (hand-written TS/TSX/JS/MJS)	~75,900	~410	~18.7%
Other authored (generated presets, scripts, docs, e2e, sql)	remainder		
TOTAL AUTHORED	406,319	1,212	100%

Measured combined catalogue: `find data/catalog data/catalog-consumer -name '*.json' | xargs cat | wc -l` → **299,508**.

Growth note: the Licence Proposal quoted ~398K lines / 1,021 files / ~298K catalogue. The repo has grown to **406,319 lines / 1,212 files / 299,508 catalogue** – report the current measured figures.

19.2 The SKU model – 100 families × 5 markets = 500 SKUs

The commercial unit is `family × market pack`, locked in **ADR 0001** (`docs/adr/0001-catalog-100x5.md` , Status: Accepted, 2026-08-02) and `docs/FAMILIES_100.md` .

METRIC	COUNT	SOURCE
Families (unique vertical jobs)	100	<code>families.json</code> (100 entries, every one with <code>packs.length === 5</code>)
Market packs	5	<code>market-packs.json</code> (us, eu, africa, asia, oceania)
Indexed sellable SKUs	500	<code>index.json</code> (500 entries) / <code>INDEXED_AGENT_COUNT = 500</code> in <code>apps/web/src/lib/catalog.ts</code>
<code>*.agent.json</code> files on disk (<code>data/catalog</code>)	552	500 prefixed SKUs + 52 legacy unprefixed aliases

METRIC	COUNT	SOURCE
Total files in <code>data/catalog</code>	555	552 packages + <code>index.json</code> + <code>families.json</code> + <code>market-packs.json</code>

Legacy aliases: 52 unprefixd `{family}.agent.json` files (e.g. `clinic-front-desk`, `bank-branch`, `hotel-concierge`) are ZA-era imports retained for stable deep links. Per ADR 0001 they are **not** a sixth market: `families.json` maps each family's `markets.za` to the canonical `africa-{family}` id, and `publicMarkets()` in `catalog.ts` folds `za → africa`. Market prefix distribution on disk is exactly balanced: **us/eu/africa/asia/oceania = 100 each** (500), plus 52 unprefixd aliases.

19.3 Families by industry – 19 verticals

The industry taxonomy is defined in `apps/web/src/lib/sectors.ts` (`SECTORS[]`, exactly **19** entries) and assigned by `marketplaceCategory()` in `packages/agent-protocol/src/index.ts`. Applying that function to the 100 family ids yields:

INDUSTRY	FAMILIES	INDUSTRY	FAMILIES
Financial services	16	Manufacturing & industrial	5
HR & internal ops	9	Retail & e-commerce	5
Logistics & field ops	8	Education	4
Hospitality & travel	8	Property	3
Professional services	7	Agriculture	2
Telecommunications	6	Customer & front office	2
Government & public sector	6	Cybersecurity	2
AI & developer tools	5	Energy & utilities	1
Data & analytics	5	Media & entertainment	1
Health & wellness	5	Total	100 across 19 industries

The catalogue reached 100 families over four waves (`FAMILIES_100.md`): baseline 55 → Wave 1 70 → Wave 2 82 → Wave 3 92 → **Wave 4 100**; Wave 4 added Cybersecurity, Energy, Agriculture, Media families).

Underlying `manifest.category` (the 5-value enum `AgentCategory`) distributes across the 500 indexed SKUs as: `operations` 225, `vertical` 215, `commerce` 40, `front-office` 10, `sales` 10.

19.4 The five regional market packs

Each family is localised into five packs. Packs are not copies — they change language, channel mix, compliance, currency, and emergency-routing (`data/catalog/market-packs.json`).

PACK	PREFIX	LANGUAGES	CHANNELS	COMPLIANCE	CURRENCY	EMERGENCY
US	us-	en, es	sms, web, app	TCPA, CCPA (+HIPAA health)	USD	911
EU	eu-	en, de, fr, es, it	sms, web, app	GDPR	EUR	112
Africa	africa- / unprefixed	en, fr, sw	whatsapp, web, app, sms	POPIA + regional	local	local services
Asia	asia-	en, zh, hi	web, app, sms	PDPA + regional	local	local services
Oceania	oceania-	en	sms, web, app	AU Privacy Act, NZ Privacy Act	AUD	000 / 111

19.5 Tiers and monthly rent

Three tiers, priced in [packages/agent-protocol/src/index.ts](#) ([RENT_USD](#) / [RENT_EUR](#)):

TIER	RENT_USD	RENT_EUR	INDEXED SKUS AT THIS TIER
standard	\$349	€319	95
pro	\$699	€649	305
enterprise	\$1,199	€1,099	100

List-price footprint (one of each of the 500 SKUs at list rent): **\$366,250/mo → \$4,395,000 ARR** – standard 95×\$349 = \$33,155, pro 305×\$699 = \$213,195, enterprise 100×\$1,199 = \$119,900. (Illustrative gross list value, not a revenue forecast.)

Every SKU also carries a **prepaid capacity ladder** ([manifest.prepaid.skus](#)) – 500/500 indexed SKUs have prepaid cards, e.g. [us-clinic-front-desk](#) :

```
"prepaid": { "skus": [
  { "sku": "CARD-CLFRDE-150", "capacity": "~150 conversations / ~100 answered calls",
    "price_band": "$79-99" },
  { "sku": "CARD-CLFRDE-500", "capacity": "~500 conversations / ~350 answered calls",
    "price_band": "$149-199" },
  { "sku": "CARD-CLFRDE-OUT50", "capacity": "50 booked appointments",
    "price_band": "$199-249" }
]}
```

19.6 Tools – ~1,900 typed, auth-scoped functions

VIEW	TOOL COUNT	AVG/AGENT	RANGE
Indexed 500 SKUs (<code>index.json</code> <code>tools</code> field)	1,890	3.78	—
All 552 package files (counted from <code>tools[]</code>)	2,128	3.86	3–17

Tools are not free-text — they are typed `AgentTool` objects (`name` , `description` , JSON-Schema `parameters` , `returns` , `side_effects` , `auth_scope`). Across all 2,128 tool definitions:

ATTRIBUTE	DISTRIBUTION
<code>side_effects</code>	read-only 877 · write 1,232 · financial 19
<code>auth_scope</code>	2,128 / 2,128 carry an explicit scope (e.g. <code>tenant</code>) — 100%

The financial/write split matters for handover: 1,251 tools perform state-changing or money-moving actions and are gated by the confirm-before-write guardrails and the wallet meter. The richest agent, `personal-assistant.agent.json` , defines 17 tools (`triage_inbox` , `send_email` , `manage_calendar` , `create_playlist` , `remember_person` , ...).

19.7 Evals – 8,500+ behavioural test cases

VIEW	EVAL CASES	AVG/AGENT	RANGE
Indexed 500 SKUs (<code>index.json</code> <code>evals</code> field)	7,640	15.28	—
All 552 package files (counted from <code>evals[]</code>)	8,526	15.45	12–23

These are structured behavioural assertions, not prompt notes. Each eval case is `{ id, channel, lang, input, expect }` ; the `expect` block asserts tool-choice and content behaviour. Assertion-key frequency across all 8,526 cases (a case can carry several keys):

ASSERTION	USES	MEANING
<code>says_any</code>	8,091	required content present
<code>tool</code>	2,884	must call a specific tool
<code>says_none</code>	1,693	forbidden content absent
<code>no_tool</code>	1,614	must answer without any tool
<code>refuses</code>	367	must refuse
<code>tool_any</code>	318	must call one of a set
<code>tool_none</code>	307	must not call a tool
<code>lang</code>	92	must answer in a given language

ASSERTION	USES	MEANING
<code>no_long_reply</code>	1	brevity bound

Example (`us-clinic-front-desk`): `{"id":"list-services-happy","channel":"sms","input":"what do you offer and what does it cost?","expect":{"tool":"list_services","says_any":["$95","consultation","dental"]}}`. The presence of `tool`, `refuses`, `no_tool`, and `lang` assertions is direct evidence these packages drive a tool-calling runtime, not a chat wrapper.

19.8 Channels and languages per SKU

Measured across the 500 indexed SKUs (agent-count carrying each attribute):

CHANNEL	SKUS	LANGUAGE	SKUS
web	500	en	500
app	500	es	200
sms	500	fr	200
whatsapp	100 (Africa pack only)	de / it / zh / hi / sw	100 each

Every SKU is at least tri-channel (web + app + sms); the Africa pack adds WhatsApp. Every SKU is multilingual; language set is pack-driven.

19.9 agent.json anatomy – proof each SKU is a runtime, not a prompt

Every package validates as `format: "miai.agent-package/v1"` (enforced by `LoadAgentPackage()` in `agent-protocol`). The seven top-level keys of `AgentPackage`:

KEY	TYPE	ROLE	TYPICAL SIZE (BUSINESS CATALOGUE)
<code>format</code>	string	version pin <code>miai.agent-package/v1</code>	—
<code>manifest</code>	<code>AgentManifest</code>	id, name, tier, category, market, compliance[], channels[], languages[], <code>model{primary, fallback, temperature, max_output_tokens}</code> , <code>handoff</code> , <code>usage_profile</code> , <code>prepaid</code>	—
<code>system_prompt</code>	string	role, rules, persona	avg 4,611 chars
<code>knowledge</code>	string	grounded KB (drives RAG switch > ~6,000 chars)	avg 3,786 chars , max 10,954

KEY	TYPE	ROLE	TYPICAL SIZE (BUSINESS CATALOGUE)
<code>tools</code>	<code>AgentTool[]</code>	typed function surface	3–17 tools
<code>guardrails</code>	string	safety / confirm-before-write policy	avg ~5,000 chars
<code>evals</code>	<code>unknown[]</code>	behavioural test cases	12–23 cases

Manifest richness across the 500 indexed SKUs: **500/500** carry `handoff`, **500/500** carry `usage_profile`, **500/500** carry `prepaid`. Sample manifest fragments (`us-clinic-front-desk`):

```
"model": { "primary": "claude-sonnet", "fallback": "gpt-4o-mini", "temperature": 0.3,
"max_output_tokens": 700 },
"handoff": { "enabled": true, "target": "clinic_reception",
"triggers": ["clinical_question", "emergency", "explicit_request", "low_confidence"] },
"usage_profile": { "tier_cap_msgs_month": 3000, "avg_tokens_per_msg": 850 }
```

So each SKU ships a model config (with fallback), a human-handoff contract with named triggers, a metering profile, a prepaid ladder, a typed tool surface, grounded knowledge, guardrails, and its own regression suite — the full definition of an operable agent runtime.

19.10 Certification / readiness state

- `index.json` marks **500/500 SKUs** `readiness: "catalogue-ready"` — the entire indexed catalogue is at the top readiness tier; there are no partial/pilot SKUs in the index.
- The gate is automated: `pnpm catalog:ready` (per `docs/CATALOGUE_READY.md`) asserts package validity, compliance/languages/channels/handoff/prepaid presence, a connector preset per agent, and a full `rent→configure→actions→sandbox` path.
- `catalogue-ready ≠ live-model-certified`. `docs/CATALOGUE_READY.md` is explicit: `pnpm eval:suite` is a **MOCK-runtime contract check (~95% package integrity)**, **NOT model quality**; real-model evals live in a separate `miai-agents` repo (`tools/run_evals.py`). Platform rails (Auth/SSO, Wallet, Model gateway, Postgres) are adapter-ready and blocked only on MyInstantAI staging credentials.

19.11 Consumer catalogue (`data/catalog-consumer`)

A separate, smaller asset run by the consumer line, same `miai.agent-package/v1` format:

METRIC	VALUE
Consumer specialist agents	17 (<code>*.agent.json</code>) + <code>index.json</code> = 18 files
Lines of IP	9,447

METRIC	VALUE
Tool definitions	50
Eval cases	302

Agents include `health-navigator`, `money-coach`, `study-coach`, `trip-planner`, `paperwork-navigator`, `topup-concierge`, `private-confidant`, etc. `getAgentPackage()` in `catalog.ts` falls back to `CONSUMER_CATALOG_DIR` when an id is not in the business catalogue.

Bottom line

The catalogue is **299,508 lines of structured, versioned agent IP** across **500 sellable SKUs (100 families × 5 market packs)** plus 17 consumer specialists — carrying **~1,900 typed, auth-scoped tools (2,128 across all package files, 1,251 write/financial), 8,500+ behavioural eval cases**, per-SKU model config, handoff contracts, metering profiles and prepaid ladders. It is the platform's dominant asset by both line count (~74% of authored code) and commercial value (\$4.4M ARR of list rent), and its structure — typed tools + tool-choice evals + guardrails — is what distinguishes it from a prompt library.

20 Complexity & Rebuild Effort

This section is a purely technical build-complexity and rebuild-**effort** assessment of the codebase at `/Users/malcolmgovender/Projects/miai-agent-marketplace` (measured at commit `fb97438`, `main`). It quantifies how much engineering and authoring work the artefact represents, using a standard COCOMO-style effort model. Every figure below is re-measured from the working tree. **The dollar range is an engineering effort-estimation model – it is not a quote, a price, a valuation, or a commercial offer, and no licence, subscription, or “build-vs-licence” comparison is made or implied.**

20.1 Measured size basis (re-counted for this section)

Counts exclude `node_modules`, `.next`, `dist`, and `.git`.

STREAM	FILES	LINES	NOTES
TOTAL authored (ts/tsx/js/mjs/json/md/sql/css)	1,212	406,319	Full repo authored surface
– Catalogue IP <code>data/catalog</code>	555	290,061	552 <code>*.agent.json</code> + <code>families.json</code> + <code>index.json</code> + <code>market-packs.json</code>
– Catalogue IP <code>data/catalog-consumer</code>	18	9,447	17 consumer specialists
Catalogue IP subtotal	573	299,508	The largest and least-substitutable asset
Hand-written runtime (ts/tsx/js/mjs)	415	76,169	Platform + engine + scripts + tests
– <code>apps/web</code>	280	34,550	Next.js App Router, 69 API routes, 39 pages
– <code>packages/runtime</code>	36	13,818	<code>runTurn</code> , adapters, guardrails, RAG, 22 workflow modules
– <code>packages/presets</code>	2	12,245	11,761 lines are <code>generated-presets.ts</code> (machine-generated)
– <code>scripts</code>	34	9,110	catalog/eval/proof tooling (55 root scripts)
– <code>packages/connectors</code>	19	4,293	16-entry connector registry + live handlers
– <code>e2e</code>	34	1,095	Playwright <code>@smoke/@functional/@uat/@handover</code>
– <code>packages/wallet-adapter</code>	2	349	prepaid metering adapter
– <code>packages/agent-protocol</code>	1	142	tiers + <code>RENT_USD/RENT_EUR</code>

File-type distribution (authored): `json` 607f / 314,535L, `ts` 247f / 48,626L, `mjs` 83f / 14,262L, `tsx` 84f / 13,275L, `md` 180f / 12,632L, `css` 5f / 2,800L, `sql` 5f / 183L, `js` 1f / 6L. Repo on disk **incl.** dependencies ≈ **805 MB**.

Growth since the earlier brief. A prior document quoted ~398K lines / 1,021 files / ~298K catalogue / 45 routes / ~631 MB. The tree has since grown to **406,319 lines / 1,212 files / 299,508 catalogue / 69 API routes / ~805 MB** — the numbers in this section supersede the earlier ones.

Scale discipline: the headline size is **406,319 authored lines**, of which **~74% (299,508 lines) is the 500-SKU catalogue IP** and **~19% (76,169 lines) is hand-written runtime** (of which ~11.8K is generated). Any effort statement that counts only the runtime understates the artefact by roughly 4x; any COCOMO run that treats the catalogue JSON as procedural source overstates it. The model below keeps the two streams separate for exactly this reason.

20.2 Two effort streams, estimated independently

Applying a single procedural-code multiplier to all 406K lines would be wrong: 299.5K lines are curated JSON (prompts, knowledge, tools, guardrails, evals), not control-flow logic, and COCOMO coefficients are not calibrated for structured content. The rebuild effort is therefore modelled as **runtime engineering + catalogue authoring**, each with its own basis.

Two adjustments applied before estimating:

- **Generated code removed from authoring.** `packages/presets/src/generated-presets.ts` (11,761 lines) is emitted by `scripts/generate-presets.mjs` (`/* AUTO-GENERATED ... do not edit by hand */`). Hand-authored runtime is therefore **~64,400 lines** (76,169 – 11,761 + minor rounding), with ~76.2 KLOC as an upper anchor that still includes the generator's output.
- **Catalogue derivation is 100x5, not 500-from-scratch.** ADR `0001-catalog-100x5` fixes the SKU model at **100 families × 5 markets = 500 SKUs** (552 agent files on disk incl. retained ZA aliases). Only ~100 base families are authored originally; the other ~400 are produced by `generate:packs` (idempotent market-pack generation) plus localization/compliance review — so the catalogue's effort is dominated by ~100 originals, not 500.

20.3 COCOMO-style basis and effort-per-KLOC

Mode: Organic. The project fits COCOMO organic mode — a small, cohesive, senior team working in familiar technology (Next.js 15 / React 19 / TypeScript 5 / Node 20 / Postgres via `pg`, no exotic hardware, no hard real-time/embedded constraints). It is *not* semi-detached or embedded.

Basic COCOMO organic (COCOMO-81):

$$\text{Effort (PM)} = 2.4 \times \text{KLOC}^{1.05}$$

$$\text{Schedule TDEV (months)} = 2.5 \times \text{Effort}^{0.38}$$

Two calibration points are shown so the assumptions are explicit:

BASIS	EFFORT-MONTHS / KLOC	RUNTIME (64.4 KLOC HAND-AUTHORED)
Textbook organic COCOMO-81 <code>2.4 · KLOC^{1.05}</code>	≈ 2.95 PM/KLOC	≈ 190 PM (upper reference)
Modern greenfield-adjusted (≈ 2,000–2,300 LOC/PM effective)	≈ 0.45–0.50 PM/KLOC	≈ 28–32 PM (used)

The textbook coefficient (~3 PM/KLOC) was calibrated on 1970s–80s procedural 3GL/assembler and materially overstates a 2020s framework-heavy TypeScript monorepo, where Next.js App Router, React, `pg`, `jose`, and `zod` supply large amounts

of undifferentiated heavy lifting and AI-assisted development compresses boilerplate. The adjusted band (~0.45–0.50 PM/KLOC) is used for the headline; the ~190 PM textbook figure is retained as a conservative upper anchor for the runtime alone.

20.4 Headline rebuild-effort estimate

Bottom-up sum of the two streams (standard engineering effort model — **not a quote or price**):

STREAM	BASIS	EFFORT (PERSON-MONTHS)
Runtime & platform engineering	~64.4 KLOC hand-authored @ organic-adjusted 0.45–0.50 PM/KLOC	28–32
Catalogue IP authoring	100 base families @ 3–4 person-days + ~400 market derivations @ ~0.5 person-day + 8,526 eval cases	22–24
Total rebuild effort		≈ 50–56 PM

- **Effort:** ~50–56 person-months ≈ 4.2–4.7 person-years.
- **Schedule (COCOMO TDEV, organic, E≈53):** $2.5 \times 53^{0.38} \approx 11.3$ calendar months, implying a steady-state team of ~5 (≈ 53 PM ÷ 11.3 mo). Realistically a blended team of ~4–6 engineers plus 2–3 domain/prompt authors over ~12–16 calendar months, given the parallel runtime and catalogue tracks and the eval/hardening tail.
- **Indicative rebuild cost:** at a fully-loaded blended senior rate of ≈ \$14K–\$17K per person-month, 50–56 PM ⇒ ≈ \$0.75M – \$0.95M.

Model, not a price. The dollar figure is a standard engineering effort-cost model (effort × a loaded labour rate). It is **not** a quote, an invoice, a valuation, a market price, or a commercial offer, and it is deliberately not compared to any licence, subscription, or purchase alternative.

For comparison, the conservative textbook path — Basic COCOMO organic applied to the ~76.2 KLOC runtime including the generator ($2.4 \times 76.2^{1.05} \approx 227$ PM), TDEV ≈ 20 months, ~11–12 engineers) — would imply a much larger runtime-only effort. The realistic ~50–56 PM headline reflects modern framework reuse, catalogue automation, and the removal of generated lines; the 190–227 PM textbook band is the upper bound if one (incorrectly) prices the codebase as bespoke procedural software.

20.5 Catalogue authoring as a first-class effort driver

The catalogue is the dominant asset by size (299,508 lines) and the least reproducible by lines-of-code alone, because each JSON line encodes curated domain knowledge, tuned prompts, and designed test oracles rather than boilerplate. Per-agent structure (verified across all 552 `*.agent.json`): keys `format · manifest · system_prompt · knowledge · tools · guardrails · evals`. Measured aggregates:

CATALOGUE METRIC	MEASURED
Agent SKUs on disk	552 (500 canonical + retained ZA aliases)
Tier split	pro 333, standard 112, enterprise 107

CATALOGUE METRIC	MEASURED
Tool bindings across catalogue	2,128 (avg 3.9/agent)
Eval cases across catalogue	8,526 (avg 15.4/agent)
Avg <code>system_prompt</code> size	~4,611 chars (~700 words)
Avg <code>knowledge</code> payload	~4,095 chars serialized

Each eval case is a structured oracle, e.g. `{ id, channel, lang, input, expect: { tool, says_any: [...] } }` — 8,526 of these are hand-designed test specs, not incidental fixtures. Authoring effort per original family therefore bundles: domain research, a ~700-word system prompt, a curated knowledge base, ~4 tool contracts, guardrails, and ~15 evals, then iteration against the eval suite. This is why the catalogue carries ~22–24 PM even though ~80% of SKUs are machine-derived from ~100 originals — the ~100 originals and the 8,526-case eval corpus are genuine human authoring.

20.6 Complexity indicators (drivers of the effort above)

These raise complexity beyond a naive LOC count and justify the organic-team schedule tail:

INDICATOR	MEASURED DETAIL	COMPLEXITY CONTRIBUTION
Model adapters	4 live modes selected by <code>MIAI_MODEL_MODE</code> — <code>openai</code> , <code>azure</code> (Azure OpenAI), <code>anthropic claude</code> , <code>gateway http</code> — plus a deterministic <code>mock</code> default (<code>packages/runtime/src/index.ts</code>)	Provider abstraction, streaming, usage accounting, fail-closed metering
Connector framework	16-entry registry (9 Phase-1 + 7 Phase-2) incl. Google Calendar, M365 Calendar, Shopify, HubSpot, Slack, Email, Teams, Xero, QuickBooks, Calendly, Zendesk, plus <code>webhook</code> / <code>mcp</code> / <code>stripe</code> / <code>whatsapp</code> / <code>woocommerce</code> ; OAuth2/PKCE, AES-GCM sealed tokens, SSRF pin, retry (<code>packages/connectors/src</code>)	Per-provider integration + auth + security hardening
Multi-tenant isolation	<code>tenant_id</code> predicate per query, per-tenant public key + knowledge base + domain allowlist; B2B OIDC Bearer + consumer OIDC/mock + embed keys + webhook HMAC	Cross-cutting; must be correct in every route
Regional compliance packs	5 market packs (US CCPA/TCPA, EU GDPR, Africa/ZA POPIA, Asia PDPA, Oceania AU/NZ); DSAR export/erase, consent, audit, retention, PII redaction, AI disclosure	Legal-surface duplication ×5 + Trust Center
Eval corpus	8,526 static+live cases, nightly <code>eval-nightly.yml</code> , <code>eval:suite</code> / <code>eval:full</code>	Large test-authoring + CI cost
Runtime engine	bounded plan→act→observe loop (≤3 steps, credit-checked), hybrid RAG that switches from prompt-stuffing to vector retrieval above ~6,000-char KB, 22 workflow modules (20 verticals + <code>i18n</code> + <code>stop-suppression</code>), guardrails	Non-trivial control logic, not CRUD

INDICATOR	MEASURED DETAIL	COMPLEXITY CONTRIBUTION
Persistence	Postgres (no ORM, raw <code>pg</code>) with JSON file-store fallback; 5 SQL migrations <code>001_init</code> → <code>005_consumer_reminders</code> ; consumer memory layers (sessions, facts, life-graph, KB, brief+reminders)	Schema design + dual-store parity
Security hardening	dual-flag mock-rail guards (<code>ALLOW MOCK RAILS</code> + <code>I_UNDERSTAND_*</code>), CSP, embed CORS + frame-ancestors, DNS-pinned SSRF, JWT alg-pinning, open-redirect guard, agent-IP redaction on public reads	Adds review + test depth beyond feature LOC
DevOps surface	multi-stage Dockerfile → Railway (<code>railway.toml</code>), gosu privilege-drop, health <code>/api/health</code> ; Azure Container Apps + Azure OpenAI target (<code>infra/azure</code> Bicep); 5 CI workflows ; 5 ADRs; 171 docs	Operability + handover overhead

20.7 Effort summary

```

Measured size ..... 406,319 lines / 1,212 files (74% catalogue IP, 19% runtime)
Runtime (authored) .. ~64.4 KLOC → 28-32 PM (organic-adjusted)
Catalogue IP ..... 100 families + 400 derivations + 8,526 evals → 22-24 PM
Total effort ..... ~50-56 person-months ≈ 4.2-4.7 person-years
Schedule ..... ~12-16 calendar months, ~4-6 engineers + 2-3 authors
Indicative cost ..... ~$0.75M - $0.95M (effort model, NOT a quote/price)
Upper anchor ..... textbook COCOMO on runtime alone ≈ 190-227 PM

```

The single largest rebuild risk is not the runtime — it is reproducing the **299,508-line catalogue IP** (100 tuned domain families, 2,128 tool contracts, 8,526 eval oracles, 5 compliance packs), which lines-of-JSON *understate* because each line is curated domain knowledge rather than boilerplate.

21 Visual Reports

Print-optimised, at-a-glance metrics dashboard. Every figure is re-measured from the repository at commit `fb97438` (branch `main`, 2026-08-25), excluding `node_modules/.next/dist/.git`. Bars are drawn with block characters so they render identically in PDF. Where a number differs from the earlier Licence Proposal, the delta is called out – the repository has grown.

21.1 Headline KPI strip

METRIC	VALUE	SOURCE OF TRUTH
Total authored lines	406,319	1,212 files (ts/tsx/js/mjs/json/md/sql/css)
Catalogue IP	299,508 lines	<code>data/catalog</code> 290,061 + <code>data/catalog-consumer</code> 9,447
Agents (SKUs) × 5 markets)	500 (100 families	552 <code>.agent.json</code> files on disk incl. 51 ZA deep-link aliases + 1 global
Consumer specialists	17	<code>data/catalog-consumer/*.agent.json</code>
API routes	69	<code>route.ts</code> under <code>apps/web/src/app/api</code>
Page routes	39	<code>page.tsx</code> under <code>apps/web/src/app</code>
Connector Actions	16	Google/M365/Slack/Shopify/HubSpot/Xero/QuickBooks/Calendly/Zendesk + Teams/webhook/MCP
Catalogue tools	2,128	summed across 552 agents (avg 3.86/agent)
Eval cases	8,526	summed across 552 agents (avg 15.45/agent)
Regions	5	US · EU · Africa (incl. ZA) · Asia · Oceania
Pricing tiers	3	standard \$349 · pro \$699 · enterprise \$1,199 /mo (<code>RENT_USD</code>)

KPI SNAPSHOT

406K lines 500 agents 69 routes 16 connectors 8,526 evals 5 regions

21.2 LOC distribution – where the 406,319 lines live

The catalogue IP is ~3× the entire hand-written platform. Any valuation that headlines only the runtime undercounts the asset by ~299K lines.

Catalogue IP (JSON, 500 agents + 17 consumer)	299,508	73.7%	
Runtime (hand-written TS/TSX/JS, excl presets)	63,357	15.6%	
Presets (generated: generated-presets.ts)	12,245	3.0%	
Other (docs · SQL · CSS · config · index JSON)	31,209	7.7%	
TOTAL AUTHORED	406,319	100%	

Runtime bucket, broken out (hand-written + generated presets = ~75.6K LOC / 408 files):

PACKAGE / AREA	LOC	FILES	NOTE
<code>apps/web</code>	34,550	280	Next.js 15 App Router, 69 API + 39 page routes
<code>packages/runtime</code>	13,818	36	<code>runTurn()</code> , model adapter, RAG, guardrails, workflows
<code>packages/presets</code>	12,245	2	generated (<code>generated-presets.ts</code> ≈ 11,761)
<code>scripts</code>	9,110	34	catalogue build, evals, proofs, cutover
<code>packages/connectors</code>	4,293	19	OAuth2/PKCE, SSRF, sealed tokens, live handlers
<code>e2e</code>	1,095	34	Playwright <code>@smoke/@functional/@uat/@handover</code>
<code>packages/wallet-adapter</code>	349	2	prepaid metering adapter
<code>packages/agent-protocol</code>	142	1	tiers + <code>RENT_USD</code> / <code>RENT_EUR</code>
Runtime subtotal	75,602	408	

21.3 Catalogue by market pack

500 canonical SKUs (100 families × 5 regions). Africa carries 51 additional legacy ZA aliases retained for deep links (ADR 0001), plus 1 `global` agent, for 552 files on disk.

Market pack	Agents	(bar scaled to Africa = 151)	
Africa (incl. ZA)	151		(100 canonical + 51 ZA aliases)
United States	100		
European Union	100		
Asia	100		
Oceania	100		
Global	1		
On-disk total	552		

Market packs encode jurisdictional compliance: US CCPA/TCPA · EU GDPR · Africa POPIA (incl. ZA) · Asia PDPA · Oceania AU/NZ.

21.4 Catalogue by tier

Tier is read from `manifest.tier`; distribution sums to all 552 files. Monthly rent from `packages/agent-protocol` `RENT_USD`.

Tier	Agents	Rent/mo	(bar scaled to Pro = 333)
Pro	333	\$ 699	
Standard	112	\$ 349	
Enterprise	107	\$1,199	
Total	552		

TIER	AGENTS	SHARE	USD/MO	EUR/MO
Standard	112	20.3%	\$349	defined (<code>RENT_EUR</code>)
Pro	333	60.3%	\$699	defined
Enterprise	107	19.4%	\$1,199	defined

21.5 API endpoints by domain (69 routes)

Each `route.ts` assigned to exactly one domain; counts sum to 69.

Domain	Routes	(bar scaled to Consumer = 15)
Consumer app	15	
Connectors & OAuth	8	
Ops & Observability	7	
Chat & Agent runtime	5	
Knowledge / RAG	5	
Catalog & Discovery	5	
Compliance & Governance	5	
Payments & Wallet	4	
Integrations & Channels	4	
Auth & Tenancy	3	
Growth & Leads	3	
Public API (v1)	3	
Rentals & Commerce	2	
TOTAL	69	

DOMAIN	REPRESENTATIVE ROUTES
Consumer app	<code>consumer/auth/*</code> , <code>consumer/chat</code> , <code>consumer/brief*</code> , <code>consumer/reminders*</code> , <code>consumer/connectors*</code> , <code>consumer/wallet</code> , <code>consumer/telegram/webhook</code>

DOMAIN	REPRESENTATIVE ROUTES
Connectors & OAuth	<code>connectors</code> , <code>connectors/credentials</code> , <code>oauth/[connector]/{start,disconnect,test}</code> , <code>oauth/{callback,status}</code> , <code>slack/channels</code>
Ops & Observability	<code>health</code> , <code>version</code> , <code>ops</code> , <code>insights</code> , <code>history/turns</code> , <code>history/trace/[correlationId]</code> , <code>proof/tool</code>
Chat & Agent runtime	<code>chat</code> , <code>app/chat</code> , <code>ask/chat</code> , <code>embed/chat</code> , <code>agents/[id]</code>
Knowledge / RAG	<code>knowledge</code> , <code>knowledge/[id]</code> , <code>knowledge/{crawl,paste,upload}</code>
Catalog & Discovery	<code>catalog</code> , <code>catalog/family/[familyId]</code> , <code>catalog/personal</code> , <code>configure</code> , <code>onboarding</code>
Compliance & Governance	<code>dsar/export</code> , <code>dsar/erase</code> , <code>consent</code> , <code>audit</code> , <code>admin</code>
Payments & Wallet	<code>payments/paystack/{init,return,webhook}</code> , <code>wallet</code>
Integrations & Channels	<code>mcp</code> , <code>mcp/tools/call</code> , <code>webhook/sink</code> , <code>embed/sri</code>
Auth & Tenancy	<code>auth/handoff</code> , <code>workspace/members</code> , <code>workspace/members/[userId]</code>
Growth & Leads	<code>ask/leads</code> , <code>custom-requests</code> , <code>custom-requests/[id]</code>
Public API (v1)	<code>v1/openapi</code> , <code>v1/rent</code> , <code>v1/embed/chat</code>
Rentals & Commerce	<code>rent</code> , <code>rentals</code>

21.6 Security controls matrix

Every control below was located in code (path cited). All read **Live** in the shipped runtime.

CONTROL	STATE	EVIDENCE (REPO PATH)
Content-Security-Policy	● Live	<code>apps/web/src/lib/csp.ts</code>
Embed CORS allowlist + <code>frame-ancestors</code>	● Live	<code>apps/web/src/lib/embed-cors.ts</code> , <code>csp.ts</code>
SSRF DNS-pinned outbound check	● Live	<code>packages/connectors/src/ssrf.ts</code>
AES-GCM sealed OAuth tokens	● Live	<code>packages/connectors/src/oauth/tokens.ts</code>

CONTROL	STATE	EVIDENCE (REPO PATH)
Webhook HMAC-SHA256 (in + out)	● Live	<code>packages/connectors/src/webhook-sig.ts</code> , <code>api/webhook/sink/route.ts</code>
B2B OIDC Bearer + JWKS / JWT alg-pin	● Live	<code>apps/web/src/lib/auth.ts</code> , <code>agents-auth.ts</code>
Dual-flag mock-rails guard	● Live	<code>apps/web/src/lib/security-flags.ts</code> (<code>ALLOW MOCK RAILS</code> + <code>I UNDERSTAND *</code>)
Open-redirect guard (<code>safeReturnPath</code>)	● Live	consumer <code>auth/callback</code> , <code>auth/logout</code> routes
Agent-IP redaction on public reads	● Live	<code>apps/web/src/lib/agent-ip.ts</code> → <code>api/agents/[id]/route.ts</code>
Per-tenant isolation (<code>tenant_id</code> predicate)	● Live	queries across <code>apps/web/src/lib</code> (no ORM; <code>pg</code>)
Embed key signing (<code>mia_pk_</code>)	● Live	embed key secret + <code>data-key</code>
Container privilege-drop (<code>gosu</code>)	● Live	multi-stage <code>Dockerfile</code> entrypoint

SECURITY POSTURE 12 / 12 controls verified in code

100% Live

21.7 Readiness – Live / Partial / Planned

The Trust Center (`apps/web/src/app/trust/page.tsx`) publishes an honest tri-state per capability. Status tags appear throughout the web app (measured raw occurrences: `live` ≈ 80, `partial` ≈ 8, `planned` ≈ 8). Posture, grouped by what is verifiable in the shipped runtime:

STATE	CAPABILITIES
● Live	Runtime engine (<code>runTurn</code> , plan→act→observe ≤3, fail-closed metering); catalogue serving (552 agents); OIDC B2B + consumer Google/mock auth; connector OAuth framework; knowledge/RAG (hybrid retrieval > ~6k chars); MCP server; embeddable JS widget + Telegram webhook; DSAR export/erase, consent, audit, retention; Paystack top-ups; Postgres persistence (5 migrations) + file-store fallback; full security stack (§21.6); Railway deploy + <code>/api/health</code>
● Partial	Individual connector Actions (mix of live vs mock per provider, dual-flag gated); WhatsApp channel; live-LLM eval scoreboard; Stripe (<code>STRIPE_SECRET_KEY</code> present — Paystack is the live money rail)
● Planned	Azure Container Apps + Azure OpenAI + App Insights migration target (<code>infra/azure</code> Bicep, <code>scripts/validate-azure.sh</code> present; Railway is the running host)

READINESS MIX (capability groups)

Live		~80% status tags
Partial		~10%
Planned		~10%

21.8 Supporting inventory (governance & delivery)

ARTEFACT	COUNT	LOCATION
ADRs	5	docs/adr/0001..0005 (README index)
Design docs	33	docs/*.md
SQL migrations	5	apps/web/migrations/001_init ... 005_consumer_reminders
CI workflows	5	ci.yml , publish-image.yml , e2e-staging.yml , eval-nightly.yml , daily-brief.yml
Non-agent catalogue index files	3	data/catalog/{families,index,market-packs}.json

21.9 Growth since Licence Proposal

The repo has materially grown; report the measured numbers, not the older quote.

DIMENSION	LICENCE PROPOSAL	MEASURED @ FB97438	Δ
Authored lines	~398,000	406,319	+8.3K
Files	1,021	1,212	+191
Catalogue LOC	~298,000	299,508	+1.5K
API routes	45	69	+24
Catalogue tools	~1,900	2,128	+228
On-disk incl. deps	~631 MB	~805 MB	+174 MB

All figures in §21 are reproducible: `find ... -name '*.json' | xargs cat | wc -l` for catalogue LOC, `find apps/web/src/app/api -name route.ts | wc -l` for routes, and the per-agent tool/eval sums via a Node pass over `data/catalog/*.agent.json`. Numbers reconcile exactly – the four LOC buckets in §21.2 sum to 406,319.

22 Handover & Cutover Plan

This closing section is the **technical** handover and cutover plan. It covers what is delivered as running software, the four cutover topologies (A–D) with engineering effort and time, the recommended sequence, the 14-day fast-track, exactly what the platform needs from the partner to go live, and the managed-Postgres prerequisite that gates the partner-tenancy topology. Every figure below was re-measured on the repository at commit `fb97438`; every mechanism is cited to a real path. Nothing in this section describes commercial arrangements — only deployment.

22.1 What is delivered technically

The deliverable is a **running, multi-tenant marketplace-plus-runtime**, not a spec. Measured across the repository (excluding `node_modules/.next/dist/.git`):

ASSET	MEASURED SIZE	WHERE
Total authored	406,319 LOC / 1,212 files (ts/tsx/js/mjs/json/md/sql/css)	whole repo
500-agent catalogue IP (largest asset)	299,508 LOC / 573 files	<code>data/catalog</code> (555 files / 290,061 LOC) + <code>data/catalog-consumer</code> (18 files / 9,447 LOC)
Hand-written platform runtime	~75,900 LOC / ~410 files	<code>apps/web</code> 34,550/280, <code>packages/runtime</code> 13,818/36, <code>packages/presets</code> (generated) 12,245/2, <code>scripts/</code> 9,110/34, <code>packages/connectors</code> 4,293/19, <code>e2e/</code> 1,095/34, <code>packages/wallet-adapter</code> 349/2, <code>packages/agent-protocol</code> 142/1
API surface	69 API route modules, 39 pages	<code>apps/web/src/app/api/**/route.ts</code> , <code>apps/web/src/app/**/page.tsx</code>
Persistence	13 tables across 5 migrations	<code>apps/web/migrations/001_init.sql</code> .. <code>005_consumer_renders.sql</code>

Concretely, handover transfers:

- **The catalogue** — 100 agent families × 5 market packs (US / EU / Africa incl. ZA / Asia / Oceania) = 500 SKUs as flat `data/catalog/{id}.agent.json` bundles (keys `format`, `manifest`, `system_prompt`, `knowledge`, `tools`, `guardrails`, `evals`), ~555 files on disk (500 SKUs + retained legacy ZA aliases for deep links per ADR `0001-catalog-100x5`), plus 17 consumer specialists. Tiering and monthly rent constants live in `packages/agent-protocol/src/index.ts` (`RENT_USD` = 349 / 699 / 1199; `RENT_EUR` mirrored).
- **The execution engine** — `@miai/runtime` (`runTurn()`, `createModelAdapter()`), the convergence hub that imports all four other workspace packages; guardrails, hybrid semantic retrieval, 15+ per-vertical workflow modules under `packages/runtime/src/workflows`, and a bounded plan→act→observe loop with fail-closed metering (zero balance returns a top-up message and makes **no** provider call).
- **Evals & docs** — the eval suite over the catalogue (`pnpm eval:suite` / nightly `eval-nightly.yml`), 5 ADRs in `docs/adr`, and 30+ runbooks/specs in `docs/` (including `MIGRATION_RUNBOOK.md`, `MIGRATION_P0.md`,

`PLATFORM_INTEGRATION.md`, `RAILWAY_DEPLOY.md`, `APP_CHANNEL.md`, `B2B_ONBOARDING.md`, `HANDOVER_TEST_PACK.md`).

- **The connector / channel surface** — `packages/connectors` (OAuth2/PKCE + AES-GCM sealed tokens; Google Cal/Gmail, Microsoft M365, Slack, Shopify, HubSpot, Xero, QuickBooks, Calendly, Zendesk, and 16 Actions incl. Teams/webhook/MCP) and the channel matrix: web chat, embeddable JS widget (`apps/web/src/app/agents/v1/agent.js` + `/api/embed/chat`), Telegram, WhatsApp, an MCP server (`/api/mcp` + `/api/mcp/tools/call`), and the native App channel (`/app/v1` + `/api/app/chat`).
- **The deployment substrate** — multi-stage `Dockerfile` (gosu privilege-drop entrypoint) → Railway (`railway.toml` , health `/api/health`), plus Azure Container Apps IaC (`infra/azure/main.bicep`) as the migration target.

Handover evidence already exists: `pnpm handover:staging` (Playwright `@smoke` / `@functional` / `@uat` / `@handover`) and `pnpm run ci` (build + typecheck + test + catalog integrity + static evals), with the last recorded full run **78/78 Playwright passed** on 2026-08-02 at commit `feb6d43` (`docs/HANDOVER_TEST_PACK.md`). The merge gate is the `quality` job; SonarCloud is advisory.

22.2 The four cutover topologies

The runtime is env-selected — `MIAI_AUTH_MODE`, `MIAI_WALLET_MODE`, `MIAI_MODEL_MODE`, `DATABASE_URL` vs file fallback — so the same image serves all four topologies. What differs is *where the runtime runs* and *how the partner reaches it*.

TOPOLOGY	DESCRIPTION	ENGINEERING EFFORT	TIME	SHIPS / DEPLOYS
A — API/MCP from partner storefront	Partner calls the versioned REST API (<code>/api/v1/*</code> , OpenAPI at <code>/api/v1/openapi</code>) and/or the MCP server (<code>/api/mcp</code> , <code>/api/mcp/tools/call</code>) from their own front end; embed/app channels via publishable keys	None partner-side; config only	Available now	Nothing to deploy; point clients at the hosted host
B — Partner domain → hosted runtime	Partner hostname resolves to the hosted runtime with partner brand/TLS/URLs; widget loader + snippet derive host at runtime	DNS + config change only	~2 weeks (recommended first launch)	DNS/Front Door + <code>APP_BASE_URL</code> / <code>NEXT_PUBLIC_APP_URL</code> + OAuth redirect URIs
C — Embedded in partner console (signed SSO handoff)	Marketplace + per-agent setup embedded in the partner console; no second login via OIDC Bearer + <code>/api/auth/handoff</code>	B + SSO handoff wiring	~3 weeks	Same host as B; adds <code>MIAI_AGENTS_AUTH_URL</code> + <code>product=agents</code> / <code>return_to</code> handoff

TOPOLOGY	DESCRIPTION	ENGINEERING EFFORT	TIME	SHIPS / DEPLOYS
D – Container into partner Azure tenancy	Runtime ships as a container into the partner's own Azure subscription; partner owns the data plane	B/C + IaC deploy + data-plane migration	Week-scale, gated on managed-Postgres first	infra/azure/main.bicep → Container Apps + Postgres + Key Vault + Files + App Insights

Topology A – REST/MCP API (now). The platform already exposes a versioned API namespace ([apps/web/src/app/api/v1/](#): [openapi](#), [embed/chat](#), [rent](#), with legacy unversioned routes retained for existing clients), the MCP server pair, the embed chat endpoint, and the App-channel SSE endpoint [POST /api/app/chat](#). Business calls authenticate with OIDC Bearer + [workspace_id](#); embed/app calls authenticate with publishable keys ([mia_pk_...](#)); webhooks use HMAC-SHA256. A partner can consume the running service from their own storefront with **nothing deployed on their side** – this topology is live today against staging (<https://miaiweb-production.up.railway.app>).

Topology B – partner domain, hosted runtime (~2 weeks, recommended first launch). The widget loader and snippet generator derive their host **at runtime** rather than baking in a build-time origin, which is what makes B a DNS-plus-config change and not a rebuild. In [apps/web/src/lib/agent-js-script.ts](#):

```
var current = document.currentScript;
var key = current && current.getAttribute("data-key");
// host is derived from the script tag's own origin, not hard-coded:
var origin = (current && current.src) ? new URL(current.src).origin : window.location.origin;
...
fetch(origin + "/api/embed/chat", { ... }); // calls back to whatever host served the script
```

and the snippet generator emits `<script src="{options.src}" data-key="{options.key}" integrity="..." ...>` – so pointing the `src` at a partner hostname is sufficient for the widget to call the partner hostname. Server-side, public URLs are read from [APP_BASE_URL](#) / [NEXT_PUBLIC_APP_URL](#). Cutover to B is therefore: publish the image, set [APP_BASE_URL](#) to the partner host, point DNS/Front Door at the runtime, and update OAuth callback URLs to [https://<partner-host>/api/oauth/callback](#) ([docs/MIGRATION_RUNBOOK.md](#), Phase 1). This is the recommended first launch because it is reversible in a DNS TTL and requires no partner-side code.

Topology C – embedded in partner console via signed SSO handoff (~3 weeks). C layers a no-second-login seam on top of B. The runtime already ships the handoff primitives: [apps/web/src/lib/agents-auth.ts](#) builds an auth URL from [MIAI_AGENTS_AUTH_URL](#) (or [NEXT_PUBLIC_MIAI_AGENTS_AUTH_URL](#)) carrying `product=agents` and an absolute `return_to`, and [GET /api/auth/handoff](#) returns `{ mode, loginUrl, consumerAppUrl }` for clients to resolve where to send the user. With the partner's OIDC issuing workspace-scoped Bearer tokens (`MIAI_AUTH_MODE=oidc`), the partner console can hand a signed session straight into the embedded marketplace, so browse → rent → configure → Live Ops happens inside their product with no separate login. The extra ~1 week over B is wiring the partner's SSO endpoint and workspace-provision call (`company → workspace_id`) into the handoff, per [docs/B2B_ONBOARDING.md](#) / [docs/PLATFORM_INTEGRATION.md §1](#).

Topology D – container in the partner's Azure tenancy (week-scale, gated). D moves the data plane into the partner's subscription. The image is already Azure-ready: [infra/azure/main.bicep](#) provisions a Container App, a PostgreSQL flexible server ([Microsoft.DBforPostgreSQL/flexibleServers](#)), Key Vault (wallet/model/OAuth/embed/state signing secrets, connector client id/secret pairs), an Azure Files share mounted at `/data`, a user-assigned managed identity with Key Vault

Secrets User, and App Insights. Because the two marketplace packages sit on a small, well-bounded internal dependency graph — `@miai/runtime` imports only `agent-protocol`, `connectors`, `presets`, `wallet-adapter`, and `presets` imports only `connectors` (verified from each `packages/*/package.json`), i.e. ~6 modules from the wider codebase plus `pg` — extraction to a standalone container is **week-scale, not a rewrite**. D is deliberately **not** the launch topology: it is gated on the managed-Postgres migration in §22.6.

22.3 Recommended sequence

```

Now → Topology A (API/MCP, zero partner deploy)
  |
  ▼ ~2 weeks
Launch on Topology B (partner domain → hosted runtime; DNS + config; reversible in a DNS TTL)
  |
  ▼ +~1 week
Layer Topology C (signed SSO handoff dissolves the login seam; marketplace embedded in partner console)
  |
  ▼ when partner wants to own the data plane
Topology D (container into partner Azure tenancy) — GATED on §22.6 managed-Postgres migration first

```

Launch on **B** (fast, reversible, no partner-side code), **layer C** to remove the second login, and hold **D** as the Azure-tenancy option that is only opened once managed Postgres is in place. A and the API/MCP surface remain available throughout for storefront integrations.

22.4 The 14-day technical fast-track

The fast-track flips one adapter family at a time so each window has an independent green/rollback signal. All windows target the partner hostname and use the staging smoke (`pnpm smoke:cutover`) and health probe (`/api/health` expecting `storePing=ok`, `database=configured`, `telemetry=appinsights|console`).

WINDOW	DAYS	OUTCOME	PRIMARY ENV / CHECKS
1. OIDC login end-to-end	1–3	Partner SSO signs a user into Agents on the partner host	<code>MIAI_AUTH_MODE=oidc</code> , <code>MIAI_OIDC_ISSUER</code> / <code>AUDIENCE</code> / <code>JWKS_URL</code> ; JWT carries <code>workspace_id</code> (<code>lib/auth.ts</code> + <code>agents-auth.ts</code>); OIDC Bearer 401 matrix
2. Wallet debit + model gateway on one hero agent	4–7	activate → chat → bill on a single hero SKU	<code>MIAI_WALLET_MODE=http</code> (+ URL/key), <code>MIAI_MODEL_MODE=gateway</code> (+ gateway URL/key, alias map); tool-calling smoke; fail-closed at zero balance
3. Custom domain + OAuth redirects + UAT	8–11	Partner host + OAuth callbacks live; UAT over a go-live shortlist	<code>APP_BASE_URL</code> / <code>NEXT_PUBLIC_APP_URL</code> ; OAuth apps → <code>https://<partner-host>/api/oauth/callback</code> (keep prior URI until cutover); <code>pnpm uat:staging</code>

WINDOW	DAYS	OUTCOME	PRIMARY ENV / CHECKS
4. Cutover rehearsal, mock rails off, Live Ops green	12–14	Rehearsed DNS flip; mock rails disabled; Live Ops/health green on partner host	Unset <code>ALLOW MOCK RAILS</code> and <code>I_UNDERSTAND MOCK RAILS IN PROD</code> ; <code>pnpm smoke:cutover</code> ; watch App Insights + <code>/api/health</code>

The env flip between staging (mock) and production is a table in the runbook:

	staging (Railway)	production (partner host)
MIAI_AUTH_MODE	mock	→ oidc
MIAI_WALLET_MODE	mock	→ http
MIAI_MODEL_MODE	mock	→ gateway
DATABASE_URL	optional/unset	→ managed Postgres
APP_BASE_URL	Railway URL	→ partner host (native + App-channel base)

Rollback is a DNS/Front Door flip back to the last-good host with mock/live creds that worked pre-cutover; **rollback RTO target = DNS TTL + 15 minutes**, so keep TTL ≤ 300s during the window (`docs/MIGRATION_RUNBOOK.md`, *Rollback*).

22.5 What is required from the partner to go live

The remaining work is **wiring the partner's identity, wallet, models, and hosting** — not new invention (`docs/MIGRATION_P0.md` / `docs/CATALOGUE_READY.md`). Without items 1–3 even as staging stubs, the first window stays infra-documentation only.

#	RAIL	EXACTLY WHAT IS NEEDED	CONSUMED BY
1	Production OIDC	Issuer, audience, JWKS URL, and a sample JWT with <code>workspace_id</code> (or <code>workspaceId</code> / <code>org_id</code>), <code>sub</code> / <code>user_id</code> , <code>roles[]</code> (incl. <code>owner</code> for the workspace creator); optional <code>product=agents</code>	<code>MIAI_OIDC_ISSUER/AUDIENC</code> <code>E/JWKS_URL</code> , <code>lib/auth.ts</code> + <code>agents-auth.ts</code>
2	Production wallet API	Base URL + API key + debit/balance/top-up contract with sample responses; pause-on-zero confirmed (<code>GET /v1/wallets/:id</code> , <code>POST .../debit {amount, idempotencyKey, reason, agentId}</code> , top-up/redeem link)	<code>MIAI_WALLET_MODE=http</code> , <code>MIAI_WALLET_API_URL/KEY</code> , <code>packages/wallet-adapter</code>
3	Model gateway	Base URL + auth + tool/function-calling support + model alias map (<code>gemini-flash</code> , <code>gpt-4o-mini</code> , <code>claude-sonnet</code> , <code>gpt-4o</code> , <code>claude-opus</code>)	<code>MIAI_MODEL_MODE=gateway</code> , <code>MIAI_MODEL_GATEWAY_URL/K</code> <code>EY</code>
4	Cloud + DNS/TLS	Cloud subscription + region + production hostname/DNS and TLS for <code>APP_BASE_URL</code> and embed (<code>/agents/v1/agent.js</code>); OAuth redirect URI updates to the new host	Front Door/DNS, <code>APP_BASE_URL</code> , OAuth callback allowlist
5	WhatsApp	WABA / BSP ownership + credentials (separate channel; WABA ownership TBD)	WhatsApp channel / webhook

Signing secrets (`oauthTokenSecretParam`, `oauthStateSecretParam`, `embedKeySecretParam`, each ≥32 chars) are generated **once**, stored, and reused on every deploy — rotating them invalidates existing embed keys and sessions and can

crash-loop the app on the boot-hardening floor ([docs/MIGRATION_RUNBOOK.md](#) , *Deploy Azure*).

22.6 Managed-Postgres prerequisite for Topology D

Topology D — partner owns the data plane — is **gated on completing the managed-Postgres migration first**. Persistence has two backends (ADR [0003-postgres-persistence](#)): when [DATABASE_URL](#) (or [MIAI_DATABASE_URL](#)) is set, the app runs versioned migrations on boot and uses **Postgres** via raw [pg](#) (no ORM) with row-level upserts; otherwise it falls back to **JSON files on the [/data](#) volume** (or in-memory for CI). The five migrations create **13 tables** that together are the data plane:

```
miai_rentals · miai_audit · miai_turns · miai_oauth_tokens · miai_knowledge_sources  
miai_workspace_members · miai_ask_leads · miai_custom_requests  
miai_consumer_memory · miai_consumer_goal · miai_consumer_person  
miai_consumer_brief · miai_consumer_reminder
```

For a partner-owned data plane, three conditions must hold before D is safe:

- 1. All durable state on managed Postgres, not the Files mount.** [DATABASE_URL](#) must point at the partner's managed Postgres so rentals, audit, turns, OAuth tokens, knowledge, workspace members and the consumer memory/goals/people/brief/reminder tables all live in the managed database. ADR 0003 is explicit that operators must prefer Postgres for multi-replica production and that "file fallback is not a substitute for backup, replication, or cross-region DR." The runbook's post-cutover backlog item is exactly this: *"Schedule P2: Postgres for OAuth/knowledge if still on Files."*
- 2. Single-replica until shared state is externalised.** [infra/azure/main.bicep](#) defaults [maxReplicas = 1](#) , and the runbook requires [maxReplicas](#) stay at 1 **until Redis is provisioned**, because chat sessions and rate limits are per-replica otherwise (optional Upstash Redis shares [miai:rl:*](#) / [miai:chan:*](#)). Horizontal scale in the partner tenancy therefore needs both managed Postgres and Redis.
- 3. Secrets and OAuth re-issued in the partner tenancy.** OAuth tokens on a file volume are not portable unless [OAUTH_TOKEN_STORE_PATH](#) + the same [OAUTH_TOKEN_SECRET](#) are copied; the runbook prefers **re-connect** in the target environment, and publishable App/embed keys issued after rent in the target are authoritative — Railway/staging demo keys must not be reused in production. Key Vault holds [database-url](#) , the three signing secrets, wallet/model keys, and connector client id/secret pairs.

Until those are done, hold at Topology B/C on the hosted runtime (single stable Front Door host so App URLs and native builds don't chase FQDNs) and treat D as the follow-on once managed Postgres — and, for scale, Redis — are in the partner's Azure subscription.

22.7 Cutover & rollback quick reference

CUTOVER (per docs/MIGRATION_RUNBOOK.md)

1. Publish image (publish-image.yml → GHCR/ACR); set containerImage + APP_BASE_URL
2. Bicep deploy (Topology D) or point DNS/Front Door at hosted runtime (Topology B/C)
3. Flip MIAI_AUTH_MODE=oidc, MIAI_WALLET_MODE=http, MIAI_MODEL_MODE=gateway
4. OAuth callback host → partner host (keep prior URI 24h)
5. Unset ALLOW MOCK_RAILS + I_UNDERSTAND MOCK_RAILS_IN_PROD
6. Set EMBED_ALLOWED_ORIGINS to production hostnames (no bare *)
7. Smoke: pnpm smoke:cutover; curl /api/health (storePing=ok, database=configured)
8. Watch App Insights + /api/health 30-60 min

ROLLBACK (RTO target = DNS TTL + 15 min; keep TTL ≤ 300s)

1. Flip DNS/Front Door back to last-good host
2. Ensure prior mock/live creds intact; re-add prior OAuth redirect URIs
3. Leave the new environment running for diagnosis; file incident notes

Data decision at cutover is **fresh start** by default (production tenants; staging was demo-only); *import rentals* only on explicit need via a Postgres dump into the target `miai_rentals` (`docs/MIGRATION_RUNBOOK.md` , *Data decision*).

A Complete Endpoint Index

All 69 route handlers under `apps/web/src/app/api` (contracts in §07):

Consumer & channels: `/api/consumer/auth/{login,callback,logout,me}`,
`/api/consumer/{chat,wallet,connectors,reminders,brief}`, `/api/consumer/connectors/{connector}/{start,disconnect}`,
`/api/consumer/reminders/dismiss`, `/api/consumer/brief/{run,run-due}`, `/api/consumer/telegram/webhook`, `/api/app/chat`,
`/api/chat`, `/api/ask/{chat,leads}` **Catalog & agents:** `/api/agents/{id}`, `/api/catalog`, `/api/catalog/family/{familyId}`,
`/api/catalog/personal`, `/api/configure`, `/api/onboarding`, `/api/custom-requests`, `/api/custom-requests/{id}` **Knowledge:**
`/api/knowledge`, `/api/knowledge/{crawl,paste,upload}`, `/api/knowledge/{id}` **Connectors & OAuth:** `/api/connectors`,
`/api/connectors/credentials`, `/api/oauth/{connector}/{start,disconnect,test}`, `/api/oauth/callback`, `/api/oauth/status`,
`/api/slack/channels` **Billing:** `/api/wallet`, `/api/rent`, `/api/rentals`, `/api/payments/paystack/{init,return,webhook}` **Embed & public**
v1: `/api/embed/{chat,sri}`, `/api/v1/embed/chat`, `/api/v1/openapi`, `/api/v1/rent` **MCP:** `/api/mcp`, `/api/mcp/tools/call` **Workspace &**
admin: `/api/workspace/members`, `/api/workspace/members/{userId}`, `/api/admin`, `/api/ops` **Ops & history:** `/api/health`,
`/api/version`, `/api/insights`, `/api/history/turns`, `/api/history/trace/{correlationId}`, `/api/proof/tool`, `/api/webhook/sink` **Identity &**
privacy: `/api/auth/handoff`, `/api/consent`, `/api/audit`, `/api/dsar/{export,erase}`

B Environment Variable Catalog

Grouped by concern (from `.env.example` + `Dockerfile`). Modes default to `mock`; production sets the live values.

GROUP	VARIABLES
Modes	<code>MIAI_AUTH_MODE</code> (mockloide) · <code>MIAI_WALLET_MODE</code> (mocklhttp) · <code>MIAI_MODEL_MODE</code> (mocklopenailazurelanthropicgateway) · <code>SANDBOX_MODE</code>
Identity / auth	<code>MIAI_OIDC_ISSUER</code> · <code>MIAI_OIDC_AUDIENCE</code> · <code>MIAI_OIDC_JWKS_URL</code> · <code>MIAI_AGENTS_AUTH_URL</code> · <code>GOOGLE_OAUTH_CLIENT_ID/SECRET</code> · <code>MIAI_SESSION_SECRET</code> · <code>OAUTH_TOKEN_SECRET</code> · <code>OAUTH_STATE_SECRET</code>
Model	<code>MIAI_MODEL_GATEWAY_URL/KEY</code> · <code>OPENAI_API_KEY</code> · <code>ANTHROPIC_API_KEY</code> · <code>AZURE_OPENAI_ENDPOINT/API_KEY/DEPLOYMENT[_LARGE]/API_VERSION</code> · <code>RUNTIME_SEMANTIC_RETRIEVAL</code> · <code>EMBEDDING_API_KEY/BASE_URL/MODEL</code>
Wallet / payments	<code>MIAI_WALLET_API_URL/KEY</code> · <code>PAYSTACK_SECRET_KEY/PUBLIC_KEY/CURRENCY</code> · <code>STRIPE_SECRET_KEY</code>
Data	<code>DATABASE_URL</code> / <code>MIAI_DATABASE_URL</code> · <code>PG_SSL_REJECT_UNAUTHORIZED</code> · <code>PGSSLROOTCERT</code> · <code>CATALOG_DIR</code> · <code>*_STORE_PATH</code> (oauth-tokens, knowledge, rentals, consumer-memory/goals/people/reminders, brief)
Connectors	<code>{GOOGLE,MICROSOFT,SLACK,SHOPIFY,HUBSPOT,XERO,QUICKBOOKS,CALENDLY,ZENDESK}_OAUTH_CLIENT_ID/SECRET</code> · <code>QUICKBOOKS_ENV</code> · <code>SLACK_DEFAULT_CHANNEL</code> · <code>WHATSAPP_TOKEN/PHONE_NUMBER_ID</code> · <code>TEAMS_TEAM_ID/CHANNEL_ID</code> · <code>HANDOFF_EMAIL_TO</code>
Embed / webhooks	<code>EMBED_KEY_SECRET</code> · <code>EMBED_ALLOWED_ORIGINS</code> · <code>ALLOW_EMBED_ORIGIN_STAR</code> (+ <code>I_UNDERSTAND_*</code>) · <code>WEBHOOK_SINK_SECRET</code> · <code>WEBHOOK_SINK_HMAC_ONLY</code> · <code>MCP_SINK_TOKEN</code>
Infra / staging	<code>APP_BASE_URL</code> · <code>NEXT_PUBLIC_APP_URL</code> · <code>PORT</code> · <code>APPLICATIONINSIGHTS_CONNECTION_STRING</code> · <code>UPSTASH_REDIS_REST_URL/TOKEN</code> · <code>ALLOW MOCK RAILS</code> (+ <code>I_UNDERSTAND MOCK RAILS IN PROD</code>)

C Dependency Manifest

apps/web runtime: `next@^15.2.8`, `react@^19`, `react-dom@^19`, `jose@^6.2.5` (JWT/JWKS/OIDC + session signing), `pg@^8.22` (Postgres client/pool — no ORM), `zod@^3.24` (request validation) + all five workspace packages via `workspace:^`. **apps/web dev:** `typescript@^5`, `tailwindcss@^3.4.1`, `postcss@^8`, `eslint@^9` + `eslint-config-next@15.1.0`, `@types/*`. **Root dev:** `@playwright/test@1.54.2`, `tsx@^4.19.3`. Package manager pinned `pnpm@9.15.0`.

Internal graph: connectors→pg; presets→connectors; runtime→{agent-protocol, connectors, presets, wallet-adapter}; agent-protocol & wallet-adapter are zero-dep leaves. **Deliberately absent:** no ORM (raw SQL + pg), no required Redis (optional Upstash), no external UI/state library — a minimal, auditable supply chain.

D In-Repo Documentation Index

docs/ (33 files) + 5 ADRs. Key entries: TECHNICAL_SPEC.md, PLATFORM_INTEGRATION.md, B2B_ONBOARDING.md, CONSUMER_AUTH.md, CONNECTOR_OAUTH.md, APP_CHANNEL.md, SANDBOX.md, RAILWAY_DEPLOY.md, MIGRATION_P0/P1/RUNBOOK.md, FAMILIES_100.md, MARKET_PACKS.md, CATALOGUE_READY.md, PRODUCTION_SCALE_500.md, TRUST_AND_COMPLIANCE.md, AUDIT_RETENTION.md, TESTING.md, HANDOVER_TEST_PACK.md, UAT_CHECKLIST.md, PILOT_PRODUCTION_BAR.md, PARTNERSHIP_KICKOFF_BRIEF.md. ADRs: 0001 catalog-100x5 · 0002 mock-rails-dual-flag · 0003 postgres-persistence · 0004 live-llm-guardrails · 0005 compliance-drafts.

Open items to confirm

- **00 readiness:** Live-connector EXECUTE is only implemented for Slack, webhook, and MCP (packages/connectors/src/live/handlers/); the other 16 providers are OAuth/preset-configured but their execute paths were not verified as live in this pass. The kickoff brief lists Slack/Google Calendar/Gmail/Calendly as staging-live — the Calendar/Gmail/Calendly execute paths were not individually confirmed in code here.
- **00 readiness:** The '~6 modules' extraction-dependency claim (topologies) is taken from the provided ground truth and was not independently traced through the two marketplace packages' import graph in this pass.
- **00 readiness:** agent.js widget loader is served via a route (apps/web/src/app/agents/v1/agent.js/route.ts, 552 bytes) rather than a static file; the stated '6.4 kB zero-dep' payload size was not measured from the served response in this pass.
- **00 readiness:** Repo on-disk size ~805 MB is the total including node_modules; the authored-only du figure could not be captured (the -exclude flags are unsupported by macOS du in the shell used), though the authored line/file counts were measured directly.
- **00 readiness:** checkBootHardening() is imported into apps/web/src/lib/security.ts from security-flags.ts / instrumentation.ts; the exact dual-flag predicate body (ALLOW MOCK_RAILS + I_UNDERSTAND MOCK_RAILS_IN_PROD) was confirmed by reference and env docs but the function implementation was not line-read in this section.
- **01 exec:** Consumer specialist count: measured 17 *.agent.json in data/catalog-consumer (+ index.json = 18 files), not 18 specialists as stated in the brief's ground truth. Report uses 17.
- **01 exec:** Tool count measured at 2,128 across the catalogue (~3.86/agent), higher than the brief's '~1,900 tools' and the TECHNICAL_SPEC's 936 — the spec is stale; 2,128 is current.
- **01 exec:** Model modes: MIAI_MODEL_MODE resolves 5 values (mock + openai + anthropic/claude + gateway/http + azure), not '4 model modes' as phrased in the brief. Report treats it as mock plus 4 live adapters.
- **01 exec:** Distinct family stems measured at ~101 (after stripping market prefixes), vs the ADR 0001 '100 families' model; the extra stem comes from the single 'global' entry and legacy ZA aliasing. The 100x5=500 SKU model still holds; the 552 files = 500 + 51 legacy ZA aliases + 1 global.
- **01 exec:** apps/web/src measured 31,554 TS/TSX LOC across 246 files (ts+tsx only); the brief's 34,550/280 for apps/web includes css/json and the wider app tree. Numbers differ by what file types are counted, not a contradiction.
- **01 exec:** Per-vertical workflow module count stated as '~20' from a directory listing of packages/runtime/src/workflows; exact count not tallied (brief says '10+'). Not independently re-counted to a single figure.
- **01 exec:** Line counts are raw wc -l (includes blank/comment lines); no distinction drawn between blank, comment, and code lines for any figure.
- **02 stats:** Catalogue file count is 573 JSON (not the '~555' in the ground-truth brief): 555 in data/catalog (552 agents + 3 meta) PLUS 18 in data/catalog-consumer. The '~555' figure refers to data/catalog only. Stated explicitly in the section.
- **02 stats:** The 52 extra agent files beyond the 500 canonical SKUs were identified by market (51 with market='africa' beyond the 100 canonical Africa SKUs, +1 with market='global' = personal-assistant.agent.json). No file id literally contains '-za', so the 'legacy ZA alias' characterization is inferred from the Africa-pack overflow + ADR 0001 deep-link-retention rationale, not from an explicit alias marker in the manifests. If the exact alias mapping matters for handover, it should be cross-checked against the catalog-build script.
- **02 stats:** Tool count has two legitimate values: index.json-declared = 1,890 (source of the '~1,900' headline) vs on-disk tool objects = 2,128 (data/catalog) / 2,180 (catalogue-wide). Likewise evals: index-declared 7,640 vs on-disk 8,526. The index appears to carry nominal/rounded readiness counts that diverge from actual file contents; the reason for the divergence (stale index vs intentional summary) was not traced to the generator.

- **02 stats:** Consumer catalogue is 17 specialist agents + 1 index.json (18 JSON files); the ground-truth brief's '17 consumer specialists' counts the index.json as an 18th. Clarified in-section.
- **02 stats:** 'du -sh' on macOS reports allocated (disk) size; the 805 MB / 452 MB / 30 MB figures are apparent-size approximations and can vary a few percent by filesystem block size. The ~30 MB authored-source figure is derived from du over the 8 tracked extensions and is an estimate, not an exact byte count.
- **02 stats:** docs/ has 171 markdown files recursively vs 33 at top level; the brief's '32 docs' matches the top-level count at an earlier snapshot (now 33). Non-blocking, noted in-section.
- **03 structure:** dist/ is git-ignored; the ESM dist/index.js layout is confirmed from each package's exports/main/types fields and from locally-built dist/ dirs present on disk, not from committed build output. A clean checkout has no dist until `build:packages` runs.
- **03 structure:** apps/web src TS/TSX measured at 31,554 LOC / 246 files; including the test/ dir and .css it is 36,551 LOC / 283 files. The lead's 34,550/280 figure sits between these two boundaries (likely src + tests, excl. some CSS); exact inclusion set for that number was not reproduced to the line.
- **03 structure:** The embeddable agent.js (referenced as ~6.4 kB zero-dep) is not a static file under apps/web/public/; it is served dynamically (lib/agent-js-script.ts + /embed + /api/embed/sri). Its exact minified byte size was not re-measured in this section.
- **03 structure:** Per-package LOC for scripts/ (~9,110) and e2e/ (~1,095) from the lead's inventory were not independently re-counted here; only the workspace apps/packages and the catalogue were re-measured.
- **03 structure:** apps/runtime and apps/connectors are confirmed as non-deployed scaffolds by their source (16 and 21 LOC) and self-describing comments, but whether any environment currently runs them was not verified against deploy config beyond the web-only Dockerfile/railway.toml.
- **04 frontend:** Widget size discrepancy: DD brief says '~6.4kB' and the agent-js-script.ts docstring says '~9KB', but the served body measures 13,449 bytes raw / 4,307 gzip / 3,630 brotli at HEAD. The documented figures are stale (source has grown); reconcile which number the pack should quote (recommend '~4.3 KB gzipped over the wire, ~13.4 KB raw, unminified').
- **04 frontend:** The DD brief refers to a '/embed chat surface' as if it were a page. There is no app/embed page route — the embed is the Shadow-DOM widget injected by /agents/v1/agent.js calling /api/embed/chat (+ /api/embed/sri), and /app/v1 (AppChatClient) is the hosted WebView. Confirm the pack's wording so it doesn't imply a Next page that doesn't exist.
- **04 frontend:** I read component structure/heads for the largest client components (CatalogGrid 1,139 LOC, ActionsPanel 678, AgentStudio 550) but did not line-by-line audit their full bodies; deep behavioural review of ActionsPanel (connector/Actions wiring) and KnowledgePanel (upload/crawl/paste) was out of scope for the frontend section and may warrant its own pass.
- **04 frontend:** Consumer specialist count is stated as '17-18' because /agents copy and the ground-truth data/catalog-consumer (18 specialists) differ slightly; the exact live count is computed at runtime from listPersonalAgents() and was not pinned to a single integer here.
- **04 frontend:** Did not measure real Lighthouse/bundle sizes or verify the production build output (no build run); client-bundle weight of the 1,139-LOC CatalogGrid + providers is unquantified.
- **04 frontend:** Accessibility was spot-checked (role=switch on ThemeToggle, role=img on charts, aria-labels on nav/widget) but no systematic a11y audit (focus traps in the widget panel, keyboard nav of the Studio step machine) was performed.
- **05 backend:** Middleware body-size cap checks the content-length header only; a chunked request omitting content-length bypasses the 413 gate and is bounded only by per-field zod max() limits — not verified whether any route enforces a hard byte cap on the read stream.

- **05 backend:** runTurn debit is post-paid within the turn: the pre-call guard only checks balance > 0, and the wallet debit runs after the model reply is generated. If a single turn's token cost exceeds the remaining balance, MockWalletAdapter.debit returns ok:false with tokensDebited:0 (reply delivered but not charged) and paused:true — the overspend on that final turn is effectively unmetered. HttpWalletAdapter backend semantics for the same case were not read.
- **05 backend:** The debit idempotencyKey is `${workspaceId}:${agentId}:${Date.now()}:${messages.length}`, which is unique per invocation rather than derived from the request/correlation id, so it does not dedupe a retried HTTP request (only an in-process re-call with identical timestamp). Whether the HTTP wallet backend adds stronger idempotency was not verified.
- **05 backend:** Did not read the connector execution layer (executeConnector / packages/connectors) or the per-vertical workflow module internals — the tool plan/act/observe loop was traced only at the runTurn orchestration level.
- **05 backend:** guardrails.ts internals (the exact patterns in checkInputGuardrails/checkOutputGuardrails) were confirmed to exist and be called but not read line-by-line.
- **05 backend:** SSE responses always return HTTP 200 and deliver failures as an in-band `error` event; behaviour of intermediary proxies/load balancers on long-lived streams (timeouts) and the Railway/Azure edge buffering beyond the x-accel-buffering:no hint was not assessed.
- **05 backend:** Consumer-line tool calls are surfaced but (per consumer-turn.ts) not yet executed against real connectors; the memory/reminder persistence reconstructs writes from tool-call args heuristically (regex on tool name), which could mis-file or miss writes — not validated against the consumer agent tool schemas.
- **06 ai:** The licence-proposal figure of a '~6,000-char prompt-stuff → vector retrieval switch' could not be located as a literal threshold anywhere in the runtime or apps/web code. The actual switch in selectKnowledgeForPromptAsync/assemblePrompt is driven by (a) embedder availability + RUNTIME_SEMANTIC_RETRIEVAL and (b) whether retrieval returns any chunk (else raw prefix slice). Verify whether 6,000 was an intended constant that was never implemented, or lives in a doc-only description.
- **06 ai:** The '200k-char docs, 20/agent' knowledge-capacity claim is not reflected in code. Measured caps: each ingested source sliced to 100,000 chars (api/knowledge/{upload,paste,crawl}); composeKnowledge budget KNOWLEDGE_MAX_CHARS default 80,000; runtime budget RUNTIME_KNOWLEDGE_CHARS default 40,000. No per-agent document-count cap (e.g. 20) was found in the knowledge store. Confirm the source of the 200k/20 figures.
- **06 ai:** MockModelAdapter (index.ts ~627-1729, ~1,100 LOC) was only skimmed at the method/intent level; its deterministic intent-matching, knowledgeHit scoring, and pickToolByIntent heuristics were not read exhaustively. It is prototype/demo logic, not the live provider path, but is large.
- **06 ai:** HttpWalletAdapter (MIAI_WALLET_MODE=http) request/response shape was read only at signature level (POST /v1/wallets/{id}/debit, 15s abort); the live wallet service contract and error mapping were not fully traced.
- **06 ai:** executeConnector internals (@miai/connectors) — how live vs stubbed execution, connector selection, and SSRF/token-sealing behave inside a tool round — are out of scope for this section and covered by the integrations/connectors section.
- **07 api:** MCP is a sink, not a server: /api/mcp exposes NO tools/list and no JSON-RPC surface; it only records + echoes accepted POSTs. The prompt's framing of an 'MCP server' overstates it. The real tool catalogue is executed internally via packages/connectors and is not published as an HTTP tool-listing endpoint (closest is GET /api/connectors).
- **07 api:** Auth in mock mode: with MIAI_AUTH_MODE=mock (the default per /api/health) requireAuth never fails and grants ['owner','operator'] outside production, so every 'Bearer+role' route is effectively open. The documented role gates only bind under a correctly configured OIDC deployment — the handover team must confirm MIAI_AUTH_MODE=oidc + OIDC issuer/JWKS are set in prod.
- **07 api:** /api/connectors GET has no in-route auth guard; in mock mode (no edge gate) it is fully public. Low sensitivity (static connector catalogue) but worth noting.

- **07 api:** The public OpenAPI spec (`/api/v1/openapi`) documents only 2 of the 69 endpoints (`embed/chat`, `rent`). The other 67 routes have no machine-readable contract; this section is currently the most complete API reference.
- **07 api:** I did not exhaustively trace every downstream store/runtime call for each route (e.g. exact response schemas, all query-param options); tables capture method, auth/role, and purpose. Full request/response field-level schemas beyond those in `lib/api-schemas.ts` would require reading each handler body in full.
- **07 api:** Rate-limit windows (30/40 per 60s) and body caps (1 MiB middleware, 4 MiB knowledge upload) are read from code constants/env defaults; actual production values depend on `MIAI_MAX_BODY_BYTES` and any env overrides not visible in the repo.
- **08 data:** `miai_consumer_brief` is keyed by `consumer_id` alone (migration 002) with no `tenant_id` column – unlike `facts/goals/people/reminders`, the daily-brief config and last-sent brief are NOT brand-isolated, so the same `consumer_id` under two brands would share/overwrite brief state. Could not find any code that compensates for this; needs product confirmation of intent.
- **08 data:** Consumer memory tables use PK (`consumer_id`, `id`) rather than (`tenant_id`, `consumer_id`, `id`). Cross-tenant isolation relies on query-level predicates + UUID id uniqueness + the tenant-scoped unique dedupe indexes, not on the primary key. Any future query that omits the `tenant_id` predicate would leak across brands. Recommend a schema hardening review.
- **08 data:** `migrate.ts splitStatements()` is a naive semicolon splitter that strips only line comments; it will mis-split any future migration containing a semicolon inside a string literal or a DO \$\$... \$\$ / PL-pgSQL function body. Current migrations are pure DDL so this is latent, not active.
- **08 data:** No down-migrations / rollback tooling exists; schema is forward-only (migration 004 performs an in-place ALTER retrofit). DR/backup/replication is referenced by ADR 0003 as 'document in runbooks' but I did not open `docs/RAILWAY_DEPLOY.md` or the runbooks to confirm those procedures exist.
- **08 data:** `dsar-erase.eraseWorkspaceData()` erases only the workspace-scoped B2B tables and redacts audit; it does not appear to delete the consumer (tenant, consumer) memory/goal/person/reminder/brief rows. Whether a separate consumer-DSAR path covers those was not verified within this section's scope.
- **08 data:** Retention/TTL enforcement for `miai_turns` and `miai_audit` is referenced (`AUDIT_CAP` for in-memory; `docs/AUDIT_RETENTION.md` for Postgres) but the actual archival/retention job was not read and could not be confirmed to run.
- **08 data:** Deployment state: could not verify from the repo whether these 5 migrations have actually been applied against the live Replit/Neon production database, nor whether Upstash Redis is provisioned in production (session durability depends on it).
- **09 integrations:** `/api/mcp` is NOT a JSON-RPC 2.0 MCP server: grep of `apps/web/src/app/api` found no `jsonrpc`, `initialize`, or `tools/list`. Both the outbound connector (`handlers/mcp.ts`) and inbound sink (`api/mcp/tools/call`) use a flat POST `/tools/call {name, arguments}` contract. The DD brief's 'document the JSON-RPC tools' cannot be satisfied as written – the surface is an HTTP tools/call bridge / Wave-4 proof sink, not a discoverable JSON-RPC tool server.
- **09 integrations:** No inbound WhatsApp channel exists. WhatsApp is only the outbound `whatsapp` connector (`whatsappSend`, Cloud API). There is no `api/.../whatsapp` webhook route to receive inbound messages, so WhatsApp is not a full conversational channel like Telegram/embed at commit fb97438.
- **09 integrations:** `calendlyBook`, `xeroRead`, and `quickbooksRead` are read/echo handlers, not transactional writes. `calendlyBook` returns the user's `scheduling_url` (does not create a booking); `xeroRead` lists invoices and echoes args; `quickbooksRead` queries invoices and echoes args. Any DD claim that these 'book' or 'write' should be qualified.
- **09 integrations:** Live health probes (`/api/oauth/[connector]/test`) exist for only 4 connectors (`slack`, `google_calendar`, `hubspot`, `email`); all others return `probe_not_supported`. There is no automated connectivity test for `Shopify`, `Zendesk`, `Xero`, `QuickBooks`, `Calendly`, `Teams`, `M365`, etc.

- **09 integrations:** Token store still accepts legacy `v1.` (HMAC integrity-only, not encrypted) and bare-plaintext token envelopes for migration; webhook verify still accepts a raw shared secret unless `allowLegacyRawSecret:false` / `WEBHOOK_SINK_HMAC_ONLY=1`. These migration paths remain live and should be closed before handover.
- **09 integrations:** Channel session history for embed/app is stored in an in-process bag (`createSessionStore` with `redisPrefix 'miai:chan:'`); whether a Redis backend is actually wired in production vs. in-memory-only was not verified in this section's scope.
- **09 integrations:** Connector package size: measured src TS = ~3,961 LOC / 12 files; with test/ = 4,293 LOC / 19 files (matches the lead's 4,293/19 figure — the 4,293 counts tests). `OAUTH_PROVIDERS` contains 18 providers and `ConnectorId` is a 24-member union, vs the 16-entry `CONNECTORS` Actions catalogue — worth reconciling in any headline connector count.
- **10 security:** B2B OIDC Bearer verification (`apps/web/src/lib/auth.ts`) calls `jwtVerify` without an explicit `algorithms` allow-list; it relies on the remote JWKS supplying only asymmetric keys. The consumer path pins `['HS256']` explicitly. Recommend adding an explicit `RS256/ES256` allow-list on the B2B path as defence-in-depth against algorithm-confusion.
- **10 security:** Embed key verification (`store.ts::resolveEmbedKey`) matches a truncated 10-hex-char (40-bit) HMAC with a non-constant-time `===` comparison. `mia_pk_` is a publishable key (lower sensitivity, further gated by `CORS/rate-limit/wallet`), but a timing-safe compare and/or longer tag would harden it.
- **10 security:** CSP retains `style-src 'unsafe-inline'` (documented as deferred in `csp.ts` for next/font + Tailwind). `Script-src` is clean (`nonce + strict-dynamic`), so residual risk is style-injection only, but it is not yet closed.
- **10 security:** The task's stated 'MCP description sanitisation' control was not found as dedicated code: the platform's `/api/mcp` is an OUTBOUND sink and does not import third-party MCP servers' tool descriptions into the model prompt, so there is no current injection vector there. Verified prompt-injection defences are HTML-stripping of web-search snippets, newline-collapse on connector free-text, parse-only structured `tool_calls`, tool results carried as `role:'tool'` data, and deterministic output guardrails. If a future connector ingests remote MCP tool descriptions into context, a strip/whitelist step should be added.
- **10 security:** `assertProductionSecrets()` only logs errors; the throwing boot gate is `assertBootHardening()` (composing `secrets+rails+persistence` checks) invoked from `instrumentation.ts`. Confirm the production deployment actually reaches the Node instrumentation `register()` path (`NEXT_RUNTIME=nodejs`) so the fail-closed throw fires — not verified at runtime here.
- **10 security:** Rate limiting was confirmed on 7 abuse-prone routes; a full audit of all 69 API routes for rate-limit coverage (e.g. the B2B `rent/admin/ops` routes) was not performed. Bearer auth gates those, but per-tenant rate limits on authenticated mutating routes were not exhaustively verified.
- **10 security:** Consumer session cookie is `sameSite=lax` with a 30-day lifetime; state-changing consumer routes rely on `same-origin + POST` rather than an explicit anti-CSRF token on each data route. Confirm no state-changing consumer action is reachable via a simple top-level GET/form navigation.
- **10 security:** Did not runtime-verify the Postgres schema-level constraints (e.g. that the `ON CONFLICT` composite keys and `NOT NULL` tenant columns exist as written in the 001..005 migrations); tenant-scoping was verified at the query layer in application code only.
- **11 compliance:** Turn-transcript (`miai_turns`) retention period is documented as TBD in `AUDIT_RETENTION.md` and not enforced by a scheduler; only a 20,000-row in-memory/file `TURN_CAP` exists. Actual production retention behaviour unresolved.
- **11 compliance:** `AUDIT_RETENTION.md` claims a Postgres `DELETE` cap of 20,000 audit rows, but `insertAuditRow()` contains no `DELETE` — Postgres audit is effectively append-only/unbounded. The true production retention/cost profile of `miai_audit` growth was not measured.
- **11 compliance:** Erase route notice states audit detail is 'redacted in memory only' while `redactWorkspaceAuditDetails` also issues a Postgres `UPDATE` — did not runtime-verify against a live Postgres instance which path executes in the deployed

config (Postgres vs file fallback).

- **11 compliance:** ai-disclosure.ts shared constants are imported by zero files (dead code); disclosure copy is duplicated as literals on a subset of surfaces. Did not exhaustively enumerate every chat/embed/WhatsApp/Telegram surface to confirm disclosure is present on ALL of them — some channels may omit it.
- **11 compliance:** Consent choice is recorded to the audit log but no code was found that consults stored consent before analytics/telemetry/processing; whether any downstream gating exists elsewhere was not exhaustively proven.
- **11 compliance:** Market-pack compliance is verified as prompt/manifest/guardrail text baked at build time; there is no runtime enforcement engine. Whether the deployed LLM reliably honours these instructions is a model-behaviour question outside static code review.
- **11 compliance:** Did not open every file in docs/compliance/ (ROPA, DPIA, breach runbook, PCI-SAQ, SOC2 index, DPA/BAA templates) line-by-line — confirmed their presence, draft banners, and ADR 0005 status but not full contents.
- **11 compliance:** PII redaction quality (false-negative rate of the regex on names/addresses/non-US-ZA phone formats) was not empirically tested.
- **12 devops:** Runtime env state is not observable from the repo: cannot confirm whether the live Railway production service currently has DATABASE_URL set (Postgres) vs file-fallback, nor the actual values of ALLOW MOCK_RAILS / OIDC / wallet / model env on either running service.
- **12 devops:** No evidence the Azure Container Apps deployment has actually been executed — infra/azure Bicep + MIGRATION_RUNBOOK exist and are validated offline via validate-azure.sh, but there is no state file, deployment output, or live Azure FQDN committed; Azure remains a documented target, not a confirmed running environment.
- **12 devops:** No in-repo continuous-deployment workflow: Railway auto-deploy is configured in the Railway dashboard (not in the repo) and Azure deploy is a manual `az deployment group create`. The publish-image.yml pushes to GHCR but nothing in .github/workflows pulls/rolls that image onto a running environment.
- **12 devops:** SonarCloud gating is unverifiable from the repo: sonar-project.properties is present but no workflow runs a Sonar scanner, so it is presumably SonarCloud automatic analysis via the GitHub App; the 'quality' required-merge-check and branch protection are GitHub repo settings not visible in the codebase.
- **12 devops:** SANDBOX_MODEL_TURN_CAP default is 1000 in packages/runtime/src/index.ts (`Number(...) || 1000`); docs/SANDBOX.md describes the cap behaviour without stating the number — confirm intended default with the team (1000 real-provider turns per process is generous for a cost-capped sandbox).
- **12 devops:** Dockerfile builder stage runs `pnpm install --frozen-lockfile` twice (once in deps, once after COPY . . in builder) — a redundant install; not a defect but a build-time optimisation opportunity (BuildKit cache mounts / single install).
- **12 devops:** Redis backend for shared state: REDIS_URL is documented as reserved/unused in v1 and only the Upstash REST client is wired; multi-replica scale on Azure (maxReplicas > 1) is blocked until Upstash is provisioned — no native Redis/Azure Cache path is implemented yet.
- **13 deps:** No `engines` field in any package.json; Node 20 is enforced only via the Dockerfile base image (node:20-bookworm-slim) and CI, not by the manifests — a local dev on a different Node major is not blocked. Recommend adding `engines.node` and `engines.pnpm`.
- **13 deps:** `eslint-config-next` is pinned at exactly 15.1.0 while `next` resolves to 15.5.22 — a lint-config/framework minor skew that should be realigned on the next Next.js bump.
- **13 deps:** Headline framework versions have floated ahead of declared caret floors (e.g. next `^15.2.8` resolves to 15.5.22, zod `^3.24.2` → 3.25.76, typescript `^5` → 5.9.3); any doc quoting the floors understates what actually installs.
- **13 deps:** apps/mobile-shell is excluded from the pnpm workspace and pins a separate React lineage (react 18.3.1 / react-native 0.76.3, Expo ~52). Its dependency tree was not resolved into the root lockfile and was not audited here beyond its package.json — a full review of the Expo island is out of scope for this section.

- **13 deps:** I did not run `pnpm audit` /CVE scanning against the ~754 resolved packages; SonarCloud is wired as advisory but no vulnerability report was produced during this review.
- **13 deps:** The embeddable widget's `agent.js` (documented as ~6.4kB zero-dep) was not located on disk at `apps/web/public` during this pass (likely generated/served dynamically); its zero-dependency claim was not independently size-verified here.
- **14 performance:** The internal 'prompt-stuff below ~6,000 chars / vector retrieval above ~6,000 chars' description is NOT supported by code: a repo-wide grep for 6000/6_000/6,000 finds no retrieval threshold. The real stuffing-vs-retrieval switch is model type + embedder presence (MockModel stuffs to the 40k budget; live models always retrieve; hybrid only when an embedding key is configured). The section documents the actual mechanism; the '~6k' figure should be corrected in any summary that repeats it.
- **14 performance:** No load/performance benchmarks exist in the repo (no k6/Artillery/throughput or latency artefacts). All performance figures given are structural (pool size, per-turn round-trip counts, rate-limit ceilings, char budgets), not measured RPS/latency/concurrency numbers. A real capacity test is needed before committing to SLAs.
- **14 performance:** Postgres pool sets only max:10 with no connectionTimeoutMillis/statement_timeout/min — behaviour under pool exhaustion or a slow query (indefinite queueing) is inferred from pg library defaults, not from explicit config; confirm intended production pool sizing per replica.
- **14 performance:** Knowledge hydrate-once loads the entire `miai_knowledge_sources` table (no WHERE) into every replica heap and is not read-through, implying per-replica memory growth with total tenant corpus and cross-replica staleness until restart. This was verified in code but its production impact (heap size at N tenants, staleness window) is unquantified.
- **14 performance:** `apps/runtime worker` (16 LOC) is a scaffold; tool loops run in-process in the Next.js request worker. The intended queue/offload architecture for Container Apps is documented as future, so real horizontal throughput of long tool-loop turns is untested.
- **14 performance:** `estimateTurnTokens` is an explicit char/4 'prototype multipliers' heuristic used when the provider omits usage; its accuracy vs real provider billing on non-usage-reporting paths is unverified.
- **14 performance:** The 1 MiB body cap in middleware checks the content-length header only; requests without/with chunked content-length are not pre-gated at the edge (relies on downstream `req.json()+Zod`). Edge behaviour under a missing content-length was not runtime-tested.
- **15 testing:** Playwright evidence is stale relative to HEAD: `docs/HANDOVER_TEST_PACK.md` records 78/78 on 2026-08-02 @ commit `feb6d43`, but the tree at HEAD `fb97438` has 33 specs / 66 static test() blocks (runtime-expanded via parameterization). No fresh full-run pass count exists at HEAD; recommend re-running `pnpm handover:staging` and updating the pack.
- **15 testing:** Current MockModel eval pass-rate and static-high-issue count are not captured in-repo at HEAD — `data/reports/eval-results.json` is generated output; the actual latest numbers (and whether the 35% floor / zero-static-high gate currently passes) could not be verified without running `pnpm eval:suite`.
- **15 testing:** Live-LLM scoreboard figures depend on `docs/reports/eval-live-*.md` and `data/wave4-live-proofs.json`, whose current contents were not enumerated; real live pass-rates per hero set and the connector-proof-count (`agentsProven/sliceTarget`) at HEAD are unverified.
- **15 testing:** The claim that CI job `quality` is a *required* merge check comes from `docs/TESTING.md`; GitHub branch-protection rules are not in the repo, so 'required' status cannot be confirmed from code alone.
- **15 testing:** SonarCloud: `sonar-project.properties` is present but no `.github/workflows` file invokes the SonarCloud scanner, so its 'advisory, not a gate' status is inferred; any external SonarCloud PR-decoration integration is not verifiable from the repo.
- **15 testing:** `playwright.config.ts` has no `webServer` block and defaults `baseUrl` to Railway staging — E2E specs require an already-running target; there is no self-contained local E2E harness, so 'green E2E' is only meaningful against a live

deployment.

- **15 testing:** Eval-case totals (8,526 B2B + 302 consumer) were counted from *.agent.json on disk (552 + 17 packs); the 552 includes ~52 legacy unprefixes ZA aliases, so per-SKU (500) eval density differs slightly from per-file counts.
- **16 docs:** OpenAPI completeness: /api/v1/openapi/route.ts was read only for the embedChat operation and the info/servers block; I confirmed the spec documents embedChat and rentAgent (per dossier line 1305/1226) but did not read the full route.ts to enumerate every documented path — the 'only two operations' characterisation is based on the ~74-line file size and dossier cross-references, not a line-by-line schema audit.
- **16 docs:** Comment-density metric is heuristic: 948 'comment-ish' lines counted by grep for lines beginning with //, / or across apps/web/src/lib/*.ts, and 286 JSDoc-opening blocks; this over-counts multi-line block-comment continuation lines and does not measure the ~410-file runtime as a whole (only the 80-file lib layer was measured).
- **16 docs:** docs/compliance/*, most migration docs, TRUST_AND_COMPLIANCE.md, and the 100 pilot files were characterised from headers/first lines and one sample (customer-support.md), not read in full — group descriptions are accurate to the headers but per-file content beyond the first lines was not exhaustively verified.
- **16 docs:** The dossier's 'Open items to confirm' ledger was read in full and its ~55 caveats are summarised second-hand; I did not independently re-verify each underlying claim (e.g. the next@15.5.22 lockfile resolution, the absence of an inbound WhatsApp webhook) against source in this section's scope.
- **16 docs:** The PRODUCTION_SCALE_410.md dangling reference was confirmed by grep (referenced in PARALLEL_WORKSTREAMS.md / PILOT_PRODUCTION_BAR.md, file absent; only PRODUCTION_SCALE_500.md exists), but I did not check every doc for other stale cross-links — additional dangling references may exist.
- **16 docs:** docs/reports/ contents were inventoried by filename/size only (not opened), so the report-type descriptions rely on filenames plus the dossier; the actual pass/fail contents of audit and certification reports were not read in this section.
- **17 hygiene:** Branch-protection settings are not in the repo — COORDINATION.md documents 'quality' as the required merge check, but whether GitHub also marks the 'secrets' (gitLeaks) job and the SonarCloud App status as required-to-merge cannot be confirmed from the code alone.
- **17 hygiene:** SonarCloud is inferred to run as the GitHub App (no workflow invokes it); the actual quality-gate thresholds/pass-fail configured in SonarCloud are not visible in the repo.
- **17 hygiene:** The eslint-config-next major/minor lag (15.1.0 vs next 15.5.22) and skipLibCheck:true are stated as low-risk but their real-world impact was not exercised (no lint/typecheck run performed in this read-only pass).
- **17 hygiene:** The '~2 any occurrences in apps/web/src' figure is a grep heuristic (': any', 'as any', '<any>') and may miss aliased/indirect any usage; packages/* were not separately counted for any.
- **17 hygiene:** Node-version divergence (CI Node 22 vs stated runtime Node 20) is drawn from ci.yml + CONTRIBUTING; the Dockerfile base image Node line was not re-read in this pass to confirm the exact runtime major.
- **18 patterns:** Loop-bound correction: the plan→act→observe loop is NOT a fixed ≤3 steps — it is `Math.min(5, Math.max(1, Number(env('RUNTIME_MAX_TOOL_ROUNDS') ?? 3)))` in packages/runtime/src/index.ts (default 3, env-tunable, hard-clamped to [1,5]). Reported accurately in §18.8.
- **18 patterns:** I read the pattern seams of packages/runtime/src/index.ts (116KB) selectively (adapter classes, factory, runTurn body, tool-round loop) but did not read all ~2,900 lines; the 10+ per-vertical workflow modules under packages/runtime/src/workflows were confirmed to exist and be invoked via finishWorkflow() but their internals were not individually reviewed for this patterns section.
- **18 patterns:** The `certified` gate semantics ('3x-majority eval pass') are quoted from the consumer-catalog.ts source comment; the actual eval-certification pipeline that sets the flag (eval suite / catalog integrity scripts) was not traced end-to-end here.
- **18 patterns:** Preset counts measured programmatically: generated-presets.ts has 551 `"agentId":` entries and 11,761 lines; index.ts HAND_OVERRIDES has 48 `agentId:` occurrences (~46 override entries after accounting for the

interface/type lines) – exact override entry count not hand-verified one-by-one.

- **18 patterns:** AES-256-GCM token sealing and the v1→v2 migration path were confirmed by grep on packages/connectors/src/oauth/tokens.ts (seal/open, createCipheriv 'aes-256-gcm') but the full key-derivation (aesKey()) was not read line-by-line.
- **19 metrics:** Consumer catalogue count: the provided ground truth says '17 consumer specialists', but data/catalog-consumer holds 17 *.agent.json files (+ index.json = 18 files total). The '18' likely counted files, not agents. Reported as 17 agents / 18 files.
- **19 metrics:** On-disk agent-file count is 552 (500 indexed + 52 legacy unprefixed aliases), but docs/CATALOGUE_READY.md's catalog:ready gate references '551/551'. The 1-file delta (likely one alias, e.g. personal-assistant, excluded from the go-live filter) was not fully traced; measured on-disk count is 552.
- **19 metrics:** Two legitimate counting views exist for tools and evals and both are reported: indexed-500 view from index.json fields (1,890 tools / 7,640 evals) vs on-disk view counting the arrays across all 552 packages (2,128 tools / 8,526 evals). The 'growth' between them is the 52 alias files. The ground-truth headlines (~1,900 tools, 8,500+ evals) map to indexed-tools and on-disk-evals respectively.
- **19 metrics:** The 19-industry family distribution was computed by re-implementing marketplaceCategory()'s regex logic in Node against family ids (the function is TS and not directly executable in the scratch script). Totals reconcile to exactly 100 families / 19 industries, but a family whose id matched none of the regexes would fall through to its category-map label rather than a dedicated industry – no such fall-through occurred in this run.
- **19 metrics:** Guardrails average size (~5,000 chars) is extrapolated from the sampled agent (5,042 chars) plus the all-file scan; a full per-file mean for guardrails specifically was not separately tabulated.
- **19 metrics:** List-price ARR (\$4.395M) is gross list rent summing one of each 500 SKUs at RENT_USD; it is an asset-valuation illustration, not a booked-revenue or occupancy figure.
- **20 cost:** Effort-per-KLOC calibration is a modelling judgement, not a measured fact: the ~2,000-2,300 LOC/PM modern-greenfield productivity and the ~\$14K-17K/PM loaded rate are industry-typical assumptions chosen to bracket the target range. Actual as-built effort/timeline was not reconstructed from git history (commit-count/author-timeline analysis was out of scope) and could differ materially from the reverse-engineered estimate.
- **20 cost:** Catalogue authoring effort (100 families @ 3-4 person-days, 400 derivations @ ~0.5 person-day) is an inferred model from the 100x5 ADR and file structure, not a measured record of hours spent; heavy tooling (generate:packs, polish:catalog, AI-assisted authoring) could push actual per-family effort lower.
- **20 cost:** The 2,128 tool count is the sum of top-level `tools` arrays across 552 agent files; a handful of files failed strict parse checks earlier are not expected but were not individually audited, so the true count could vary by a small margin.
- **20 cost:** COCOMO-81 basic coefficients were used (no COCOMO II effort-multipliers/scale-factors calibration and no function-point cross-check), so the runtime upper anchor (190-227 PM) is a coarse reference rather than a tuned estimate.
- **20 cost:** Test/eval and hardening effort is folded into the two streams rather than costed separately; a full lifecycle model (requirements, design, QA, security review, docs) could allocate additional person-months not itemised here.
- **21 visual:** Connector Action count of 16 is taken from lead-verified ground truth; a code grep of live handlers surfaced a broader identifier set (google_calendar/contacts/drive/tasks, m365_calendar, teams, slack, shopify, hubspot, xero, quickbooks, calendly, zendesk, webhook, mcp, plus notion/spotify/todoist/youtube strings). No single authoritative ACTIONS registry/array was found to pin the exact live-vs-planned Action total – the '16 Live Actions' split should be confirmed against the connectors registry before publication.
- **21 visual:** Readiness Live/Partial/Planned percentages are derived from raw status-tag string occurrences in apps/web/src (live ~80, partial ~8, planned ~8), not from a structured capability manifest. The exact per-feature tri-state should be read

directly from the Trust Center data model (apps/web/src/app/trust/page.tsx and its data source) for an authoritative count.

- **21 visual:** The 'Other' LOC bucket (31,209) is computed as the residual (total 406,319 minus catalogue 299,508 minus runtime 75,602) rather than measured directly; it aggregates docs (.md), SQL, CSS, config JSON and catalogue index files. A direct per-extension breakdown was not produced.
- **21 visual:** Total authored 406,319 lines / 1,212 files is the lead-verified aggregate; component buckets were independently measured and reconcile exactly to it, but the top-line file count (1,212) itself was not re-counted in this pass.
- **21 visual:** EUR tier prices (RENT_EUR) are confirmed to exist in packages/agent-protocol but their exact numeric values were not extracted; only USD (349/699/1199) values were read.
- **22 partnership:** Handover Playwright scenario count is inconsistent in-repo: README.md line 66 labels `pnpm handover:staging` as '76 scenarios', while docs/HANDOVER_TEST_PACK.md records '78/78 Playwright passed' (2026-08-02, commit feb6d43). I cited 78/78 from the test pack; the exact current count should be re-confirmed by running `pnpm handover:staging`.
- **22 partnership:** Topology time estimates (A now / B ~2 weeks / C ~3 weeks / D week-scale) and the 14-day window breakdown are the lead-provided verified plan and the runbook's phased/30-day framing, not an independently timed engineering estimate against a specific partner environment; actual duration depends on when partner OIDC/wallet/model/DNS/WABA credentials land.
- **22 partnership:** The '~6 modules' extraction figure is grounded in the internal workspace dependency graph (runtime → agent-protocol/connectors/presets/wallet-adapter; presets → connectors; + pg), but no repo doc states a literal '6 modules' extraction count or a formal standalone-container extraction runbook — D's effort is inferred from the bounded dep graph plus the existing Azure Bicep, not from a dedicated extraction guide.
- **22 partnership:** Redis for multi-replica scale in Topology D is optional Upstash (REST) today; the Bicep template (infra/azure/main.bicep) provisions Postgres + Files but does not itself provision a Redis/cache resource, so standing up Redis in the partner tenancy is additional IaC not yet in-repo.
- **22 partnership:** The last recorded green handover/CI run is dated 2026-08-02 at commit feb6d43; the current HEAD is fb97438. A fresh `pnpm run ci` + `pnpm handover:staging` should be run at handover to re-establish the evidence at the delivered commit.